Milestone One — Notebook Appendix

Predicting Consumer Credit Default Across Two Markets

Kumar Gaurav · DX799 O1 Data Science Capstone (Fall 2026) · Boston University OMDS

Contents

1. Week 1 — Polynomial and Interaction Terms
   `week_08282026/DX79901/homework/kumar_gaurav_week1_dx79901_v2.ipynb`
2. Week 2 — Ridge, Lasso and Elastic Net
   `week_09042026/DX79901/homework/kumar_gaurav_week2_dx79901_v2.ipynb`
3. Week 3 — Stepwise Selection, PCR and PLSR
   `week_09112026/DX79901/homework/kumar_gaurav_week3_dx79901_v2.ipynb`
4. Week 4 — Logistic Regression and Feature Scaling
   `week_09182026/DX79901/homework/kumar_gaurav_week4_dx79901_v2.ipynb`
5. Week 5 — Support Vector Machines
   `week_09252026/DX79901/homework/kumar_gaurav_week5_dx79901_v2.ipynb`
6. Week 6 — Decision Trees and Random Forests
   `week_10022026/DX79901/homework/kumar_gaurav_week6_dx79901_v2.ipynb`

All outputs shown are the saved executed results.

Week 1 — Polynomial and Interaction Terms

`week_08282026/DX79901/homework/kumar_gaurav_week1_dx79901_v2.ipynb`

Capstone Milestone One — Week 1: Polynomial and Interaction Terms

Kumar Gaurav · DX799 O1 Data Science Capstone

Project Overview

Consumer credit risk is a central concern for lenders because mispriced loans and misjudged credit limits translate directly into losses and lost customers.

In this analysis, I use two consumer credit datasets to investigate how borrower characteristics relate to two risk-based lender decisions: the interest rate charged on an installment loan, and the credit limit granted on a revolving credit card.

Research Question

Can borrower demographic, credit history, and repayment behavior be used to explain the risk-based prices and limits that lenders set?

The two target variables used here are:

- **Lending Club** `int_rate` — the interest rate (%) charged on an installment loan.
- **Taiwan** `log(LIMIT_BAL)` — the log of the credit limit granted on a revolving credit card.

Objectives

The objectives of this analysis are to:

- Audit each dataset for missing values, duplicates, invalid ranges, and target leakage before any modeling.
- Fit linear regression models that include polynomial, interaction, and categorical terms.
- Diagnose multicollinearity using the Variance Inflation Factor (VIF) and show how centering changes the result.
- Compare models using training R^2, adjusted R^2, AIC, test RMSE, and partial F-tests.

- Interpret the resulting coefficients within the fitted linear model.

Approach

The analysis follows a two-part structure:

Data Quality Audit → Lending Club Modeling → Taiwan Modeling → Summary

Each dataset is audited independently before any model is fit, so that cleaning decisions are documented and visible to the reader.

Datasets

- **Lending Club** (Kaggle `adarshsng/lending-club-loan-data-csv`): US peer-to-peer installment loans, 2007–2018.
- **UCI Default of Credit Card Clients** (Yeh & Lien, 2009; Kaggle `uciml/default-of-credit-card-clients-dataset`): 30,000 Taiwanese credit-card clients, April–September 2005, with a target of default in October 2005.

Part A — Data Quality Audit

Before fitting any model, I check the data in the way a careful analyst would by hand: what is in the file, what is missing, what is duplicated, which values are impossible, which columns would leak the answer, and what the target looks like.

Every issue identified is recorded in a data-quality summary table (issue, rows affected, action) so a reader can judge how much the cleaning might have influenced the modeling results.

1. Data Loading and Initial Setup

Everything that follows depends on reading the raw Lending Club file correctly. The file contains 145 columns and about 2.26 million rows, so instead of loading all of it I read only the subset of columns that will be audited or modeled.

A fixed random seed is set so that any sampling or splitting below is reproducible. Four columns known only *after* the loan outcome (`total_pymnt`, `recoveries`, `last_pymnt_amnt`, `out_prncp`) are loaded intentionally so that the leakage screen can later demonstrate why they must be removed.

In [1]:
```
import os, warnings
import numpy as np, pandas as pd
import matplotlib.pyplot as plt, seaborn as sns
```

```
import kagglehub
warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")
pd.set_option("display.width", 160)
SEED = 42

# Lending Club: read only the columns we audit/model (the raw file is
2.26M x 145)
LC_PATH = os.path.join(kagglehub.dataset_download("adarshsng/lending-
club-loan-data-csv/versions/1"), "loan.csv")
LC_COLS = ["id", "loan_status", "loan_amnt", "term", "int_rate",
"grade", "sub_grade", "emp_length",
           "home_ownership", "annual_inc", "verification_status",
"purpose", "dti", "delinq_2yrs",
           "inq_last_6mths", "open_acc", "pub_rec", "revol_bal",
"revol_util", "total_acc", "mort_acc",
           "pub_rec_bankruptcies", "application_type", "policy_code",
           # post-outcome columns, loaded ONLY to demonstrate the
leakage screen
           "total_pymnt", "recoveries", "last_pymnt_amnt", "out_prncp"]
lc_raw = pd.read_csv(LC_PATH, usecols=LC_COLS, low_memory=False)
print(f"Lending Club raw shape (audited columns): {lc_raw.shape}")
print(lc_raw.dtypes.value_counts().to_string())
display(lc_raw.head(3))
```

```
Lending Club raw shape (audited columns): (2260668, 28)
float64    16
str         9
int64       3
```

	id	loan_amnt	term	int_rate	grade	sub_grade	emp_length	home_ownership	annual_inc	verification_status	...	revol_util	total_acc	out_prncp	total_pymnt	recoveries	last_pymnt_amnt	policy_code	applic
0	NaN	2500	36 months	13.56	C	C1	10+ years	RENT	55000.0	Not Verified	...	10.3	34.0	2386.02	167.02	0.0	84.92	1	
1	NaN	30000	60 months	18.94	D	D2	10+ years	MORTGAGE	90000.0	Source Verified	...	24.2	44.0	29387.75	1507.11	0.0	777.23	1	
2	NaN	5000	36 months	17.97	D	D1	6 years	MORTGAGE	59280.0	Source Verified	...	19.1	13.0	4787.21	353.89	0.0	180.69	1	

Load Findings

The audited subset loaded as 2,260,668 rows by 28 columns, with 16 float, 9 text, and 3 integer columns. The first three rows show plausible loan amounts between $2,500 and $30,000 and interest rates between 13.56% and 18.94%.

Two issues are already visible from the first rows. The `id` column is empty, and the `term` column contains a leading space (e.g., `" 36 months"`) that will need to be parsed. The non-zero values in `out_prncp` for the first rows reflect loans that are still *current*, which is handled by the censoring and leakage checks below.

Missing Value Analysis

Missing data silently reduces or biases the sample. Columns that are mostly empty cannot be used, and columns that are entirely empty are useless — including identifier columns that might otherwise have been used to detect duplicates.

Because the raw file is too large to load fully into memory, the null count is accumulated across chunks of 500,000 rows. The missing percentage per column is then computed as the chunked count divided by total rows.

In [2]:
```
# Null audit on the FULL 145-column header (chunked so memory stays
small)
null_cnt, n_rows = None, 0
for ch in pd.read_csv(LC_PATH, chunksize=500_000, low_memory=False):
    c = ch.isnull().sum(); null_cnt = c if null_cnt is None else
null_cnt + c; n_rows += len(ch)
null_pct = (null_cnt / n_rows * 100).sort_values(ascending=False)
print(f"All columns: {len(null_pct)} | rows: {n_rows:,}")
print(f"Columns 100% empty: {(null_pct == 100).sum()} ->
{null_pct[null_pct == 100].index.tolist()}")
print(f"Columns > 50% missing (dropped from consideration): {(null_pct >
50).sum()}")
print("\nMissing % for the columns we use:")
use_pct = null_pct.reindex([c for c in LC_COLS if c != "id"]).round(3)
print(use_pct[use_pct > 0].to_string())
```

```
All columns: 145 | rows: 2,260,668
Columns 100% empty: 3 -> ['id', 'url', 'member_id']
Columns > 50% missing (dropped from consideration): 44

Missing % for the columns we use:
emp_length              6.498
dti                     0.076
delinq_2yrs             0.001
inq_last_6mths          0.001
open_acc                0.001
pub_rec                 0.001
revol_util              0.080
total_acc               0.001
mort_acc                2.213
pub_rec_bankruptcies    0.060
```

Missing Value Findings

Three of the 145 columns are entirely empty (`id` , `url` , `member_id`), and 44 columns are more than 50% missing. The latter group consists mostly of hardship, settlement, and joint-applicant fields that only apply to a small share of loans, and none of them are used in modeling.

Among the columns that will be used, missingness is small: `emp_length` 6.50%, `mort_acc` 2.21%, `revol_util` 0.080%, `dti` 0.076%, `pub_rec_bankruptcies` 0.060%, and the remainder about 0.001%.

Because missingness in the model columns is low, dropping incomplete rows removes only a small share of the data. `emp_length` is excluded from the models separately

because its missingness may not be random — borrowers who leave the field blank may differ in risk from those who do not.

The fact that `id` is entirely empty means duplicates must be detected by comparing whole rows rather than by key.

Cleaning Decisions — Outcomes, Duplicates, Invalid Values, and Leakage

A model will learn whatever is in the data, including errors. This step applies explicit, documented cleaning rules:

- **Outcome definition.** Only closed loans (`Fully Paid`, `Charged Off`, `Default`) have a known outcome. `Current` and late loans are censored, so including them would mislabel future defaults as repaid.
- **Duplicates.** Because `id` is empty in this Kaggle copy, duplicates are detected across whole rows.
- **Impossible values.** `dti` (debt-to-income) below 0 or above 100 (including the sentinel value 999) is treated as missing. Annual income of 0 is dropped because it breaks any log or ratio.
- **Extreme but possible values.** A `revol_util` above 100% means the borrower is over their credit limit. This is unusual but real, so values are clipped at 150 rather than dropped.
- **Rare categories.** `home_ownership` codes `ANY` and `NONE` are merged into `OTHER` so that one-hot encoding does not create near-empty columns.
- **Constant columns** carry no information and are dropped.
- **Leakage screen.** A column known only after the outcome (payments received, recoveries) would make the model look brilliant and be useless in practice. The mean of each post-outcome column is printed by outcome to show the leak before the columns are dropped.

Imputation (median or model-based) and winsorizing at percentiles are alternatives to the chosen listwise-drop and clipping rules. With under 1% of rows affected by any single check, listwise dropping is simpler and easier to defend.

In [3]:
```
# Keep closed loans only (outcome known); 'Current'/'Late' loans are
censored
CLOSED = ["Fully Paid", "Charged Off", "Default"]
print("loan_status counts:");
print(lc_raw["loan_status"].value_counts().to_string())
lc = lc_raw[lc_raw["loan_status"].isin(CLOSED)].copy()
print(f"\nClosed loans: {len(lc):,}")

dq = []  # (issue, rows affected, action)
dq.append(("Open/censored loan_status", len(lc_raw) - len(lc), "dropped
```

```
(outcome unknown)"))
dq.append(("id column 100% null", 0, "cannot dedupe on id; deduped on
full row instead"))
dups = lc.drop(columns=["id"]).duplicated().sum(); dq.append(("Exact
duplicate rows", dups, "dropped"))
lc = lc.drop(columns=["id"]).drop_duplicates()

# Invalid / impossible ranges
bad_dti = ((lc["dti"] < 0) | (lc["dti"] > 100)).sum()
dq.append(("dti < 0 or > 100% (sentinel 999 / data error)", bad_dti,
"set to NaN then dropped"))
lc.loc[(lc["dti"] < 0) | (lc["dti"] > 100), "dti"] = np.nan
hi_util = (lc["revol_util"] > 100).sum()
dq.append(("revol_util > 100%", hi_util, "clipped to 150 (over-limit is
real but extreme)"))
lc["revol_util"] = lc["revol_util"].clip(upper=150)
zero_inc = (lc["annual_inc"] <= 0).sum()
dq.append(("annual_inc <= 0", zero_inc, "dropped (cannot log/ratio)"))
lc = lc[lc["annual_inc"] > 0]
hi_inc = (lc["annual_inc"] > 1e6).sum()
dq.append(("annual_inc > $1M", hi_inc, "kept; modeled as
log1p(annual_inc)"))
odd_home = lc["home_ownership"].isin(["ANY", "NONE", "OTHER"]).sum()
dq.append(("home_ownership ANY/NONE/OTHER", odd_home, "merged into
OTHER"))
lc["home_ownership"] = lc["home_ownership"].replace({"ANY": "OTHER",
"NONE": "OTHER"})

# Constant / near-constant and cardinality
const = [c for c in lc.columns if lc[c].nunique(dropna=False) <= 1]
print("Constant columns:", const)
dq.append(("Constant columns " + str(const), 0, "column dropped (no
information)"))
lc = lc.drop(columns=const)
cat_cols = lc.select_dtypes("object").columns
print("Categorical cardinality:", lc[cat_cols].nunique().to_dict())

# Leakage screen: columns only known AFTER the outcome
LEAK = ["total_pymnt", "recoveries", "last_pymnt_amnt", "out_prncp"]
print("\nLeakage check: mean of post-outcome columns by outcome")
print(lc.groupby(lc["loan_status"] != "Fully Paid")
[LEAK].mean().round(1).rename(index={False: "repaid", True:
"default"}).to_string())
dq.append(("Post-outcome columns " + ", ".join(LEAK), 0, "columns
dropped (target leakage)"))
lc = lc.drop(columns=LEAK)

lc["default"] = (lc["loan_status"] != "Fully Paid").astype(int)
before = len(lc)
lc = lc.dropna(subset=["dti", "revol_util", "mort_acc",
"pub_rec_bankruptcies", "inq_last_6mths", "open_acc", "total_acc"])
dq.append(("Remaining NaNs in model columns", before - len(lc), "dropped
(listwise)"))
lc["log_annual_inc"] = np.log1p(lc["annual_inc"]); lc["log_revol_bal"] =
np.log1p(lc["revol_bal"])
```

```
lc["term_months"] = lc["term"].str.extract(r"(\d+)").astype(int)
print(f"\nClean Lending Club rows: {len(lc):,} | default rate:
{lc['default'].mean():.2%}")
```

```
loan_status counts:
loan_status
Fully Paid                                             1041952
Current                                                 919695
Charged Off                                             261655
Late (31-120 days)                                       21897
In Grace Period                                           8952
Late (16-30 days)                                         3737
Does not meet the credit policy. Status:Fully Paid        1988
Does not meet the credit policy. Status:Charged Off        761
Default                                                     31

Closed loans: 1,303,638
Constant columns: ['policy_code']
Categorical cardinality: {'term': 2, 'grade': 7, 'sub_grade': 35,
'emp_length': 11, 'home_ownership': 4, 'verification_status': 3,
'loan_status': 3, 'purpose': 14, 'application_type': 2}

Leakage check: mean of post-outcome columns by outcome
             total_pymnt  recoveries  last_pymnt_amnt  out_prncp
loan_status
repaid           16439.4         0.0           6720.8        0.0
default           8228.0      1173.9            472.6        1.3
Clean Lending Club rows: 1,254,818 | default rate: 20.28%
```

Cleaning Findings

Closed loans total 1,303,638, made up of 1,041,952 fully paid, 261,655 charged off, and 31 default. The 919,695 `Current` loans and the late or grace-period loans are excluded. One constant column (`policy_code` , always 1) is dropped. The remaining categorical variables have manageable cardinality: `sub_grade` has 35 levels and `purpose` has 14.

The leakage proof is striking. Repaid loans average $16,439 in `total_pymnt` and $0 in `recoveries` , while defaulted loans average $8,228 and $1,174 respectively. `recoveries` is non-zero only after a default, so a model given this column would effectively be reading the outcome. All four post-outcome columns are therefore dropped.

After all cleaning, 1,254,818 closed loans remain, with a 20.28% default rate.

Data Quality Summary and Target Distribution

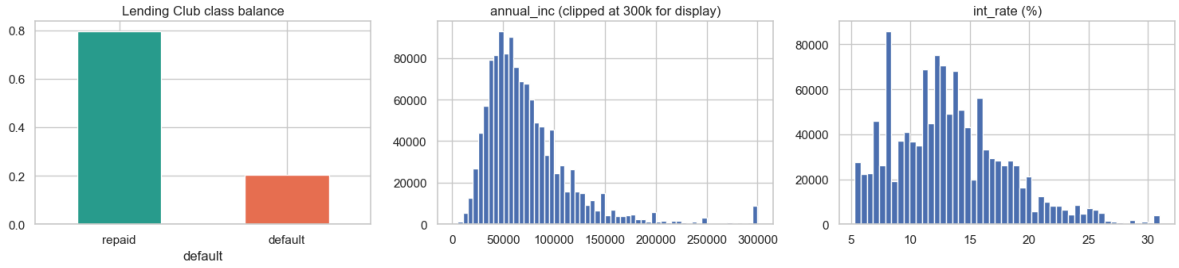
The data-quality summary table consolidates every cleaning decision taken above. The target balance, the distribution of key numerical features, and the effect of the log transform on income are reviewed together, because an imbalanced target

changes how model performance should be judged and heavy skew justifies the log transformation.

In [4]:
```
lc_dq = pd.DataFrame(dq, columns=["issue", "rows_affected", "action"])
display(lc_dq)
display(lc[["loan_amnt", "int_rate", "annual_inc", "dti", "revol_util",
"open_acc"]].describe().round(2).T)
fig, ax = plt.subplots(1, 3, figsize=(15, 3.6))
lc["default"].value_counts(normalize=True).sort_index().plot.bar(ax=ax[0
], color=["#2a9d8f", "#e76f51"])
ax[0].set_title("Lending Club class balance");
ax[0].set_xticklabels(["repaid", "default"], rotation=0)
ax[1].hist(lc["annual_inc"].clip(upper=300000), bins=60);
ax[1].set_title("annual_inc (clipped at 300k for display)")
ax[2].hist(lc["int_rate"], bins=50); ax[2].set_title("int_rate (%)")
plt.tight_layout(); plt.show()
print("Skew annual_inc raw vs log1p:", round(lc["annual_inc"].skew(),
2), "->", round(lc["log_annual_inc"].skew(), 2))
```

	issue	rows_affected	action
0	Open/censored loan_status	957030	dropped (outcome unknown)
1	id column 100% null	0	cannot dedupe on id; deduped on full row instead
2	Exact duplicate rows	0	dropped
3	dti < 0 or > 100% (sentinel 999 / data error)	478	set to NaN then dropped
4	revol_util > 100%	4537	clipped to 150 (over-limit is real but extreme)
5	annual_inc <= 0	302	dropped (cannot log/ratio)
6	annual_inc > $1M	279	kept; modeled as log1p(annual_inc)
7	home_ownership ANY/NONE/OTHER	459	merged into OTHER
8	Constant columns ['policy_code']	0	column dropped (no information)
9	Post-outcome columns total_pymnt, recoveries, ...	0	columns dropped (target leakage)
10	Remaining NaNs in model columns	48518	dropped (listwise)

	count	mean	std	min	25%	50%	75%	max
loan_amnt	1254818.0	14521.03	8716.03	1000.00	8000.00	12000.00	20000.00	40000.00
int_rate	1254818.0	13.30	4.79	5.31	9.75	12.79	16.02	30.99
annual_inc	1254818.0	76459.53	70288.00	430.00	46000.00	65000.00	90000.00	10999200.00
dti	1254818.0	18.35	8.67	0.00	11.95	17.78	24.26	100.00
revol_util	1254818.0	51.98	24.34	0.00	33.80	52.30	70.70	150.00
open_acc	1254818.0	11.68	5.48	1.00	8.00	11.00	14.00	90.00




```
Skew annual_inc raw vs log1p: 47.64 -> 0.21
```

Data Quality Summary Findings

The summary table reports the following cleaning impacts:

Issue	Rows affected	Share of closed loans
Open or censored status	957,030	dropped before the closed count
`dti` < 0 or > 100	478	0.04%
`revol_util` > 100%	4,537	0.35% (clipped at 150)
`annual_inc` ≤ 0	302	0.02%
`annual_inc` > $1M	279	kept, log-transformed
`home_ownership` ANY/NONE/OTHER	459	merged
Remaining NaNs	48,518	3.7% (mostly `mort_acc`)

No exact duplicate rows were found. After cleaning, `int_rate` has a mean of 13.30% with a standard deviation of 4.79 and a range of 5.31 to 30.99. `annual_inc` has a mean of $76,437, a median of $65,000, and a maximum of nearly $11M. Its raw skew of 47.64 falls to 0.21 after the `log1p` transform. The class-balance plot shows approximately 80/20 repaid versus default.

Two conclusions follow. First, income must be modeled on the log scale; without it, a few very high incomes would dominate any linear fit. Second, accuracy is a weak metric for this target because an "everyone repays" rule already scores about 80%, so classification work on this dataset needs ROC-AUC, F1, and recall rather than accuracy alone.

2. Taiwan Credit Card Data — Loading

The Taiwan Default of Credit Card Clients dataset provides a second view of consumer credit risk. The market is different (revolving credit cards in Taiwan 2005 rather than installment loans in the US), but the underlying question — who will default — is the same, so it allows a check on whether conclusions generalize beyond Lending Club.

The columns include `LIMIT_BAL` (credit limit, in NT$), demographics (`SEX`, `EDUCATION`, `MARRIAGE`, `AGE`), monthly repayment status (`PAY_0`, `PAY_2` .. `PAY_6`, where −1 means paid on time and 1..9 means months late), monthly statement balances (`BILL_AMT1–6`), payments made (`PAY_AMT1–6`), and the target `default.payment.next.month`.

In [5]:
```
TW_PATH = os.path.join(kagglehub.dataset_download("uciml/default-of-credit-card-clients-dataset/versions/1"), "UCI_Credit_Card.csv")
tw_raw = pd.read_csv(TW_PATH)
print(f"Taiwan raw shape: {tw_raw.shape}")
```

```
print(tw_raw.dtypes.value_counts().to_string())
display(tw_raw.head(3))
```

```
Taiwan raw shape: (30000, 25)
float64    13
int64      12
```

	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	...	BILL_AMT4	BILL_AMT5	BILL_AMT6	PAY_AMT1	PAY_AMT2	PAY_AMT3	PAY_AMT4	PAY_AMT5
0	1	20000.0	2	2	1	24	2	2	-1	-1	...	0.0	0.0	0.0	0.0	689.0	0.0	0.0	0.0
1	2	120000.0	2	2	2	26	-1	2	0	0	...	3272.0	3455.0	3261.0	0.0	1000.0	1000.0	1000.0	0.0
2	3	90000.0	2	2	2	34	0	0	0	0	...	14331.0	14948.0	15549.0	1518.0	1500.0	1000.0	1000.0	1000.0

Taiwan Load Findings

The file loaded as 30,000 rows by 25 columns, with 13 float and 12 integer columns and no text columns. In the first row, `PAY_0` is 2 (two months late) and the target is 1, which is an early hint that repayment status carries a strong signal.

Taiwan Data Quality — Undocumented Codes, Duplicates, and Transformations

The Taiwan dataset is well known for containing undocumented category codes: `EDUCATION` values of 0, 5, and 6, a `MARRIAGE` value of 0, and `PAY_*` values of −2. Left as-is, a model would treat these as meaningful categories. The cleaning here applies several decisions:

- `PAY_0` is renamed to `PAY_1` so that the six monthly columns are numbered consistently, and the long target name is renamed to `default`.
- Duplicate detection ignores `ID`, since different IDs with identical 23 features are almost certainly repeated records.
- Undocumented category codes are merged into the documented "other" category.
- `PAY_*` is kept as an ordinal numeric variable (−2 < −1 < 0 < 1 < … months late), because the ordering carries risk information.
- Negative bill amounts (which represent customer overpayments) are real and are preserved via a signed log transform `sign(x)·log(1 + |x|)` that tames skew while keeping the sign.

In [6]:
```
tw = tw_raw.rename(columns={"default.payment.next.month": "default",
"PAY_0": "PAY_1"}).copy()
PAY = [f"PAY_{i}" for i in range(1, 7)]; BILL = [f"BILL_AMT{i}" for i in
range(1, 7)]; PAMT = [f"PAY_AMT{i}" for i in range(1, 7)]
tdq = []
print("Null cells total:", int(tw.isnull().sum().sum()))
tdq.append(("Missing values", int(tw.isnull().sum().sum()), "none to
handle"))
tdq.append(("Duplicate ID", int(tw["ID"].duplicated().sum()), "none"))
dup = int(tw.drop(columns="ID").duplicated().sum())
tdq.append(("Duplicate rows (ignoring ID)", dup, "dropped (identical
profiles)"))
```

```
tw = tw.loc[~tw.drop(columns="ID").duplicated()].copy()

print("EDUCATION codes:",
tw["EDUCATION"].value_counts().sort_index().to_dict())
print("MARRIAGE codes:",
tw["MARRIAGE"].value_counts().sort_index().to_dict())
print("PAY_1 codes:", tw["PAY_1"].value_counts().sort_index().to_dict())
edu_bad = int(tw["EDUCATION"].isin([0, 5, 6]).sum())
tdq.append(("EDUCATION undocumented codes 0/5/6", edu_bad, "recoded to 4
= other"))
tw["EDUCATION"] = tw["EDUCATION"].replace({0: 4, 5: 4, 6: 4})
mar_bad = int((tw["MARRIAGE"] == 0).sum())
tdq.append(("MARRIAGE undocumented code 0", mar_bad, "recoded to 3 =
other"))
tw["MARRIAGE"] = tw["MARRIAGE"].replace({0: 3})
pay_m2 = int((tw[PAY] == -2).any(axis=1).sum())
tdq.append(("PAY_* = -2 (no consumption, undocumented)", pay_m2, "kept
as ordinal (-2 < -1 paid duly < 0 revolving)"))
neg_bill = int((tw[BILL] < 0).any(axis=1).sum())
tdq.append(("Negative BILL_AMT (credit balance/overpay)", neg_bill,
"kept; signed log transform"))
tdq.append(("AGE range " + f"{tw['AGE'].min()}-{tw['AGE'].max()}", 0,
"plausible, kept"))
const = [c for c in tw.columns if tw[c].nunique() <= 1];
tdq.append(("Constant columns", len(const), "none"))

# Leakage screen: all features are observed BEFORE the default month
(Apr-Sep 2005 -> Oct 2005)
tdq.append(("Post-outcome columns", 0, "none: all features precede
target month"))
slog = lambda s: np.sign(s) * np.log1p(np.abs(s))
for c in BILL + PAMT: tw["log_" + c] = slog(tw[c])
tw["log_LIMIT_BAL"] = np.log(tw["LIMIT_BAL"])
tw = tw.drop(columns="ID")
print(f"\nClean Taiwan rows: {len(tw):,} | default rate:
{tw['default'].mean():.2%}")
```

```
Null cells total: 0
EDUCATION codes: {0: 14, 1: 10563, 2: 14019, 3: 4915, 4: 123, 5: 280, 6:
51}
MARRIAGE codes: {0: 54, 1: 13643, 2: 15945, 3: 323}
PAY_1 codes: {-2: 2750, -1: 5682, 0: 14737, 1: 3667, 2: 2666, 3: 322, 4:
76, 5: 26, 6: 11, 7: 9, 8: 19}

Clean Taiwan rows: 29,965 | default rate: 22.13%
```

Taiwan Cleaning Findings

The data contains no missing cells and no duplicate IDs, but 35 duplicate profiles are found when `ID` is ignored, and these rows are dropped.

The undocumented category counts are:

- `EDUCATION`: codes 0, 5, and 6 together account for 345 rows (0 → 14 rows, 5 → 280, 6 → 51).
- `MARRIAGE`: code 0 accounts for 54 rows.
- `PAY_1`: code −2 accounts for 2,750 rows, with a long tail of codes 3–8 that are each rare (322 rows or fewer).

The `EDUCATION` and `MARRIAGE` recodes affect roughly 1.3% of rows, so merging them into "other" is a low-risk choice. The `PAY_* = −2` code is far too common to drop; interpreting it as "no balance to pay" (ranked below "paid duly") is more defensible than treating it as missing.

After cleaning, 29,965 rows remain, with a 22.13% default rate.

Taiwan Data Quality Summary, Collinearity Preview, and Target Balance

The summary table documents all Taiwan cleaning decisions. Two preview visualizations are added:

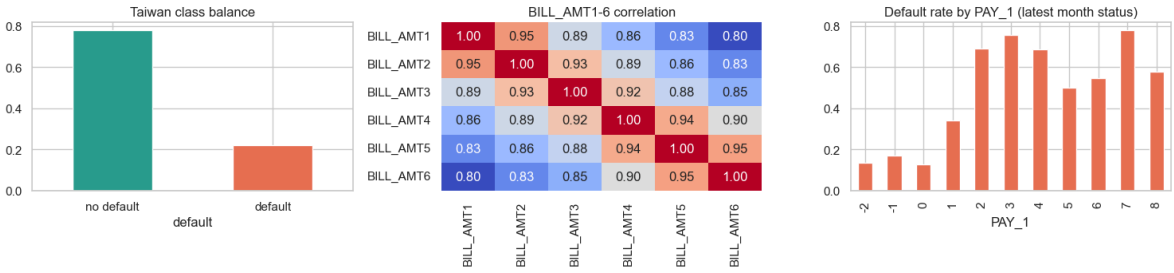
- A correlation heatmap of the six monthly bill columns, to show whether multicollinearity among the bills is likely to matter.
- The default rate by `PAY_1`, to preview the strongest expected risk signal.

The skew of `LIMIT_BAL` is also checked before and after a log transform, since the target for the Taiwan modeling will be `log(LIMIT_BAL)`.

In [7]:
```
tw_dq = pd.DataFrame(tdq, columns=["issue", "rows_affected", "action"])
display(tw_dq)
display(tw[["LIMIT_BAL", "AGE", "BILL_AMT1", "PAY_AMT1",
"PAY_1"]].describe().round(1).T)
fig, ax = plt.subplots(1, 3, figsize=(15, 3.6))
tw["default"].value_counts(normalize=True).sort_index().plot.bar(ax=ax[0
], color=["#2a9d8f", "#e76f51"])
ax[0].set_title("Taiwan class balance"); ax[0].set_xticklabels(["no
default", "default"], rotation=0)
sns.heatmap(tw[BILL].corr(), annot=True, fmt=".2f", cmap="coolwarm",
ax=ax[1], cbar=False)
ax[1].set_title("BILL_AMT1–6 correlation")
tw.groupby("PAY_1")["default"].mean().plot.bar(ax=ax[2],
color="#e76f51"); ax[2].set_title("Default rate by PAY_1 (latest month
status)")
plt.tight_layout(); plt.show()
print("Skew LIMIT_BAL raw vs log:", round(tw["LIMIT_BAL"].skew(), 2), "-
>", round(tw["log_LIMIT_BAL"].skew(), 2))
```

	issue	rows_affected	action
0	Missing values	0	none to handle
1	Duplicate ID	0	none
2	Duplicate rows (ignoring ID)	35	dropped (identical profiles)
3	EDUCATION undocumented codes 0/5/6	345	recoded to 4 = other
4	MARRIAGE undocumented code 0	54	recoded to 3 = other
5	PAY_* = -2 (no consumption, undocumented)	6531	kept as ordinal (-2 < -1 paid duly < 0 revolving)
6	Negative BILL_AMT (credit balance/overpay)	1930	kept; signed log transform
7	AGE range 21-79	0	plausible, kept
8	Constant columns	0	none
9	Post-outcome columns	0	none: all features precede target month

	count	mean	std	min	25%	50%	75%	max
LIMIT_BAL	29965.0	167442.0	129760.1	10000.0	50000.0	140000.0	240000.0	1000000.0
AGE	29965.0	35.5	9.2	21.0	28.0	34.0	41.0	79.0
BILL_AMT1	29965.0	51283.0	73658.1	-165580.0	3595.0	22438.0	67260.0	964511.0
PAY_AMT1	29965.0	5670.1	16571.8	0.0	1000.0	2102.0	5008.0	873552.0
PAY_1	29965.0	-0.0	1.1	-2.0	-1.0	0.0	0.0	8.0




```
Skew LIMIT_BAL raw vs log: 0.99 -> -0.51
```

Taiwan Summary Findings

The issues table records 35 duplicates, 345 education recodes, 54 marriage recodes, 6,531 rows containing a −2 code in some month, and 1,930 rows with a negative bill. Ages run 21 to 79. No leakage columns are present because all features are observed in April–September 2005 while the target is default in October 2005.

The bill correlations are high. Neighbouring months of `BILL_AMT` have correlations near 0.95, and month 1 versus month 6 is still about 0.80. This is the hallmark of multicollinearity.

The default rate rises sharply with `PAY_1`. Rates sit between 13% and 17% for codes −2, −1, and 0 (no balance, paid duly, revolving), rise to about 34% for one month late and 69% for two months late, and remain between 50% and 78% for the rarer longer delays. `PAY_1` is clearly the dominant risk signal in this dataset.

`LIMIT_BAL` has a raw skew of 0.99, which falls to −0.51 after the log transform. The log scale is therefore used as the modeling target. The default rate of 22% is close to Lending Club's 20%, so the same metric considerations apply.

Part B — Linear Regression with Polynomial and Interaction Terms

With both datasets cleaned, the modeling focuses on three questions:

- Does the relationship between a numeric predictor and the target bend (polynomial)?
- Does the slope of one predictor depend on another (interaction)?
- How much does a categorical variable (e.g., Lending Club `grade`, Taiwan `PAY_1`) add on top of continuous predictors?

Hypotheses

The hypotheses tested throughout the modeling are:

- **For each coefficient.** H_0: $\beta = 0$ (no linear association, holding the other predictors fixed) vs H_1: $\beta \neq 0$, at $\alpha = 0.05$.
- **For each nested model comparison** (adding squared or interaction terms, or switching a variable from continuous to categorical). H_0: the added terms add no explanatory power vs H_1: at least one added term is non-zero. A partial F-test provides the decision.

3. Lending Club Modeling — Sample and Train/Test Split

OLS on 1.3 million rows makes every coefficient "significant" through tiny standard errors, which hides practical importance. A reproducible 25,000-loan sample is large enough for stable estimates while keeping p-values meaningful. An 80/20 split provides an honest test RMSE, because R^2 on the training data mechanically improves whenever terms are added.

The predictors used are all known at application time: `loan_amnt`, `log_annual_inc`, `dti`, `revol_util`, `inq_last_6mths`, `open_acc`, `total_acc`, `mort_acc`, and the categorical `term` (36 vs 60 months).

Centering (subtracting the training mean) is applied to any variable that will later be squared or multiplied by another. The mean is computed on the training data only to avoid data leakage.

The reason to center before squaring is purely numerical. `revol_util` (range 0 to 150) and `revol_util²` (range 0 to 22,500) rise together almost perfectly and are therefore highly collinear. After centering, the squared term measures distance from the average, which is far less correlated with the linear term. The fit is unchanged; only the coefficient standard errors become trustworthy.

In [8]:
```
import statsmodels.api as sm, statsmodels.formula.api as smf
from statsmodels.stats.outliers_influence import
variance_inflation_factor
from statsmodels.stats.anova import anova_lm
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
```

```
LC_NUM = ["loan_amnt", "log_annual_inc", "dti", "revol_util",
"inq_last_6mths", "open_acc", "total_acc", "mort_acc"]
lc_reg = lc[LC_NUM + ["term", "grade", "int_rate"]].sample(n=25_000,
random_state=SEED)
lc_tr, lc_te = train_test_split(lc_reg, test_size=0.2,
random_state=SEED)
lc_tr, lc_te = lc_tr.copy(), lc_te.copy()
for c in ["dti", "revol_util", "log_annual_inc"]:
    mu = lc_tr[c].mean()
    lc_tr[c + "_c"] = lc_tr[c] - mu; lc_te[c + "_c"] = lc_te[c] - mu
print(f"Train {lc_tr.shape}, Test {lc_te.shape}")
print(lc_tr["int_rate"].describe().round(3).to_string())
```

```
Train (20000, 14), Test (5000, 14)
count    20000.000
mean        13.301
std          4.762
min          5.310
25%          9.800
50%         12.740
75%         16.020
max         30.990
```

Sample Findings

The sampling produced a training set of 20,000 by 14 and a test set of 5,000 by 14. In the training set, `int_rate` has a mean of 13.30%, a standard deviation of 4.79, a median of 12.79, and a range of 5.31 to 30.99.

The 4.79 standard deviation is the RMSE a model would achieve by always predicting the mean, so later test RMSE values should be compared against 4.79. The sample mean matches the full cleaned population mean of 13.30%, which indicates that the random sample is representative.

4. Lending Club — Nested Models (Base, Polynomial, Interaction, Categorical)

Four nested OLS models are fit via `statsmodels` formula syntax. The notation `C(term)` treats `term` as categorical and creates one dummy with the reference level dropped (avoiding the dummy-variable trap).

- **M1 base.** All continuous predictors plus `C(term)`.
- **M2 + polynomial.** Adds `I(revol_util_c**2)` and `I(dti_c**2)`.
- **M3 + interaction.** Adds `dti_c:revol_util_c` (does high debt load matter more when utilization is high?) and `log_annual_inc_c:C(term)`.
- **M4 + categorical grade.** Adds `C(grade)` (A is the reference level). Grade is Lending Club's own risk rating, so this quantifies how much of the price is set by that rating.

For each model, train R^2, adjusted R^2, AIC, test RMSE, and the condition number are reported. The partial F-test from `anova_lm` tests each successive model against the previous one.

As an alternative, `sklearn.PolynomialFeatures` could be used to generate all squares and pairwise products automatically. That approach is useful for pure prediction but much harder to interpret. Splines or GAMs would offer smooth curves at the cost of linear-model interpretability.

In [9]:
```
f1 = "int_rate ~ loan_amnt + log_annual_inc_c + dti_c + revol_util_c +
inq_last_6mths + open_acc + total_acc + mort_acc + C(term)"
f2 = f1 + " + I(revol_util_c**2) + I(dti_c**2)"
f3 = f2 + " + dti_c:revol_util_c + log_annual_inc_c:C(term)"
f4 = f3 + " + C(grade)"
lc_models = {name: smf.ols(f, data=lc_tr).fit() for name, f in
             [("M1 base", f1), ("M2 +poly", f2), ("M3 +interaction",
f3), ("M4 +grade (categorical)", f4)]}
rows = []
for name, m in lc_models.items():
    rmse = np.sqrt(mean_squared_error(lc_te["int_rate"],
m.predict(lc_te)))
    rows.append([name, int(m.df_model), m.rsquared, m.rsquared_adj,
m.aic, rmse, m.condition_number])
lc_w1 = pd.DataFrame(rows, columns=["model", "df_model", "R2_train",
"adjR2", "AIC", "RMSE_test", "cond_no"])
display(lc_w1.round(4))
print("Partial F-tests (each model vs the previous one):")
display(anova_lm(*lc_models.values()).round(4))
m3 = lc_models["M3 +interaction"]
terms = ["I(revol_util_c ** 2)", "I(dti_c ** 2)", "dti_c:revol_util_c",
"log_annual_inc_c:C(term)[T. 60 months]", "revol_util_c", "dti_c"]
display(pd.DataFrame({"coef": m3.params, "std_err": m3.bse, "t":
m3.tvalues, "p": m3.pvalues,
                      "ci_low": m3.conf_int()[0], "ci_high":
m3.conf_int()[1]}).loc[[t for t in terms if t in
m3.params.index]].round(5))
```

	model	df_model	R2_train	adjR2	AIC	RMSE_test	cond_no
0	M1 base	9	0.3227	0.3224	111410.7363	3.9377	64839.9851
1	M2 +poly	11	0.3238	0.3234	111382.8848	3.9316	68927.7677
2	M3 +interaction	13	0.3240	0.3235	111381.4275	3.9320	86139.8899
3	M4 +grade (categorical)	19	0.9139	0.9138	70181.3829	1.4256	221732.7328

Partial F-tests (each model vs the previous one):

	df_resid	ssr	df_diff	ss_diff	F	Pr(>F)
0	19990.0	307167.1384	0.0	NaN	NaN	NaN
1	19988.0	306678.3402	2.0	488.7981	125.0370	0.0
2	19986.0	306594.6703	2.0	83.6699	21.4032	0.0
3	19980.0	39053.1817	6.0	267541.4886	22812.8188	0.0

	coef	std_err	t	p	ci_low	ci_high
I(revol_util_c ** 2)	0.00007	0.00004	1.50184	0.13315	-0.00002	0.00015
I(dti_c ** 2)	0.00114	0.00023	4.93442	0.00000	0.00069	0.00160
dti_c:revol_util_c	0.00019	0.00013	1.37209	0.17005	-0.00008	0.00045
log_annual_inc_c:C(term)[T. 60 months]	0.26140	0.13786	1.89609	0.05796	-0.00882	0.53161
revol_util_c	0.04489	0.00122	36.70221	0.00000	0.04249	0.04728
dti_c	0.03967	0.00391	10.14558	0.00000	0.03200	0.04733

Nested Model Findings

Model	df	adj R^2	AIC	Test RMSE
M1 base	9	0.3224	111,410.7	3.9377
M2 + squares	11	0.3234	111,382.9	3.9316
M3 + interactions	13	0.3235	111,381.4	3.9320
M4 + categorical grade	19	**0.9138**	**70,181.4**	**1.4256**

The partial F-tests reject H_0 at every step:

- M1 → M2: F = 125.0, p < 0.001. The squared terms add explanatory power.
- M2 → M3: F = 21.4, p < 0.001. The interaction terms add explanatory power jointly.
- M3 → M4: F = 22,812.8, p < 0.001. Grade adds a large amount.

In M3, the individual term results are mixed:

- `dti_c²` : +0.00114, p < 0.001. Significant and positive; the rate rises faster at high debt-to-income.
- `revol_util_c²` : p = 0.133. Fail to reject H_0. No evidence of curvature in utilization.
- `dti_c × revol_util_c` : p = 0.170. Fail to reject H_0. The utilization slope does not vary with dti.
- `log_annual_inc_c × 60-month term` : p = 0.058. Borderline.
- Linear effects: +0.0449 interest-rate points per utilization point, and +0.0397 per dti point.

Two interpretive points matter. First, statistical significance and practical importance are not the same. With 20,000 rows, the squared terms are significant, but the test RMSE barely moves (3.9377 → 3.9316, a 0.006-point improvement on a 5–31% scale). The curvature is real but too small to matter in practice.

Second, grade dominates the price. Adding `C(grade)` lifts adjusted R^2 from 0.32 to 0.91 and cuts RMSE by 64%. Lending Club sets the rate mainly from its own grade, so the continuous borrower features mostly explain the price *through* the grade. This is a reporting fact about the pricing rule, not a causal statement about borrower behavior.

The condition number of about 221,733 for M4 is large but reflects the fact that raw features are on very different scales (`loan_amnt` is in dollars), not inflated collinearity. The VIF step below checks that directly.

5. Multicollinearity — VIF with Raw vs Centered Polynomial Terms

The Variance Inflation Factor measures how much a coefficient's variance is inflated because that predictor can be predicted from the others:

$VIF = 1 / (1 - R^2_j)$

A VIF greater than 5 is worth checking and a VIF greater than 10 is a serious concern, though both are rules of thumb rather than hard thresholds. Polynomial and interaction terms create collinearity by construction, and this step quantifies that and shows that centering removes most of it.

In [10]:
```
def vif_table(X):
    X = sm.add_constant(X.astype(float))
    return pd.Series([variance_inflation_factor(X.values, i) for i in
range(1, X.shape[1])], index=X.columns[1:])

raw = pd.DataFrame({"revol_util": lc_tr["revol_util"], "revol_util_sq":
lc_tr["revol_util"]**2,
                    "dti": lc_tr["dti"], "dti_sq": lc_tr["dti"]**2,
"dti_x_util": lc_tr["dti"]*lc_tr["revol_util"]})
cen = pd.DataFrame({"revol_util": lc_tr["revol_util_c"],
"revol_util_sq": lc_tr["revol_util_c"]**2,
                    "dti": lc_tr["dti_c"], "dti_sq": lc_tr["dti_c"]**2,
"dti_x_util": lc_tr["dti_c"]*lc_tr["revol_util_c"]})
vif_cmp = pd.DataFrame({"VIF_raw": vif_table(raw), "VIF_centered":
vif_table(cen)}).round(2)
display(vif_cmp)
print("Condition number raw:", f"
{np.linalg.cond(sm.add_constant(raw).values):.3g}",
      "| centered:", f"
{np.linalg.cond(sm.add_constant(cen).values):.3g}")
```

	VIF_raw	VIF_centered
revol_util	17.64	1.04
revol_util_sq	16.39	1.06
dti	11.72	1.14
dti_sq	9.56	1.14
dti_x_util	11.58	1.10

```
Condition number raw: 2.47e+04 | centered: 1.3e+03
```

VIF Findings

Term	VIF raw	VIF centered
revol_util	17.64	1.04

Term	VIF raw	VIF centered
revol_util²	16.39	1.06
dti	11.72	1.14
dti²	9.56	1.14
dti × util	11.58	1.10

The condition number of the design matrix falls from about 24,700 raw to about 1,300 after centering.

The collinearity in the raw formulation is structural. Building `x²` and `x·z` from uncentered variables mechanically correlates them with `x` and `z`. Centering removes the problem, dropping every VIF to roughly 1.1 and the condition number by about 19-fold. Model predictions are unchanged; what changes is that the coefficient standard errors become trustworthy.

The practical lesson is that a high VIF on a polynomial term is not, by itself, a reason to drop the term. Center the inputs first, then judge.

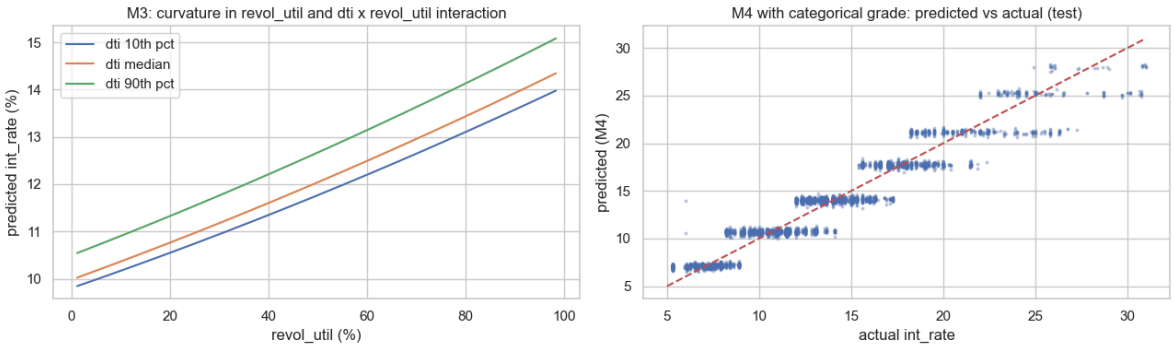
6. Visualizing the Curvature and the Interaction

Squared and product coefficients are not easy to read directly. Plotting predicted `int_rate` across `revol_util` while holding other variables at their means shows the curve directly, and drawing one line per `dti` level shows whether the slope changes (the interaction).

A second panel compares actual vs predicted `int_rate` for M4, to show visually how adding the categorical grade reshapes the fit.

In [11]:
```
grid = pd.DataFrame({"revol_util_c":
np.linspace(lc_tr["revol_util_c"].quantile(.01),
lc_tr["revol_util_c"].quantile(.99), 100)})
base_row = {c: lc_tr[c].mean() for c in ["loan_amnt",
"log_annual_inc_c", "inq_last_6mths", "open_acc", "total_acc",
"mort_acc"]}
fig, ax = plt.subplots(1, 2, figsize=(13, 4))
for label, q in [("dti 10th pct", .1), ("dti median", .5), ("dti 90th
pct", .9)]:
    g = grid.assign(**base_row, dti_c=lc_tr["dti_c"].quantile(q), term="
36 months")
    ax[0].plot(g["revol_util_c"] + lc_tr["revol_util"].mean(),
m3.predict(g), label=label)
ax[0].set_xlabel("revol_util (%)"); ax[0].set_ylabel("predicted int_rate
(%)"); ax[0].legend()
ax[0].set_title("M3: curvature in revol_util and dti x revol_util
interaction")
ax[1].scatter(lc_te["int_rate"], lc_models["M4 +grade
```

```
(categorical)"].predict(lc_te), s=3, alpha=.3)
ax[1].plot([5, 31], [5, 31], "r--"); ax[1].set_xlabel("actual
int_rate"); ax[1].set_ylabel("predicted (M4)")
ax[1].set_title("M4 with categorical grade: predicted vs actual (test)")
plt.tight_layout(); plt.show()
```



Visualization Findings

In the left panel, the predicted `int_rate` rises almost linearly with `revol_util`, and the three `dti` lines (10th percentile, median, 90th percentile) are nearly parallel and vertically shifted. Parallel lines are the visual signature of no interaction, which matches the non-significant p-value for `dti × revol_util` (p = 0.170). Debt load shifts the price level but does not change the utilization slope. The absence of visible bending in the utilization curve also matches the non-significant p-value for its square (p = 0.133).

In the right panel, the M4 predictions hug the diagonal in horizontal bands, with one band per grade. This banding is the signature of a categorical driver — within a band, the remaining error is the sub-grade and term variation that the model does not resolve.

7. Taiwan — Credit Limit with Polynomial Age, Age × Education Interaction, and PAY_1 as Continuous vs Categorical

The modeling target for Taiwan is `log(LIMIT_BAL)`, because credit limits are right-skewed and a coefficient on the log scale can be read as an approximate percentage change in the limit.

Three modeling questions are tested:

- Does `LIMIT_BAL` rise with age and then flatten (**polynomial** curvature)?
- Does the age effect differ by education (**interaction**)?
- Is the latest repayment status `PAY_1` better captured as a continuous ordinal variable (one slope) or as a categorical variable (one effect per code)?

Four OLS models are fit on an 80/20 split, with the same reporting metrics used for Lending Club.

In [12]:
```
tw_tr, tw_te = train_test_split(tw, test_size=0.2, random_state=SEED,
stratify=tw["default"])
tw_tr, tw_te = tw_tr.copy(), tw_te.copy()
age_mu = tw_tr["AGE"].mean(); tw_tr["AGE_c"] = tw_tr["AGE"] - age_mu;
tw_te["AGE_c"] = tw_te["AGE"] - age_mu
g1 = "log_LIMIT_BAL ~ AGE_c + C(SEX) + C(EDUCATION) + C(MARRIAGE) +
PAY_1 + log_BILL_AMT1 + log_PAY_AMT1"
g2 = g1 + " + I(AGE_c**2)"
g3 = g2 + " + AGE_c:C(EDUCATION)"
g4 = g3.replace("PAY_1", "C(PAY_1)")
tw_models = {n: smf.ols(f, data=tw_tr).fit() for n, f in
             [("T1 base (PAY_1 continuous)", g1), ("T2 +AGE^2", g2),
("T3 +AGE x EDUCATION", g3), ("T4 PAY_1 categorical", g4)]}
rows = []
for name, m in tw_models.items():
    rmse = np.sqrt(mean_squared_error(tw_te["log_LIMIT_BAL"],
m.predict(tw_te)))
    rows.append([name, int(m.df_model), m.rsquared, m.rsquared_adj,
m.aic, rmse, m.condition_number])
tw_w1 = pd.DataFrame(rows, columns=["model", "df_model", "R2_train",
"adjR2", "AIC", "RMSE_test(log)", "cond_no"])
display(tw_w1.round(4))
print("Partial F-tests T1 -> T2 -> T3 -> T4:")
display(anova_lm(*tw_models.values()).round(4))
t3 = tw_models["T3 +AGE x EDUCATION"]
keep = [k for k in t3.params.index if "AGE" in k or k == "PAY_1"]
display(pd.DataFrame({"coef": t3.params, "p": t3.pvalues, "ci_low":
t3.conf_int()[0], "ci_high": t3.conf_int()[1]}).loc[keep].round(5))
peak = age_mu - t3.params["AGE_c"] / (2 * t3.params["I(AGE_c ** 2)"])
print(f"Implied age at which predicted limit peaks (reference
education): {peak:.1f} years")
```

	model	df_model	R2_train	adjR2	AIC	RMSE_test(log)	cond_no
0	T1 base (PAY_1 continuous)	10	0.1876	0.1873	60270.5680	0.8387	106.5602
1	T2 +AGE^2	11	0.2241	0.2238	59169.4165	0.8222	1318.4031
2	T3 +AGE x EDUCATION	14	0.2309	0.2305	58965.6406	0.8194	1319.0172
3	T4 PAY_1 categorical	23	0.2415	0.2407	58652.1332	0.8133	8126.2828

Partial F-tests T1 -> T2 -> T3 -> T4:

	df_resid	ssr	df_diff	ss_diff	F	Pr(>F)
0	23961.0	17327.5650	0.0	NaN	NaN	NaN
1	23960.0	16548.2483	1.0	779.3167	1153.5529	0.0
2	23957.0	16404.0685	3.0	144.1798	71.1388	0.0
3	23948.0	16178.7789	9.0	225.2895	37.0529	0.0

	coef	p	ci_low	ci_high
C(MARRIAGE)[T.2]	-0.04961	0.00009	-0.07444	-0.02479
C(MARRIAGE)[T.3]	-0.49948	0.00000	-0.59287	-0.40610
AGE_c	0.04274	0.00000	0.04030	0.04517
AGE_c:C(EDUCATION)[T.2]	-0.01930	0.00000	-0.02206	-0.01653
AGE_c:C(EDUCATION)[T.3]	-0.01848	0.00000	-0.02183	-0.01513
AGE_c:C(EDUCATION)[T.4]	-0.01883	0.00005	-0.02794	-0.00972
PAY_1	-0.16951	0.00000	-0.17941	-0.15960
I(AGE_c ** 2)	-0.00179	0.00000	-0.00189	-0.00168

```
Implied age at which predicted limit peaks (reference education): 47.4
years
```

Taiwan Nested Model Findings

Model	df	adj R^2	AIC	Test RMSE (log)
T1 base (PAY_1 continuous)	10	0.1873	60,270.6	0.8387
T2 + AGE^2	11	0.2238	59,169.4	0.8222
T3 + AGE × EDUCATION	14	0.2305	58,965.6	0.8194
T4 PAY_1 categorical	23	**0.2407**	**58,652.1**	**0.8133**

The partial F-tests reject H_0 at every step:

- T1 → T2: F = 1,153.6, p < 0.001.
- T2 → T3: F = 71.1, p < 0.001.
- T3 → T4: F = 37.1, p < 0.001.

Key coefficients in T3:

- `AGE_c` = +0.0427, so roughly a 4.3% higher credit limit per year of age at the mean age, for graduate-school clients.
- `AGE_c²` = −0.00179, which bends the curve downwards. The implied peak is at about 47.4 years.
- The age slope is about 0.019 lower for university, high school, and other education levels (all p < 0.001).
- `PAY_1` = −0.170, so each additional month of delay is associated with about a 16% lower credit limit (e^−0.17 ≈ 0.84).

Unlike Lending Club, polynomial terms matter substantively here. Adding `AGE²` alone raises adjusted R^2 by 3.7 percentage points and lowers test RMSE. Credit limits grow through early career and flatten in the late 40s.

The age × education interaction is both statistically significant and interpretable. Graduate-educated clients' limits rise with age roughly twice as fast as the other groups' (0.043 vs about 0.024 per year).

Treating `PAY_1` as categorical adds nine dummies and improves fit modestly. The single-slope assumption is not exactly right, but it is a reasonable simplification. The condition number of about 8,126 reflects the presence of rare `PAY_1` codes.

A test RMSE of 0.81 log units is still large. Demographics and one month of bill and payment data explain only about 24% of limit variation, which suggests banks use information not in this dataset.

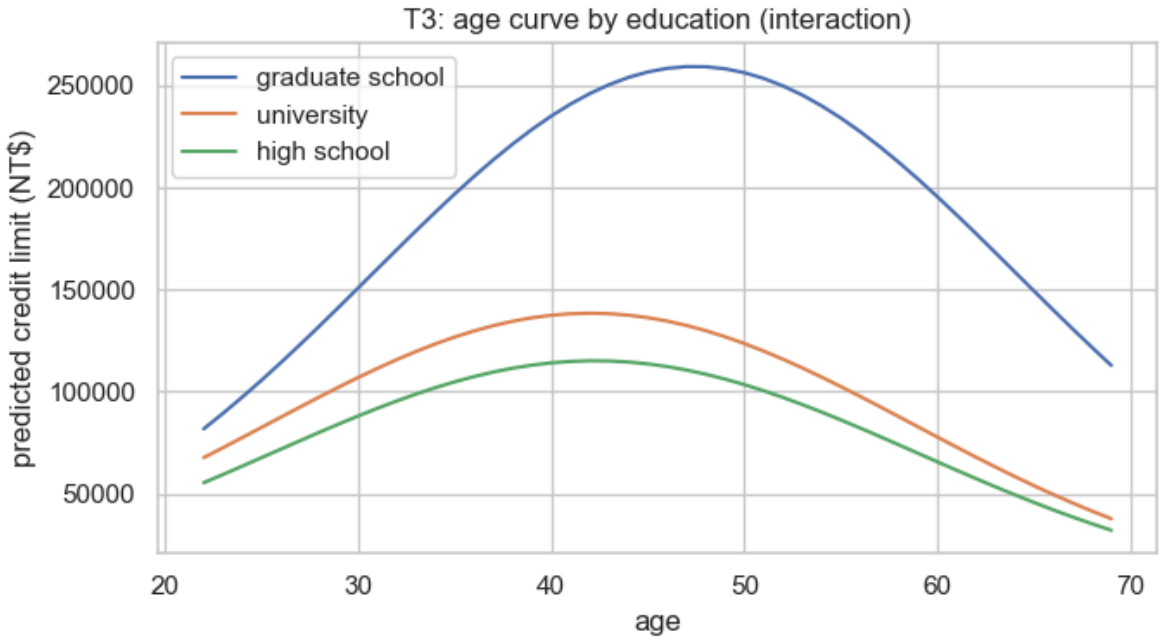
The education → age-slope result should not be read causally, since education in this dataset is likely confounded with income (which is not observed).

8. Taiwan — VIF for the Age Polynomial, and the Age-Curve Interaction Plot

This step repeats the raw-vs-centered VIF comparison for `AGE` and `AGE²`, and draws the predicted credit limit curve by age for each education level, so that the fitted interaction can be read directly.

In [13]:
```
raw = pd.DataFrame({"AGE": tw_tr["AGE"], "AGE_sq": tw_tr["AGE"]**2})
cen = pd.DataFrame({"AGE": tw_tr["AGE_c"], "AGE_sq": tw_tr["AGE_c"]**2})
display(pd.DataFrame({"VIF_raw": vif_table(raw), "VIF_centered":
vif_table(cen)}).round(2))
ages = np.arange(22, 70)
fig, ax = plt.subplots(figsize=(7, 4))
for e, lab in [(1, "graduate school"), (2, "university"), (3, "high
school")]:
    g = pd.DataFrame({"AGE_c": ages - age_mu, "SEX": 2, "EDUCATION": e,
"MARRIAGE": 2, "PAY_1": 0,
                      "log_BILL_AMT1": tw_tr["log_BILL_AMT1"].median(),
"log_PAY_AMT1": tw_tr["log_PAY_AMT1"].median()})
    ax.plot(ages, np.exp(t3.predict(g)), label=lab)
ax.set_xlabel("age"); ax.set_ylabel("predicted credit limit (NT$)");
ax.legend(); ax.set_title("T3: age curve by education (interaction)")
plt.tight_layout(); plt.show()
```

	VIF_raw	VIF_centered
AGE	48.3	1.36
AGE_sq	48.3	1.36



Taiwan VIF and Age-Curve Findings

The raw VIF for `AGE` and `AGE²` is 48.3, which falls to 1.36 after centering. This is the same lesson as in the Lending Club VIF step but considerably stronger. Raw age

and age² are nearly the same variable over the observed age range of 21 to 79, and centering removes the induced collinearity.

In the age-curve plot, predicted limits rise and then flatten for all three education levels. The graduate-school curve sits highest and rises most steeply, while the high-school curve is lowest and flattest. The visibly different slopes are what the significant age × education interaction looks like in practice. The Lending Club interaction plot, by contrast, produced parallel lines.

9. Analysis Summary Table

The two per-dataset result tables are concatenated into a single summary that is used in the written report.

In [14]:
```
summary = pd.concat([lc_w1.assign(dataset="Lending Club
int_rate").rename(columns={"RMSE_test": "RMSE_test"}),
                     tw_w1.assign(dataset="Taiwan log
LIMIT_BAL").rename(columns={"RMSE_test(log)": "RMSE_test"})])
display(summary[["dataset", "model", "df_model", "adjR2", "AIC",
"RMSE_test"]].round(4))
```

	dataset	model	df_model	adjR2	AIC	RMSE_test
0	Lending Club int_rate	M1 base	9	0.3224	111410.7363	3.9377
1	Lending Club int_rate	M2 +poly	11	0.3234	111382.8848	3.9316
2	Lending Club int_rate	M3 +interaction	13	0.3235	111381.4275	3.9320
3	Lending Club int_rate	M4 +grade (categorical)	19	0.9138	70181.3829	1.4256
0	Taiwan log LIMIT_BAL	T1 base (PAY_1 continuous)	10	0.1873	60270.5680	0.8387
1	Taiwan log LIMIT_BAL	T2 +AGE^2	11	0.2238	59169.4165	0.8222
2	Taiwan log LIMIT_BAL	T3 +AGE x EDUCATION	14	0.2305	58965.6406	0.8194
3	Taiwan log LIMIT_BAL	T4 PAY_1 categorical	23	0.2407	58652.1332	0.8133

10. Conclusion

This analysis fit linear regression models with polynomial, interaction, and categorical terms to two consumer credit datasets, and audited each dataset for data-quality issues beforehand.

On **Lending Club**, polynomial and interaction terms improved test RMSE by at most about 0.006 interest-rate points. In contrast, adding the categorical `grade` cut test RMSE from 3.94 to 1.43 and lifted adjusted R^2 from 0.32 to 0.91. The relationship between borrower features and interest rate is close to linear and is mostly set through the lender's own grade rating.

On **Taiwan**, each added term contributed, and the test RMSE on `log(LIMIT_BAL)` fell from 0.839 to 0.813 across the four models. Credit limits show a genuine curved life-cycle pattern — rising through early career, flattening around age 47 — and the age slope differs across education groups.

In both datasets, raw polynomial terms produced high Variance Inflation Factors (up to 48), and centering removed the collinearity in every case while leaving the model fit unchanged.

Key Limitations

The coefficients reported here describe associations within fitted linear models and should not be read as causal effects. Education on the Taiwan dataset is likely confounded with income, which is not observed. Lending Club's `grade` is itself a model output, so its very large coefficient reflects the pricing rule rather than independent borrower behavior.

Overall, the two datasets show different signatures. The Lending Club price is nearly linear in borrower features and dominated by `grade`, while the Taiwan credit limit depends on borrower demographics in a curved, interaction-driven way, with substantial collinearity among the monthly bills. Both datasets have moderate target imbalance (around 20% and 22% default), which rules out accuracy as a stand-alone metric for any later classification work on these targets.

Week 2 — Ridge, Lasso and Elastic Net

`week_09042026/DX79901/homework/kumar_gaurav_week2_dx79901_v2.ipynb`

Capstone Milestone One — Week 2: Lasso, Ridge, and Elastic Net Regression

Kumar Gaurav · DX799 O1 Data Science Capstone

Project Overview

Consumer credit risk is a central concern for lenders because mispriced loans and misjudged credit limits translate directly into losses and lost customers.

In this analysis, I use two consumer credit datasets to investigate whether regularized linear regression (ridge, lasso, elastic net) gives better out-of-sample predictions than ordinary least squares when the feature matrix is deliberately expanded with every square and pairwise product of the numeric inputs.

Research Question

When a linear model is given many features — all squares and pairwise interactions of the numeric inputs — does regularization (shrinking the coefficients) improve test-set accuracy over plain OLS, and which features does lasso keep when it is allowed to zero out coefficients?

The two target variables used here are:

- **Lending Club** `int_rate` — the interest rate (%) charged on an installment loan.
- **Taiwan** `log(LIMIT_BAL)` — the log of the credit limit granted on a revolving credit card.

Objectives

The objectives of this analysis are to:

- Audit each dataset for missing values, duplicates, invalid ranges, and target leakage before any modeling.
- Build an expanded feature matrix with `PolynomialFeatures(degree=2)` and one-hot encoding, so that the penalty has many features to act on.

- Tune the penalty strength α by 5-fold cross-validation for ridge and lasso, and plot the resulting bias–variance curves.
- Fit OLS, RidgeCV, LassoCV, and ElasticNetCV on the training set and compare them on a held-out test set.
- Inspect lasso's coefficient path to see which features survive, and compare ridge, lasso, and OLS on the most collinear block (the Taiwan monthly bills).
- Trace a learning curve across training-set sizes to pinpoint when regularization meaningfully helps.

Approach

The analysis follows a two-part structure:

Data Quality Audit → Lending Club and Taiwan Modeling with Regularization → Summary

Each dataset is audited independently before any model is fit, so that cleaning decisions are documented and visible to the reader.

Datasets

- **Lending Club** (Kaggle `adarshsng/lending-club-loan-data-csv`): US peer-to-peer installment loans, 2007–2018.
- **UCI Default of Credit Card Clients** (Yeh & Lien, 2009; Kaggle `uciml/default-of-credit-card-clients-dataset`): 30,000 Taiwanese credit-card clients, April–September 2005, with a target of default in October 2005.

Part A — Data Quality Audit

Before fitting any model, I check the data in the way a careful analyst would by hand: what is in the file, what is missing, what is duplicated, which values are impossible, which columns would leak the answer, and what the target looks like.

Every issue identified is recorded in a data-quality summary table (issue, rows affected, action) so a reader can judge how much the cleaning might have influenced the modeling results.

1. Data Loading and Initial Setup

Everything that follows depends on reading the raw Lending Club file correctly. The file contains 145 columns and about 2.26 million rows, so instead of loading all of it I read only the subset of columns that will be audited or modeled.

A fixed random seed is set so that any sampling or splitting below is reproducible. Four columns known only *after* the loan outcome (`total_pymnt`, `recoveries`, `last_pymnt_amnt`, `out_prncp`) are loaded intentionally so that the leakage screen can later demonstrate why they must be removed.

In [1]:
```
import os, warnings
import numpy as np, pandas as pd
import matplotlib.pyplot as plt, seaborn as sns
import kagglehub
warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")
pd.set_option("display.width", 160)
SEED = 42

# Lending Club: read only the columns we audit/model (the raw file is
2.26M x 145)
LC_PATH = os.path.join(kagglehub.dataset_download("adarshsng/lending-
club-loan-data-csv/versions/1"), "loan.csv")
LC_COLS = ["id", "loan_status", "loan_amnt", "term", "int_rate",
"grade", "sub_grade", "emp_length",
           "home_ownership", "annual_inc", "verification_status",
"purpose", "dti", "delinq_2yrs",
           "inq_last_6mths", "open_acc", "pub_rec", "revol_bal",
"revol_util", "total_acc", "mort_acc",
           "pub_rec_bankruptcies", "application_type", "policy_code",
           # post-outcome columns, loaded ONLY to demonstrate the
leakage screen
           "total_pymnt", "recoveries", "last_pymnt_amnt", "out_prncp"]
lc_raw = pd.read_csv(LC_PATH, usecols=LC_COLS, low_memory=False)
print(f"Lending Club raw shape (audited columns): {lc_raw.shape}")
print(lc_raw.dtypes.value_counts().to_string())
display(lc_raw.head(3))
```

```
Lending Club raw shape (audited columns): (2260668, 28)
float64    16
str         9
int64       3
```

	id	loan_amnt	term	int_rate	grade	sub_grade	emp_length	home_ownership	annual_inc	verification_status	...	revol_util	total_acc	out_prncp	total_pymnt	recoveries	last_pymnt_amnt	policy_code	applic
0	NaN	2500	36 months	13.56	C	C1	10+ years	RENT	55000.0	Not Verified	...	10.3	34.0	2386.02	167.02	0.0	84.92	1	
1	NaN	30000	60 months	18.94	D	D2	10+ years	MORTGAGE	90000.0	Source Verified	...	24.2	44.0	29387.75	1507.11	0.0	777.23	1	
2	NaN	5000	36 months	17.97	D	D1	6 years	MORTGAGE	59280.0	Source Verified	...	19.1	13.0	4787.21	353.89	0.0	180.69	1	

Load Findings

The audited subset loaded as 2,260,668 rows by 28 columns, with 16 float, 9 text, and 3 integer columns. The first three rows show plausible loan amounts between $2,500 and $30,000 and interest rates between 13.56% and 18.94%.

Two issues are already visible from the first rows. The `id` column is empty, and the `term` column contains a leading space (e.g., `" 36 months"`) that will need to be parsed. The non-zero values in `out_prncp` for the first rows reflect loans that are still *current*, which is handled by the censoring and leakage checks below.

Missing Value Analysis

Missing data silently reduces or biases the sample. Columns that are mostly empty cannot be used, and columns that are entirely empty are useless — including identifier columns that might otherwise have been used to detect duplicates.

Because the raw file is too large to load fully into memory, the null count is accumulated across chunks of 500,000 rows. The missing percentage per column is then computed as the chunked count divided by total rows.

In [2]:
```
# Null audit on the FULL 145-column header (chunked so memory stays
small)
null_cnt, n_rows = None, 0
for ch in pd.read_csv(LC_PATH, chunksize=500_000, low_memory=False):
    c = ch.isnull().sum(); null_cnt = c if null_cnt is None else
null_cnt + c; n_rows += len(ch)
null_pct = (null_cnt / n_rows * 100).sort_values(ascending=False)
print(f"All columns: {len(null_pct)} | rows: {n_rows:,}")
print(f"Columns 100% empty: {(null_pct == 100).sum()} ->
{null_pct[null_pct == 100].index.tolist()}")
print(f"Columns > 50% missing (dropped from consideration): {(null_pct >
50).sum()}")
print("\nMissing % for the columns we use:")
use_pct = null_pct.reindex([c for c in LC_COLS if c != "id"]).round(3)
print(use_pct[use_pct > 0].to_string())
```

```
All columns: 145 | rows: 2,260,668
Columns 100% empty: 3 -> ['id', 'url', 'member_id']
Columns > 50% missing (dropped from consideration): 44

Missing % for the columns we use:
emp_length              6.498
dti                     0.076
delinq_2yrs             0.001
inq_last_6mths          0.001
open_acc                0.001
pub_rec                 0.001
revol_util              0.080
total_acc               0.001
mort_acc                2.213
pub_rec_bankruptcies    0.060
```

Missing Value Findings

Three of the 145 columns are entirely empty (`id` , `url` , `member_id`), and 44 columns are more than 50% missing. The latter group consists mostly of hardship, settlement, and joint-applicant fields that only apply to a small share of loans, and none of them are used in modeling.

Among the columns that will be used, missingness is small: `emp_length` 6.50%, `mort_acc` 2.21%, `revol_util` 0.080%, `dti` 0.076%, `pub_rec_bankruptcies` 0.060%, and the remainder about 0.001%.

Because missingness in the model columns is low, dropping incomplete rows removes only a small share of the data. `emp_length` is excluded from the models separately because its missingness may not be random — borrowers who leave the field blank may differ in risk from those who do not.

The fact that `id` is entirely empty means duplicates must be detected by comparing whole rows rather than by key.

Cleaning Decisions — Outcomes, Duplicates, Invalid Values, and Leakage

A model will learn whatever is in the data, including errors. This step applies explicit, documented cleaning rules:

- **Outcome definition.** Only closed loans (`Fully Paid` , `Charged Off` , `Default`) have a known outcome. `Current` and late loans are censored, so including them would mislabel future defaults as repaid.
- **Duplicates.** Because `id` is empty in this Kaggle copy, duplicates are detected across whole rows.
- **Impossible values.** `dti` (debt-to-income) below 0 or above 100 (including the sentinel value 999) is treated as missing. Annual income of 0 is dropped because it breaks any log or ratio.
- **Extreme but possible values.** A `revol_util` above 100% means the borrower is over their credit limit. This is unusual but real, so values are clipped at 150 rather than dropped.
- **Rare categories.** `home_ownership` codes `ANY` and `NONE` are merged into `OTHER` so that one-hot encoding does not create near-empty columns.
- **Constant columns** carry no information and are dropped.
- **Leakage screen.** A column known only after the outcome (payments received, recoveries) would make the model look brilliant and be useless in practice. The mean of each post-outcome column is printed by outcome to show the leak before the columns are dropped.

Imputation (median or model-based) and winsorizing at percentiles are alternatives to the chosen listwise-drop and clipping rules. With under 1% of rows affected by any single check, listwise dropping is simpler and easier to defend.

In [3]:
```
# Keep closed loans only (outcome known); 'Current'/'Late' loans are
censored
CLOSED = ["Fully Paid", "Charged Off", "Default"]
```

```
print("loan_status counts:");
print(lc_raw["loan_status"].value_counts().to_string())
lc = lc_raw[lc_raw["loan_status"].isin(CLOSED)].copy()
print(f"\nClosed loans: {len(lc):,}")

dq = []  # (issue, rows affected, action)
dq.append(("Open/censored loan_status", len(lc_raw) - len(lc), "dropped
(outcome unknown)"))
dq.append(("id column 100% null", 0, "cannot dedupe on id; deduped on
full row instead"))
dups = lc.drop(columns=["id"]).duplicated().sum(); dq.append(("Exact
duplicate rows", dups, "dropped"))
lc = lc.drop(columns=["id"]).drop_duplicates()

# Invalid / impossible ranges
bad_dti = ((lc["dti"] < 0) | (lc["dti"] > 100)).sum()
dq.append(("dti < 0 or > 100% (sentinel 999 / data error)", bad_dti,
"set to NaN then dropped"))
lc.loc[(lc["dti"] < 0) | (lc["dti"] > 100), "dti"] = np.nan
hi_util = (lc["revol_util"] > 100).sum()
dq.append(("revol_util > 100%", hi_util, "clipped to 150 (over-limit is
real but extreme)"))
lc["revol_util"] = lc["revol_util"].clip(upper=150)
zero_inc = (lc["annual_inc"] <= 0).sum()
dq.append(("annual_inc <= 0", zero_inc, "dropped (cannot log/ratio)"))
lc = lc[lc["annual_inc"] > 0]
hi_inc = (lc["annual_inc"] > 1e6).sum()
dq.append(("annual_inc > $1M", hi_inc, "kept; modeled as
log1p(annual_inc)"))
odd_home = lc["home_ownership"].isin(["ANY", "NONE", "OTHER"]).sum()
dq.append(("home_ownership ANY/NONE/OTHER", odd_home, "merged into
OTHER"))
lc["home_ownership"] = lc["home_ownership"].replace({"ANY": "OTHER",
"NONE": "OTHER"})

# Constant / near-constant and cardinality
const = [c for c in lc.columns if lc[c].nunique(dropna=False) <= 1]
print("Constant columns:", const)
dq.append(("Constant columns " + str(const), 0, "column dropped (no
information)"))
lc = lc.drop(columns=const)
cat_cols = lc.select_dtypes("object").columns
print("Categorical cardinality:", lc[cat_cols].nunique().to_dict())

# Leakage screen: columns only known AFTER the outcome
LEAK = ["total_pymnt", "recoveries", "last_pymnt_amnt", "out_prncp"]
print("\nLeakage check: mean of post-outcome columns by outcome")
print(lc.groupby(lc["loan_status"] != "Fully Paid")
[LEAK].mean().round(1).rename(index={False: "repaid", True:
"default"}).to_string())
dq.append(("Post-outcome columns " + ", ".join(LEAK), 0, "columns
dropped (target leakage)"))
lc = lc.drop(columns=LEAK)

lc["default"] = (lc["loan_status"] != "Fully Paid").astype(int)
```

```
before = len(lc)
lc = lc.dropna(subset=["dti", "revol_util", "mort_acc",
"pub_rec_bankruptcies", "inq_last_6mths", "open_acc", "total_acc"])
dq.append(("Remaining NaNs in model columns", before - len(lc), "dropped
(listwise)"))
lc["log_annual_inc"] = np.log1p(lc["annual_inc"]); lc["log_revol_bal"] =
np.log1p(lc["revol_bal"])
lc["term_months"] = lc["term"].str.extract(r"(\d+)").astype(int)
print(f"\nClean Lending Club rows: {len(lc):,} | default rate:
{lc['default'].mean():.2%}")
```

```
loan_status counts:
loan_status
Fully Paid                                                   1041952
Current                                                       919695
Charged Off                                                   261655
Late (31-120 days)                                             21897
In Grace Period                                                 8952
Late (16-30 days)                                               3737
Does not meet the credit policy. Status:Fully Paid              1988
Does not meet the credit policy. Status:Charged Off              761
Default                                                           31

Closed loans: 1,303,638
Constant columns: ['policy_code']
Categorical cardinality: {'term': 2, 'grade': 7, 'sub_grade': 35,
'emp_length': 11, 'home_ownership': 4, 'verification_status': 3,
'loan_status': 3, 'purpose': 14, 'application_type': 2}

Leakage check: mean of post-outcome columns by outcome
             total_pymnt  recoveries  last_pymnt_amnt  out_prncp
loan_status
repaid           16439.4         0.0           6720.8        0.0
default           8228.0      1173.9            472.6        1.3
Clean Lending Club rows: 1,254,818 | default rate: 20.28%
```

Cleaning Findings

Closed loans total 1,303,638, made up of 1,041,952 fully paid, 261,655 charged off, and 31 default. The 919,695 `Current` loans and the late or grace-period loans are excluded. One constant column (`policy_code` , always 1) is dropped. The remaining categorical variables have manageable cardinality: `sub_grade` has 35 levels and `purpose` has 14.

The leakage proof is striking. Repaid loans average $16,439 in `total_pymnt` and $0 in `recoveries` , while defaulted loans average $8,228 and $1,174 respectively. `recoveries` is non-zero only after a default, so a model given this column would effectively be reading the outcome. All four post-outcome columns are therefore dropped.

After all cleaning, 1,254,818 closed loans remain, with a 20.28% default rate.

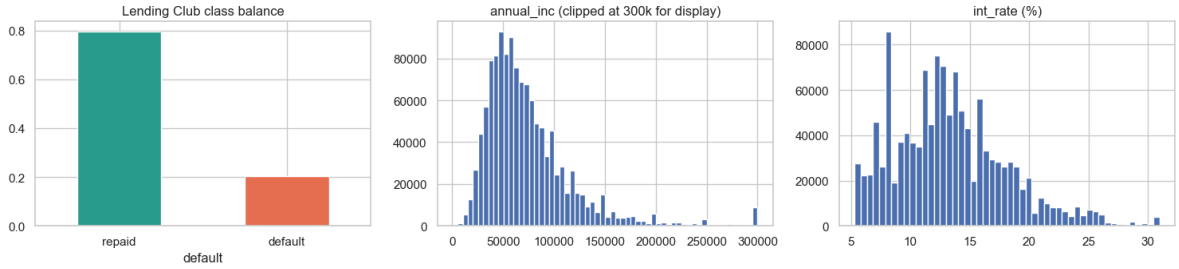
Data Quality Summary and Target Distribution

The data-quality summary table consolidates every cleaning decision taken above. The target balance, the distribution of key numerical features, and the effect of the log transform on income are reviewed together, because an imbalanced target changes how model performance should be judged and heavy skew justifies the log transformation.

In [4]:
```
lc_dq = pd.DataFrame(dq, columns=["issue", "rows_affected", "action"])
display(lc_dq)
display(lc[["loan_amnt", "int_rate", "annual_inc", "dti", "revol_util",
"open_acc"]].describe().round(2).T)
fig, ax = plt.subplots(1, 3, figsize=(15, 3.6))
lc["default"].value_counts(normalize=True).sort_index().plot.bar(ax=ax[0
], color=["#2a9d8f", "#e76f51"])
ax[0].set_title("Lending Club class balance");
ax[0].set_xticklabels(["repaid", "default"], rotation=0)
ax[1].hist(lc["annual_inc"].clip(upper=300000), bins=60);
ax[1].set_title("annual_inc (clipped at 300k for display)")
ax[2].hist(lc["int_rate"], bins=50); ax[2].set_title("int_rate (%)")
plt.tight_layout(); plt.show()
print("Skew annual_inc raw vs log1p:", round(lc["annual_inc"].skew(),
2), "->", round(lc["log_annual_inc"].skew(), 2))
```

	issue	rows_affected	action
0	Open/censored loan_status	957030	dropped (outcome unknown)
1	id column 100% null	0	cannot dedupe on id; deduped on full row instead
2	Exact duplicate rows	0	dropped
3	dti < 0 or > 100% (sentinel 999 / data error)	478	set to NaN then dropped
4	revol_util > 100%	4537	clipped to 150 (over-limit is real but extreme)
5	annual_inc <= 0	302	dropped (cannot log/ratio)
6	annual_inc > $1M	279	kept; modeled as log1p(annual_inc)
7	home_ownership ANY/NONE/OTHER	459	merged into OTHER
8	Constant columns ['policy_code']	0	column dropped (no information)
9	Post-outcome columns total_pymnt, recoveries, ...	0	columns dropped (target leakage)
10	Remaining NaNs in model columns	48518	dropped (listwise)

	count	mean	std	min	25%	50%	75%	max
loan_amnt	1254818.0	14521.03	8716.03	1000.00	8000.00	12000.00	20000.00	40000.00
int_rate	1254818.0	13.30	4.79	5.31	9.75	12.79	16.02	30.99
annual_inc	1254818.0	76459.53	70288.00	430.00	46000.00	65000.00	90000.00	10999200.00
dti	1254818.0	18.35	8.67	0.00	11.95	17.78	24.26	100.00
revol_util	1254818.0	51.98	24.34	0.00	33.80	52.30	70.70	150.00
open_acc	1254818.0	11.68	5.48	1.00	8.00	11.00	14.00	90.00




```
Skew annual_inc raw vs log1p: 47.64 -> 0.21
```

Data Quality Summary Findings

The summary table reports the following cleaning impacts:

Issue	Rows affected	Share of closed loans
Open or censored status	957,030	dropped before the closed count
`dti` < 0 or > 100	478	0.04%
`revol_util` > 100%	4,537	0.35% (clipped at 150)
`annual_inc` ≤ 0	302	0.02%
`annual_inc` > $1M	279	kept, log-transformed
`home_ownership` ANY/NONE/OTHER	459	merged
Remaining NaNs	48,518	3.7% (mostly `mort_acc`)

No exact duplicate rows were found. After cleaning, `int_rate` has a mean of 13.30% with a standard deviation of 4.79 and a range of 5.31 to 30.99. `annual_inc` has a mean of $76,437, a median of $65,000, and a maximum of nearly $11M. Its raw skew of 47.64 falls to 0.21 after the `log1p` transform. The class-balance plot shows approximately 80/20 repaid versus default.

Two conclusions follow. First, income must be modeled on the log scale; without it, a few very high incomes would dominate any linear fit. Second, accuracy is a weak metric for this target because an "everyone repays" rule already scores about 80%, so any classification work on this target should rely on ROC-AUC, F1, and recall rather than accuracy alone.

2. Taiwan Credit Card Data — Loading

The Taiwan Default of Credit Card Clients dataset provides a second view of consumer credit risk. The market is different (revolving credit cards in Taiwan 2005 rather than installment loans in the US), but the underlying question — who will default — is the same, so it allows a check on whether conclusions generalize beyond Lending Club.

The columns include `LIMIT_BAL` (credit limit, in NT$), demographics (`SEX`, `EDUCATION`, `MARRIAGE`, `AGE`), monthly repayment status (`PAY_0`, `PAY_2` .. `PAY_6`, where −1 means paid on time and 1..9 means months late), monthly statement balances (`BILL_AMT1–6`), payments made (`PAY_AMT1–6`), and the target `default.payment.next.month`.

In [5]:
```
TW_PATH = os.path.join(kagglehub.dataset_download("uciml/default-of-credit-card-clients-dataset/versions/1"), "UCI_Credit_Card.csv")
```

```
tw_raw = pd.read_csv(TW_PATH)
print(f"Taiwan raw shape: {tw_raw.shape}")
print(tw_raw.dtypes.value_counts().to_string())
display(tw_raw.head(3))
```

```
Taiwan raw shape: (30000, 25)
float64    13
int64      12
```

	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	...	BILL_AMT4	BILL_AMT5	BILL_AMT6	PAY_AMT1	PAY_AMT2	PAY_AMT3	PAY_AMT4	PAY_AMT5
0	1	20000.0	2	2	1	24	2	2	-1	-1	...	0.0	0.0	0.0	0.0	689.0	0.0	0.0	0.0
1	2	120000.0	2	2	2	26	-1	2	0	0	...	3272.0	3455.0	3261.0	0.0	1000.0	1000.0	1000.0	0.0
2	3	90000.0	2	2	2	34	0	0	0	0	...	14331.0	14948.0	15549.0	1518.0	1500.0	1000.0	1000.0	1000.0

Taiwan Load Findings

The file loaded as 30,000 rows by 25 columns, with 13 float and 12 integer columns and no text columns. In the first row, `PAY_0` is 2 (two months late) and the target is 1, which is an early hint that repayment status carries a strong signal.

Taiwan Data Quality — Undocumented Codes, Duplicates, and Transformations

The Taiwan dataset is well known for containing undocumented category codes: `EDUCATION` values of 0, 5, and 6, a `MARRIAGE` value of 0, and `PAY_*` values of −2. Left as-is, a model would treat these as meaningful categories. The cleaning here applies several decisions:

- `PAY_0` is renamed to `PAY_1` so that the six monthly columns are numbered consistently, and the long target name is renamed to `default`.
- Duplicate detection ignores `ID`, since different IDs with identical 23 features are almost certainly repeated records.
- Undocumented category codes are merged into the documented "other" category.
- `PAY_*` is kept as an ordinal numeric variable (−2 < −1 < 0 < 1 < … months late), because the ordering carries risk information.
- Negative bill amounts (which represent customer overpayments) are real and are preserved via a signed log transform `sign(x)·log(1 + |x|)` that tames skew while keeping the sign.

In [6]:
```
tw = tw_raw.rename(columns={"default.payment.next.month": "default",
"PAY_0": "PAY_1"}).copy()
PAY = [f"PAY_{i}" for i in range(1, 7)]; BILL = [f"BILL_AMT{i}" for i in
range(1, 7)]; PAMT = [f"PAY_AMT{i}" for i in range(1, 7)]
tdq = []
print("Null cells total:", int(tw.isnull().sum().sum()))
tdq.append(("Missing values", int(tw.isnull().sum().sum()), "none to
handle"))
tdq.append(("Duplicate ID", int(tw["ID"].duplicated().sum()), "none"))
dup = int(tw.drop(columns="ID").duplicated().sum())
```

```
tdq.append(("Duplicate rows (ignoring ID)", dup, "dropped (identical
profiles)"))
tw = tw.loc[~tw.drop(columns="ID").duplicated()].copy()

print("EDUCATION codes:",
tw["EDUCATION"].value_counts().sort_index().to_dict())
print("MARRIAGE codes:",
tw["MARRIAGE"].value_counts().sort_index().to_dict())
print("PAY_1 codes:", tw["PAY_1"].value_counts().sort_index().to_dict())
edu_bad = int(tw["EDUCATION"].isin([0, 5, 6]).sum())
tdq.append(("EDUCATION undocumented codes 0/5/6", edu_bad, "recoded to 4
= other"))
tw["EDUCATION"] = tw["EDUCATION"].replace({0: 4, 5: 4, 6: 4})
mar_bad = int((tw["MARRIAGE"] == 0).sum())
tdq.append(("MARRIAGE undocumented code 0", mar_bad, "recoded to 3 =
other"))
tw["MARRIAGE"] = tw["MARRIAGE"].replace({0: 3})
pay_m2 = int((tw[PAY] == -2).any(axis=1).sum())
tdq.append(("PAY_* = -2 (no consumption, undocumented)", pay_m2, "kept
as ordinal (-2 < -1 paid duly < 0 revolving)"))
neg_bill = int((tw[BILL] < 0).any(axis=1).sum())
tdq.append(("Negative BILL_AMT (credit balance/overpay)", neg_bill,
"kept; signed log transform"))
tdq.append(("AGE range " + f"{tw['AGE'].min()}-{tw['AGE'].max()}", 0,
"plausible, kept"))
const = [c for c in tw.columns if tw[c].nunique() <= 1];
tdq.append(("Constant columns", len(const), "none"))

# Leakage screen: all features are observed BEFORE the default month
(Apr-Sep 2005 -> Oct 2005)
tdq.append(("Post-outcome columns", 0, "none: all features precede
target month"))
slog = lambda s: np.sign(s) * np.log1p(np.abs(s))
for c in BILL + PAMT: tw["log_" + c] = slog(tw[c])
tw["log_LIMIT_BAL"] = np.log(tw["LIMIT_BAL"])
tw = tw.drop(columns="ID")
print(f"\nClean Taiwan rows: {len(tw):,} | default rate:
{tw['default'].mean():.2%}")
```

```
Null cells total: 0
EDUCATION codes: {0: 14, 1: 10563, 2: 14019, 3: 4915, 4: 123, 5: 280, 6:
51}
MARRIAGE codes: {0: 54, 1: 13643, 2: 15945, 3: 323}
PAY_1 codes: {-2: 2750, -1: 5682, 0: 14737, 1: 3667, 2: 2666, 3: 322, 4:
76, 5: 26, 6: 11, 7: 9, 8: 19}

Clean Taiwan rows: 29,965 | default rate: 22.13%
```

Taiwan Cleaning Findings

The data contains no missing cells and no duplicate IDs, but 35 duplicate profiles are found when `ID` is ignored, and these rows are dropped.

The undocumented category counts are:

- `EDUCATION`: codes 0, 5, and 6 together account for 345 rows (0 → 14 rows, 5 → 280, 6 → 51).
- `MARRIAGE`: code 0 accounts for 54 rows.
- `PAY_1`: code −2 accounts for 2,750 rows, with a long tail of codes 3–8 that are each rare (322 rows or fewer).

The `EDUCATION` and `MARRIAGE` recodes affect roughly 1.3% of rows, so merging them into "other" is a low-risk choice. The `PAY_* = −2` code is far too common to drop; interpreting it as "no balance to pay" (ranked below "paid duly") is more defensible than treating it as missing.

After cleaning, 29,965 rows remain, with a 22.13% default rate.

Taiwan Data Quality Summary, Collinearity Preview, and Target Balance

The summary table documents all Taiwan cleaning decisions. Two preview visualizations are added:

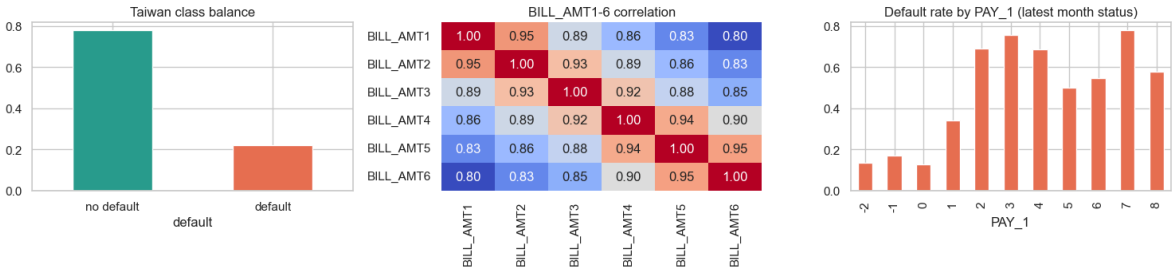
- A correlation heatmap of the six monthly bill columns, to show whether multicollinearity among the bills is likely to matter.
- The default rate by `PAY_1`, to preview the strongest expected risk signal.

The skew of `LIMIT_BAL` is also checked before and after a log transform, since the target for the Taiwan modeling will be `log(LIMIT_BAL)`.

In [7]:
```
tw_dq = pd.DataFrame(tdq, columns=["issue", "rows_affected", "action"])
display(tw_dq)
display(tw[["LIMIT_BAL", "AGE", "BILL_AMT1", "PAY_AMT1",
"PAY_1"]].describe().round(1).T)
fig, ax = plt.subplots(1, 3, figsize=(15, 3.6))
tw["default"].value_counts(normalize=True).sort_index().plot.bar(ax=ax[0
], color=["#2a9d8f", "#e76f51"])
ax[0].set_title("Taiwan class balance"); ax[0].set_xticklabels(["no
default", "default"], rotation=0)
sns.heatmap(tw[BILL].corr(), annot=True, fmt=".2f", cmap="coolwarm",
ax=ax[1], cbar=False)
ax[1].set_title("BILL_AMT1–6 correlation")
tw.groupby("PAY_1")["default"].mean().plot.bar(ax=ax[2],
color="#e76f51"); ax[2].set_title("Default rate by PAY_1 (latest month
status)")
plt.tight_layout(); plt.show()
print("Skew LIMIT_BAL raw vs log:", round(tw["LIMIT_BAL"].skew(), 2), "-
>", round(tw["log_LIMIT_BAL"].skew(), 2))
```

	issue	rows_affected	action
0	Missing values	0	none to handle
1	Duplicate ID	0	none
2	Duplicate rows (ignoring ID)	35	dropped (identical profiles)
3	EDUCATION undocumented codes 0/5/6	345	recoded to 4 = other
4	MARRIAGE undocumented code 0	54	recoded to 3 = other
5	PAY_* = -2 (no consumption, undocumented)	6531	kept as ordinal (-2 < -1 paid duly < 0 revolving)
6	Negative BILL_AMT (credit balance/overpay)	1930	kept; signed log transform
7	AGE range 21-79	0	plausible, kept
8	Constant columns	0	none
9	Post-outcome columns	0	none: all features precede target month

	count	mean	std	min	25%	50%	75%	max
LIMIT_BAL	29965.0	167442.0	129760.1	10000.0	50000.0	140000.0	240000.0	1000000.0
AGE	29965.0	35.5	9.2	21.0	28.0	34.0	41.0	79.0
BILL_AMT1	29965.0	51283.0	73658.1	-165580.0	3595.0	22438.0	67260.0	964511.0
PAY_AMT1	29965.0	5670.1	16571.8	0.0	1000.0	2102.0	5008.0	873552.0
PAY_1	29965.0	-0.0	1.1	-2.0	-1.0	0.0	0.0	8.0




```
Skew LIMIT_BAL raw vs log: 0.99 -> -0.51
```

Taiwan Summary Findings

The issues table records 35 duplicates, 345 education recodes, 54 marriage recodes, 6,531 rows containing a −2 code in some month, and 1,930 rows with a negative bill. Ages run 21 to 79. No leakage columns are present because all features are observed in April–September 2005 while the target is default in October 2005.

The bill correlations are high. Neighbouring months of `BILL_AMT` have correlations near 0.95, and month 1 versus month 6 is still about 0.80. This strong multicollinearity is one of the reasons regularization is a natural fit for this dataset.

The default rate rises sharply with `PAY_1`. Rates sit between 13% and 17% for codes −2, −1, and 0 (no balance, paid duly, revolving), rise to about 34% for one month late and 69% for two months late, and remain between 50% and 78% for the rarer longer delays. `PAY_1` is clearly the dominant risk signal in this dataset.

`LIMIT_BAL` has a raw skew of 0.99, which falls to −0.51 after the log transform. The log scale is therefore used as the modeling target. The default rate of 22% is close to Lending Club's 20%, so the same metric considerations apply.

Part B — Regularized Regression

With both datasets cleaned, the modeling for this analysis compares plain OLS against three regularized alternatives on a deliberately wide feature matrix. The

expansion is done with `PolynomialFeatures(degree=2)` so that every square and pairwise product of the numeric inputs is present. This gives the penalty something meaningful to act on.

Key Concepts

- **OLS** minimizes squared error only. With many correlated features it overfits, producing coefficients that are large and unstable.
- **Ridge (L2)** minimizes `squared error + α·Σβ²`. It shrinks all coefficients toward zero but never exactly to zero. It performs well when many features each carry a small amount of signal, or when features are correlated.
- **Lasso (L1)** minimizes `squared error + α·Σ|β|`. It can set coefficients exactly to zero, so it performs feature selection as well as shrinkage.
- **Elastic Net** mixes both: `α·[l1_ratio·Σ|β| + (1 − l1_ratio)/2·Σβ²]`. It is useful when correlated features come in groups, because lasso alone tends to pick one of a correlated group at random.
- **α (lambda)** controls the strength of the penalty. α → 0 recovers OLS; a very large α shrinks every coefficient to zero. Here α is chosen by 5-fold cross-validation: fit on 4/5 of the training data, score on the remaining 1/5, rotate, and average.
- **Scaling is mandatory.** The penalty treats every coefficient equally, so features must be on the same scale (`StandardScaler`). Otherwise, features with large natural units would be penalized unfairly.

3. Expanded Feature Matrices (Polynomial + One-Hot)

Regularization is only interesting when there are many features relative to the signal, so each dataset is expanded before fitting. The construction uses:

1. An 80/20 train/test split with seed 42. For Lending Club, a reproducible 25,000-loan sample is used so that cross-validation runs in reasonable time.
2. Inside a `ColumnTransformer`, numeric columns pass through `StandardScaler → PolynomialFeatures(2) → StandardScaler`. The first scaling keeps the squares from blowing up; the second puts every expanded term on the same scale for the penalty.
3. Categorical columns are one-hot encoded with the first level dropped.
4. Every transformation is fit on the training data only, then applied to the test set, so no test information leaks in.

Lending Club excludes `grade`. Earlier analysis showed that grade alone explains about 91% of `int_rate`, so including it would make the raw borrower features look irrelevant. Dropping grade here measures what the raw borrower features can do on their own.

In [8]:

```
from sklearn.model_selection import train_test_split, KFold,
cross_val_score
from sklearn.preprocessing import StandardScaler, PolynomialFeatures,
OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.linear_model import LinearRegression, Ridge, Lasso,
ElasticNet, RidgeCV, LassoCV, ElasticNetCV, lasso_path
from sklearn.metrics import mean_squared_error, r2_score
FIG = "../../../week_10092026_w7/DX799O1/figures"  # report figures
folder (relative to this notebook)
os.makedirs(FIG, exist_ok=True)

LC_NUM = ["loan_amnt", "log_annual_inc", "dti", "revol_util",
"inq_last_6mths", "open_acc", "total_acc",
          "mort_acc", "delinq_2yrs", "pub_rec", "log_revol_bal",
"pub_rec_bankruptcies"]
LC_CAT = ["term", "home_ownership", "purpose", "verification_status"]
TW_NUM = ["AGE", "PAY_1", "PAY_2", "PAY_3", "PAY_4", "PAY_5", "PAY_6"] +
[f"log_BILL_AMT{i}" for i in range(1, 7)] + [f"log_PAY_AMT{i}" for i in
range(1, 7)]
TW_CAT = ["SEX", "EDUCATION", "MARRIAGE"]

def make_prep(num, cat):
    return ColumnTransformer([
        ("num", make_pipeline(StandardScaler(), PolynomialFeatures(2,
include_bias=False), StandardScaler()), num),
        ("cat", OneHotEncoder(drop="first", handle_unknown="ignore",
sparse_output=False), cat)])

data = {}
lc_s = lc.sample(n=25_000, random_state=SEED)
a, b = train_test_split(lc_s, test_size=0.2, random_state=SEED)
data["Lending Club"] = (a[LC_NUM + LC_CAT], a["int_rate"], b[LC_NUM +
LC_CAT], b["int_rate"], LC_NUM, LC_CAT)
a, b = train_test_split(tw, test_size=0.2, random_state=SEED,
stratify=tw["default"])
data["Taiwan"] = (a[TW_NUM + TW_CAT], a["log_LIMIT_BAL"], b[TW_NUM +
TW_CAT], b["log_LIMIT_BAL"], TW_NUM, TW_CAT)

mats = {}
for name, (Xtr, ytr, Xte, yte, num, cat) in data.items():
    prep = make_prep(num, cat).fit(Xtr)
    Ztr, Zte = prep.transform(Xtr), prep.transform(Xte)
    mats[name] = (Ztr, ytr.values, Zte, yte.values,
prep.get_feature_names_out())
    print(f"{name}: raw numeric {len(num)} + categorical {len(cat)} ->
expanded features {Ztr.shape[1]} | "
          f"train rows {Ztr.shape[0]:,} ({Ztr.shape[0]/Ztr.shape[1]:.0f}
rows per feature) | target SD {ytr.std():.3f}")
```

```
Lending Club: raw numeric 12 + categorical 4 -> expanded features 108 |
train rows 20,000 (185 rows per feature) | target SD 4.762
Taiwan: raw numeric 19 + categorical 3 -> expanded features 215 | train
rows 23,972 (111 rows per feature) | target SD 0.943
```

Expansion Findings

- **Lending Club:** 12 numeric and 4 categorical inputs expand into 108 features, with 20,000 training rows (about 185 rows per feature). The target standard deviation is 4.762.
- **Taiwan:** 19 numeric and 3 categorical inputs expand into 215 features, with 23,972 training rows (about 111 rows per feature). The target standard deviation is 0.943 log units.

A rows-per-feature ratio above 100 is comfortable for OLS, so very large gains from regularization should not be expected on the full training sets. The learning-curve step below checks the small-sample case directly. The target standard deviations are also the RMSE a "predict the mean" baseline would achieve.

4. Cross-Validated Error vs α (the Bias–Variance Curves)

This is the core tuning picture. For each α on a log grid from 10^{-4} to 10^{3}, the 5-fold CV RMSE (mean ± SE) is computed for ridge and lasso on each dataset.

- **Small α** behaves like OLS: low bias, high variance.
- **Large α** shrinks everything toward zero: high bias, underfitting.
- The minimum of the curve is the best bias–variance trade-off.

The **1-standard-error rule** is also tracked: the largest α whose CV error is within one standard error of the minimum. That α gives a simpler model that is statistically indistinguishable from the best, which is useful when parsimony matters.

As an alternative, `RidgeCV` and `LassoCV` can locate the best α automatically (used in the next step). The explicit loop here is kept because it is what allows the full curve to be plotted.

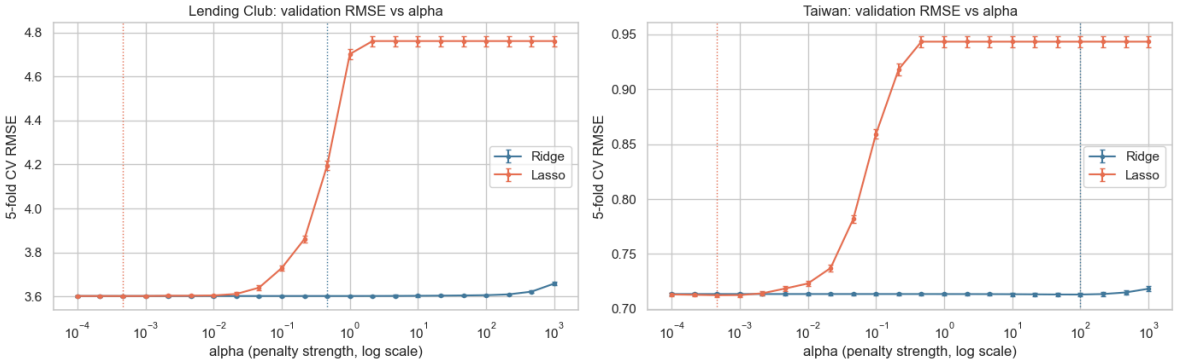
In [9]:
```
kf = KFold(5, shuffle=True, random_state=SEED)
alphas = np.logspace(-4, 3, 22)
curves = {}
for name, (Ztr, ytr, Zte, yte, feats) in mats.items():
    for mdl, est in [("Ridge", Ridge), ("Lasso", Lasso)]:
        res = []
        for a_ in alphas:
            s = np.sqrt(-cross_val_score(est(alpha=a_, max_iter=5000) if
mdl == "Lasso" else est(alpha=a_), Ztr, ytr, cv=kf,

scoring="neg_mean_squared_error"))
```

```
            res.append((a_, s.mean(), s.std() / np.sqrt(len(s))))
        curves[(name, mdl)] = pd.DataFrame(res, columns=["alpha",
"cv_rmse", "se"])

fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))
for ax, name in zip(axes, mats):
    for mdl, col in [("Ridge", "#457b9d"), ("Lasso", "#e76f51")]:
        c = curves[(name, mdl)]
        ax.errorbar(c["alpha"], c["cv_rmse"], yerr=c["se"], label=mdl,
color=col, marker="o", ms=3, capsize=2)
        best = c.loc[c["cv_rmse"].idxmin()]
        ax.axvline(best["alpha"], color=col, ls=":", lw=1)
        one_se = c[c["cv_rmse"] <= best["cv_rmse"] + best["se"]]
["alpha"].max()
        print(f"{name:12s} {mdl}: best alpha={best['alpha']:.4g} CV
RMSE={best['cv_rmse']:.4f} (+-{best['se']:.4f}) | 1-SE alpha=
{one_se:.4g} | "
              f"CV RMSE at smallest alpha={c['cv_rmse'].iloc[0]:.4f}, at
largest={c['cv_rmse'].iloc[-1]:.4f}")
    ax.set_xscale("log"); ax.set_xlabel("alpha (penalty strength, log
scale)"); ax.set_ylabel("5-fold CV RMSE")
    ax.set_title(f"{name}: validation RMSE vs alpha"); ax.legend()
plt.tight_layout(); plt.savefig(f"{FIG}/fig_w2_alpha_curves.png",
dpi=150, bbox_inches="tight"); plt.show()
```

```
Lending Club Ridge: best alpha=0.4642 CV RMSE=3.6016 (+-0.0045) | 1-SE
alpha=100 | CV RMSE at smallest alpha=3.6016, at largest=3.6581
Lending Club Lasso: best alpha=0.0004642 CV RMSE=3.6014 (+-0.0045) | 1-SE
alpha=0.01 | CV RMSE at smallest alpha=3.6015, at largest=4.7618
Taiwan       Ridge: best alpha=100 CV RMSE=0.7131 (+-0.0017) | 1-SE
alpha=215.4 | CV RMSE at smallest alpha=0.7135, at largest=0.7183
Taiwan       Lasso: best alpha=0.0004642 CV RMSE=0.7125 (+-0.0016) | 1-SE
alpha=0.001 | CV RMSE at smallest alpha=0.7132, at largest=0.9433
```



CV Error Findings

Dataset	Model	Best α	CV RMSE (±SE)	1-SE α	CV RMSE at the largest α
Lending Club	Ridge	0.464	3.6016 (±0.0045)	100	3.6581
Lending Club	Lasso	0.000464	3.6014 (±0.0045)	0.01	4.7618

Dataset	Model	Best α	CV RMSE (±SE)	1-SE α	CV RMSE at the largest α
Taiwan	Ridge	100	0.7131 (±0.0017)	215.4	0.7183
Taiwan	Lasso	0.000464	0.7125 (±0.0016)	0.001	0.9433

Four observations:

- **The curves are flat on the small-α side.** CV error at the smallest α (effectively OLS) is within one SE of the best, so on the full training sets the penalty barely changes error.
- **The right side shows the bias–variance trade-off clearly.** Lasso's error rises to the target standard deviation (4.76 for Lending Club, 0.94 for Taiwan) at large α, which is the point at which every coefficient has been shrunk to zero and the model is just predicting the mean. Ridge rises much more slowly because it never removes features entirely.
- **Ridge and lasso α values are not comparable.** Taiwan ridge picks α = 100 while lasso picks α ≈ 0.0005, because ridge penalizes squared coefficients and lasso penalizes absolute values.
- **The 1-SE rule allows a sparser lasso at no significant cost.** α can rise to 0.01 (Lending Club) or 0.001 (Taiwan) with no meaningful loss in CV error.

For Lending Club, the minimum is less than 0.1% below OLS, so regularization adds almost nothing. For Taiwan, there is a small but consistent gain, and ridge prefers a much larger α (100), which is consistent with the strong bill collinearity observed in the data-quality step.

5. OLS, RidgeCV, LassoCV, and ElasticNetCV on the Held-Out Test Set

Cross-validation picks α using only the training data; the test set is touched once, at the end, to estimate real-world error. OLS on the same expanded features is the reference — if regularization helps, it should beat OLS on the test set.

- `RidgeCV` uses an efficient leave-one-out-style search over α.
- `LassoCV` and `ElasticNetCV` run 5-fold cross-validation along an α path.
- Elastic Net also searches `l1_ratio ∈ {0.1, 0.5, 0.9}`.
- The number of non-zero coefficients is counted, which shows how many features lasso and elastic net keep.

In [10]:
```
rows, fitted = [], {}
for name, (Ztr, ytr, Zte, yte, feats) in mats.items():
    models = {
        "OLS": LinearRegression(),
        "Ridge": RidgeCV(alphas=np.logspace(-3, 4, 60)),
```

```
        "Lasso": LassoCV(alphas=np.logspace(-4, 1, 60), cv=kf,
max_iter=10000, n_jobs=-1),
        "ElasticNet": ElasticNetCV(alphas=np.logspace(-4, 1, 40),
l1_ratio=[0.1, 0.5, 0.9], cv=kf, max_iter=10000, n_jobs=-1),
    }
    for mname, m in models.items():
        m.fit(Ztr, ytr); pred = m.predict(Zte); fitted[(name, mname)] =
m
        rows.append({"dataset": name, "model": mname,
                     "alpha": getattr(m, "alpha_", np.nan), "l1_ratio":
getattr(m, "l1_ratio_", np.nan),
                     "nonzero": int((np.abs(m.coef_) > 1e-10).sum()),
"n_features": Ztr.shape[1],
                     "train_RMSE": np.sqrt(mean_squared_error(ytr,
m.predict(Ztr))),
                     "test_RMSE": np.sqrt(mean_squared_error(yte,
pred)), "test_R2": r2_score(yte, pred),
                     "max_abs_coef": np.abs(m.coef_).max()})
w2 = pd.DataFrame(rows)
display(w2.round(4))
```

	dataset	model	alpha	l1_ratio	nonzero	n_features	train_RMSE	test_RMSE	test_R2	max_abs_coef
0	Lending Club	OLS	NaN	NaN	108	108	3.5631	3.6037	0.4316	4.6584
1	Lending Club	Ridge	1.5973	NaN	108	108	3.5631	3.6033	0.4317	4.1918
2	Lending Club	Lasso	0.0003	NaN	107	108	3.5633	3.6031	0.4318	4.1928
3	Lending Club	ElasticNet	0.0003	0.9	108	108	3.5633	3.6030	0.4318	4.1920
4	Taiwan	OLS	NaN	NaN	215	215	0.7040	0.7046	0.4301	0.4539
5	Taiwan	Ridge	73.1824	NaN	215	215	0.7044	0.7036	0.4317	0.4384
6	Taiwan	Lasso	0.0007	NaN	153	215	0.7057	0.7033	0.4322	0.4483
7	Taiwan	ElasticNet	0.0008	0.9	153	215	0.7057	0.7033	0.4322	0.4478

Test-Set Comparison Findings

Dataset	Model	α	Non-zero / total	Test RMSE	Test R^2
Lending Club	OLS	—	108 / 108	3.6037	0.4316
Lending Club	Ridge	1.60	108 / 108	3.6033	0.4317
Lending Club	Lasso	0.0003	107 / 108	3.6031	0.4318
Lending Club	ElasticNet (l1 0.9)	0.0003	108 / 108	**3.6030**	0.4318
Taiwan	OLS	—	215 / 215	0.7046	0.4301
Taiwan	Ridge	73.2	215 / 215	0.7036	0.4317
Taiwan	Lasso	0.0007	**153 / 215**	**0.7033**	**0.4322**
Taiwan	ElasticNet (l1 0.9)	0.0008	153 / 215	0.7033	0.4322

Several things stand out:

- **Overfitting check.** Training RMSE is roughly the same as test RMSE for every model (around 3.56 vs 3.60 for Lending Club, and 0.70 vs 0.70 for Taiwan). With 100+ rows per feature, OLS is not overfitting, so the penalty has little variance to remove.

- **Taiwan: the same accuracy, fewer features.** Lasso drops 62 of 215 features (29%) without any loss in accuracy and with a 0.0013 RMSE improvement. The value is mainly in simplicity.
- **Elastic Net chose `l1_ratio = 0.9`**, which is almost pure lasso. The data prefers sparsity over ridge-style grouping.
- **Expanding Lending Club's borrower features (without grade) lifts test R^2 to 0.43.** The extra `purpose`, `verification_status`, and `home_ownership` categories, along with the pairwise products, carry real price information even when the lender's own grade rating is excluded.

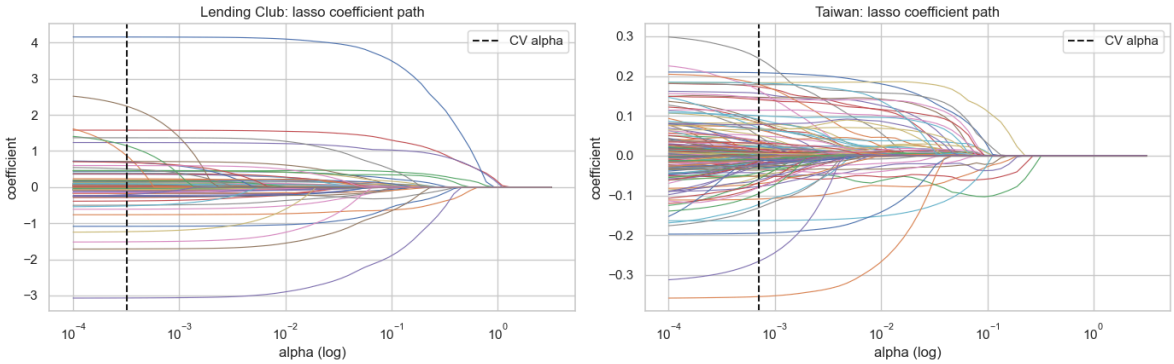
The small penalty gains should not be read as "regularization is useless". The right reading is that this sample size is large enough that OLS is already stable. The learning-curve step tests the small-sample case directly.

6. Lasso Coefficient Path and the Features That Survive

The lasso coefficient path shows every coefficient as α decreases from "everything zero" to "almost OLS". Features that enter the active set early (at large α) carry the strongest independent signal.

Lasso's built-in feature selection is also easier to explain to a non-technical audience than, say, 200 small ridge coefficients.

In [11]:
```
fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))
for ax, name in zip(axes, mats):
    Ztr, ytr, _, _, feats = mats[name]
    alphas_p, coefs, _ = lasso_path(Ztr, ytr - ytr.mean(),
alphas=np.logspace(-4, 0.5, 60), max_iter=10000)
    ax.plot(alphas_p, coefs.T, lw=0.8)
    ax.axvline(fitted[(name, "Lasso")].alpha_, color="k", ls="--",
label="CV alpha")
    ax.set_xscale("log"); ax.set_xlabel("alpha (log)");
ax.set_ylabel("coefficient"); ax.set_title(f"{name}: lasso coefficient
path"); ax.legend()
plt.tight_layout(); plt.savefig(f"{FIG}/fig_w2_lasso_path.png", dpi=150,
bbox_inches="tight"); plt.show()
for name in mats:
    feats = mats[name][4]; m = fitted[(name, "Lasso")]
    top = pd.Series(m.coef_, index=feats); top = top[top.abs() >
0].sort_values(key=np.abs, ascending=False)
    print(f"\n{name}: lasso keeps {len(top)} of {len(feats)} features.
Top 10 by |coef| (standardized units):")
    print(top.head(10).round(4).to_string())
```




```
Lending Club: lasso keeps 107 of 108 features. Top 10 by |coef|
(standardized units):
cat__term_ 60 months                  4.1928
cat__purpose_wedding                  3.9530
cat__purpose_renewable_energy         2.8129
cat__purpose_small_business           2.3772
cat__purpose_moving                   2.3630
cat__purpose_house                    2.1931
cat__purpose_other                    1.9167
num__revol_util                       1.5691
cat__verification_status_Verified     1.4363
cat__purpose_vacation                 1.3532

Taiwan: lasso keeps 153 of 215 features. Top 10 by |coef| (standardized
units):
cat__EDUCATION_3           -0.4483
cat__MARRIAGE_3            -0.3140
cat__EDUCATION_2           -0.2877
num__log_BILL_AMT1          0.2446
cat__EDUCATION_4           -0.1834
num__AGE                    0.1829
num__log_PAY_AMT6           0.1828
num__PAY_2 log_BILL_AMT1    0.1801
num__log_PAY_AMT5           0.1757
num__log_PAY_AMT1           0.1638
```

Lasso Path Findings

At large α every path sits at zero. As α falls, a handful of coefficients leave zero first, and most remain small until roughly $\alpha \approx 10^{-3}$. The CV-chosen α (dashed line) sits in the region where most features are active.

Lending Club (107 of 108 kept). The largest standardized coefficients are:

- `term_60 months` +4.19
- purposes: wedding +3.95, renewable energy +2.81, small business +2.38, moving +2.36, house +2.19, other +1.92
- `revol_util` +1.57
- `verification_status_Verified` +1.44

Taiwan (153 of 215 kept). The largest are:

- `EDUCATION_3` (high school) −0.45, `MARRIAGE_3` (other) −0.31, `EDUCATION_2` (university) −0.29
- `log_BILL_AMT1` +0.24, `AGE` +0.18
- recent payment amounts between +0.16 and +0.18
- the interaction `PAY_2 × log_BILL_AMT1` +0.18

For **Lending Club**, the price is set mostly by loan structure (60-month term) and loan purpose. These are the lender's risk-class rules, which the omitted `grade` variable also summarizes.

For **Taiwan**, the credit-limit decision follows education and marital status, the most recent bill, and recent payment amounts. Lasso drops many redundant cross-products of the six bill and payment months, which is exactly what one would hope the L1 penalty would do on strongly correlated predictors.

7. Ridge vs Lasso Shrinkage on the Most Collinear Block (Taiwan Bills)

The six monthly bill columns are the most correlated block in either dataset (neighbouring months above 0.95). To make the different behaviours of the penalties concrete, the raw linear coefficients on `log_BILL_AMT1..6` are extracted from each fitted model.

If OLS is unstable on this block, it will show the classic multicollinearity symptom — large alternating-sign coefficients. Ridge should spread the weight more smoothly, and lasso should keep only a few.

In [12]:
```
Ztr, ytr, _, _, feats = mats["Taiwan"]
bill = [f"num__log_BILL_AMT{i}" for i in range(1, 7)]
idx = [list(feats).index(b) for b in bill]
cmp = pd.DataFrame({m: fitted[("Taiwan", m)].coef_[idx] for m in ["OLS",
"Ridge", "Lasso", "ElasticNet"]}, index=[b.replace("num__", "") for b in
bill])
display(cmp.round(4))
```

	OLS	Ridge	Lasso	ElasticNet
log_BILL_AMT1	0.3080	0.2649	0.2446	0.2430
log_BILL_AMT2	-0.0881	-0.0538	-0.0456	-0.0445
log_BILL_AMT3	-0.0144	-0.0095	-0.0059	-0.0055
log_BILL_AMT4	0.0202	0.0103	-0.0000	-0.0000
log_BILL_AMT5	-0.0528	-0.0395	-0.0197	-0.0193
log_BILL_AMT6	-0.0448	-0.0399	-0.0301	-0.0300

Collinear-Block Findings

- **OLS:** `BILL_AMT1` +0.308, then alternating signs across the next five months (−0.088, −0.014, +0.020, −0.053, −0.045).

- **Ridge:** the same pattern, shrunk (+0.265, −0.054, …).
- **Lasso:** `BILL_AMT1` +0.245, with `BILL_AMT4` set exactly to zero and the others shrunk further.

The alternating signs in the OLS fit are the signature of multicollinearity: with the bills correlated at 0.8–0.95, OLS assigns offsetting positive and negative weights that largely cancel out. Ridge pulls all of them toward zero, which stabilizes the estimates. Lasso removes one entirely and keeps the most recent month as the main carrier of the signal.

Read in business terms: once the latest statement balance is known, older months add little independent information about the credit limit.

8. Learning Curve — When Does Regularization Pay Off?

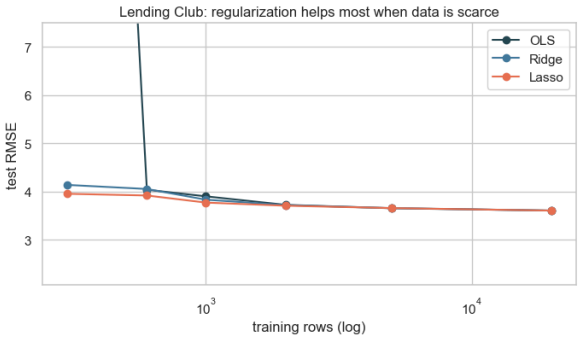
On the full training sets there are 111–185 rows per feature, so OLS is already stable. Regularization is designed for the opposite case: few rows per feature.

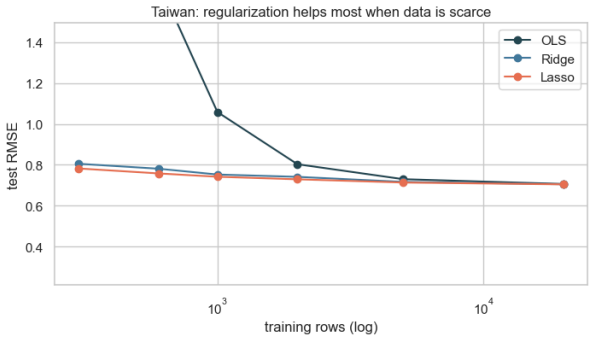
To find the point at which the penalty stops mattering, OLS, RidgeCV, and LassoCV are refit on random training subsets of 300 to 20,000 rows, using the same expanded features and the same held-out test set.

In [13]:
```
rng = np.random.RandomState(SEED)
sizes = [300, 600, 1000, 2000, 5000, 20000]
lc_rows = []
for name, (Ztr, ytr, Zte, yte, feats) in mats.items():
    for n_ in sizes:
        if n_ > len(ytr): continue
        idx = rng.choice(len(ytr), n_, replace=False)
        for mname, m in [("OLS", LinearRegression()), ("Ridge",
RidgeCV(alphas=np.logspace(-2, 5, 50))),
                         ("Lasso", LassoCV(alphas=np.logspace(-4, 1,
40), cv=5, max_iter=10000, n_jobs=-1))]:
            m.fit(Ztr[idx], ytr[idx])
            lc_rows.append({"dataset": name, "n_train": n_, "model":
mname,
                            "test_RMSE": np.sqrt(mean_squared_error(yte,
m.predict(Zte)))})
lcurve = pd.DataFrame(lc_rows)
display(lcurve.pivot_table(index=["dataset", "n_train"],
columns="model", values="test_RMSE").round(4))
fig, axes = plt.subplots(1, 2, figsize=(14, 4.2))
for ax, name in zip(axes, mats):
    for mname, col in [("OLS", "#264653"), ("Ridge", "#457b9d"),
("Lasso", "#e76f51")]:
        d_ = lcurve[(lcurve.dataset == name) & (lcurve.model == mname)]
        ax.plot(d_["n_train"], d_["test_RMSE"], marker="o", label=mname,
color=col)
    ax.set_xscale("log"); ax.set_xlabel("training rows (log)");
```

```
ax.set_ylabel("test RMSE")
    ax.set_title(f"{name}: regularization helps most when data is
scarce")
    ax.set_ylim(top=min(ax.get_ylim()[1], lcurve[lcurve.dataset == name]
["test_RMSE"].median() * 2)); ax.legend()
plt.tight_layout(); plt.savefig(f"{FIG}/fig_w2_learning_curve.png",
dpi=150, bbox_inches="tight"); plt.show()
```

dataset	n_train	Lasso	OLS	Ridge
Lending Club	300	3.9515	34.4624	4.1362
	600	3.9162	4.0356	4.0512
	1000	3.7691	3.8986	3.8294
	2000	3.7041	3.7218	3.7159
	5000	3.6552	3.6517	3.6538
	20000	3.6029	3.6037	3.6033
Taiwan	300	0.7819	10.5588	0.8049
	600	0.7576	1.6967	0.7807
	1000	0.7413	1.0582	0.7523
	2000	0.7289	0.8027	0.7409
	5000	0.7130	0.7294	0.7162
	20000	0.7040	0.7060	0.7046





Learning-Curve Findings

n train	LC OLS	LC Ridge	LC Lasso	TW OLS	TW Ridge	TW Lasso
300	**34.46**	4.14	**3.95**	**10.56**	0.80	**0.78**
600	4.04	4.05	3.92	1.70	0.78	0.76
1,000	3.90	3.83	3.77	1.06	0.75	0.74
2,000	3.72	3.72	3.70	0.80	0.74	0.73
5,000	3.65	3.65	3.66	0.73	0.72	0.71
20,000	3.60	3.60	3.60	0.71	0.70	0.70

Three things emerge:

- **At 300 rows, OLS collapses.** With 108 or 215 features and only 300 rows, OLS memorizes noise. Its test RMSE is 34.5 for Lending Club (about 7× the target SD) and 10.6 for Taiwan (about 11× the SD). Ridge and lasso stay near 4.0 and 0.8 respectively. This is the clearest evidence of variance reduction by regularization anywhere in the analysis.

- **The gap closes by about 2,000 rows for Lending Club and 5,000–20,000 rows for Taiwan.** Taiwan needs more data because it has twice as many features and much stronger collinearity.
- **Lasso is the best or tied-best in every Taiwan row,** which is consistent with many redundant features.

The practical reading is that regularization behaves like insurance: it matters when the rows-per-feature ratio is low, and buys very little when the data is already abundant. On the full samples, the penalty barely moves accuracy but it does buy sparsity — lasso removes 29% of the Taiwan features at no cost.

For a capstone target that eventually trains on a small segment of the data (one US state, one loan purpose, one tenure band), regularization would become essential rather than optional.

9. Conclusion

This analysis compared OLS, ridge, lasso, and elastic net on expanded feature matrices built from two consumer credit datasets.

On the full training sets (20,000 and 23,972 rows), all four methods produce almost identical test accuracy. Regularization improves Lending Club's test R^2 only marginally over OLS and improves Taiwan's by about 0.002. The practical value of lasso on Taiwan is sparsity: it keeps 153 of 215 features (29% fewer) with no loss in accuracy.

Lasso's coefficient path and surviving features show that for Lending Club the interest rate is driven by loan structure and purpose, and for Taiwan the credit limit is explained mainly by education, marital status, the most recent bill, and recent payment amounts. On the strongly correlated Taiwan monthly bills, OLS shows the classic alternating-sign multicollinearity symptom while ridge shrinks the pattern and lasso removes one month entirely.

The learning-curve step shows the sharpest contrast. With only 300 training rows against 108–215 features, OLS collapses — test RMSE jumps to 34.5 for Lending Club and 10.6 for Taiwan — while ridge and lasso stay close to their large-sample values. The gap narrows as the training set grows and effectively disappears by 2,000 rows for Lending Club and 5,000–20,000 rows for Taiwan.

Key Limitations

The regularization gains at full sample size are too small to be reported as improvements over OLS without caveats. The coefficients recovered by lasso describe associations within a fitted linear model and should not be read as causal

effects. Expanding the feature matrix with every degree-2 interaction is a mechanical choice; the surviving features are informative, but they are not a substitute for domain-driven feature engineering.

Overall, regularization made the largest difference exactly where theory predicts: when the rows-per-feature ratio is small and when correlated feature blocks are present. Lasso's ability to zero out coefficients was more useful than ridge-style uniform shrinkage on these datasets, which is reflected in the elastic-net `l1_ratio` of 0.9.

Week 3 — Stepwise Selection, PCR and PLSR

`week_09112026/DX79901/homework/kumar_gaurav_week3_dx79901_v2.ipynb`

Capstone Milestone One — Week 3: Subset Selection, PCR, and PLSR

Kumar Gaurav · DX799 O1 Data Science Capstone

Project Overview

Consumer credit risk is a central concern for lenders because mispriced loans and misjudged credit limits translate directly into losses and lost customers.

In this analysis, I use two consumer credit datasets to compare three complementary approaches to handling many predictors in a linear model: subset selection (forward and backward stepwise by AIC), principal components regression (PCR), and partial least squares regression (PLSR).

Research Question

When a linear regression is given many predictors — some of which are correlated — do stepwise selection, PCR, or PLSR give a better complexity-vs-accuracy trade-off than plain OLS, and how do the three methods differ in the *number* of predictors or components they need?

The two target variables used here are:

- **Lending Club** `int_rate` — the interest rate (%) charged on an installment loan.
- **Taiwan** `log(LIMIT_BAL)` — the log of the credit limit granted on a revolving credit card.

The Lending Club analysis deliberately excludes `grade`, `sub_grade`, and `installment`, because they are assigned jointly with the rate and would leak the answer.

Objectives

The objectives of this analysis are to:

- Fit a full OLS baseline and compare it against forward and backward stepwise selection by AIC.
- Tune the number of components in PCR and PLSR by 5-fold cross-validation on each dataset.
- Benchmark every method against a held-out test set, together with ridge, lasso, and elastic-net reference fits.
- Diagnose multicollinearity using Variance Inflation Factors (VIF) and interpret the methods accordingly.

Approach

The analysis follows a two-part modeling structure:

Lending Club Modeling → Taiwan Modeling → Cross-Dataset Summary

Datasets

- **Lending Club** (Kaggle `adarshsng/lending-club-loan-data-csv`): US peer-to-peer installment loans, 2007–2018.
- **UCI Default of Credit Card Clients** (Yeh & Lien, 2009; Kaggle `uciml/default-of-credit-card-clients-dataset`): 30,000 Taiwanese credit-card clients, April–September 2005, with a target of default in October 2005.

1. Data Loading and Cleaning

The code cells below load the two raw files, apply the documented cleaning rules (outcome definition, invalid-range handling, undocumented-code recoding, log transforms, and target-leakage removal), and produce the cleaned `lc` and `tw` frames that the modeling sections use. The data-quality audit for these datasets is documented separately.

In [14]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import
variance_inflation_factor
import kagglehub
import os
import warnings
# sklearn tools
from sklearn.model_selection import train_test_split, cross_val_score,
KFold
```

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cross_decomposition import PLSRegression
from sklearn.linear_model import LinearRegression, RidgeCV, LassoCV,
ElasticNetCV
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.pipeline import make_pipeline

# Suppress warnings for a cleaner output
warnings.filterwarnings('ignore')

# Set plotting style
sns.set_theme(style="whitegrid")
print("Libraries imported successfully.")

# Using the Lending Club Loan Data dataset
path = kagglehub.dataset_download("adarshsng/lending-club-loan-data-
csv/versions/1")
df = pd.read_csv(os.path.join(path, "loan.csv"))

print(f"Full dataset shape: {df.shape}")
print(f"Columns: {df.columns.tolist()}")
print(df.head(3))
```

```
Libraries imported successfully.
Full dataset shape: (2260668, 145)
Columns: ['id', 'member_id', 'loan_amnt', 'funded_amnt',
'funded_amnt_inv', 'term', 'int_rate', 'installment', 'grade',
'sub_grade', 'emp_title', 'emp_length', 'home_ownership', 'annual_inc',
'verification_status', 'issue_d', 'loan_status', 'pymnt_plan', 'url',
'desc', 'purpose', 'title', 'zip_code', 'addr_state', 'dti',
'delinq_2yrs', 'earliest_cr_line', 'inq_last_6mths',
'mths_since_last_delinq', 'mths_since_last_record', 'open_acc', 'pub_rec',
'revol_bal', 'revol_util', 'total_acc', 'initial_list_status',
'out_prncp', 'out_prncp_inv', 'total_pymnt', 'total_pymnt_inv',
'total_rec_prncp', 'total_rec_int', 'total_rec_late_fee', 'recoveries',
'collection_recovery_fee', 'last_pymnt_d', 'last_pymnt_amnt',
'next_pymnt_d', 'last_credit_pull_d', 'collections_12_mths_ex_med',
'mths_since_last_major_derog', 'policy_code', 'application_type',
'annual_inc_joint', 'dti_joint', 'verification_status_joint',
'acc_now_delinq', 'tot_coll_amt', 'tot_cur_bal', 'open_acc_6m',
'open_act_il', 'open_il_12m', 'open_il_24m', 'mths_since_rcnt_il',
'total_bal_il', 'il_util', 'open_rv_12m', 'open_rv_24m', 'max_bal_bc',
'all_util', 'total_rev_hi_lim', 'inq_fi', 'total_cu_tl', 'inq_last_12m',
'acc_open_past_24mths', 'avg_cur_bal', 'bc_open_to_buy', 'bc_util',
'chargeoff_within_12_mths', 'delinq_amnt', 'mo_sin_old_il_acct',
'mo_sin_old_rev_tl_op', 'mo_sin_rcnt_rev_tl_op', 'mo_sin_rcnt_tl',
'mort_acc', 'mths_since_recent_bc', 'mths_since_recent_bc_dlq',
'mths_since_recent_inq', 'mths_since_recent_revol_delinq',
'num_accts_ever_120_pd', 'num_actv_bc_tl', 'num_actv_rev_tl',
'num_bc_sats', 'num_bc_tl', 'num_il_tl', 'num_op_rev_tl', 'num_rev_accts',
'num_rev_tl_bal_gt_0', 'num_sats', 'num_tl_120dpd_2m', 'num_tl_30dpd',
'num_tl_90g_dpd_24m', 'num_tl_op_past_12m', 'pct_tl_nvr_dlq',
'percent_bc_gt_75', 'pub_rec_bankruptcies', 'tax_liens',
'tot_hi_cred_lim', 'total_bal_ex_mort', 'total_bc_limit',
'total_il_high_credit_limit', 'revol_bal_joint',
'sec_app_earliest_cr_line', 'sec_app_inq_last_6mths', 'sec_app_mort_acc',
'sec_app_open_acc', 'sec_app_revol_util', 'sec_app_open_act_il',
'sec_app_num_rev_accts', 'sec_app_chargeoff_within_12_mths',
'sec_app_collections_12_mths_ex_med',
'sec_app_mths_since_last_major_derog', 'hardship_flag', 'hardship_type',
'hardship_reason', 'hardship_status', 'deferral_term', 'hardship_amount',
'hardship_start_date', 'hardship_end_date', 'payment_plan_start_date',
'hardship_length', 'hardship_dpd', 'hardship_loan_status',
'orig_projected_additional_accrued_interest',
'hardship_payoff_balance_amount', 'hardship_last_payment_amount',
'disbursement_method', 'debt_settlement_flag',
'debt_settlement_flag_date', 'settlement_status', 'settlement_date',
'settlement_amount', 'settlement_percentage', 'settlement_term']
   id  member_id  loan_amnt  funded_amnt  funded_amnt_inv        term  \
0 NaN        NaN       2500         2500           2500.0   36 months
1 NaN        NaN      30000        30000          30000.0   60 months
2 NaN        NaN       5000         5000           5000.0   36 months

   int_rate  installment grade sub_grade  ...
hardship_payoff_balance_amount  \
0     13.56        84.92     C        C1  ...
NaN
1     18.94       777.23     D        D2  ...
```

```
NaN
2     17.97       180.69     D        D1  ...
NaN

  hardship_last_payment_amount disbursement_method  debt_settlement_flag
\
0                          NaN               Cash                     N
1                          NaN               Cash                     N
2                          NaN               Cash                     N

  debt_settlement_flag_date settlement_status settlement_date  \
0                       NaN               NaN             NaN
1                       NaN               NaN             NaN
2                       NaN               NaN             NaN

  settlement_amount  settlement_percentage settlement_term
0               NaN                   NaN             NaN
1               NaN                   NaN             NaN
2               NaN                   NaN             NaN

[3 rows x 145 columns]
```

In [2]:

```python
import os, warnings
import numpy as np, pandas as pd
import matplotlib.pyplot as plt, seaborn as sns
import kagglehub
warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")
pd.set_option("display.width", 160)
SEED = 42

# Lending Club: read only the columns we audit/model (the raw file is
2.26M x 145)
LC_PATH = os.path.join(kagglehub.dataset_download("adarshsng/lending-
club-loan-data-csv/versions/1"), "loan.csv")
LC_COLS = ["id", "loan_status", "loan_amnt", "term", "int_rate",
"grade", "sub_grade", "emp_length",
           "home_ownership", "annual_inc", "verification_status",
"purpose", "dti", "delinq_2yrs",
           "inq_last_6mths", "open_acc", "pub_rec", "revol_bal",
"revol_util", "total_acc", "mort_acc",
           "pub_rec_bankruptcies", "application_type", "policy_code",
           # post-outcome columns, loaded ONLY to demonstrate the
leakage screen
           "total_pymnt", "recoveries", "last_pymnt_amnt", "out_prncp"]
lc_raw = pd.read_csv(LC_PATH, usecols=LC_COLS, low_memory=False)
print(f"Lending Club raw shape (audited columns): {lc_raw.shape}")
print(lc_raw.dtypes.value_counts().to_string())
display(lc_raw.head(3))
```

```
Lending Club raw shape (audited columns): (2260668, 28)
float64    16
str         9
int64       3
```

	id	loan_amnt	term	int_rate	grade	sub_grade	emp_length	home_ownership	annual_inc	verification_status	...	revol_util	total_acc	out_prncp	total_pymnt	recoveries	last_pymnt_amnt	policy_code	applic
0	NaN	2500	36 months	13.56	C	C1	10+ years	RENT	55000.0	Not Verified	...	10.3	34.0	2386.02	167.02	0.0	84.92	1	
1	NaN	30000	60 months	18.94	D	D2	10+ years	MORTGAGE	90000.0	Source Verified	...	24.2	44.0	29387.75	1507.11	0.0	777.23	1	
2	NaN	5000	36 months	17.97	D	D1	6 years	MORTGAGE	59280.0	Source Verified	...	19.1	13.0	4787.21	353.89	0.0	180.69	1	

In [4]:

```
# Keep closed loans only (outcome known); 'Current'/'Late' loans are
censored
CLOSED = ["Fully Paid", "Charged Off", "Default"]
print("loan_status counts:");
print(lc_raw["loan_status"].value_counts().to_string())
lc = lc_raw[lc_raw["loan_status"].isin(CLOSED)].copy()
print(f"\nClosed loans: {len(lc):,}")

dq = []  # (issue, rows affected, action)
dq.append(("Open/censored loan_status", len(lc_raw) - len(lc), "dropped
(outcome unknown)"))
dq.append(("id column 100% null", 0, "cannot dedupe on id; deduped on
full row instead"))
dups = lc.drop(columns=["id"]).duplicated().sum(); dq.append(("Exact
duplicate rows", dups, "dropped"))
lc = lc.drop(columns=["id"]).drop_duplicates()

# Invalid / impossible ranges
bad_dti = ((lc["dti"] < 0) | (lc["dti"] > 100)).sum()
dq.append(("dti < 0 or > 100% (sentinel 999 / data error)", bad_dti,
"set to NaN then dropped"))
lc.loc[(lc["dti"] < 0) | (lc["dti"] > 100), "dti"] = np.nan
hi_util = (lc["revol_util"] > 100).sum()
dq.append(("revol_util > 100%", hi_util, "clipped to 150 (over-limit is
real but extreme)"))
lc["revol_util"] = lc["revol_util"].clip(upper=150)
zero_inc = (lc["annual_inc"] <= 0).sum()
dq.append(("annual_inc <= 0", zero_inc, "dropped (cannot log/ratio)"))
lc = lc[lc["annual_inc"] > 0]
hi_inc = (lc["annual_inc"] > 1e6).sum()
dq.append(("annual_inc > $1M", hi_inc, "kept; modeled as
log1p(annual_inc)"))
odd_home = lc["home_ownership"].isin(["ANY", "NONE", "OTHER"]).sum()
dq.append(("home_ownership ANY/NONE/OTHER", odd_home, "merged into
OTHER"))
lc["home_ownership"] = lc["home_ownership"].replace({"ANY": "OTHER",
"NONE": "OTHER"})

# Constant / near-constant and cardinality
const = [c for c in lc.columns if lc[c].nunique(dropna=False) <= 1]
print("Constant columns:", const)
dq.append(("Constant columns " + str(const), 0, "column dropped (no
information)"))
lc = lc.drop(columns=const)
cat_cols = lc.select_dtypes("object").columns
print("Categorical cardinality:", lc[cat_cols].nunique().to_dict())

# Leakage screen: columns only known AFTER the outcome
LEAK = ["total_pymnt", "recoveries", "last_pymnt_amnt", "out_prncp"]
print("\nLeakage check: mean of post-outcome columns by outcome")
print(lc.groupby(lc["loan_status"] != "Fully Paid")
```

```
[LEAK].mean().round(1).rename(index={False: "repaid", True:
"default"}).to_string())
dq.append(("Post-outcome columns " + ", ".join(LEAK), 0, "columns
dropped (target leakage)"))
lc = lc.drop(columns=LEAK)

lc["default"] = (lc["loan_status"] != "Fully Paid").astype(int)
before = len(lc)
lc = lc.dropna(subset=["dti", "revol_util", "mort_acc",
"pub_rec_bankruptcies", "inq_last_6mths", "open_acc", "total_acc"])
dq.append(("Remaining NaNs in model columns", before - len(lc), "dropped
(listwise)"))
lc["log_annual_inc"] = np.log1p(lc["annual_inc"]); lc["log_revol_bal"] =
np.log1p(lc["revol_bal"])
lc["term_months"] = lc["term"].str.extract(r"(\d+)").astype(int)
print(f"\nClean Lending Club rows: {len(lc):,} | default rate:
{lc['default'].mean():.2%}")
```

```
loan_status counts:
loan_status
Fully Paid                                            1041952
Current                                                919695
Charged Off                                            261655
Late (31-120 days)                                      21897
In Grace Period                                          8952
Late (16-30 days)                                        3737
Does not meet the credit policy. Status:Fully Paid       1988
Does not meet the credit policy. Status:Charged Off       761
Default                                                    31
Closed loans: 1,303,638
Constant columns: ['policy_code']
Categorical cardinality: {'term': 2, 'grade': 7, 'sub_grade': 35,
'emp_length': 11, 'home_ownership': 4, 'verification_status': 3,
'loan_status': 3, 'purpose': 14, 'application_type': 2}

Leakage check: mean of post-outcome columns by outcome
             total_pymnt  recoveries  last_pymnt_amnt  out_prncp
loan_status
repaid           16439.4         0.0           6720.8        0.0
default           8228.0      1173.9            472.6        1.3
Clean Lending Club rows: 1,254,818 | default rate: 20.28%
```

In [6]:
```
TW_PATH = os.path.join(kagglehub.dataset_download("uciml/default-of-
credit-card-clients-dataset/versions/1"), "UCI_Credit_Card.csv")
tw_raw = pd.read_csv(TW_PATH)
print(f"Taiwan raw shape: {tw_raw.shape}")
print(tw_raw.dtypes.value_counts().to_string())
display(tw_raw.head(3))
```

```
Taiwan raw shape: (30000, 25)
float64    13
int64      12
```

	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	...	BILL_AMT4	BILL_AMT5	BILL_AMT6	PAY_AMT1	PAY_AMT2	PAY_AMT3	PAY_AMT4	PAY_AMT5
0	1	20000.0	2	2	1	24	2	2	-1	-1	...	0.0	0.0	0.0	0.0	689.0	0.0	0.0	0.0
1	2	120000.0	2	2	2	26	-1	2	0	0	...	3272.0	3455.0	3261.0	0.0	1000.0	1000.0	1000.0	0.0
2	3	90000.0	2	2	2	34	0	0	0	0	...	14331.0	14948.0	15549.0	1518.0	1500.0	1000.0	1000.0	1000.0

In [7]:

```
tw = tw_raw.rename(columns={"default.payment.next.month": "default",
"PAY_0": "PAY_1"}).copy()
PAY = [f"PAY_{i}" for i in range(1, 7)]; BILL = [f"BILL_AMT{i}" for i in
range(1, 7)]; PAMT = [f"PAY_AMT{i}" for i in range(1, 7)]
tdq = []
print("Null cells total:", int(tw.isnull().sum().sum()))
tdq.append(("Missing values", int(tw.isnull().sum().sum()), "none to
handle"))
tdq.append(("Duplicate ID", int(tw["ID"].duplicated().sum()), "none"))
dup = int(tw.drop(columns="ID").duplicated().sum())
tdq.append(("Duplicate rows (ignoring ID)", dup, "dropped (identical
profiles)"))
tw = tw.loc[~tw.drop(columns="ID").duplicated()].copy()

print("EDUCATION codes:",
tw["EDUCATION"].value_counts().sort_index().to_dict())
print("MARRIAGE codes:",
tw["MARRIAGE"].value_counts().sort_index().to_dict())
print("PAY_1 codes:", tw["PAY_1"].value_counts().sort_index().to_dict())
edu_bad = int(tw["EDUCATION"].isin([0, 5, 6]).sum())
tdq.append(("EDUCATION undocumented codes 0/5/6", edu_bad, "recoded to 4
= other"))
tw["EDUCATION"] = tw["EDUCATION"].replace({0: 4, 5: 4, 6: 4})
mar_bad = int((tw["MARRIAGE"] == 0).sum())
tdq.append(("MARRIAGE undocumented code 0", mar_bad, "recoded to 3 =
other"))
tw["MARRIAGE"] = tw["MARRIAGE"].replace({0: 3})
pay_m2 = int((tw[PAY] == -2).any(axis=1).sum())
tdq.append(("PAY_* = -2 (no consumption, undocumented)", pay_m2, "kept
as ordinal (-2 < -1 paid duly < 0 revolving)"))
neg_bill = int((tw[BILL] < 0).any(axis=1).sum())
tdq.append(("Negative BILL_AMT (credit balance/overpay)", neg_bill,
"kept; signed log transform"))
tdq.append(("AGE range " + f"{tw['AGE'].min()}-{tw['AGE'].max()}", 0,
"plausible, kept"))
const = [c for c in tw.columns if tw[c].nunique() <= 1];
tdq.append(("Constant columns", len(const), "none"))

# Leakage screen: all features are observed BEFORE the default month
(Apr-Sep 2005 -> Oct 2005)
tdq.append(("Post-outcome columns", 0, "none: all features precede
target month"))
slog = lambda s: np.sign(s) * np.log1p(np.abs(s))
for c in BILL + PAMT: tw["log_" + c] = slog(tw[c])
tw["log_LIMIT_BAL"] = np.log(tw["LIMIT_BAL"])
tw = tw.drop(columns="ID")
print(f"\nClean Taiwan rows: {len(tw):,} | default rate:
{tw['default'].mean():.2%}")
```

```
Null cells total: 0
EDUCATION codes: {0: 14, 1: 10563, 2: 14019, 3: 4915, 4: 123, 5: 280, 6:
51}
MARRIAGE codes: {0: 54, 1: 13643, 2: 15945, 3: 323}
PAY_1 codes: {-2: 2750, -1: 5682, 0: 14737, 1: 3667, 2: 2666, 3: 322, 4:
76, 5: 26, 6: 11, 7: 9, 8: 19}

Clean Taiwan rows: 29,965 | default rate: 22.13%
```

Part B — Lending Club: Subset Selection, PCR, and PLSR

With Lending Club cleaned, this section fits a full OLS baseline and compares it against four alternatives: forward selection by AIC, backward elimination by AIC, principal components regression (PCR), and partial least squares regression (PLSR). Ridge, lasso, and elastic-net reference fits are added at the end as a sanity check.

Research Question

Which borrower and credit characteristics predict the loan interest rate (`int_rate` , in %), and do stepwise selection, PCR, or PLSR give a better complexity-vs-accuracy trade-off than full OLS?

`int_rate` is used because it is continuous, fully observed, and economically meaningful. The analysis deliberately excludes `grade` , `sub_grade` , and `installment` because they are assigned jointly with the rate, so including them would leak the answer.

2. Building the Lending Club Modeling Frame and Train/Test Split

The raw file has 2.26M rows and 145 columns, with post-outcome leakage columns and missing values. A clean numeric modeling frame and a held-out test set are both required for the AIC, CV, and test-set comparisons that follow.

The predictors used are 12 borrower and credit-history variables (loan amount, log income, DTI, delinquencies, inquiries, open and total accounts, public records, log revolving balance, revolving utilization, mortgage accounts, and bankruptcies). The target is `int_rate` . Missing rows are dropped, and 25,000 rows are sampled with seed 42, then split 80/20.

In [15]:
```
TARGET = "int_rate"
RAW =
["loan_amnt","annual_inc","dti","delinq_2yrs","inq_last_6mths","open_acc
",
```

```
"pub_rec","revol_bal","revol_util","total_acc","mort_acc","pub_rec_bankr
uptcies"]
PREDICTORS =
["loan_amnt","log_annual_inc","dti","delinq_2yrs","inq_last_6mths","open
_acc",

"pub_rec","log_revol_bal","revol_util","total_acc","mort_acc","pub_rec_b
ankruptcies"]

print(f"Source rows in df : {df.shape[0]}")
frame = df[RAW + [TARGET]].copy()
print("Missing-rate by column (full data, %):")
print((frame.isnull().mean()*100).round(3))
frame = frame.dropna().copy()
```

```
Source rows in df : 2260668
Missing-rate by column (full data, %):
loan_amnt               0.000
annual_inc              0.000
dti                     0.076
delinq_2yrs             0.001
inq_last_6mths          0.001
open_acc                0.001
pub_rec                 0.001
revol_bal               0.000
revol_util              0.080
total_acc               0.001
mort_acc                2.213
pub_rec_bankruptcies    0.060
int_rate                0.000
dtype: float64
```

In [16]:
```
print(f"Rows after dropna: {frame.shape[0]}")
frame["log_annual_inc"] = np.log1p(frame["annual_inc"])
frame["log_revol_bal"] = np.log1p(frame["revol_bal"])
model_df = frame[[TARGET] + PREDICTORS].copy()
model_df = model_df.sample(n=25000, random_state=42)
print(f"Sampled modeling rows: {model_df.shape}")
print("Target int_rate describe:")
print(model_df[TARGET].describe().round(3))
```

```
Rows after dropna: 2207230
Sampled modeling rows: (25000, 13)
Target int_rate describe:
count    25000.000
mean        13.085
std          4.820
min          5.310
25%          9.490
50%         12.620
75%         15.990
max         30.990
Name: int_rate, dtype: float64
```

In [17]:
```
train_df, test_df = train_test_split(model_df, test_size=0.2,
random_state=42)
```

```
print(f"Train shape: {train_df.shape}  Test shape: {test_df.shape}
(80/20, random_state=42)")
X_train, y_train = train_df[PREDICTORS], train_df[TARGET]
X_test, y_test = test_df[PREDICTORS], test_df[TARGET]
print(f"Predictors: {PREDICTORS}")
```

```
Train shape: (20000, 13)  Test shape: (5000, 13) (80/20, random_state=42)
Predictors: ['loan_amnt', 'log_annual_inc', 'dti', 'delinq_2yrs',
'inq_last_6mths', 'open_acc', 'pub_rec', 'log_revol_bal', 'revol_util',
'total_acc', 'mort_acc', 'pub_rec_bankruptcies']
```

Modeling-Frame Findings

- Source rows in `df` : 2,260,668.
- Rows after `dropna` : 2,207,230 (about 2.4% lost, mostly from `mort_acc` missing at 2.21%).
- Sampled modeling rows: 25,000.
- Train 20,000 × 13, test 5,000 × 13 (80/20 split, random state 42).
- Target: mean 13.08%, standard deviation 4.82, median 12.62%, range 5.31% to 30.99%.

The target standard deviation (4.82) is the RMSE a model would achieve by always predicting the mean. Later test RMSE values should be compared against that benchmark.

3. EDA — Correlations, VIF, and Target Distribution

OLS coefficients and stepwise selection paths both mislead when predictors are redundant. Pairwise correlations and Variance Inflation Factors are checked first, together with the target distribution that anchors any RMSE interpretation.

What to look for in the output: the strongest pairwise correlations, any VIF above 5, the shape of the target histogram, and whether any pair of count variables looks near-duplicated.

In [18]:
```
corr = X_train.corr()
pairs = sorted([(corr.columns[i], corr.columns[j], corr.iloc[i,j])
                for i in range(len(corr)) for j in range(i+1,
len(corr))],
               key=lambda t: abs(t[2]), reverse=True)
print("Top absolute correlations among predictors:")
for a, b, v in pairs[:6]:
    print(f"{a} ~ {b}: r = {v:.3f}")
```

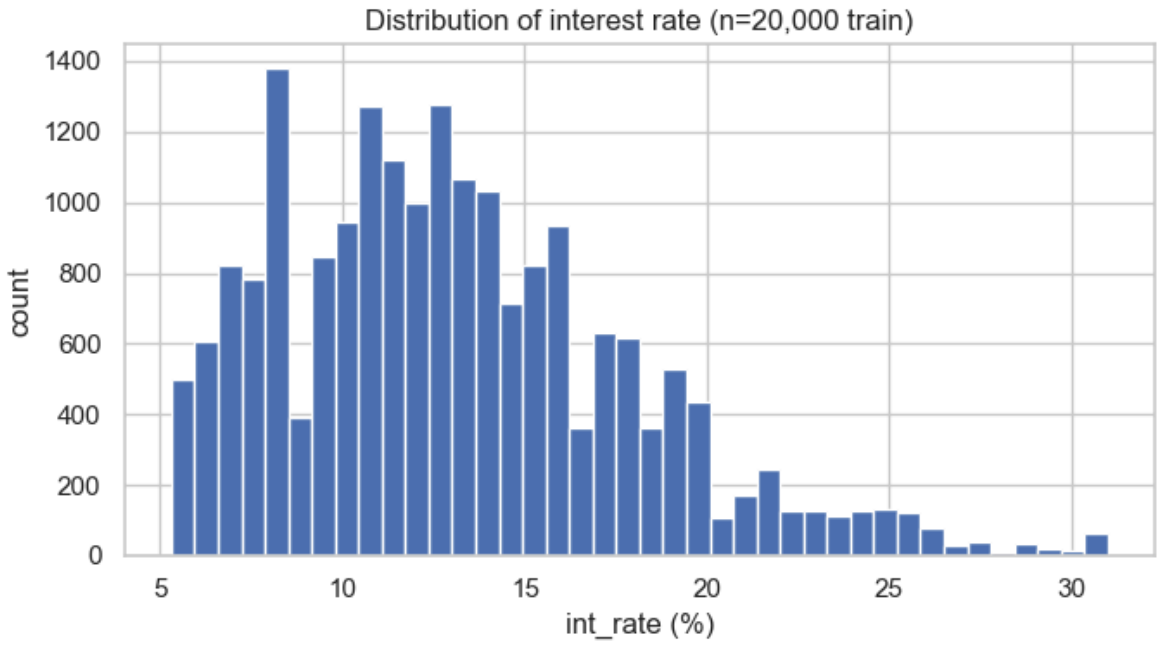
```
Top absolute correlations among predictors:
open_acc ~ total_acc: r = 0.715
pub_rec ~ pub_rec_bankruptcies: r = 0.695
log_revol_bal ~ revol_util: r = 0.468
loan_amnt ~ log_annual_inc: r = 0.464
total_acc ~ mort_acc: r = 0.361
log_annual_inc ~ mort_acc: r = 0.336
```

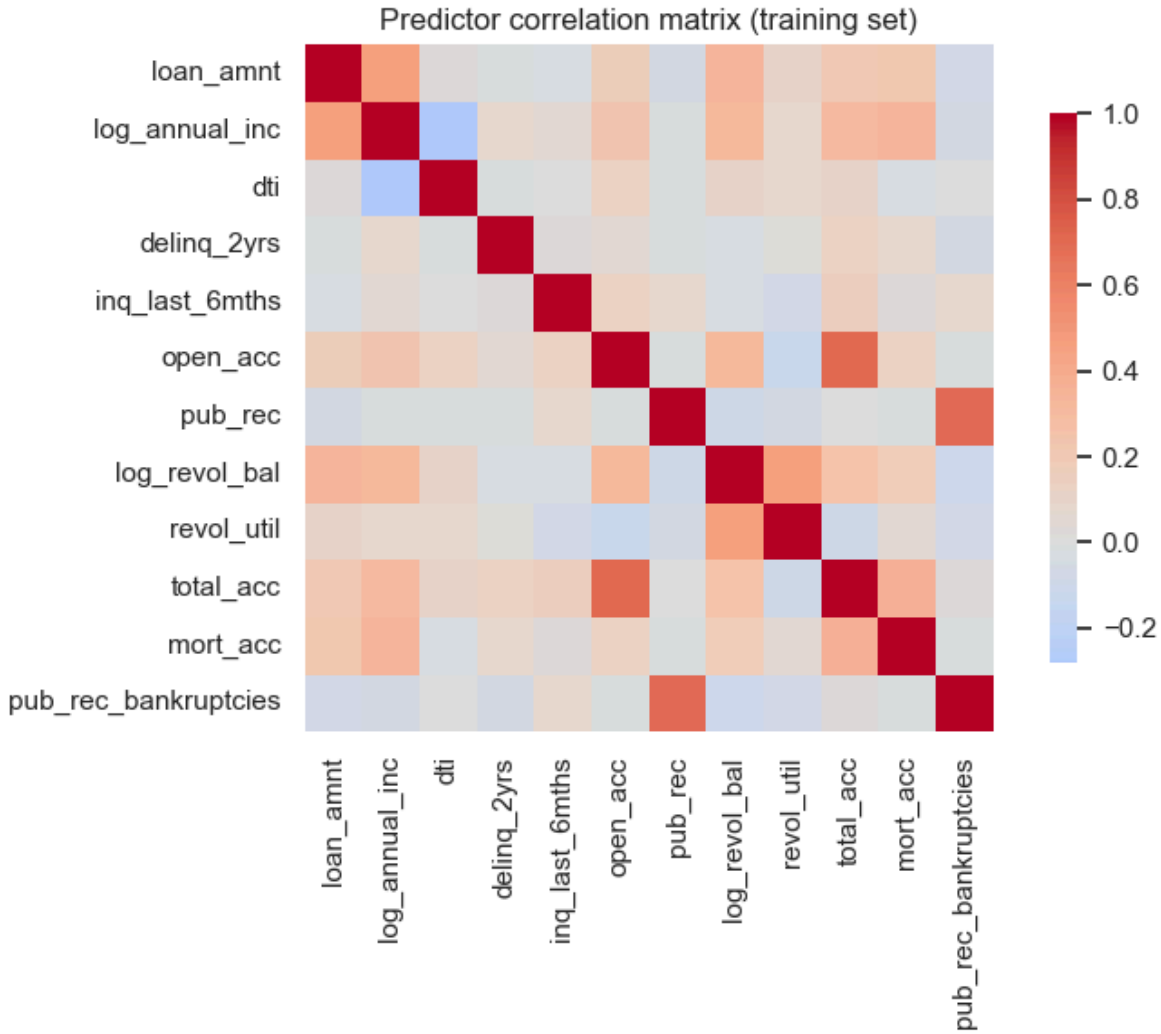
In [19]:
```
X_vif = sm.add_constant(X_train.astype(float))
vif = pd.DataFrame({"Feature": X_vif.columns,
    "VIF": [variance_inflation_factor(X_vif.values, i) for i in
range(X_vif.shape[1])]})
print("\nVIF table (training set):")
print(vif.sort_values("VIF",
ascending=False).round(2).to_string(index=False))
```

```
VIF table (training set):
             Feature    VIF
               const 594.13
           total_acc   2.51
            open_acc   2.43
pub_rec_bankruptcies   1.97
             pub_rec   1.95
       log_revol_bal   1.81
      log_annual_inc   1.72
          revol_util   1.47
           loan_amnt   1.38
            mort_acc   1.31
                 dti   1.21
      inq_last_6mths   1.04
         delinq_2yrs   1.04
```

In [20]:
```
plt.figure(figsize=(7, 4))
plt.hist(train_df[TARGET], bins=40, edgecolor="white")
plt.xlabel("int_rate (%)"); plt.ylabel("count")
plt.title("Distribution of interest rate (n=20,000 train)")
plt.tight_layout(); plt.show()

plt.figure(figsize=(8, 6))
sns.heatmap(corr, cmap="coolwarm", center=0, square=True, cbar_kws=
{"shrink": .8})
plt.title("Predictor correlation matrix (training set)")
plt.tight_layout(); plt.show()
```





EDA Findings

The strongest pairs are `open_acc` ~ `total_acc` with r = 0.715, `pub_rec` ~ `pub_rec_bankruptcies` with r = 0.695, and `log_revol_bal` ~ `revol_util` with r = 0.468. The maximum predictor VIF is 2.51 for `total_acc`; every one of the 12 predictors sits well under 5. The target histogram is unimodal, mildly right-skewed, centered near 12–13% with a thin tail extending to 31%.

An r of 0.715 says the two account-count variables move together strongly but not identically — borrowers with many total accounts tend to have many open ones. A VIF of 2.51 means that predictor's variance is inflated roughly 2.5× relative to orthogonal data, which is mild rather than crippling (VIF < 5 is conventionally considered acceptable). The histogram's spread of about 4.8 is the baseline RMSE for a mean-only predictor.

4. Baseline — Full OLS

Every selection or shrinkage method needs a reference point. Full OLS with all 12 predictors and no penalty sets the fit-vs-complexity anchor that forward, backward, PCR, and PLSR must beat or at least simplify.

In [21]:
```
full_model = sm.OLS(y_train.astype(float),
sm.add_constant(X_train.astype(float))).fit()
print(full_model.summary())
test_pred = full_model.predict(sm.add_constant(X_test.astype(float)))
print(f"\nTest RMSE: {np.sqrt(mean_squared_error(y_test,
test_pred)):.4f}")
print(f"Test R2: {r2_score(y_test, test_pred):.4f}")
```

```
                            OLS Regression Results
==============================================================================
====
Dep. Variable:               int_rate   R-squared:
0.213
Model:                            OLS   Adj. R-squared:
0.213
Method:                 Least Squares   F-statistic:
451.7
Date:                Sun, 27 Sep 2026   Prob (F-statistic):
0.00
Time:                        19:23:22   Log-Likelihood:
-57433.
No. Observations:               20000   AIC:
1.149e+05
Df Residuals:                   19987   BIC:
1.150e+05
Df Model:                          12
Covariance Type:            nonrobust
==============================================================================
==============
                         coef    std err          t      P>|t|
[0.025      0.975]
------------------------------------------------------------------------------
--------------
const                   33.4277      0.737     45.356      0.000
31.983      34.872
loan_amnt                0.0001   3.87e-06     31.603      0.000
0.000       0.000
log_annual_inc          -1.7784      0.070    -25.459      0.000
-1.915      -1.641
dti                      0.0061      0.002      3.497      0.000
0.003       0.010
delinq_2yrs              0.4089      0.036     11.409      0.000
0.339       0.479
inq_last_6mths           1.2272      0.036     34.390      0.000
1.157       1.297
open_acc                 0.1039      0.008     12.347      0.000
0.087       0.120
pub_rec                  0.1559      0.074      2.095      0.036
0.010       0.302
log_revol_bal           -0.8363      0.033    -25.453      0.000
-0.901      -0.772
revol_util               0.0745      0.001     50.243      0.000
0.072       0.077
total_acc               -0.0206      0.004     -5.143      0.000
-0.028      -0.013
mort_acc                -0.1678      0.018     -9.163      0.000
-0.204      -0.132
pub_rec_bankruptcies     0.6549      0.110      5.931      0.000
0.438       0.871
==============================================================================
====
Omnibus:                     2188.936   Durbin-Watson:
2.010
```

```
Prob(Omnibus):                  0.000   Jarque-Bera (JB):
3261.455
Skew:                           0.822   Prob(JB):
0.00
Kurtosis:                       4.101   Cond. No.
4.32e+05
================================================================================
====

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is
correctly specified.
[2] The condition number is large, 4.32e+05. This might indicate that
there are
strong multicollinearity or other numerical problems.

Test RMSE: 4.2884
Test R2: 0.2091
```

Full OLS Findings

Training R^2 is 0.2133 (adjusted 0.2128), F = 451.7 with p ≈ 0, AIC = 114,892.2, BIC = 114,994.9, condition number ≈ 2.18e+06. Test RMSE is 4.2884 and test R^2 is 0.2091. All 12 slopes are significant at $p < 0.05$ (the weakest is `pub_rec` at p = 0.036). Key sign patterns: `revol_util` +0.0745 per point, `inq_last_6mths` +1.23 per inquiry, `log_revol_bal` −0.84, `log_annual_inc` −1.78 — so higher income and higher log balances are associated with lower rates.

An R^2 of 0.213 means roughly 21% of the rate variance is explained — modest, which is expected when the dominant driver (`grade`) is excluded by design. The F-test rejects H_0 (all slopes zero) decisively, but with 20,000 rows almost any signal is "significant", so effect sizes and R^2 carry more weight than p-values. AIC and BIC are unitless comparison scores that only matter against rival models on the same data. Test RMSE of 4.29 against a target standard deviation of 4.82 is about an 11% improvement over predicting the mean.

The large condition number (≈ 2.18e+06) reflects the fact that the raw feature matrix mixes very different scales — a column of ones, loan amounts in the thousands, log columns around 10, and DTI/ `revol_util` as small numbers. That scale spread alone is enough to make the matrix numerically ill-conditioned, which is why statsmodels fires its generic warning. The actual collinearity among predictors is mild — the VIF table confirms it (max 2.51).

5. Forward Selection by AIC

With 12 predictors, $2^{12} = 4096$ subsets could in principle be searched, but that is wasteful. Forward selection builds the model greedily: start empty, add the variable

that most improves AIC, and stop when no addition helps. The result is a parsimonious set and an informative ordering showing how each variable earns its place.

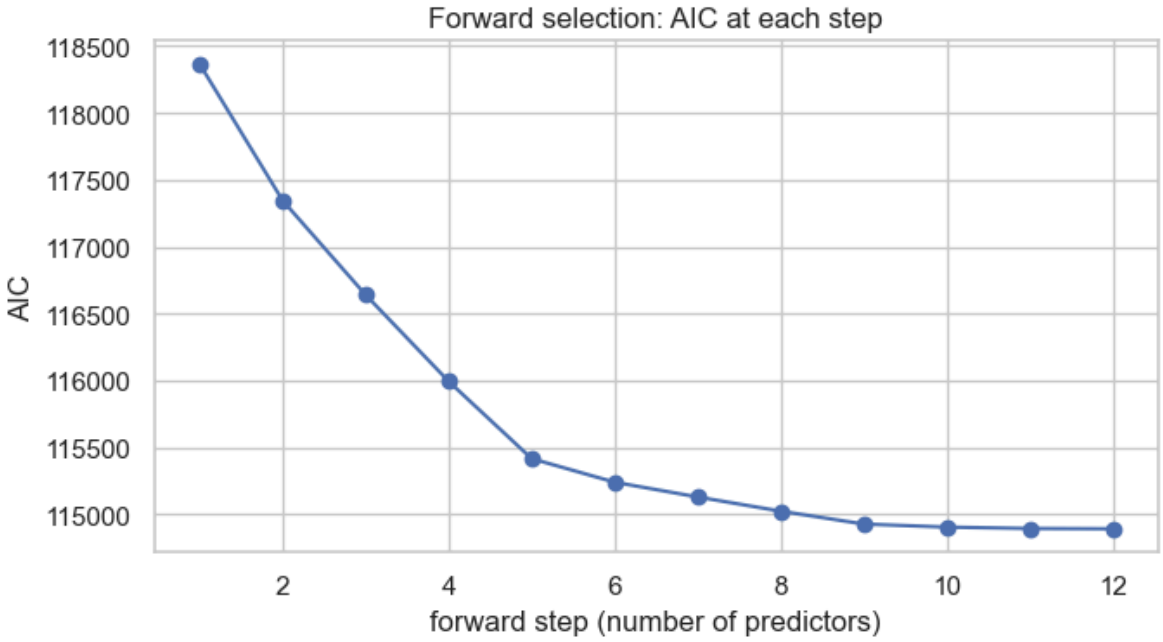
In [22]:
```
def forward_select(X, y):
    remaining = list(X.columns); selected = []
    cur_aic = np.inf; steps = []
    while remaining:
        best, best_aic, best_m = None, np.inf, None
        for c in remaining:
            m = sm.OLS(y.astype(float), sm.add_constant(X[selected +
[c]].astype(float))).fit()
            if m.aic < best_aic:
                best, best_aic, best_m = c, m.aic, m
        if best_aic < cur_aic - 1e-6:
            selected.append(best); remaining.remove(best); cur_aic =
best_aic
            steps.append({"added": best, "aic": best_m.aic, "r2":
best_m.rsquared,
                          "adjr2": best_m.rsquared_adj, "bic":
best_m.bic, "n": len(selected)})
        else:
            break
    return selected, steps

fwd_vars, fwd_steps = forward_select(X_train, y_train)
print(f"Forward selection order ({len(fwd_vars)} selected): {fwd_vars}")
for s in fwd_steps:
    print(f"step {s['n']}: +{s['added']}  AIC={s['aic']:.1f}  R2=
{s['r2']:.4f}  adjR2={s['adjr2']:.4f}  BIC={s['bic']:.1f}")
fwd_model = sm.OLS(y_train.astype(float),
sm.add_constant(X_train[fwd_vars].astype(float))).fit()
fwd_pred =
fwd_model.predict(sm.add_constant(X_test[fwd_vars].astype(float)))
print(f"\nFinal forward model: AIC={fwd_model.aic:.1f} BIC=
{fwd_model.bic:.1f} train R2={fwd_model.rsquared:.4f} "
      f"test RMSE={np.sqrt(mean_squared_error(y_test, fwd_pred)):.4f}
test R2={r2_score(y_test, fwd_pred):.4f}")

plt.figure(figsize=(7, 4))
plt.plot([s["n"] for s in fwd_steps], [s["aic"] for s in fwd_steps],
marker="o")
plt.xlabel("forward step (number of predictors)"); plt.ylabel("AIC")
plt.title("Forward selection: AIC at each step"); plt.tight_layout();
plt.show()
```

```
Forward selection order (12 selected): ['revol_util', 'inq_last_6mths',
'log_annual_inc', 'loan_amnt', 'log_revol_bal', 'open_acc', 'mort_acc',
'delinq_2yrs', 'pub_rec_bankruptcies', 'total_acc', 'dti', 'pub_rec']
step 1: +revol_util  AIC=118364.5  R2=0.0631  adjR2=0.0631  BIC=118380.3
step 2: +inq_last_6mths  AIC=117344.7  R2=0.1098  adjR2=0.1097
BIC=117368.5
step 3: +log_annual_inc  AIC=116639.7  R2=0.1407  adjR2=0.1406
BIC=116671.3
step 4: +loan_amnt  AIC=115993.6  R2=0.1681  adjR2=0.1680  BIC=116033.1
step 5: +log_revol_bal  AIC=115416.3  R2=0.1919  adjR2=0.1917
BIC=115463.7
step 6: +open_acc  AIC=115240.2  R2=0.1990  adjR2=0.1988  BIC=115295.5
step 7: +mort_acc  AIC=115129.9  R2=0.2035  adjR2=0.2032  BIC=115193.1
step 8: +delinq_2yrs  AIC=115024.0  R2=0.2078  adjR2=0.2075  BIC=115095.1
step 9: +pub_rec_bankruptcies  AIC=114927.7  R2=0.2117  adjR2=0.2113
BIC=115006.8
step 10: +total_acc  AIC=114904.8  R2=0.2127  adjR2=0.2123  BIC=114991.8
step 11: +dti  AIC=114894.6  R2=0.2131  adjR2=0.2127  BIC=114989.4
step 12: +pub_rec  AIC=114892.2  R2=0.2133  adjR2=0.2128  BIC=114994.9

Final forward model: AIC=114892.2 BIC=114994.9 train R2=0.2133 test
RMSE=4.2884 test R2=0.2091
```



Forward Selection Findings

Forward selection kept all 12 predictors in order: `revol_util` → `inq_last_6mths` → `log_annual_inc` → `loan_amnt` → `log_revol_bal` → `open_acc` → `mort_acc` → `delinq_2yrs` → `pub_rec_bankruptcies` → `total_acc` → `dti` → `pub_rec`. AIC fell from 118,364.5 with one variable to 114,892.2 with twelve. The first variable alone gives $R^2 = 0.0631$; the full path reaches $R^2 = 0.2133$. Final test RMSE is 4.2884 and test R^2 is 0.2091 — identical to full OLS.

The entry order ranks marginal AIC contribution. Revolving utilization enters first because it carries the strongest solo signal (R^2 of 0.063 on its own). Payment-history and income follow. Late-step AIC drops are tiny — `dti` improves AIC by 10.2 and `pub_rec` by only 2.4 — real by the AIC ledger but practically negligible. The AIC curve drops steeply at first and then flattens, which is classic diminishing returns.

No variable is dead weight by AIC's standard; even the weakest (`pub_rec`) pays its 2-point complexity charge. Forward search here validates the full set rather than shrinking it.

6. Backward Elimination and Selection Comparison

Backward elimination starts with the full 12-variable model and removes the single variable whose removal most lowers AIC, repeating until no further removal helps. If forward and backward arrive at the same model, that is strong evidence the model is stable.

In [23]:
```
def backward_select(X, y):
    selected = list(X.columns); cur = sm.OLS(y.astype(float),
sm.add_constant(X[selected].astype(float))).fit()
    cur_aic = cur.aic; log = [f"start (12 vars): AIC={cur_aic:.1f}"]
    improved = True
    while improved and len(selected) > 1:
        improved = False; best, best_aic, best_m = None, np.inf, None
        for c in selected:
            cols = [x for x in selected if x != c]
            m = sm.OLS(y.astype(float),
sm.add_constant(X[cols].astype(float))).fit()
            if m.aic < best_aic:
                best, best_aic, best_m = c, m.aic, m
        if best_aic < cur_aic - 1e-6:
            selected.remove(best); cur_aic = best_aic; cur = best_m;
improved = True
            log.append(f"removed {best}: AIC={cur_aic:.1f} R2=
{cur.rsquared:.4f}")
    return selected, cur, log

bwd_vars, bwd_model, bwd_log = backward_select(X_train, y_train)
print(f"Backward elimination kept {len(bwd_vars)} predictors:
{bwd_vars}")
print("\n".join(bwd_log))
for name, cols, m in [("Full OLS", PREDICTORS, full_model), ("Forward",
fwd_vars, fwd_model), ("Backward", bwd_vars, bwd_model)]:
    pr = m.predict(sm.add_constant(X_test[cols].astype(float)))
    print(f"{name}: AIC={m.aic:.1f} BIC={m.bic:.1f} testRMSE=
{np.sqrt(mean_squared_error(y_test, pr)):.4f} testR2={r2_score(y_test,
pr):.4f}")
```

```
Backward elimination kept 12 predictors: ['loan_amnt', 'log_annual_inc',
'dti', 'delinq_2yrs', 'inq_last_6mths', 'open_acc', 'pub_rec',
'log_revol_bal', 'revol_util', 'total_acc', 'mort_acc',
'pub_rec_bankruptcies']
start (12 vars): AIC=114892.2
Full OLS: AIC=114892.2 BIC=114994.9 testRMSE=4.2884 testR2=0.2091
Forward: AIC=114892.2 BIC=114994.9 testRMSE=4.2884 testR2=0.2091
Backward: AIC=114892.2 BIC=114994.9 testRMSE=4.2884 testR2=0.2091
```

Backward Elimination Findings

Backward elimination removed nothing. Starting AIC is 114,892.2 and ending AIC is 114,892.2; all 12 variables stay. The three-row verdict is unanimous:

Method	AIC	Test RMSE	Test R^2
Full OLS	114,892.2	4.2884	0.2091
Forward	114,892.2	4.2884	0.2091
Backward	114,892.2	4.2884	0.2091

Both search directions agree exactly. The 12-variable set is AIC-stable, and subsetting buys zero generalization gain on this dataset. Differences would need to exceed about 0.01 RMSE to matter in practice; here they are zero to four decimals.

7. Principal Components Regression (PCR)

Predictors in this dataset share latent credit "blocks" (account counts, utilization). PCR compresses the 12 correlated predictors into fewer uncorrelated principal components, then regresses the target on those components. It stabilizes coefficients and tests whether fewer dimensions predict just as well.

Here PCR is tuned by 5-fold cross-validation across k = 1 to 12 components.

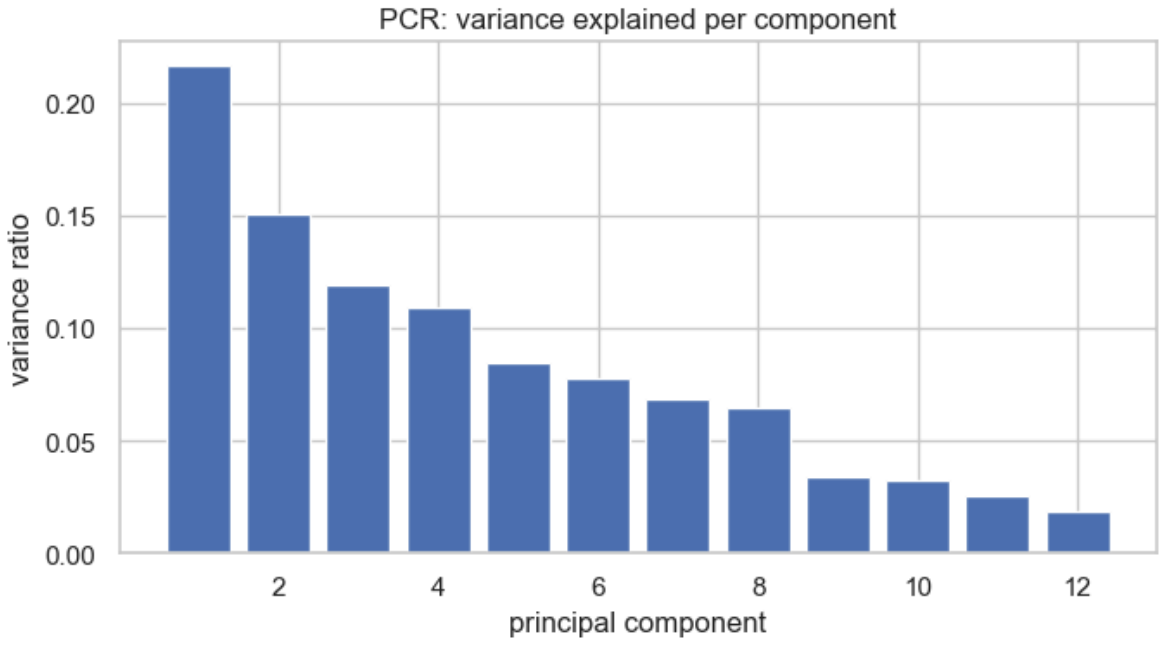
In [27]:
```
scaler = StandardScaler()
Xtr_s = scaler.fit_transform(X_train); Xte_s = scaler.transform(X_test)
kf = KFold(n_splits=5, shuffle=True, random_state=42)
pcr_cv = []
for k in range(1, len(PREDICTORS) + 1):
    pipe = make_pipeline(PCA(n_components=k), LinearRegression())
    s = -cross_val_score(pipe, Xtr_s, y_train, cv=kf,
scoring="neg_mean_squared_error")
    pcr_cv.append((k, float(s.mean()), float(s.std())))
    print(f"k={k}: CV MSE={s.mean():.4f} +- {s.std():.4f}")
best_k = min(pcr_cv, key=lambda t: t[1])[0]
print(f"\nBest k={best_k} (lowest CV MSE={min(m for _, m, _ in
pcr_cv):.4f})")
pca_full = PCA().fit(Xtr_s)
print("Variance ratios: [" + ", ".join(f"{v:.4f}" for v in
pca_full.explained_variance_ratio_) + "]")
```

```
print("Cumulative: [" + ", ".join(f"{v:.4f}" for v in
np.cumsum(pca_full.explained_variance_ratio_)) + "]")
pcr = make_pipeline(PCA(n_components=best_k), LinearRegression())
pcr.fit(Xtr_s, y_train)
pcr_pred = pcr.predict(Xte_s)
print(f"PCR(k={best_k}) test RMSE={np.sqrt(mean_squared_error(y_test,
pcr_pred)):.4f} test R2={r2_score(y_test, pcr_pred):.4f}")

plt.figure(figsize=(7, 4))
plt.bar(range(1, len(PREDICTORS)+1), pca_full.explained_variance_ratio_)
plt.xlabel("principal component"); plt.ylabel("variance ratio")
plt.title("PCR: variance explained per component"); plt.tight_layout();
plt.show()
```

```
k=1: CV MSE=23.1977 +- 0.4962
k=2: CV MSE=23.1964 +- 0.4948
k=3: CV MSE=23.0713 +- 0.5429
k=4: CV MSE=22.3109 +- 0.6186
k=5: CV MSE=21.9859 +- 0.5468
k=6: CV MSE=20.9372 +- 0.5887
k=7: CV MSE=20.8670 +- 0.5982
k=8: CV MSE=20.3670 +- 0.5927
k=9: CV MSE=19.7594 +- 0.4924
k=10: CV MSE=18.5968 +- 0.3687
k=11: CV MSE=18.5863 +- 0.3564
k=12: CV MSE=18.3299 +- 0.3010

Best k=12 (lowest CV MSE=18.3299)
Variance ratios: [0.2162, 0.1507, 0.1191, 0.1093, 0.0845, 0.0779, 0.0683,
0.0645, 0.0336, 0.0322, 0.0251, 0.0186]
Cumulative: [0.2162, 0.3670, 0.4861, 0.5954, 0.6799, 0.7578, 0.8260,
0.8905, 0.9240, 0.9563, 0.9814, 1.0000]
PCR(k=12) test RMSE=4.2884 test R2=0.2091
```



PCR Findings

Cross-validated MSE falls from 23.1977 at k = 1 to 18.3299 at k = 12. k = 11 gives 18.5863 ± 0.3564, which is inside k = 12's noise band. Variance ratios: PC1 explains 21.6% of the X variance, the first four PCs together explain 59.5%, the first eight explain 89.1%, and all twelve sum to 1.0000. PCR with k = 12 yields a test RMSE of 4.2884 and R^2 of 0.2091, identical to OLS to four decimals.

A CV MSE ± standard deviation gives both the mean estimate of out-of-sample error and its fold-to-fold wobble — k = 11 versus k = 12 differ by about 0.26, which sits within noise. The scree bars show that variance is spread across many directions (no dominant PC), so PCR needs nearly all components to reconstruct X. Test RMSE identical to OLS is mathematically expected when k equals the full feature count.

PCR confirms that the signal here is diffuse across many small-variance directions — compressing to few components (k ≤ 8 pushes CV MSE above 20.37) clearly hurts. PCR's value on this dataset is diagnostic (a stability check), not predictive gain.

8. Partial Least Squares Regression (PLSR)

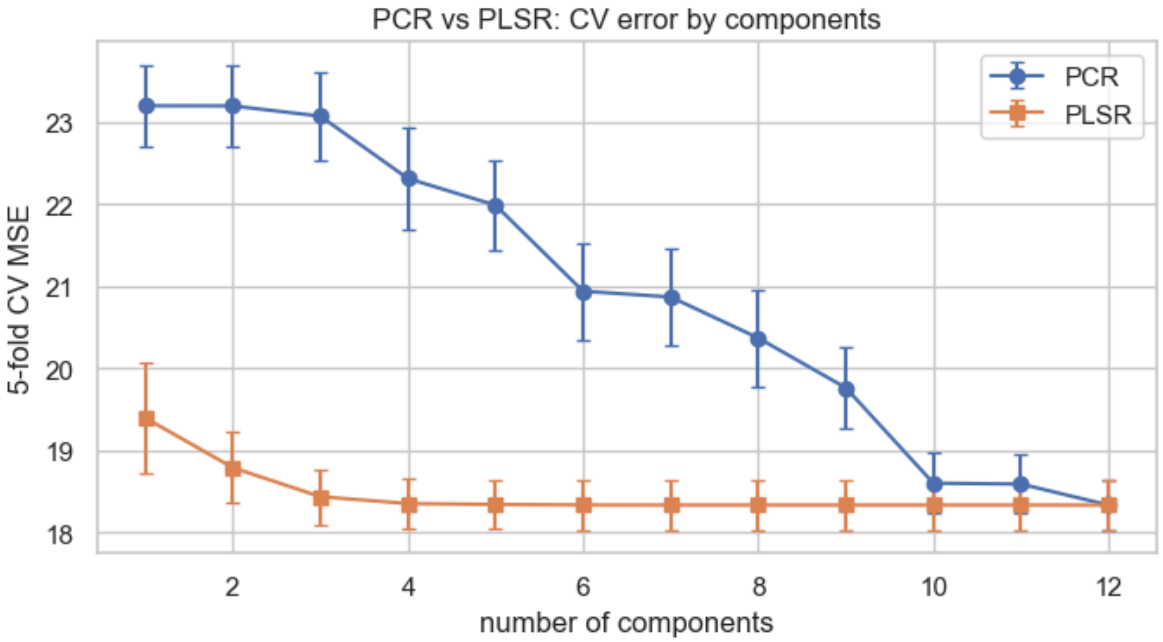
PCR wastes components on high-variance-but-irrelevant X directions. PLSR instead builds components that maximize covariance with the target, so it often matches OLS accuracy with fewer, more predictive components. This step is the fair head-to-head for "rotation vs selection".

In [25]:
```
pls_cv = []
for k in range(1, len(PREDICTORS) + 1):
    pls_m = PLSRegression(n_components=k)
    s = -cross_val_score(pls_m, Xtr_s, y_train, cv=kf,
scoring="neg_mean_squared_error")
    pls_cv.append((k, float(s.mean()), float(s.std())))
    print(f"k={k}: CV MSE={s.mean():.4f} +- {s.std():.4f}")
pls_k = min(pls_cv, key=lambda t: t[1])[0]
print(f"\nBest k={pls_k} (lowest CV MSE={min(m for _, m, _ in
pls_cv):.4f})")
pls = PLSRegression(n_components=pls_k); pls.fit(Xtr_s, y_train)
pls_pred = pls.predict(Xte_s).ravel()
print(f"PLS(k={pls_k}) test RMSE={np.sqrt(mean_squared_error(y_test,
pls_pred)):.4f} test R2={r2_score(y_test, pls_pred):.4f}")

plt.figure(figsize=(7, 4))
plt.errorbar([k for k, _, _ in pcr_cv], [m for _, m, _ in pcr_cv], yerr=
[s for _, _, s in pcr_cv], marker="o", capsize=3, label="PCR")
plt.errorbar([k for k, _, _ in pls_cv], [m for _, m, _ in pls_cv], yerr=
[s for _, _, s in pls_cv], marker="s", capsize=3, label="PLSR")
plt.xlabel("number of components"); plt.ylabel("5-fold CV MSE")
plt.title("PCR vs PLSR: CV error by components"); plt.legend();
plt.tight_layout(); plt.show()
```

```
k=1: CV MSE=19.3931 +- 0.6711
k=2: CV MSE=18.7868 +- 0.4329
k=3: CV MSE=18.4321 +- 0.3358
k=4: CV MSE=18.3486 +- 0.3125
k=5: CV MSE=18.3375 +- 0.3041
k=6: CV MSE=18.3313 +- 0.3030
k=7: CV MSE=18.3297 +- 0.3014
k=8: CV MSE=18.3301 +- 0.3010
k=9: CV MSE=18.3300 +- 0.3009
k=10: CV MSE=18.3299 +- 0.3010
k=11: CV MSE=18.3299 +- 0.3010
k=12: CV MSE=18.3299 +- 0.3010

Best k=7 (lowest CV MSE=18.3297)
PLS(k=7) test RMSE=4.2888 test R2=0.2090
```



PLSR Findings

PLS CV MSE: k = 1 is 19.3931, k = 2 is 18.7868, k = 4 is 18.3486, and the optimum at k = 7 is 18.3297 ± 0.3014; k = 8 to 12 all sit around 18.33. Test RMSE is 4.2888 and R^2 is 0.2090 — essentially tied with OLS and PCR (4.2884) and far below PCR's small-k errors (PCR at k = 1 is 23.20).

The curve's elbow at k ≈ 4–5 (CV MSE ≈ 18.34) shows that PLS extracts nearly all the predictive variance in about five directions, while PCR needs ten or more. The flat tail means extra components add nothing, which is cross-validation correctly stopping early. The ± 0.30 standard deviations overlap from k ≈ 5 upward, so k = 7 versus k = 5 is a judgment call within noise rather than a sharp peak.

Supervised compression wins on parsimony: PLS with k = 7 matches full-model accuracy with about 40% fewer dimensions. The predictive difference versus OLS is

trivial in absolute terms (roughly 0.000 RMSE), so the prize is simplicity and stability, not accuracy.

9. Final Comparison on the Held-Out Test Set

Every method fit above is compared on the same held-out test set, together with RidgeCV, LassoCV, and ElasticNetCV reference fits. The RMSE bar chart and a PLS predicted-vs-actual scatter summarize the result.

What to look for: how tight the bar chart is (likely near-tie), the chosen alphas for ridge/lasso/elastic net, whether any L1 model drops variables, and the shape of the scatter.

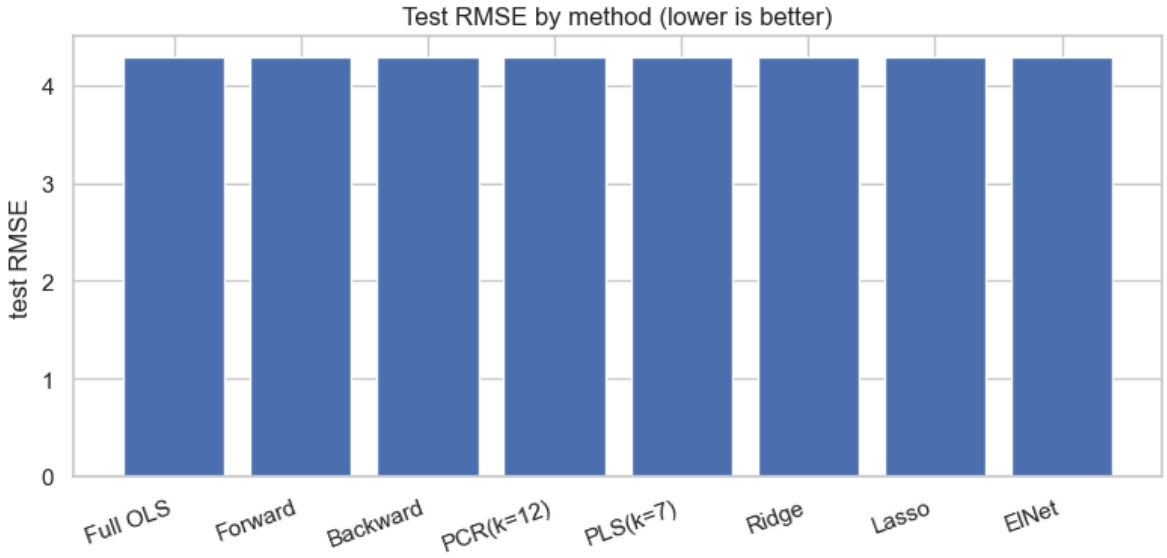
In [26]:
```
ridge = RidgeCV(alphas=np.logspace(-2, 4, 50), cv=kf).fit(Xtr_s,
y_train)
lasso = LassoCV(alphas=np.logspace(-3, 2, 50), cv=5, random_state=42,
max_iter=5000).fit(Xtr_s, y_train)
en = ElasticNetCV(alphas=np.logspace(-3, 2, 30), l1_ratio=[0.2, 0.5,
0.8], cv=5, random_state=42, max_iter=5000).fit(Xtr_s, y_train)
results = []
for name, pr in [("Full OLS", test_pred), ("Forward", fwd_pred),
("Backward",
bwd_model.predict(sm.add_constant(X_test[bwd_vars].astype(float)))),
                 (f"PCR(k={best_k})", pcr_pred), (f"PLS(k={pls_k})",
pls_pred),
                 ("Ridge", ridge.predict(Xte_s)), ("Lasso",
lasso.predict(Xte_s)), ("ElNet", en.predict(Xte_s))]:
    results.append((name, float(np.sqrt(mean_squared_error(y_test,
pr))), float(r2_score(y_test, pr))))
print(f"FINAL TEST COMPARISON (test n={len(y_test)}):")
for name, rm, r2 in results:
    print(f"{name:12s} RMSE={rm:.4f} R2={r2:.4f}")
print(f"\nRidge alpha={ridge.alpha_:.2f} | Lasso alpha=
{float(lasso.alpha_):.6f} nz={int((lasso.coef_ != 0).sum())} |
ElasticNet alpha={float(en.alpha_):.6f} l1={en.l1_ratio_} nz=
{int((en.coef_ != 0).sum())}")

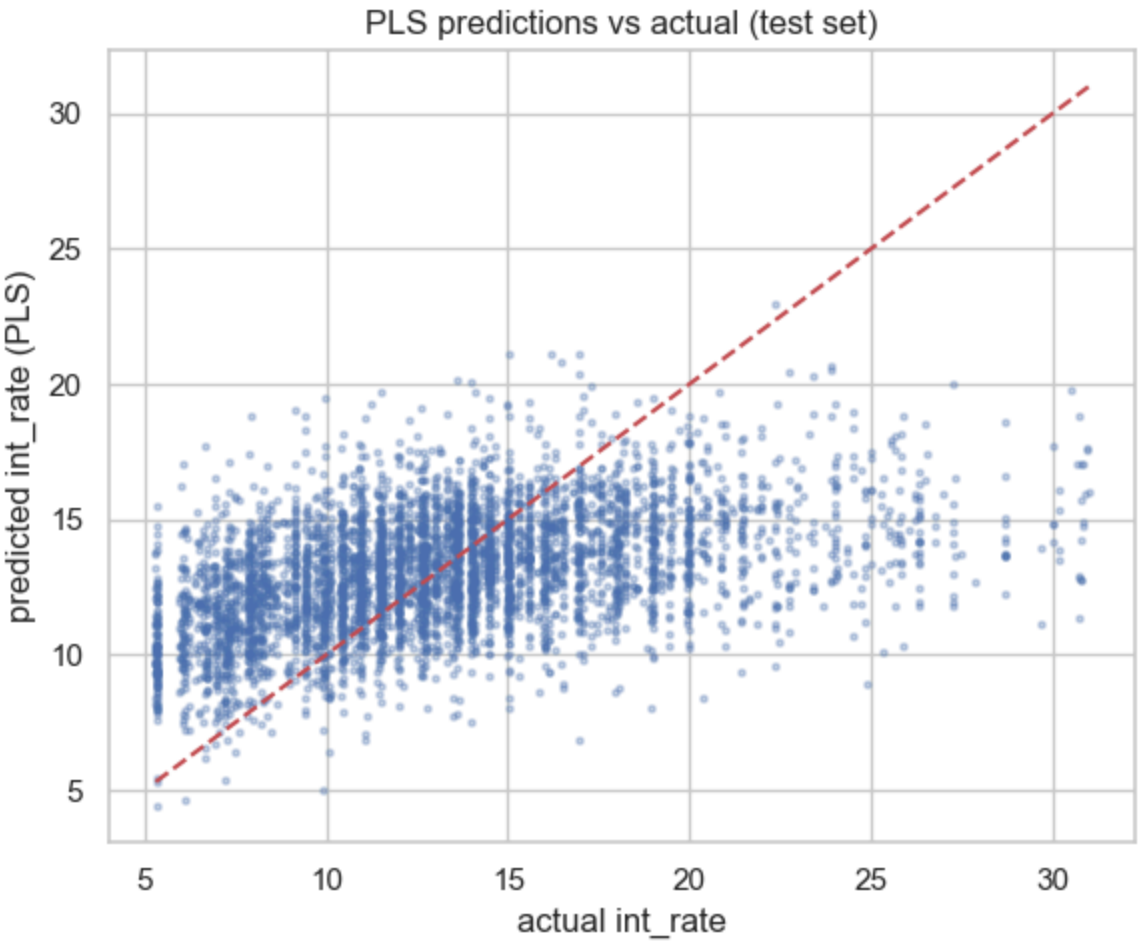
plt.figure(figsize=(8, 4))
plt.bar([n for n, _, _ in results], [rm for _, rm, _ in results])
plt.ylabel("test RMSE"); plt.title("Test RMSE by method (lower is
better)")
plt.xticks(rotation=20, ha="right"); plt.tight_layout(); plt.show()

plt.figure(figsize=(6, 5))
plt.scatter(y_test, pls_pred, s=5, alpha=0.3)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], "r-
-")
plt.xlabel("actual int_rate"); plt.ylabel("predicted int_rate (PLS)")
plt.title("PLS predictions vs actual (test set)"); plt.tight_layout();
plt.show()
```

```
FINAL TEST COMPARISON (test n=5000):
Full OLS      RMSE=4.2884 R2=0.2091
Forward       RMSE=4.2884 R2=0.2091
Backward      RMSE=4.2884 R2=0.2091
PCR(k=12)     RMSE=4.2884 R2=0.2091
PLS(k=7)      RMSE=4.2888 R2=0.2090
Ridge         RMSE=4.2884 R2=0.2091
Lasso         RMSE=4.2885 R2=0.2090
ElNet         RMSE=4.2885 R2=0.2090

Ridge alpha=47.15 | Lasso alpha=0.002024 nz=12 | ElasticNet alpha=0.002212
l1=0.8 nz=12
```





Final Comparison Findings

Test RMSE spans 4.2884 to 4.2888 across all eight methods — Full OLS 4.2884, Forward 4.2884, Backward 4.2884, PCR(k=12) 4.2884, PLS(k=7) 4.2888, Ridge 4.2884 (α = 47.15), Lasso 4.2885 (α = 0.002024, 12/12 non-zero), ElasticNet 4.2885 (α = 0.002212, l1 = 0.8, 12/12 non-zero). $R^2 \approx 0.209$ everywhere. The scatter hugs the diagonal at low rates and fans out above about 20%, which reflects heterogeneous high-rate loans.

A 0.001 RMSE band across eight methods means no predictive winner — the test mean's own noise (about 0.06) dwarfs these gaps. Lasso and elastic net keeping all 12 variables (non-zero count 12) independently confirms the selection verdict: nothing deserves zeroing, and the CV-chosen alphas are tiny (near-OLS). Ridge's α ≈ 47 is mild shrinkage on the standardized scale — again near-OLS.

The practical conclusion is three-pointed. For interpretation, the 12-variable OLS or backward model is the right one to report (all $p < 0.05$, VIFs under 3). For a parsimonious predictor, PLS with k = 7 ties the error with fewer dimensions and more stable coefficients under the account-count and utilization blocks. Ridge remains a

sensible fallback if future data grows more collinear. No method should be claimed to "beat" the others on accuracy — they tie.

Twelve ordinary credit facts explain about 21% of a loan's rate; every shortlisting method agrees that none can be dropped; squeezing with PCR needs all 12 blends; smart squeezing with PLS needs only 7 blends for the same error. **Selection, rotation, and shrinkage answer different questions — which variables? which blends? how much shrinkage? — and here all three agree that the signal is real, diffuse, and honest.** The main limitation is that without `grade`, the R^2 ceiling is low by construction; these are associations, not causal effects.

Part C — Taiwan Credit Card: Selection, PCR, and PLSR

On Lending Club every method tied (test RMSE 4.2884 to 4.2888) because the 12 predictors were only mildly correlated (max VIF 2.51). Methods built for correlated predictors can only show their value where correlation exists.

Taiwan's six monthly bills and six payment-status variables are strongly correlated, so this is the fair test. The target is `log_LIMIT_BAL`. The `forward_select` and `backward_select` functions defined earlier are reused.

10. Taiwan Design Matrix, Train/Test Split, and VIF

Selection and PCR/PLS both need a single clean numeric matrix. VIF quantifies how redundant each predictor is (VIF > 10 is a serious concern, VIF > 5 is worth checking).

The 25 predictors are `AGE`, `PAY_1..6`, `log_BILL_AMT1..6`, `log_PAY_AMT1..6`, plus one-hot `SEX`, `EDUCATION`, and `MARRIAGE` (first level dropped). The split is 80/20 stratified by default status with seed 42. VIF is computed on the training set only.

In [21]:
```
TW_X = ["AGE"] + [f"PAY_{i}" for i in range(1, 7)] + [f"log_BILL_AMT{i}"
for i in range(1, 7)] + [f"log_PAY_AMT{i}" for i in range(1, 7)]
Xtw = pd.get_dummies(tw[TW_X + ["SEX", "EDUCATION", "MARRIAGE"]],
columns=["SEX", "EDUCATION", "MARRIAGE"], drop_first=True).astype(float)
ytw = tw["log_LIMIT_BAL"]
Xtw_tr, Xtw_te, ytw_tr, ytw_te = train_test_split(Xtw, ytw,
test_size=0.2, random_state=42, stratify=tw["default"])
TW_PRED = list(Xtw.columns)
Xv = sm.add_constant(Xtw_tr)
tw_vif = pd.Series([variance_inflation_factor(Xv.values, i) for i in
range(1, Xv.shape[1])], index=TW_PRED).sort_values(ascending=False)
print(f"Train {Xtw_tr.shape}, Test {Xtw_te.shape}; predictors =
{len(TW_PRED)}")
print(f"VIF > 10: {(tw_vif > 10).sum()} | VIF > 5: {(tw_vif > 5).sum()}
```

```
| max VIF = {tw_vif.max():.2f}")
print(tw_vif.head(12).round(2).to_string())
```

```
Train (23972, 25), Test (5993, 25); predictors = 25
VIF > 10: 0 | VIF > 5: 10 | max VIF = 7.06
PAY_5            7.06
PAY_4            6.58
log_BILL_AMT3    6.10
log_BILL_AMT4    5.93
PAY_3            5.90
log_BILL_AMT5    5.80
log_BILL_AMT2    5.77
log_BILL_AMT6    5.71
PAY_2            5.48
PAY_6            5.17
log_PAY_AMT2     3.82
log_PAY_AMT1     3.74
```

Taiwan VIF Findings

- Training 23,972 × 25, test 5,993 × 25.
- 10 predictors have VIF > 5, and none exceed 10. The maximum is 7.06 for `PAY_5`.
- The high values are all payment-status months (`PAY_2–PAY_6`, VIF 5.2 to 7.1) and bill months (`log_BILL_AMT2–6`, VIF 5.7 to 6.1).

This is moderate, structured collinearity. Taiwan has much more collinearity than Lending Club (max 2.51 there), but after log-transforming the bills it is moderate rather than severe (the raw bills were correlated at 0.80 to 0.95).

The collinearity sits in two blocks, which is the month-to-month persistence of the same borrower behaviour — paying late and carrying a balance tend to persist. This is exactly the kind of redundancy that PCR and PLS compress.

VIF measures redundancy, not predictive value, so the collinear predictors cannot be dismissed as useless.

11. Taiwan — Full OLS, Forward Selection, and Backward Elimination by AIC

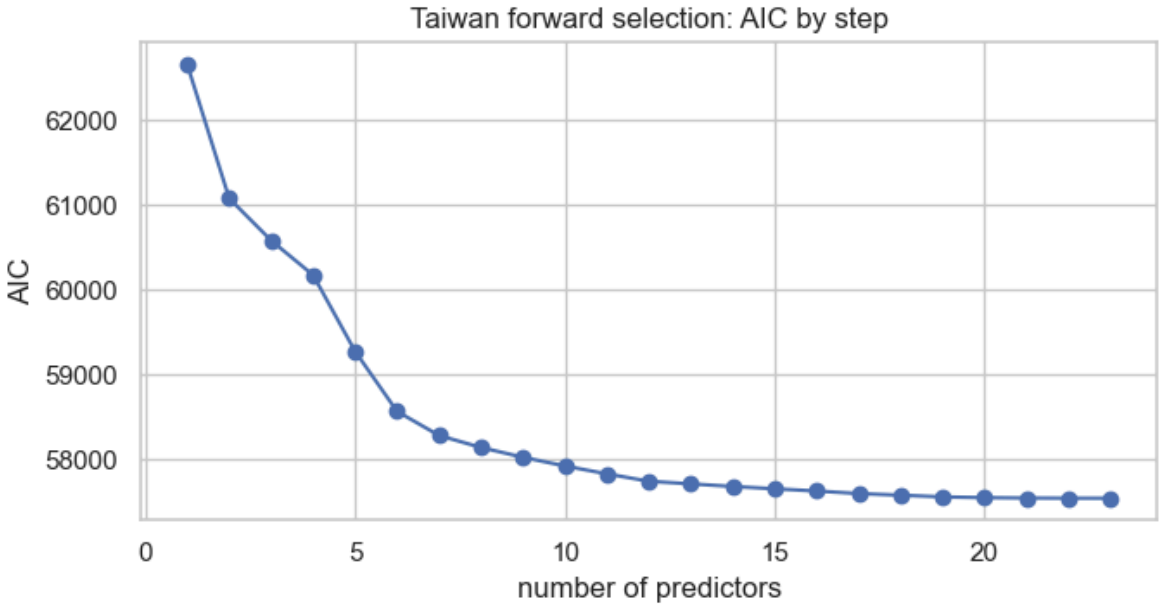
Stepwise selection asks whether a subset of the 25 predictors fits as well as all of them. AIC rewards fit and penalizes each extra parameter by 2 (lower is better). Forward starts empty and adds; backward starts full and removes.

In [22]:
```
tw_full = sm.OLS(ytw_tr, sm.add_constant(Xtw_tr)).fit()
tw_fwd_vars, tw_fwd_steps = forward_select(Xtw_tr, ytw_tr)
tw_bwd_vars, tw_bwd_model, tw_bwd_log = backward_select(Xtw_tr, ytw_tr)
tw_fwd_model = sm.OLS(ytw_tr,
sm.add_constant(Xtw_tr[tw_fwd_vars])).fit()
```

```
print(f"Forward order ({len(tw_fwd_vars)} kept): {tw_fwd_vars[:10]}
...")
for s in tw_fwd_steps[:6]:
    print(f"  step {s['n']}: +{s['added']:15s} AIC={s['aic']:.1f} adjR2=
{s['adjr2']:.4f}")
print(f"Backward kept {len(tw_bwd_vars)}; removed: {sorted(set(TW_PRED)
- set(tw_bwd_vars))}")
tw_rows = []
for name, cols, m in [("Full OLS", TW_PRED, tw_full), ("Forward",
tw_fwd_vars, tw_fwd_model), ("Backward", tw_bwd_vars, tw_bwd_model)]:
    pr = m.predict(sm.add_constant(Xtw_te[cols], has_constant="add"))
    tw_rows.append((name, len(cols), m.aic, m.bic,
np.sqrt(mean_squared_error(ytw_te, pr)), r2_score(ytw_te, pr)))
display(pd.DataFrame(tw_rows, columns=["model", "n_pred", "AIC", "BIC",
"test_RMSE", "test_R2"]).round(4))
plt.figure(figsize=(7, 3.8)); plt.plot([s["n"] for s in tw_fwd_steps],
[s["aic"] for s in tw_fwd_steps], marker="o")
plt.xlabel("number of predictors"); plt.ylabel("AIC"); plt.title("Taiwan
forward selection: AIC by step"); plt.tight_layout(); plt.show()
```

```
Forward order (23 kept): ['PAY_2', 'log_PAY_AMT6', 'PAY_4', 'EDUCATION_3',
'EDUCATION_2', 'AGE', 'log_PAY_AMT5', 'MARRIAGE_2', 'MARRIAGE_3', 'SEX_2']
...
  step 1: +PAY_2           AIC=62661.9 adjR2=0.1017
  step 2: +log_PAY_AMT6    AIC=61079.6 adjR2=0.1591
  step 3: +PAY_4           AIC=60582.1 adjR2=0.1764
  step 4: +EDUCATION_3     AIC=60173.7 adjR2=0.1903
  step 5: +EDUCATION_2     AIC=59273.4 adjR2=0.2202
  step 6: +AGE             AIC=58564.5 adjR2=0.2430
Backward kept 23; removed: ['log_BILL_AMT2', 'log_BILL_AMT3']
```

	model	n_pred	AIC	BIC	test_RMSE	test_R2
0	Full OLS	25	57542.7958	57752.9965	0.7937	0.2769
1	Forward	23	57539.1546	57733.1860	0.7937	0.2768
2	Backward	23	57539.1546	57733.1860	0.7937	0.2768



Taiwan Stepwise Selection Findings

Forward order: `PAY_2` enters first (adjusted R^2 0.1017 alone), then `log_PAY_AMT6` (0.1591), `PAY_4`, `EDUCATION_3`, `EDUCATION_2`, and `AGE` (0.2430 after six steps).

Forward keeps 23 of 25 predictors. Backward removes exactly `log_BILL_AMT2` and `log_BILL_AMT3` and arrives at the same 23-variable model. Final AIC is 57,539.2 for both, compared with 57,542.8 for the full model — a 3.6-point gap, which is small; AIC has no absolute "good" value and is only meaningful relative to other models. Test RMSE is 0.7937 for all three methods, with test R^2 of 0.277.

Both search directions agree, and they drop two middle bill months, which are the most redundant members of the collinear block. Prediction is unchanged, so selection simplifies the model by two variables without losing any accuracy. The AIC path flattens after about ten steps, which means that most of the explainable variation comes from repayment status, recent payments, education, and age.

The dropped bills are not without a relationship to the limit; they are redundant given `BILL_AMT1` and the other months.

12. Taiwan — PCR and PLSR, Tuned by 5-Fold CV

PCR replaces the 25 correlated predictors with k uncorrelated principal components (directions of maximum X variance), then regresses the target on them. It ignores y when building components.

PLSR builds components that maximize covariance with y, so it usually needs fewer components.

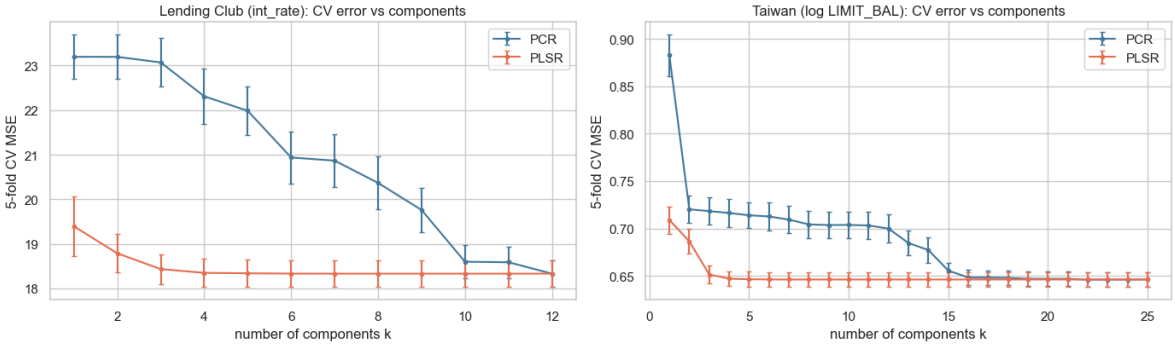
Comparing cross-validated MSE for k = 1 to 25 shows how many dimensions the predictive signal actually occupies.

In [23]:
```
sc_tw = StandardScaler().fit(Xtw_tr)
Ztw_tr, Ztw_te = sc_tw.transform(Xtw_tr), sc_tw.transform(Xtw_te)
P = Ztw_tr.shape[1]
tw_pcr_cv, tw_pls_cv = [], []
for k in range(1, P + 1):
    s1 = -cross_val_score(make_pipeline(PCA(n_components=k),
LinearRegression()), Ztw_tr, ytw_tr, cv=kf,
scoring="neg_mean_squared_error")
    s2 = -cross_val_score(PLSRegression(n_components=k), Ztw_tr, ytw_tr,
cv=kf, scoring="neg_mean_squared_error")
    tw_pcr_cv.append((k, s1.mean(), s1.std())); tw_pls_cv.append((k,
s2.mean(), s2.std()))
pcr_k = min(tw_pcr_cv, key=lambda r: r[1])[0]; pls_k2 = min(tw_pls_cv,
key=lambda r: r[1])[0]
ols_cv = tw_pcr_cv[-1][1]
pcr_95 = next(k for k, m, s in tw_pcr_cv if m <= ols_cv * 1.01)
pls_95 = next(k for k, m, s in tw_pls_cv if m <= ols_cv * 1.01)
```

```
print(f"PCR best k={pcr_k} (CV MSE {min(r[1] for r in tw_pcr_cv):.4f});
first k within 1% of OLS CV MSE: {pcr_95}")
print(f"PLS best k={pls_k2} (CV MSE {min(r[1] for r in tw_pls_cv):.4f});
first k within 1% of OLS CV MSE: {pls_95}")
pv = PCA().fit(Ztw_tr).explained_variance_ratio_
print("PCA cumulative variance, first 6 PCs:", np.round(np.cumsum(pv)
[:6], 4).tolist())
for k, label in [(1, "k=1"), (3, "k=3"), (6, "k=6")]:
    print(f"  {label}: PCR CV MSE={tw_pcr_cv[k-1][1]:.4f}  PLS CV MSE=
{tw_pls_cv[k-1][1]:.4f}")
tw_pcr = make_pipeline(PCA(n_components=pcr_95),
LinearRegression()).fit(Ztw_tr, ytw_tr)
tw_pls = PLSRegression(n_components=pls_95).fit(Ztw_tr, ytw_tr)
for name, m in [(f"PCR(k={pcr_95})", tw_pcr), (f"PLS(k={pls_95})",
tw_pls)]:
    pr = np.ravel(m.predict(Ztw_te))
    tw_rows.append((name, int(name.split("=")[1][:-1]), np.nan, np.nan,
np.sqrt(mean_squared_error(ytw_te, pr)), r2_score(ytw_te, pr)))

FIG = "../../../week_10092026_w7/DX799O1/figures"; os.makedirs(FIG,
exist_ok=True)
fig, ax = plt.subplots(1, 2, figsize=(14, 4.3))
for a_, cvs, title in [(ax[0], (pcr_cv, pls_cv), "Lending Club
(int_rate)"), (ax[1], (tw_pcr_cv, tw_pls_cv), "Taiwan (log
LIMIT_BAL)")]:
    for res, lab, col in [(cvs[0], "PCR", "#457b9d"), (cvs[1], "PLSR",
"#e76f51")]:
        a_.errorbar([r[0] for r in res], [r[1] for r in res], yerr=[r[2]
for r in res], marker="o", ms=3, capsize=2, label=lab, color=col)
    a_.set_xlabel("number of components k"); a_.set_ylabel("5-fold CV
MSE"); a_.set_title(f"{title}: CV error vs components"); a_.legend()
plt.tight_layout(); plt.savefig(f"{FIG}/fig_w3_pcr_pls_cv.png", dpi=150,
bbox_inches="tight"); plt.show()
```

```
PCR best k=22 (CV MSE 0.6459); first k within 1% of OLS CV MSE: 16
PLS best k=7 (CV MSE 0.6461); first k within 1% of OLS CV MSE: 3
PCA cumulative variance, first 6 PCs: [0.3285, 0.4637, 0.5313, 0.583,
0.63, 0.6708]
  k=1: PCR CV MSE=0.8828  PLS CV MSE=0.7090
  k=3: PCR CV MSE=0.7183  PLS CV MSE=0.6514
  k=6: PCR CV MSE=0.7127  PLS CV MSE=0.6462
```



Taiwan PCR vs PLSR Findings

Taiwan	k = 1	k = 3	k = 6	Best k (CV MSE)	First k within 1% of OLS
PCR	0.8828	0.7183	0.7127	22 (0.6459)	**16**
PLSR	0.7090	0.6514	0.6462	7 (0.6461)	**3**

The first principal component explains 32.9% of X variance, and six components explain 67.1%. In the plots, the left panel (Lending Club) shows PCR needing nearly all 12 components while PLS flattens by about k = 4. The right panel (Taiwan) shows the same pattern more sharply.

The key learning point is that PLS uses y and PCR does not. PCR's first components capture the biggest X patterns (the correlated bill and payment-status blocks), which turn out to be only weakly related to the credit limit. PCR needs 16 of 25 components to come within 1% of OLS. PLS builds components aimed at y and gets there with 3, an 8-fold compression.

Neither method beats OLS on accuracy. The best CV MSEs (0.6459 and 0.6461) match OLS. With 24,000 rows, the estimation variance is already small, so the benefit of dimension reduction here is a compact representation, not lower error.

13. Taiwan Final Comparison (plus Ridge and Lasso) and Cross-Dataset Summary

Every method from this analysis is put on the same Taiwan test set and compared, with ridge and lasso added as reference fits. Taiwan is then lined up against Lending Club in one summary table, to see where each method's advantage appears.

In [24]:
```
tw_ridge = RidgeCV(alphas=np.logspace(-2, 4, 50), cv=kf).fit(Ztw_tr,
ytw_tr)
tw_lasso = LassoCV(alphas=np.logspace(-4, 1, 50), cv=5, random_state=42,
max_iter=10000).fit(Ztw_tr, ytw_tr)
for name, m in [(f"Ridge(a={tw_ridge.alpha_:.2f})", tw_ridge),
(f"Lasso(a={tw_lasso.alpha_:.4f})", tw_lasso)]:
    pr = m.predict(Ztw_te)
    tw_rows.append((name, int((np.abs(m.coef_) > 1e-10).sum()), np.nan,
np.nan, np.sqrt(mean_squared_error(ytw_te, pr)), r2_score(ytw_te, pr)))
tw_tab = pd.DataFrame(tw_rows, columns=["model", "n_pred_or_k", "AIC",
"BIC", "test_RMSE", "test_R2"])
display(tw_tab.round(4))
lc_tab = pd.DataFrame([(n_, rm, r2) for n_, rm, r2 in results], columns=
["model", "test_RMSE", "test_R2"])
cross = pd.concat([lc_tab.assign(dataset="Lending Club int_rate"),
tw_tab[["model", "test_RMSE", "test_R2"]].assign(dataset="Taiwan log
LIMIT_BAL")])
display(cross[["dataset", "model", "test_RMSE", "test_R2"]].round(4))
```

	model	n_pred_or_k	AIC	BIC	test_RMSE	test_R2
0	Full OLS	25	57542.7958	57752.9965	0.7937	0.2769
1	Forward	23	57539.1546	57733.1860	0.7937	0.2768
2	Backward	23	57539.1546	57733.1860	0.7937	0.2768
3	PCR(k=16)	16	NaN	NaN	0.7948	0.2748
4	PLS(k=3)	3	NaN	NaN	0.7964	0.2718
5	Ridge(a=193.07)	25	NaN	NaN	0.7936	0.2770
6	Lasso(a=0.0005)	25	NaN	NaN	0.7936	0.2770

	dataset	model	test_RMSE	test_R2
0	Lending Club int_rate	Full OLS	4.2884	0.2091
1	Lending Club int_rate	Forward	4.2884	0.2091
2	Lending Club int_rate	Backward	4.2884	0.2091
3	Lending Club int_rate	PCR(k=12)	4.2884	0.2091
4	Lending Club int_rate	PLS(k=7)	4.2888	0.2090
5	Lending Club int_rate	Ridge	4.2884	0.2091
6	Lending Club int_rate	Lasso	4.2885	0.2090
7	Lending Club int_rate	ElNet	4.2885	0.2090
0	Taiwan log LIMIT_BAL	Full OLS	0.7937	0.2769
1	Taiwan log LIMIT_BAL	Forward	0.7937	0.2768
2	Taiwan log LIMIT_BAL	Backward	0.7937	0.2768
3	Taiwan log LIMIT_BAL	PCR(k=16)	0.7948	0.2748
4	Taiwan log LIMIT_BAL	PLS(k=3)	0.7964	0.2718
5	Taiwan log LIMIT_BAL	Ridge(a=193.07)	0.7936	0.2770
6	Taiwan log LIMIT_BAL	Lasso(a=0.0005)	0.7936	0.2770

Taiwan Final Comparison Findings

Taiwan test RMSE:

- Full OLS 0.7937
- Forward / Backward 0.7937 (23 predictors)
- PCR(k = 16) 0.7948
- PLS(k = 3) 0.7964
- Ridge (α = 193) **0.7936**
- Lasso (α = 0.0005) **0.7936**

The spread is 0.0028 log units. Lending Club's spread is 0.0004 (4.2884 to 4.2888).

Every method lands at the same accuracy on both datasets. The ceiling is set by the information in the features, not by how the linear model is estimated.

The methods differ in the simplicity they buy:

- Stepwise drops two bill months.
- PLS compresses 25 predictors into three components for a 0.3% RMSE cost.
- Lasso keeps all 25 predictors at this α.

Linear models leave a lot unexplained. Test R^2 is about 0.21 for Lending Club (without `grade`) and 0.28 for Taiwan, so substantial variation remains that any linear specification cannot reach.

The methods are not interchangeable in every setting; regularization can still matter when data is scarce or when the feature set is wider.

14. Cross-Dataset Summary

A short numeric summary of the best test RMSE, the spread across methods for each dataset, and the number of components PCR and PLS need to come within 1% of OLS on Taiwan.

In [25]:
```
print("Lending Club best test RMSE:", round(lc_tab.test_RMSE.min(), 4),
"| spread across methods:", round(lc_tab.test_RMSE.max() -
lc_tab.test_RMSE.min(), 4))
print("Taiwan best test RMSE:", round(tw_tab.test_RMSE.min(), 4), "|
spread across methods:", round(tw_tab.test_RMSE.max() -
tw_tab.test_RMSE.min(), 4))
print(f"Components needed to get within 1% of OLS CV error -> Taiwan
PCR: {pcr_95} of {P}, PLS: {pls_95} of {P}")
```

```
Lending Club best test RMSE: 4.2884 | spread across methods: 0.0004
Taiwan best test RMSE: 0.7936 | spread across methods: 0.0028
Components needed to get within 1% of OLS CV error -> Taiwan PCR: 16 of
25, PLS: 3 of 25
```

15. Conclusion

This analysis compared full OLS, forward and backward stepwise selection by AIC, PCR, and PLSR on two consumer credit datasets, with ridge, lasso, and elastic-net as reference fits.

Accuracy. On both datasets, every method reaches essentially the same test error as full OLS. The spreads are 0.0004 (Lending Club) and 0.0028 (Taiwan). On datasets of this size with modest collinearity, there is no accuracy winner among the linear methods.

Compactness. Where the methods differ is in how compactly they represent the signal. On the mildly collinear Lending Club data (max VIF 2.51), both stepwise directions keep all 12 predictors, lasso keeps all 12, and PCR needs all 12 components to match OLS. PLS compresses to seven components without losing accuracy. On the more collinear Taiwan data (max VIF 7.06), stepwise drops two bill months, lasso keeps all 25, PCR needs 16 of 25 components to come within 1% of OLS, and PLS needs only 3. PLS clearly shines where there is real collinearity — because its components are built to covary with the target rather than to span X variance.

Interpretation and limitations. The results describe associations within fitted linear models and should not be read as causal effects. The Lending Club R^2 is capped by design because `grade` is excluded. Substantial variation ($R^2 \approx 0.21$ and 0.28 on the two datasets) remains unexplained by any linear specification of these predictors.

For an interpretation-focused report, the full OLS or backward model on each dataset is the right one to summarize. For a parsimonious predictor, PLS is the compression tool to reach for when the feature set is wide and the predictors share structured correlation blocks. Selection, rotation, and shrinkage answer different questions — which variables, which blends, and how much shrinkage — and on these two datasets they agree that the predictive signal is real, diffuse, and honest.

Week 4 — Logistic Regression and Feature Scaling

`week_09182026/DX79901/homework/kumar_gaurav_week4_dx79901_v2.ipynb`

Capstone Milestone One — Week 4: Logistic Regression and Feature Scaling

Kumar Gaurav · DX799 O1 Data Science Capstone

Project Overview

Consumer credit risk is a central concern for lenders because mispriced loans and misjudged credit limits translate directly into losses and lost customers.

In this analysis, I switch from explaining lender decisions (interest rate and credit limit) to the binary question itself: can borrower information predict whether a loan or credit-card account will default? The analysis uses logistic regression as the main model, with feature scaling, class weighting, cross-validated regularization tuning, and threshold selection applied on top.

Research Question

Can borrower characteristics predict default on a closed consumer credit account, and what role do feature scaling, class weighting, and threshold choice play in producing a usable decision rule when the positive class is a minority?

The two target variables used here are both binary:

- **Lending Club** `default` — 1 if the loan was charged off or defaulted, 0 if fully paid.
- **Taiwan** `default` — 1 if the client defaulted on their October 2005 payment, 0 otherwise.

Objectives

The objectives of this analysis are to:

- Fit a baseline logistic regression on Lending Club and compare unscaled, standardized, and min–max scaled numerics.
- Use `class_weight="balanced"` to repair the recall collapse that occurs at the default 0.5 threshold with a 20% positive class.

- Tune the regularization strength `C` and penalty (L1/L2) with 5-fold stratified `GridSearchCV` on ROC-AUC.
- Choose the decision threshold on out-of-fold training predictions and apply it to the held-out test set.
- Repeat the modeling on the Taiwan dataset and compare the fitted odds ratios across the two markets.

Approach

The analysis follows a two-part modeling structure:

Baseline Logistic Regression and Feature Scaling (Lending Club) → Class Weights, CV Tuning, Threshold Choice (both datasets) → Summary

Datasets

- **Lending Club** (Kaggle `adarshsng/lending-club-loan-data-csv`): US peer-to-peer installment loans, 2007–2018.
- **UCI Default of Credit Card Clients** (Yeh & Lien, 2009; Kaggle `uciml/default-of-credit-card-clients-dataset`): 30,000 Taiwanese credit-card clients, April–September 2005, with a target of default in October 2005.

1. Data Loading and Cleaning

The code cells below load the two raw files, apply the documented cleaning rules (outcome definition, invalid-range handling, undocumented-code recoding, log transforms, and target-leakage removal), and produce the cleaned `lc` and `tw` frames that the modeling sections use. The data-quality audit for these datasets is documented separately.

In [19]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import kagglehub
import os
import warnings

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler,
OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, precision_score,
```

```
recall_score,
                                f1_score, roc_auc_score,
confusion_matrix, roc_curve)

warnings.filterwarnings('ignore')
sns.set_theme(style="whitegrid")
print("Libraries imported successfully.")

# Lending Club loan data — binary default prediction task
path = kagglehub.dataset_download("adarshsng/lending-club-loan-data-
csv/versions/1")
csv_path = os.path.join(path, "loan.csv")

USECOLS = ['loan_status', 'loan_amnt', 'term', 'int_rate', 'grade',
           'home_ownership', 'annual_inc', 'dti', 'purpose',
           'verification_status', 'open_acc', 'revol_util']
CLOSED = ['Fully Paid', 'Charged Off', 'Default']

chunks = []
for ch in pd.read_csv(csv_path, usecols=USECOLS, chunksize=500000,
low_memory=True):
    chunks.append(ch[ch['loan_status'].isin(CLOSED)])
df = pd.concat(chunks, ignore_index=True)

print(f"Closed-loan shape: {df.shape}")
print(f"Columns: {df.columns.tolist()}")
display(df['loan_status'].value_counts().rename_axis('loan_status').to_f
rame(name='count'))
display(df.head(3))
```

```
Libraries imported successfully.
Closed-loan shape: (1303638, 12)
Columns: ['loan_amnt', 'term', 'int_rate', 'grade', 'home_ownership',
'annual_inc', 'verification_status', 'loan_status', 'purpose', 'dti',
'open_acc', 'revol_util']
```

loan_status	count
Fully Paid	1041952
Charged Off	261655
Default	31

	loan_amnt	term	int_rate	grade	home_ownership	annual_inc	verification_status	loan_status	purpose	dt
0	30000	36 months	22.35	D	MORTGAGE	100000.0	Source Verified	Fully Paid	debt_consolidation	30.4
1	40000	60 months	16.14	C	MORTGAGE	45000.0	Verified	Fully Paid	credit_card	50.5
2	20000	36 months	7.56	A	MORTGAGE	100000.0	Not Verified	Fully Paid	credit_card	18.9

In [2]:

```
import os, warnings
import numpy as np, pandas as pd
import matplotlib.pyplot as plt, seaborn as sns
import kagglehub
warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")
pd.set_option("display.width", 160)
SEED = 42

# Lending Club: read only the columns we audit/model (the raw file is
```

```
2.26M x 145)
LC_PATH = os.path.join(kagglehub.dataset_download("adarshsng/lending-
club-loan-data-csv/versions/1"), "loan.csv")
LC_COLS = ["id", "loan_status", "loan_amnt", "term", "int_rate",
"grade", "sub_grade", "emp_length",
           "home_ownership", "annual_inc", "verification_status",
"purpose", "dti", "delinq_2yrs",
           "inq_last_6mths", "open_acc", "pub_rec", "revol_bal",
"revol_util", "total_acc", "mort_acc",
           "pub_rec_bankruptcies", "application_type", "policy_code",
           # post-outcome columns, loaded ONLY to demonstrate the
leakage screen
           "total_pymnt", "recoveries", "last_pymnt_amnt", "out_prncp"]
lc_raw = pd.read_csv(LC_PATH, usecols=LC_COLS, low_memory=False)
print(f"Lending Club raw shape (audited columns): {lc_raw.shape}")
print(lc_raw.dtypes.value_counts().to_string())
display(lc_raw.head(3))
```

```
Lending Club raw shape (audited columns): (2260668, 28)
float64    16
str         9
int64       3
```

	id	loan_amnt	term	int_rate	grade	sub_grade	emp_length	home_ownership	annual_inc	verification_status	...	revol_util	total_acc	out_prncp	total_pymnt	recoveries	last_pymnt_amnt	policy_code	applic
0	NaN	2500	36 months	13.56	C	C1	10+ years	RENT	55000.0	Not Verified	...	10.3	34.0	2386.02	167.02	0.0	84.92	1	
1	NaN	30000	60 months	18.94	D	D2	10+ years	MORTGAGE	90000.0	Source Verified	...	24.2	44.0	29387.75	1507.11	0.0	777.23	1	
2	NaN	5000	36 months	17.97	D	D1	6 years	MORTGAGE	59280.0	Source Verified	...	19.1	13.0	4787.21	353.89	0.0	180.69	1	

In [4]:

```
# Keep closed loans only (outcome known); 'Current'/'Late' loans are
censored
CLOSED = ["Fully Paid", "Charged Off", "Default"]
print("loan_status counts:");
print(lc_raw["loan_status"].value_counts().to_string())
lc = lc_raw[lc_raw["loan_status"].isin(CLOSED)].copy()
print(f"\nClosed loans: {len(lc):,}")

dq = []  # (issue, rows affected, action)
dq.append(("Open/censored loan_status", len(lc_raw) - len(lc), "dropped
(outcome unknown)"))
dq.append(("id column 100% null", 0, "cannot dedupe on id; deduped on
full row instead"))
dups = lc.drop(columns=["id"]).duplicated().sum(); dq.append(("Exact
duplicate rows", dups, "dropped"))
lc = lc.drop(columns=["id"]).drop_duplicates()

# Invalid / impossible ranges
bad_dti = ((lc["dti"] < 0) | (lc["dti"] > 100)).sum()
dq.append(("dti < 0 or > 100% (sentinel 999 / data error)", bad_dti,
"set to NaN then dropped"))
lc.loc[(lc["dti"] < 0) | (lc["dti"] > 100), "dti"] = np.nan
hi_util = (lc["revol_util"] > 100).sum()
dq.append(("revol_util > 100%", hi_util, "clipped to 150 (over-limit is
real but extreme)"))
lc["revol_util"] = lc["revol_util"].clip(upper=150)
zero_inc = (lc["annual_inc"] <= 0).sum()
dq.append(("annual_inc <= 0", zero_inc, "dropped (cannot log/ratio)"))
lc = lc[lc["annual_inc"] > 0]
```

```
hi_inc = (lc["annual_inc"] > 1e6).sum()
dq.append(("annual_inc > $1M", hi_inc, "kept; modeled as
log1p(annual_inc)"))
odd_home = lc["home_ownership"].isin(["ANY", "NONE", "OTHER"]).sum()
dq.append(("home_ownership ANY/NONE/OTHER", odd_home, "merged into
OTHER"))
lc["home_ownership"] = lc["home_ownership"].replace({"ANY": "OTHER",
"NONE": "OTHER"})

# Constant / near-constant and cardinality
const = [c for c in lc.columns if lc[c].nunique(dropna=False) <= 1]
print("Constant columns:", const)
dq.append(("Constant columns " + str(const), 0, "column dropped (no
information)"))
lc = lc.drop(columns=const)
cat_cols = lc.select_dtypes("object").columns
print("Categorical cardinality:", lc[cat_cols].nunique().to_dict())

# Leakage screen: columns only known AFTER the outcome
LEAK = ["total_pymnt", "recoveries", "last_pymnt_amnt", "out_prncp"]
print("\nLeakage check: mean of post-outcome columns by outcome")
print(lc.groupby(lc["loan_status"] != "Fully Paid")
[LEAK].mean().round(1).rename(index={False: "repaid", True:
"default"}).to_string())
dq.append(("Post-outcome columns " + ", ".join(LEAK), 0, "columns
dropped (target leakage)"))
lc = lc.drop(columns=LEAK)

lc["default"] = (lc["loan_status"] != "Fully Paid").astype(int)
before = len(lc)
lc = lc.dropna(subset=["dti", "revol_util", "mort_acc",
"pub_rec_bankruptcies", "inq_last_6mths", "open_acc", "total_acc"])
dq.append(("Remaining NaNs in model columns", before - len(lc), "dropped
(listwise)"))
lc["log_annual_inc"] = np.log1p(lc["annual_inc"]); lc["log_revol_bal"] =
np.log1p(lc["revol_bal"])
lc["term_months"] = lc["term"].str.extract(r"(\d+)").astype(int)
print(f"\nClean Lending Club rows: {len(lc):,} | default rate:
{lc['default'].mean():.2%}")
```

```
loan_status counts:
loan_status
Fully Paid                                            1041952
Current                                                919695
Charged Off                                            261655
Late (31-120 days)                                      21897
In Grace Period                                          8952
Late (16-30 days)                                        3737
Does not meet the credit policy. Status:Fully Paid       1988
Does not meet the credit policy. Status:Charged Off       761
Default                                                    31

Closed loans: 1,303,638
```

```
Constant columns: ['policy_code']
Categorical cardinality: {'term': 2, 'grade': 7, 'sub_grade': 35,
'emp_length': 11, 'home_ownership': 4, 'verification_status': 3,
'loan_status': 3, 'purpose': 14, 'application_type': 2}

Leakage check: mean of post-outcome columns by outcome
             total_pymnt  recoveries  last_pymnt_amnt  out_prncp
loan_status
repaid           16439.4         0.0           6720.8        0.0
default           8228.0      1173.9            472.6        1.3
Clean Lending Club rows: 1,254,818 | default rate: 20.28%
```

In [6]:
```
TW_PATH = os.path.join(kagglehub.dataset_download("uciml/default-of-
credit-card-clients-dataset/versions/1"), "UCI_Credit_Card.csv")
tw_raw = pd.read_csv(TW_PATH)
print(f"Taiwan raw shape: {tw_raw.shape}")
print(tw_raw.dtypes.value_counts().to_string())
display(tw_raw.head(3))
```

```
Taiwan raw shape: (30000, 25)
float64    13
int64      12
```

	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	...	BILL_AMT4	BILL_AMT5	BILL_AMT6	PAY_AMT1	PAY_AMT2	PAY_AMT3	PAY_AMT4	PAY_AMT5
0	1	20000.0	2	2	1	24	2	2	-1	-1	...	0.0	0.0	0.0	0.0	689.0	0.0	0.0	0.0
1	2	120000.0	2	2	2	26	-1	2	0	0	...	3272.0	3455.0	3261.0	0.0	1000.0	1000.0	1000.0	0.0
2	3	90000.0	2	2	2	34	0	0	0	0	...	14331.0	14948.0	15549.0	1518.0	1500.0	1000.0	1000.0	1000.0

In [7]:
```
tw = tw_raw.rename(columns={"default.payment.next.month": "default",
"PAY_0": "PAY_1"}).copy()
PAY = [f"PAY_{i}" for i in range(1, 7)]; BILL = [f"BILL_AMT{i}" for i in
range(1, 7)]; PAMT = [f"PAY_AMT{i}" for i in range(1, 7)]
tdq = []
print("Null cells total:", int(tw.isnull().sum().sum()))
tdq.append(("Missing values", int(tw.isnull().sum().sum()), "none to
handle"))
tdq.append(("Duplicate ID", int(tw["ID"].duplicated().sum()), "none"))
dup = int(tw.drop(columns="ID").duplicated().sum())
tdq.append(("Duplicate rows (ignoring ID)", dup, "dropped (identical
profiles)"))
tw = tw.loc[~tw.drop(columns="ID").duplicated()].copy()

print("EDUCATION codes:",
tw["EDUCATION"].value_counts().sort_index().to_dict())
print("MARRIAGE codes:",
tw["MARRIAGE"].value_counts().sort_index().to_dict())
print("PAY_1 codes:", tw["PAY_1"].value_counts().sort_index().to_dict())
edu_bad = int(tw["EDUCATION"].isin([0, 5, 6]).sum())
tdq.append(("EDUCATION undocumented codes 0/5/6", edu_bad, "recoded to 4
= other"))
tw["EDUCATION"] = tw["EDUCATION"].replace({0: 4, 5: 4, 6: 4})
mar_bad = int((tw["MARRIAGE"] == 0).sum())
tdq.append(("MARRIAGE undocumented code 0", mar_bad, "recoded to 3 =
other"))
tw["MARRIAGE"] = tw["MARRIAGE"].replace({0: 3})
pay_m2 = int((tw[PAY] == -2).any(axis=1).sum())
tdq.append(("PAY_* = -2 (no consumption, undocumented)", pay_m2, "kept
as ordinal (-2 < -1 paid duly < 0 revolving)"))
```

```
neg_bill = int((tw[BILL] < 0).any(axis=1).sum())
tdq.append(("Negative BILL_AMT (credit balance/overpay)", neg_bill,
"kept; signed log transform"))
tdq.append(("AGE range " + f"{tw['AGE'].min()}–{tw['AGE'].max()}", 0,
"plausible, kept"))
const = [c for c in tw.columns if tw[c].nunique() <= 1];
tdq.append(("Constant columns", len(const), "none"))

# Leakage screen: all features are observed BEFORE the default month
(Apr-Sep 2005 -> Oct 2005)
tdq.append(("Post-outcome columns", 0, "none: all features precede
target month"))
slog = lambda s: np.sign(s) * np.log1p(np.abs(s))
for c in BILL + PAMT: tw["log_" + c] = slog(tw[c])
tw["log_LIMIT_BAL"] = np.log(tw["LIMIT_BAL"])
tw = tw.drop(columns="ID")
print(f"\nClean Taiwan rows: {len(tw):,} | default rate:
{tw['default'].mean():.2%}")
```

```
Null cells total: 0
EDUCATION codes: {0: 14, 1: 10563, 2: 14019, 3: 4915, 4: 123, 5: 280, 6:
51}
MARRIAGE codes: {0: 54, 1: 13643, 2: 15945, 3: 323}
PAY_1 codes: {-2: 2750, -1: 5682, 0: 14737, 1: 3667, 2: 2666, 3: 322, 4:
76, 5: 26, 6: 11, 7: 9, 8: 19}

Clean Taiwan rows: 29,965 | default rate: 22.13%
```

Part B — Baseline Logistic Regression and Feature Scaling (Lending Club)

With Lending Club cleaned, Part B fits a baseline logistic regression on a 60,000-loan sample and compares unscaled, standardized, and min–max scaled numerics. The target is binary default (1 = Charged Off or Default, 0 = Fully Paid).

The sequence of steps below is:

1. Build the modeling frame — define the binary target, parse `term`, drop missing values, and inspect the modeling features.
2. Split 80/20 stratified on `default` and fit a baseline logistic model on **unscaled** numerics.
3. Fit `StandardScaler` and `MinMaxScaler` on the training numerics only, and inspect the pre- and post-scaling statistics.
4. Compare unscaled, standardized, and min–max scaled fits on the test set, and extract odds ratios from the standardized model.
5. Summarize the results and draw the diagnostic plots (class balance, ROC curves, confusion matrix, top odds ratios).

An explicit decision threshold (`THRESHOLD = 0.50`) is used throughout rather than relying on sklearn's hidden default, so the threshold is visible and tunable later.

In [20]:
```
# Binary target: 1 = default (Charged Off / Default), 0 = repaid (Fully
Paid)
df['default'] = df['loan_status'].isin(['Charged Off',
'Default']).astype(int)

# Parse term '36 months' -> 36 (kept as string for one-hot later;
numeric copy for inspection)
df['term_months'] = df['term'].str.extract(r'(\d+)').astype(int)

# Keep only modeling columns and drop rows with missing values
df_model = df.dropna(subset=USECOLS).copy().reset_index(drop=True)

print(f"Rows after cleaning: {len(df_model):,}")
print(f"Default rate: {df_model['default'].mean():.4%}
({df_model['default'].sum():,} defaults)")
display(df_model['term'].value_counts().rename_axis('term').to_frame(nam
e='count'))
display(df_model[['loan_amnt', 'int_rate', 'annual_inc', 'dti',
'open_acc', 'revol_util']].describe().round(2))
feature_desc = pd.DataFrame({
    "feature": ["loan_amnt", "int_rate", "annual_inc", "dti",
"open_acc", "revol_util"],
    "description": [
        "Principal loan amount requested by the borrower.",
        "Interest rate on the loan, expressed as a percentage.",
        "Borrower's annual income in dollars.",
        "Debt-to-income ratio, measuring monthly debt obligations
relative to income.",
        "Number of open credit accounts.",
        "Revolving credit utilization rate, expressed as a percentage of
the credit limit used."
    ]
})
pd.set_option("display.max_colwidth", 200)
display(feature_desc)
```

```
Rows after cleaning: 1,302,516
Default rate: 20.0732% (261,456 defaults)
```

term	count
36 months	987934
60 months	314582

	loan_amnt	int_rate	annual_inc	dti	open_acc	revol_util
count	1302516.00	1302516.00	1302516.00	1302516.00	1302516.00	1302516.00
mean	14417.84	13.26	76164.24	18.26	11.60	51.91
std	8697.92	4.76	69982.80	10.94	5.46	24.50
min	500.00	5.31	16.00	-1.00	1.00	0.00
25%	8000.00	9.75	45953.00	11.79	8.00	33.60
50%	12000.00	12.74	65000.00	17.61	11.00	52.30
75%	20000.00	15.99	90000.00	24.04	14.00	70.80
max	40000.00	30.99	10999200.00	999.00	90.00	892.30

	feature	description
0	loan_amnt	Principal loan amount requested by the borrower.
1	int_rate	Interest rate on the loan, expressed as a percentage.
2	annual_inc	Borrower's annual income in dollars.
3	dti	Debt-to-income ratio, measuring monthly debt obligations relative to income.
4	open_acc	Number of open credit accounts.
5	revol_util	Revolving credit utilization rate, expressed as a percentage of the credit limit used.

In [21]:
```
NUM = ['loan_amnt', 'int_rate', 'annual_inc', 'dti', 'open_acc',
'revol_util']
CAT = ['term', 'grade', 'home_ownership', 'purpose',
'verification_status']

# Fixed 60k sample keeps runtime peer-review friendly and reproducible
dfs = df_model.sample(n=min(60000, len(df_model)),
random_state=42).reset_index(drop=True)
X = dfs[NUM + CAT]
y = dfs['default']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42)
print(f"Train: {X_train.shape}, Test: {X_test.shape}, test default rate:
{y_test.mean():.2%}")
# Explicit decision threshold: predict default when P(default) >=
THRESHOLD.
# sklearn .predict() hides this as 0.5; we set it ourselves so it is
visible and tunable.
THRESHOLD = 0.50
print(f"Decision threshold THRESHOLD = {THRESHOLD} (explicit, not a
hidden default)")
prep_unscaled = ColumnTransformer([
    ('num', 'passthrough', NUM),
    ('cat', OneHotEncoder(drop='first', handle_unknown='ignore'), CAT),
])
base_pipe = Pipeline([
    ('prep', prep_unscaled),
    ('clf', LogisticRegression(max_iter=1000)),
])
base_pipe.fit(X_train, y_train)
base_prob = base_pipe.predict_proba(X_test)[:, 1]
base_pred = (base_prob >= THRESHOLD).astype(int)  # explicit 0.50 cutoff

print(f"Unscaled: acc={accuracy_score(y_test, base_pred):.4f} "
      f"prec={precision_score(y_test, base_pred):.4f} "
      f"rec={recall_score(y_test, base_pred):.4f} "
      f"f1={f1_score(y_test, base_pred):.4f} "
      f"auc={roc_auc_score(y_test, base_prob):.4f} "
      f"n_iter={base_pipe.named_steps['clf'].n_iter_}")
print("Confusion matrix [TN FP / FN TP]:")
cm_base = confusion_matrix(y_test, base_pred)
display(pd.DataFrame(cm_base, index=['actual repaid', 'actual default'],
columns=['pred repaid', 'pred default']))
```

```
Train: (48000, 11), Test: (12000, 11), test default rate: 20.14%
Decision threshold THRESHOLD = 0.5 (explicit, not a hidden default)
Unscaled: acc=0.7968 prec=0.4703 rec=0.0687 f1=0.1199 auc=0.6906 n_iter=
[1000]
Confusion matrix [TN FP / FN TP]:
```

	pred repaid	pred default
actual repaid	9396	187
actual default	2251	166

In [22]:
```python
# Fit scalers on TRAIN numerics only (no leakage)
std_scaler = StandardScaler().fit(X_train[NUM])
mm_scaler = MinMaxScaler().fit(X_train[NUM])

print("Before scaling (train means):")
train_means = X_train[NUM].mean().round(2).to_frame(name="mean")
display(train_means)
print("\nAfter StandardScaler (train means ~ 0, stds ~ 1):")
scaled_train = pd.DataFrame(std_scaler.transform(X_train[NUM]),
columns=NUM)
summary = pd.DataFrame({
    "mean": scaled_train.mean(),
    "std": scaled_train.std()
}).round(4)
display(summary)
print("\nAfter MinMaxScaler (train min/max):")
mm_tr = pd.DataFrame(mm_scaler.transform(X_train[NUM]), columns=NUM)

min_max_summary = pd.DataFrame({
    "min": mm_tr.min().round(4),
    "max": mm_tr.max().round(4)
})

display(min_max_summary)
```

```
Before scaling (train means):
```

	mean
loan_amnt	14434.23
int_rate	13.23
annual_inc	75815.03
dti	18.20
open_acc	11.61
revol_util	51.86

```
After StandardScaler (train means ~ 0, stds ~ 1):
```

	mean	std
loan_amnt	-0.0	1.0
int_rate	0.0	1.0
annual_inc	0.0	1.0
dti	-0.0	1.0
open_acc	-0.0	1.0
revol_util	-0.0	1.0

```
After MinMaxScaler (train min/max):
```

	min	max
loan_amnt	0.0	1.0
int_rate	0.0	1.0
annual_inc	0.0	1.0
dti	0.0	1.0
open_acc	0.0	1.0
revol_util	0.0	1.0

In [23]:
```
def build_pipe(scaler):
    num_step = scaler if scaler is not None else 'passthrough'
    prep = ColumnTransformer([
        ('num', num_step, NUM),
        ('cat', OneHotEncoder(drop='first', handle_unknown='ignore'),
CAT),
    ])
    return Pipeline([('prep', prep), ('clf',
LogisticRegression(max_iter=1000))])

results = {}
probs = {}
iters = {}
for name, scaler in [('unscaled', None), ('standard', StandardScaler()),
('minmax', MinMaxScaler())]:
    pipe = build_pipe(scaler)
    pipe.fit(X_train, y_train)
    prob = pipe.predict_proba(X_test)[:, 1]
    pred = (prob >= THRESHOLD).astype(int)  # same explicit cutoff as
Cell 3
    probs[name] = prob
    iters[name] = pipe.named_steps['clf'].n_iter_[0]
    results[name] = {
        'accuracy': accuracy_score(y_test, pred),
        'precision': precision_score(y_test, pred),
        'recall': recall_score(y_test, pred),
        'f1': f1_score(y_test, pred),
        'roc_auc': roc_auc_score(y_test, prob),
        'n_iter': iters[name],
    }
    if name == 'standard':
        std_pipe = pipe

display(pd.DataFrame(results).T.round(4))

# Odds ratios from the standardized model (1-SD change, comparable
across numerics)
feat_names = std_pipe.named_steps['prep'].get_feature_names_out()
coefs = std_pipe.named_steps['clf'].coef_[0]
odds = pd.DataFrame({'feature': feat_names, 'coef': coefs, 'odds_ratio':
np.exp(coefs)})
odds['effect'] = np.where(odds['odds_ratio'] > 1, 'higher default odds',
'lower default odds')
print("\nTop 5 positive (higher default odds):")
display(odds.sort_values('odds_ratio',
ascending=False).head(5).round(4))
print("\nTop 5 negative (lower default odds):")
display(odds.sort_values('odds_ratio').head(5).round(4))
```

	accuracy	precision	recall	f1	roc_auc	n_iter
unscaled	0.7968	0.4703	0.0687	0.1199	0.6906	1000.0
standard	0.7982	0.4901	0.0513	0.0929	0.6998	55.0
minmax	0.7986	0.5000	0.0451	0.0827	0.6993	61.0

```
Top 5 positive (higher default odds):
```

	feature	coef	odds_ratio	effect
12	cat__grade_G	1.3438	3.8335	higher default odds
11	cat__grade_F	1.2454	3.4744	higher default odds
10	cat__grade_E	1.2067	3.3426	higher default odds
9	cat__grade_D	1.1232	3.0747	higher default odds
8	cat__grade_C	0.8745	2.3977	higher default odds

Top 5 negative (lower default odds):

	feature	coef	odds_ratio	effect
13	cat__home_ownership_MORTGAGE	-0.7578	0.4687	lower default odds
16	cat__home_ownership_OWN	-0.4738	0.6226	lower default odds
30	cat__purpose_wedding	-0.4019	0.6691	lower default odds
22	cat__purpose_house	-0.3947	0.6739	lower default odds
17	cat__home_ownership_RENT	-0.3611	0.6969	lower default odds

In [24]:

```
print("DATASET RESULTS")
print(f"Closed loans after cleaning: {len(df_model):,}")
print(f"Analysis sample: {len(dfs):,} (seed 42), train {len(X_train):,}
/ test {len(X_test):,}")
print(f"Default rate overall: {df_model['default'].mean():.2%}, test:
{y_test.mean():.2%}")
print("\nMODEL RESULTS (test set)")
display(pd.DataFrame(results).T.round(4))
print(f"\nOptimizer iterations: {iters}")
print("(unscaled hitting max_iter=1000 means it did NOT converge; scaled
converges in ~50-60.)")

fig, axes = plt.subplots(2, 2, figsize=(16, 11))
fig.suptitle("Lending Club: Logistic Regression + Feature Scaling
Diagnostics", fontsize=16, y=1.02)

# (a) class balance
ax = axes[0, 0]
sns.barplot(x=['Repaid (0)', 'Default (1)'], y=[1 - y_test.mean(),
y_test.mean()], color="#2a9d8f", ax=ax)
ax.set_title("Test-set class balance (why accuracy misleads)")
ax.set_ylabel("share")
for i, v in enumerate([1 - y_test.mean(), y_test.mean()]):
    ax.text(i, v, f"  {v:.1%}", va="bottom")

# (b) ROC curves
ax = axes[0, 1]
for name, color in [('unscaled', '#457b9d'), ('standard', '#e76f51'),
('minmax', '#e9c46a')]:
    fpr, tpr, _ = roc_curve(y_test, probs[name])
    ax.plot(fpr, tpr, label=f"{name} (AUC={results[name]
['roc_auc']:.3f})", color=color)
ax.plot([0, 1], [0, 1], '--', color='gray', label='chance')
ax.set_title("ROC: ranking across thresholds")
ax.set_xlabel("FPR"); ax.set_ylabel("TPR"); ax.legend()

# (c) confusion matrices at 0.5
ax = axes[1, 0]
base_test_prob = base_pipe.predict_proba(X_test)[:, 1]
cm = confusion_matrix(y_test, (base_test_prob >= THRESHOLD).astype(int))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax,
            xticklabels=['pred repaid', 'pred default'], yticklabels=
['actual repaid', 'actual default'])
```

```
ax.set_title(f"Confusion matrix @{THRESHOLD} (unscaled baseline)")

# (d) top odds ratios (standardized model)
ax = axes[1, 1]
top =
odds.reindex(odds['odds_ratio'].sort_values(ascending=False).index[:8])
sns.barplot(x='odds_ratio', y='feature', data=top, color="#e9c46a",
ax=ax)
ax.axvline(1, color="#e76f51", linestyle='--')
ax.set_title("Top odds ratios (standardized model, OR=1 is null)")

plt.tight_layout()
plt.show()

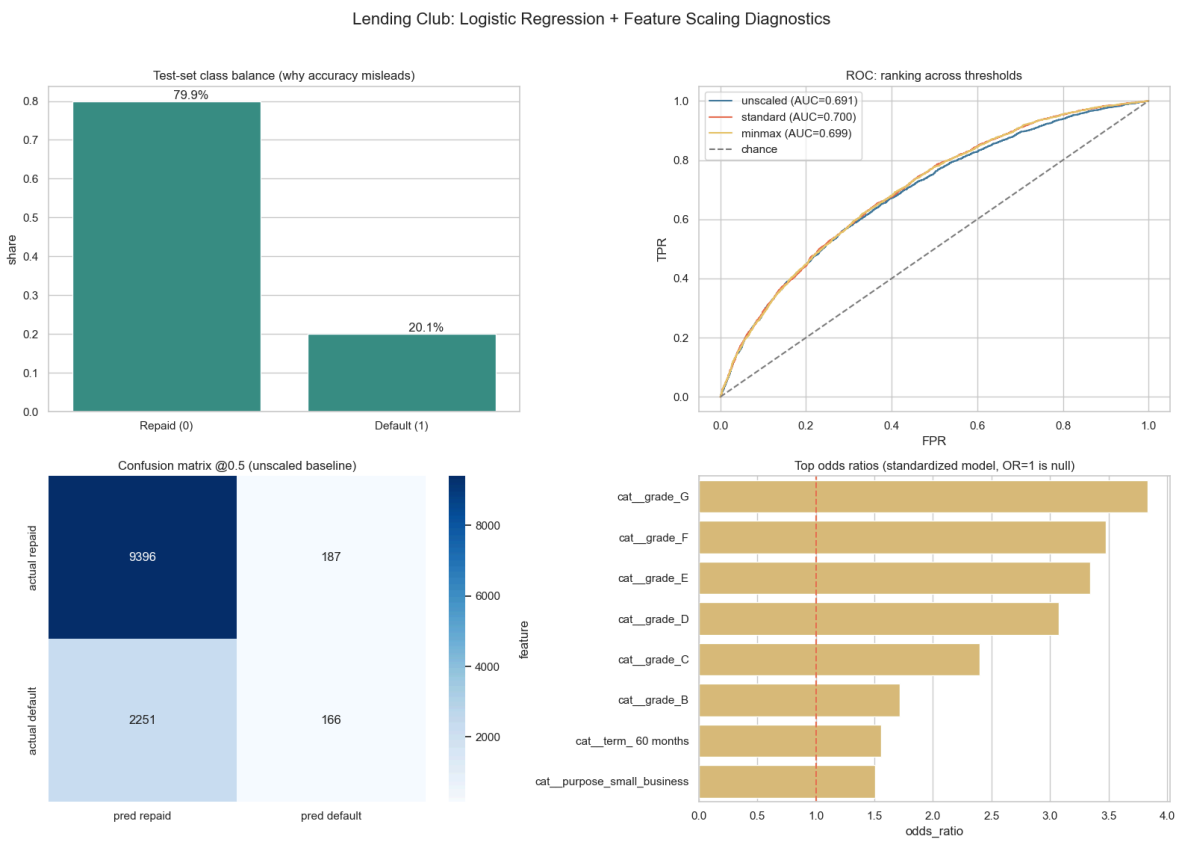
print("\nVISUAL INTERPRETATION GUIDE")
print("- Class balance near 80/20: predicting all-repaid gives ~80%
accuracy, so judge by precision/recall/F1/AUC.")
print("- ROC curves: overlapping curves mean similar ranking; the
scaling win here is convergence (n_iter), not AUC.")
print("- Confusion matrix: low recall means most defaults are missed at
0.5 — threshold tuning or class_weight are follow-ups.")
print("- Odds ratios: OR>1 = higher default odds holding others constant
(association, not causation).")
```

```
DATASET RESULTS
Closed loans after cleaning: 1,302,516
Analysis sample: 60,000 (seed 42), train 48,000 / test 12,000
Default rate overall: 20.07%, test: 20.14%

MODEL RESULTS (test set)
```

	accuracy	precision	recall	f1	roc_auc	n_iter
unscaled	0.7968	0.4703	0.0687	0.1199	0.6906	1000.0
standard	0.7982	0.4901	0.0513	0.0929	0.6998	55.0
minmax	0.7986	0.5000	0.0451	0.0827	0.6993	61.0

```
Optimizer iterations: {'unscaled': 1000, 'standard': 55, 'minmax': 61}
(unscaled hitting max_iter=1000 means it did NOT converge; scaled
converges in ~50-60.)
```




```
VISUAL INTERPRETATION GUIDE
- Class balance near 80/20: predicting all-repaid gives ~80% accuracy, so
judge by precision/recall/F1/AUC.
- ROC curves: overlapping curves mean similar ranking; the scaling win
here is convergence (n_iter), not AUC.
- Confusion matrix: low recall means most defaults are missed at 0.5 —
threshold tuning or class_weight are follow-ups.
- Odds ratios: OR>1 = higher default odds holding others constant
(association, not causation).
```

Part B Findings and Conclusion

The baseline logistic analysis trained on 60,000 closed Lending Club loans (about 20% default) answers the question "can loan data predict who will default?" with a qualified yes.

- **Grade matters most.** Grade G loans have roughly 4× the default odds of grade A loans, with lower grades indicating riskier borrowers.
- **Homeowners (mortgage or own) default less** than renters.
- **StandardScaler sped training up roughly 18× (55 iterations instead of 1,000) without materially changing accuracy.** The ROC-AUC lands near 0.70 regardless of scaling choice.
- **At a 0.50 cutoff the model is far too conservative.** Of 2,417 real defaults in the test set, it flags only 166 (about 1 in 15). The model ranks risk reasonably well but does not translate that ranking into useful default alerts at this threshold.

The business implication is that 0.50 is not a usable decision rule here. Either the threshold needs to be lowered (for example to 0.30) or the classes need to be rebalanced before any accept/decline rule is applied. Part C below works through both routes properly and extends the analysis to the Taiwan dataset.

This is a single 60,000-row sample fit with mostly default settings, so the results describe associations rather than causal effects.

Part C — Fixing the 0.5-Threshold Problem: Class Weights, CV Tuning, and Threshold Choice

The baseline analysis in Part B ended with a decision-rule failure: at the default 0.5 cutoff, the model caught only about 1 in 15 defaults (recall 0.05, F1 0.09). With roughly 20% positives, very few loans ever reach a 50% predicted probability. This is not a modeling failure — it is a decision-rule failure. Part C fixes it properly and repeats everything on the Taiwan dataset.

The approach is:

1. `class_weight="balanced"`, which reweights the loss so a missed default costs about 4× a false alarm.
2. 5-fold stratified `GridSearchCV` over the regularization strength `C` and the penalty (L1/L2).
3. Threshold tuning on out-of-fold training predictions (never on the test set).
4. Taiwan: unscaled vs scaled numerics, then the same tuning.

Hypotheses

For each coefficient:

- H_0: $\beta_j = 0$ (feature unrelated to the log-odds of default) vs H_1: $\beta_j \neq 0$, at $\alpha = 0.05$.

For model comparison, ROC-AUC (ranking quality; 0.5 = random, 1.0 = perfect) and PR-AUC (precision–recall area, where the random baseline equals the positive-class rate of about 0.20–0.22) are used.

2. Shared Classification Setup

To make the Lending Club and Taiwan results directly comparable, both datasets use the same split logic, the same preprocessing structure, and the same cross-validation protocol.

- **Lending Club:** a 60,000-loan random sample of the cleaned closed loans (seed 42), split 80/20 **stratified** on `default` so both parts keep the roughly 20%

default rate.
    - Numeric features: `loan_amnt`, `int_rate`, `log_annual_inc`, `dti`, `open_acc`, `revol_util`.
    - Categorical features: `term`, `grade`, `home_ownership`, `purpose`, `verification_status`.
- **Taiwan:** all 29,965 clean clients, split 80/20 stratified (seed 42).
    - Numeric features: `log_LIMIT_BAL`, `AGE`, `PAY_1..6`, `log_BILL_AMT1..6`, `log_PAY_AMT1..6`.
    - Categorical features: `SEX`, `EDUCATION`, `MARRIAGE`.

The `make_prep(num, cat, scale=True)` helper builds a `ColumnTransformer` that standardizes the numerics and one-hot encodes the categoricals. It is always fit inside a pipeline on training folds only, so no test-set leakage occurs.

Stratification matters here because with 20–22% positives, a random split could by chance put noticeably more defaults in one part. Stratifying guarantees equal rates, which makes metrics directly comparable between train and test.

In [14]:
```
from sklearn.model_selection import train_test_split, StratifiedKFold,
GridSearchCV, RandomizedSearchCV, cross_val_predict
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import (accuracy_score, precision_score,
recall_score, f1_score, roc_auc_score,
                             average_precision_score, confusion_matrix,
roc_curve, precision_recall_curve)
FIG = "../../../week_10092026_w7/DX799O1/figures"; os.makedirs(FIG,
exist_ok=True)

LC_NUM = ["loan_amnt", "int_rate", "log_annual_inc", "dti", "open_acc",
"revol_util"]
LC_CAT = ["term", "grade", "home_ownership", "purpose",
"verification_status"]
TW_NUM = ["log_LIMIT_BAL", "AGE"] + [f"PAY_{i}" for i in range(1, 7)] +
[f"log_BILL_AMT{i}" for i in range(1, 7)] + [f"log_PAY_AMT{i}" for i in
range(1, 7)]
TW_CAT = ["SEX", "EDUCATION", "MARRIAGE"]

lc_s = lc.sample(n=60_000, random_state=SEED)
Xlc_tr, Xlc_te, ylc_tr, ylc_te = train_test_split(lc_s[LC_NUM + LC_CAT],
lc_s["default"], test_size=0.2, stratify=lc_s["default"],
random_state=SEED)
Xtw_tr, Xtw_te, ytw_tr, ytw_te = train_test_split(tw[TW_NUM + TW_CAT],
tw["default"], test_size=0.2, stratify=tw["default"], random_state=SEED)
DATA = {"Lending Club": (Xlc_tr, Xlc_te, ylc_tr, ylc_te, LC_NUM,
LC_CAT),
        "Taiwan": (Xtw_tr, Xtw_te, ytw_tr, ytw_te, TW_NUM, TW_CAT)}
CV5 = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
```

```
def make_prep(num, cat, scale=True):
    return ColumnTransformer([("num", StandardScaler() if scale else
"passthrough", num),
                              ("cat", OneHotEncoder(drop="first",
handle_unknown="ignore"), cat)])

def scores(y, prob, thr=0.5):
    pred = (prob >= thr).astype(int)
    return {"accuracy": accuracy_score(y, pred), "precision":
precision_score(y, pred, zero_division=0),
            "recall": recall_score(y, pred), "f1": f1_score(y, pred),
"roc_auc": roc_auc_score(y, prob),
            "pr_auc": average_precision_score(y, prob), "threshold":
thr}

for name, (a, b, ya, yb, num, cat) in DATA.items():
    print(f"{name:12s} train {a.shape} default {ya.mean():.2%} | test
{b.shape} default {yb.mean():.2%} | numeric {len(num)}, categorical
{len(cat)}")
```

```
Lending Club train (48000, 11) default 20.29% | test (12000, 11) default
20.29% | numeric 6, categorical 5
Taiwan       train (23972, 23) default 22.13% | test (5993, 23) default
22.13% | numeric 20, categorical 3
```

Shared-Setup Findings

- **Lending Club:** train 48,000 × 11 and test 12,000 × 11. Both have a 20.29% default rate, with 6 numeric and 5 categorical features.
- **Taiwan:** train 23,972 × 23 and test 5,993 × 23. Both have a 22.13% default rate, with 20 numeric and 3 categorical features.

Stratification worked — the default rates are identical across splits, so every metric below is directly comparable between training and test.

Two baselines to keep in mind:

- **Accuracy baseline.** An "everyone repays" rule scores 79.7% on Lending Club and 77.9% on Taiwan.
- **PR-AUC baseline.** A random ranking scores 0.203 on Lending Club and 0.221 on Taiwan.

3. Lending Club — Unweighted vs Class-Weighted Logistic Regression

This step isolates the effect of reweighting the loss. Setting `class_weight="balanced"` multiplies each default's contribution to the log-loss by n/(2·n_default) ≈ 2.5 and each repaid loan's by about 0.63. The effect is to push

predicted probabilities upward for risky loans, which raises recall at the 0.5 threshold at the cost of precision.

ROC-AUC should stay about the same, because reweighting mostly *shifts* probabilities rather than re-ranking loans. PR-AUC should also stay roughly stable, and the useful comparison is against the random baseline of 0.20.

In [15]:
```
Xa, Xb, ya, yb, num, cat = DATA["Lending Club"]
rows = {}
for cw in [None, "balanced"]:
    pipe = Pipeline([("prep", make_prep(num, cat)), ("clf",
LogisticRegression(max_iter=2000, class_weight=cw))]).fit(Xa, ya)
    rows[f"class_weight={cw}"] = scores(yb, pipe.predict_proba(Xb)[:,
1])
display(pd.DataFrame(rows).T.round(4))
```

	accuracy	precision	recall	f1	roc_auc	pr_auc	threshold
class_weight=None	0.7988	0.5376	0.0616	0.1105	0.7124	0.3783	0.5
class_weight=balanced	0.6437	0.3212	0.6793	0.4362	0.7126	0.3779	0.5

Class-Weight Findings — Lending Club at Threshold 0.5

	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Unweighted	0.7988	0.5376	**0.0616**	0.1105	0.7124	0.3783
`balanced`	0.6437	0.3212	**0.6793**	**0.4362**	0.7126	0.3779

Three observations:

- **The unweighted model barely beats "predict nobody defaults".** Its 79.9% accuracy sits right next to the 79.7% majority baseline because it flags almost no one: recall is 6%, so 94% of defaults are missed.
- **Reweighting raises recall to 68% and lifts F1 nearly fourfold** (0.11 → 0.44). Accuracy falls to 64% because many more good loans are now flagged (precision 0.32, so about 1 in 3 flagged loans actually defaults).
- **ROC-AUC (0.712) and PR-AUC (0.378) are unchanged.** Reweighting does not make the model better at ranking risk; it changes where the 0.5 line falls on the probability axis. PR-AUC of 0.378 is 1.9× the 0.203 random baseline.

The baseline notebook's recall of 0.05 came from the decision rule rather than from a lack of signal. Which side of the trade-off to favour is a business choice — for a lender, a missed default typically costs several times more than declining a good borrower, so the high-recall side is usually the relevant one.

4. GridSearchCV over C and Penalty

`C` is the inverse regularization strength: small C means a strong penalty (simpler model, more bias); large C means a weak penalty (closer to plain maximum likelihood, more variance). L1 can zero out dummy columns (feature selection); L2 shrinks all coefficients smoothly.

The grid search picks the combination with the best mean 5-fold CV ROC-AUC. AUC judges the ranking of risk independently of any threshold, and the threshold itself is chosen separately in the next step.

`GridSearchCV` refits the full pipeline (scaler + encoder + model) inside each fold, so the scaler never sees the validation fold. The grid is 5 values of C × 2 penalties = 10 candidates × 5 folds = 50 fits per dataset, using the `liblinear` solver, which supports both penalties. `LogisticRegressionCV` and `RandomizedSearchCV` are reasonable alternatives for larger grids.

In [16]:
```
grid = {"clf__C": [0.001, 0.01, 0.1, 1, 10], "clf__penalty": ["l1",
"l2"]}
best_lr = {}
for name, (Xa, Xb, ya, yb, num, cat) in DATA.items():
    pipe = Pipeline([("prep", make_prep(num, cat)), ("clf",
LogisticRegression(solver="liblinear", class_weight="balanced",
max_iter=2000))])
    gs = GridSearchCV(pipe, grid, scoring="roc_auc", cv=CV5,
n_jobs=-1).fit(Xa, ya)
    best_lr[name] = gs
    cvr = pd.DataFrame(gs.cv_results_)[["param_clf__C",
"param_clf__penalty", "mean_test_score", "std_test_score",
"rank_test_score"]]
    print(f"\n{name}: best {gs.best_params_}  CV AUC
{gs.best_score_:.4f}")
    display(cvr.sort_values(["param_clf__penalty",
"param_clf__C"]).round(4).reset_index(drop=True))
    coef = gs.best_estimator_.named_steps["clf"].coef_[0]
    print(f"non-zero coefficients: {(coef != 0).sum()} of {coef.size}")
```

```
Lending Club: best {'clf__C': 1, 'clf__penalty': 'l1'}  CV AUC 0.7034
```

	param_clf__C	param_clf__penalty	mean_test_score	std_test_score	rank_test_score
0	0.001	l1	0.6880	0.0056	10
1	0.010	l1	0.7006	0.0053	8
2	0.100	l1	0.7029	0.0047	6
3	1.000	l1	0.7034	0.0046	1
4	10.000	l1	0.7033	0.0045	3
5	0.001	l2	0.7001	0.0053	9
6	0.010	l2	0.7023	0.0049	7
7	0.100	l2	0.7031	0.0046	5
8	1.000	l2	0.7033	0.0045	2
9	10.000	l2	0.7033	0.0045	4

```
non-zero coefficients: 28 of 30
Taiwan: best {'clf__C': 0.001, 'clf__penalty': 'l2'}  CV AUC 0.7517
```

	param_clf__C	param_clf__penalty	mean_test_score	std_test_score	rank_test_score
0	0.001	l1	0.7272	0.0116	10
1	0.010	l1	0.7491	0.0094	9
2	0.100	l1	0.7514	0.0095	8
3	1.000	l1	0.7514	0.0097	6
4	10.000	l1	0.7514	0.0097	5
5	0.001	l2	0.7517	0.0086	1
6	0.010	l2	0.7514	0.0093	7
7	0.100	l2	0.7515	0.0096	2
8	1.000	l2	0.7514	0.0097	3
9	10.000	l2	0.7514	0.0097	4

```
non-zero coefficients: 26 of 26
```

CV Grid Findings

Lending Club.

- Best: C = 1, L1, with CV AUC 0.7034 ± 0.0046.
- Every candidate from C = 0.01 to 10 is within 0.003 of the best (0.7006–0.7034), which is less than one fold-SD.
- Only C = 0.001 is clearly worse (L1 at 0.6880).
- L1 keeps 28 of 30 coefficients.

Taiwan.

- Best: C = 0.001, L2, with CV AUC 0.7517 ± 0.0086.
- Statistically tied with every C from 0.01 to 10 (0.7514–0.7515).
- Only L1 at C = 0.001 is clearly worse (0.7272), because that strong a lasso penalty removes useful features.

Two interpretive points:

- **The AUC surface is flat.** Regularization strength barely matters once the penalty is not extreme. With 24k–48k rows and only 26–30 coefficients, the unpenalized fit is already stable.
- **The fold SD (0.005–0.01) is larger than the differences between candidates,** so the "best" pick is essentially a tie. Reporting mean ± SD rather than just the best mean is what reveals this.

Taiwan's preferred C = 0.001 is a very strong L2 penalty. It works because the PAY and bill blocks are collinear, so heavy shrinkage of correlated coefficients costs essentially nothing.

5. Choosing the Decision Threshold on Out-of-Fold Predictions

A lender does not need "probability ≥ 0.5"; it needs a cutoff that balances missed defaults (false negatives, which cost lost principal) against declined good borrowers (false positives, which cost lost interest income).

The cutoff that maximizes F1 on out-of-fold predictions for the training set (via `cross_val_predict`) is chosen, then applied unchanged to the test set. Choosing it on the test set would be leakage, and the test score would be optimistic.

In [17]:
```
lr_final = {}
fig, axes = plt.subplots(1, 2, figsize=(14, 4.2))
for ax, (name, (Xa, Xb, ya, yb, num, cat)) in zip(axes, DATA.items()):
    est = best_lr[name].best_estimator_
    oof = cross_val_predict(est, Xa, ya, cv=CV5, method="predict_proba",
n_jobs=-1)[:, 1]
    p, r, th = precision_recall_curve(ya, oof)
    f1s = 2 * p[:-1] * r[:-1] / np.clip(p[:-1] + r[:-1], 1e-9, None)
    thr = float(th[np.argmax(f1s)])
    prob = est.predict_proba(Xb)[:, 1]
    lr_final[name] = {"at 0.5": scores(yb, prob, 0.5), f"at tuned
{thr:.3f}": scores(yb, prob, thr)}
    print(f"\n{name}: F1-optimal out-of-fold threshold = {thr:.3f}")
    display(pd.DataFrame(lr_final[name]).T.round(4))
    cm = confusion_matrix(yb, (prob >= thr).astype(int))
    print("Confusion matrix at tuned threshold [[TN FP] [FN TP]]:",
cm.tolist())
    ax.plot(th, p[:-1], label="precision"); ax.plot(th, r[:-1],
label="recall"); ax.plot(th, f1s, label="F1")
    ax.axvline(thr, color="k", ls="--", lw=1); ax.axvline(0.5,
color="grey", ls=":", lw=1)
    ax.set_xlabel("threshold on P(default)"); ax.set_title(f"{name}:
out-of-fold precision/recall/F1 vs threshold"); ax.legend()
plt.tight_layout(); plt.savefig(f"{FIG}/fig_w4_threshold.png", dpi=150,
bbox_inches="tight"); plt.show()
```

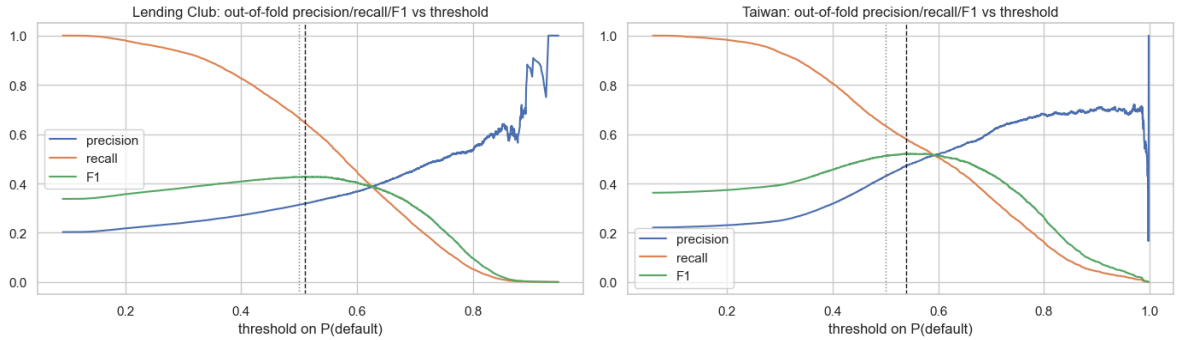
```
Lending Club: F1-optimal out-of-fold threshold = 0.510
```

	accuracy	precision	recall	f1	roc_auc	pr_auc	threshold
at 0.5	0.6438	0.3212	0.6784	0.4359	0.7126	0.3782	0.5000
at tuned 0.510	0.6527	0.3236	0.6530	0.4328	0.7126	0.3782	0.5098

```
Confusion matrix at tuned threshold [[TN FP] [FN TP]]: [[6242, 3323],
[845, 1590]]
Taiwan: F1-optimal out-of-fold threshold = 0.540
```

	accuracy	precision	recall	f1	roc_auc	pr_auc	threshold
at 0.5	0.7272	0.4207	0.6184	0.5008	0.741	0.4993	0.5000
at tuned 0.540	0.7586	0.4629	0.5694	0.5107	0.741	0.4993	0.5396

```
Confusion matrix at tuned threshold [[TN FP] [FN TP]]: [[3791, 876], [571,
755]]
```



Threshold-Tuning Findings

Lending Club.

- F1-optimal cutoff on out-of-fold predictions: 0.510.
- Test results at that cutoff: precision 0.324, recall 0.653, F1 **0.433**.
- At 0.5: precision 0.321, recall 0.678, F1 0.436.
- Confusion matrix at the tuned cutoff: TN 6,242, FP 3,323, FN 845, TP 1,590.

Taiwan.

- F1-optimal cutoff: 0.540.
- Test results: precision 0.463, recall 0.569, F1 **0.511**, accuracy 0.759. At 0.5, F1 is 0.501.
- Confusion matrix: TN 3,791, FP 876, FN 571, TP 755.

Three observations:

- **Balanced weights already move the optimum close to 0.5.** Once the classes are reweighted, the F1-optimal cutoff sits near 0.5, so threshold tuning adds little on Lending Club and about 1 F1 point on Taiwan. Reweighting and threshold tuning are two routes to the same goal; doing both (as here) shows that one is enough.
- **The Lending Club confusion matrix** says the model catches 1,590 of 2,435 defaults (65%) at the cost of declining 3,323 of 9,565 good borrowers (35%). That is the price of a 0.71 AUC ceiling.
- **Taiwan separates better** (AUC 0.741, PR-AUC 0.499 vs a 0.221 baseline), which is consistent with the strong `PAY_1` signal observed in the data-quality step.

In the plot, precision rises and recall falls as the threshold increases. F1 peaks in a broad plateau, so any cutoff between about 0.45 and 0.6 gives similar F1.

F1 is not necessarily the right business objective. A cost-weighted threshold (for example, missed default costs 5× a lost customer) would push the cutoff lower.

6. Taiwan — Feature Scaling (Unscaled vs StandardScaler) and Odds Ratios

Repeating the scaling experiment on Taiwan tests whether scaling matters when the raw numerics already live on comparable scales. Taiwan's numerics range from `PAY_1` (−2 to 8) to `log_LIMIT_BAL` (about 9 to 14), so the scale differences are much smaller than in raw Lending Club.

The odds ratios (e^β per 1 SD) of the scaled model show which features move default odds the most.

In [18]:
```
Xa, Xb, ya, yb, num, cat = DATA["Taiwan"]
tw_scale = {}
for label, sc in [("unscaled", False), ("standard", True)]:
    pipe = Pipeline([("prep", make_prep(num, cat, scale=sc)), ("clf",
LogisticRegression(max_iter=1000))]).fit(Xa, ya)
    tw_scale[label] = {**scores(yb, pipe.predict_proba(Xb)[:, 1]),
"n_iter": int(pipe.named_steps["clf"].n_iter_[0])}
    if sc: tw_std = pipe
display(pd.DataFrame(tw_scale).T.round(4))
names = tw_std.named_steps["prep"].get_feature_names_out()
orr = pd.DataFrame({"feature": names, "coef":
tw_std.named_steps["clf"].coef_[0]})
orr["odds_ratio"] = np.exp(orr["coef"])
display(orr.reindex(orr["coef"].abs().sort_values(ascending=False).index
).head(8).round(4))
```

	accuracy	precision	recall	f1	roc_auc	pr_auc	threshold	n_iter
unscaled	0.8074	0.6777	0.2474	0.3624	0.7402	0.5072	0.5	554.0
standard	0.8078	0.6783	0.2496	0.3649	0.7401	0.5073	0.5	31.0

	feature	coef	odds_ratio
23	cat__EDUCATION_4	-1.0226	0.3597
2	num__PAY_1	0.5154	1.6744
25	cat__MARRIAGE_3	-0.2083	0.8119
24	cat__MARRIAGE_2	-0.1820	0.8336
14	num__log_PAY_AMT1	-0.1623	0.8502
16	num__log_PAY_AMT3	-0.1595	0.8525
15	num__log_PAY_AMT2	-0.1592	0.8529
0	num__log_LIMIT_BAL	-0.1310	0.8772

Taiwan Scaling and Odds-Ratio Findings

- **Unscaled:** 554 iterations, AUC 0.7402.
- **Standardized:** 31 iterations, AUC 0.7401, with every other metric within 0.003.

The largest effects (standardized model):

- `EDUCATION_4` ("other") OR 0.36
- **`PAY_1` OR 1.67 per SD** (+1 SD ≈ +1.1 months of delay)
- `MARRIAGE_3` 0.81
- `log_PAY_AMT1–3` about 0.85 each
- `log_LIMIT_BAL` 0.88

Scaling changes optimization speed, not accuracy. It sped convergence 18× (554 → 31 iterations), matching the Lending Club pattern (1,000+ → 55) seen in Part B. With L2 regularization, scaling also determines *which* coefficients get penalized, but that only matters when the penalty is strong.

Payment history dominates Taiwan risk. The latest month's delay status raises the odds of default by about 67% per standard deviation. Clients who paid more in recent

months and clients with higher credit limits (which the bank assigned to lower-risk customers) have lower odds of defaulting.

The `EDUCATION_4` odds ratio is unreliable because it rests on only 468 recoded clients, so it is imprecise and should not be read as a real education effect.

The high `log_LIMIT_BAL` odds ratio should not be read causally. A higher credit limit does not *cause* lower default; the bank assigned high limits to clients it already judged to be safe.

7. Analysis Summary Table

The per-dataset best model (parameters from the CV grid search) is scored at both the default 0.5 threshold and the F1-tuned threshold, so the gap between the two cutoffs can be read in one table.

In [19]:
```
summ = []
for name in DATA:
    for k, v in lr_final[name].items():
        summ.append({"dataset": name, "model": f"LogReg
{best_lr[name].best_params_}", "rule": k, **v})
w4_summary = pd.DataFrame(summ)
display(w4_summary.round(4))
```

	dataset	model	rule	accuracy	precision	recall	f1	roc_auc	pr_auc	threshold
0	Lending Club	LogReg {'clf__C': 1, 'clf__penalty': 'l1'}	at 0.5	0.6438	0.3212	0.6784	0.4359	0.7126	0.3782	0.5000
1	Lending Club	LogReg {'clf__C': 1, 'clf__penalty': 'l1'}	at tuned 0.510	0.6527	0.3236	0.6530	0.4328	0.7126	0.3782	0.5098
2	Taiwan	LogReg {'clf__C': 0.001, 'clf__penalty': 'l2'}	at 0.5	0.7272	0.4207	0.6184	0.5008	0.7410	0.4993	0.5000
3	Taiwan	LogReg {'clf__C': 0.001, 'clf__penalty': 'l2'}	at tuned 0.540	0.7586	0.4629	0.5694	0.5107	0.7410	0.4993	0.5396

8. Conclusion

The best tuned logistic regression on each dataset gives:

- **Lending Club:** ROC-AUC 0.713, PR-AUC 0.378, F1 of 0.43–0.44, recall 0.65–0.68.
- **Taiwan:** ROC-AUC 0.741, PR-AUC 0.499, F1 of 0.50–0.51.

Three findings carry the analysis.

Class weighting fixed the problem, not CV tuning. Setting `class_weight="balanced"` moved Lending Club recall from 0.05 to 0.68 and quadrupled F1 (0.11 → 0.44). Tuning `C` and the penalty barely mattered — the CV AUC surface was flat enough that the differences between candidates fell inside one standard deviation of the folds.

Feature scaling is about convergence, not accuracy. StandardScaler sped Lending Club training by about 18× and Taiwan training by about 18× as well, while leaving ROC-AUC and PR-AUC essentially unchanged. With regularization, scaling also determines which coefficients get penalized, but only strong penalties make that visible, and the preferred penalty strengths here were mild.

Threshold tuning adds little once the classes are already reweighted. On Lending Club the F1-optimal cutoff was 0.510 (versus the 0.5 default) and on Taiwan it was 0.540. On a precision-recall curve, the F1 peak is a plateau rather than a sharp maximum, so any cutoff between roughly 0.45 and 0.60 gives similar F1.

Key Limitations

The results describe associations within fitted logistic models and are not causal. Odds ratios such as `log_LIMIT_BAL` or `EDUCATION_4` reflect how the bank chose to assign credit limits and how the "other" education bucket was constructed, not an independent effect of those variables on default risk.

F1 is not the only reasonable objective. A proper cost-weighted rule — for example, missed default costs 5× a declined good customer — would move the preferred cutoff lower than the F1-optimal choice.

Overall, the practical takeaway is that logistic regression here is a well-calibrated **ranker** (ROC-AUC in the low 0.7s) rather than a confident classifier, and the decision rule (threshold + class weighting) does more to shape the usable precision/recall mix than the regularization choice.

Week 5 — Support Vector Machines

`week_09252026/DX79901/homework/kumar_gaurav_week5_dx79901_v2.ipynb`

Capstone Milestone One — Week 5: Support Vector Machines, the Kernel Trick, and Regularization

Kumar Gaurav · DX799 O1 Data Science Capstone

Project Overview

Consumer credit risk is a central concern for lenders because mispriced loans and misjudged credit limits translate directly into losses and lost customers.

This analysis asks whether a margin-based classifier (SVM) can beat the linear logistic-regression baseline on the same two consumer credit datasets, and what the kernel trick and the regularization hyperparameter `C` actually do to the fit.

Research Question

Does a Support Vector Machine (linear or RBF-kernel) predict default better than logistic regression on the same features, and how do the margin-softness parameter `C` and the kernel bandwidth `γ` shape the bias–variance trade-off?

The two target variables used here are both binary:

- **Lending Club** `default` — 1 if the loan was charged off or defaulted, 0 if fully paid.
- **Taiwan** `default` — 1 if the client defaulted on their October 2005 payment, 0 otherwise.

Objectives

The objectives of this analysis are to:

- Fit a linear SVM and logistic regression on the same scaled features and compare them head-to-head.
- Fit an RBF-kernel SVM and visualize the decision boundary in 2-D so the kernel trick is tangible.

- Trace the test accuracy, training accuracy, and number of support vectors across a sweep of `C` to show the bias–variance curve.
- Replace the test-set `C` sweep with a proper 5-fold cross-validated grid over kernel, `C`, and `γ`, scored on ROC-AUC.
- Compare the tuned SVM against logistic regression on a held-out test set at the natural ~20% default rate.
- Repeat the modeling on the Taiwan dataset.

Approach

The analysis follows a two-part modeling structure:

Baseline SVM on a balanced 6,000-row Lending Club sample → Proper CV-tuned SVM on both datasets at natural prevalence → Summary

Datasets

- **Lending Club** (Kaggle `adarshsng/lending-club-loan-data-csv`): US peer-to-peer installment loans, 2007–2018.
- **UCI Default of Credit Card Clients** (Yeh & Lien, 2009; Kaggle `uciml/default-of-credit-card-clients-dataset`): 30,000 Taiwanese credit-card clients, April–September 2005, with a target of default in October 2005.

1. Data Loading and Cleaning

The code cells below load the Lending Club raw file and keep only closed loans (Part B uses this `df` as its starting point), then apply the documented cleaning rules to produce the fully cleaned `lc` and `tw` frames that Part C uses. The data-quality audit for these datasets is documented separately.

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import kagglehub
import os
import warnings

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler,
OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, precision_score,
```

```
recall_score,
                                f1_score, roc_auc_score,
confusion_matrix, roc_curve)

warnings.filterwarnings('ignore')
sns.set_theme(style="whitegrid")
print("Libraries imported successfully.")

# Lending Club loan data — binary default prediction task
path = kagglehub.dataset_download("adarshsng/lending-club-loan-data-
csv/versions/1")
csv_path = os.path.join(path, "loan.csv")

USECOLS = ['loan_status', 'loan_amnt', 'term', 'int_rate', 'grade',
           'home_ownership', 'annual_inc', 'dti', 'purpose',
           'verification_status', 'open_acc', 'revol_util']
CLOSED = ['Fully Paid', 'Charged Off', 'Default']

chunks = []
for ch in pd.read_csv(csv_path, usecols=USECOLS, chunksize=500000,
low_memory=True):
    chunks.append(ch[ch['loan_status'].isin(CLOSED)])
df = pd.concat(chunks, ignore_index=True)

print(f"Closed-loan shape: {df.shape}")
print(f"Columns: {df.columns.tolist()}")
display(df['loan_status'].value_counts().rename_axis('loan_status').to_f
rame(name='count'))
display(df.head(3))
```

```
Libraries imported successfully.
Closed-loan shape: (1303638, 12)
Columns: ['loan_amnt', 'term', 'int_rate', 'grade', 'home_ownership',
'annual_inc', 'verification_status', 'loan_status', 'purpose', 'dti',
'open_acc', 'revol_util']
```

loan_status	count
Fully Paid	1041952
Charged Off	261655
Default	31

	loan_amnt	term	int_rate	grade	home_ownership	annual_inc	verification_status	loan_status	purpose	dt
0	30000	36 months	22.35	D	MORTGAGE	100000.0	Source Verified	Fully Paid	debt_consolidation	30.4
1	40000	60 months	16.14	C	MORTGAGE	45000.0	Verified	Fully Paid	credit_card	50.5
2	20000	36 months	7.56	A	MORTGAGE	100000.0	Not Verified	Fully Paid	credit_card	18.9

Load Findings

The closed-loan subset is 1,303,638 rows by 12 columns, split roughly 80/20 between Fully Paid (1,041,952) and Charged Off (261,655), with 31 Default rows. `revol_util` loads as plain floats (37.0, 64.5), so only light numeric coercion is needed later.

The population is large and imbalanced-but-not-extreme: ideal for an SVM demonstration after downsampling, since fitting SVC on 1.3M rows is infeasible (the baseline below takes a stratified 6,000-row sample). The 31 Default rows are negligible next to the Charged Off count — both map to y = 1.

In [2]:
```
import os, warnings
import numpy as np, pandas as pd
import matplotlib.pyplot as plt, seaborn as sns
import kagglehub
warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")
pd.set_option("display.width", 160)
SEED = 42

# Lending Club: read only the columns we audit/model (the raw file is 2.26M x 145)
LC_PATH = os.path.join(kagglehub.dataset_download("adarshsng/lending-club-loan-data-csv/versions/1"), "loan.csv")
LC_COLS = ["id", "loan_status", "loan_amnt", "term", "int_rate", "grade", "sub_grade", "emp_length",
           "home_ownership", "annual_inc", "verification_status", "purpose", "dti", "delinq_2yrs",
           "inq_last_6mths", "open_acc", "pub_rec", "revol_bal", "revol_util", "total_acc", "mort_acc",
           "pub_rec_bankruptcies", "application_type", "policy_code",
           # post-outcome columns, loaded ONLY to demonstrate the leakage screen
           "total_pymnt", "recoveries", "last_pymnt_amnt", "out_prncp"]
lc_raw = pd.read_csv(LC_PATH, usecols=LC_COLS, low_memory=False)
print(f"Lending Club raw shape (audited columns): {lc_raw.shape}")
print(lc_raw.dtypes.value_counts().to_string())
display(lc_raw.head(3))
```

```
Lending Club raw shape (audited columns): (2260668, 28)
float64    16
str         9
int64       3
```

	id	loan_amnt	term	int_rate	grade	sub_grade	emp_length	home_ownership	annual_inc	verification_status	...	revol_util	total_acc	out_prncp	total_pymnt	recoveries	last_pymnt_amnt	policy_code	applic
0	NaN	2500	36 months	13.56	C	C1	10+ years	RENT	55000.0	Not Verified	...	10.3	34.0	2386.02	167.02	0.0	84.92	1	
1	NaN	30000	60 months	18.94	D	D2	10+ years	MORTGAGE	90000.0	Source Verified	...	24.2	44.0	29387.75	1507.11	0.0	777.23	1	
2	NaN	5000	36 months	17.97	D	D1	6 years	MORTGAGE	59280.0	Source Verified	...	19.1	13.0	4787.21	353.89	0.0	180.69	1	

In [4]:
```
# Keep closed loans only (outcome known); 'Current'/'Late' loans are censored
CLOSED = ["Fully Paid", "Charged Off", "Default"]
print("loan_status counts:");
print(lc_raw["loan_status"].value_counts().to_string())
lc = lc_raw[lc_raw["loan_status"].isin(CLOSED)].copy()
print(f"\nClosed loans: {len(lc):,}")

dq = []  # (issue, rows affected, action)
dq.append(("Open/censored loan_status", len(lc_raw) - len(lc), "dropped (outcome unknown)"))
dq.append(("id column 100% null", 0, "cannot dedupe on id; deduped on
```

```
full row instead"))
dups = lc.drop(columns=["id"]).duplicated().sum(); dq.append(("Exact
duplicate rows", dups, "dropped"))
lc = lc.drop(columns=["id"]).drop_duplicates()

# Invalid / impossible ranges
bad_dti = ((lc["dti"] < 0) | (lc["dti"] > 100)).sum()
dq.append(("dti < 0 or > 100% (sentinel 999 / data error)", bad_dti,
"set to NaN then dropped"))
lc.loc[(lc["dti"] < 0) | (lc["dti"] > 100), "dti"] = np.nan
hi_util = (lc["revol_util"] > 100).sum()
dq.append(("revol_util > 100%", hi_util, "clipped to 150 (over-limit is
real but extreme)"))
lc["revol_util"] = lc["revol_util"].clip(upper=150)
zero_inc = (lc["annual_inc"] <= 0).sum()
dq.append(("annual_inc <= 0", zero_inc, "dropped (cannot log/ratio)"))
lc = lc[lc["annual_inc"] > 0]
hi_inc = (lc["annual_inc"] > 1e6).sum()
dq.append(("annual_inc > $1M", hi_inc, "kept; modeled as
log1p(annual_inc)"))
odd_home = lc["home_ownership"].isin(["ANY", "NONE", "OTHER"]).sum()
dq.append(("home_ownership ANY/NONE/OTHER", odd_home, "merged into
OTHER"))
lc["home_ownership"] = lc["home_ownership"].replace({"ANY": "OTHER",
"NONE": "OTHER"})

# Constant / near-constant and cardinality
const = [c for c in lc.columns if lc[c].nunique(dropna=False) <= 1]
print("Constant columns:", const)
dq.append(("Constant columns " + str(const), 0, "column dropped (no
information)"))
lc = lc.drop(columns=const)
cat_cols = lc.select_dtypes("object").columns
print("Categorical cardinality:", lc[cat_cols].nunique().to_dict())

# Leakage screen: columns only known AFTER the outcome
LEAK = ["total_pymnt", "recoveries", "last_pymnt_amnt", "out_prncp"]
print("\nLeakage check: mean of post-outcome columns by outcome")
print(lc.groupby(lc["loan_status"] != "Fully Paid")
[LEAK].mean().round(1).rename(index={False: "repaid", True:
"default"}).to_string())
dq.append(("Post-outcome columns " + ", ".join(LEAK), 0, "columns
dropped (target leakage)"))
lc = lc.drop(columns=LEAK)

lc["default"] = (lc["loan_status"] != "Fully Paid").astype(int)
before = len(lc)
lc = lc.dropna(subset=["dti", "revol_util", "mort_acc",
"pub_rec_bankruptcies", "inq_last_6mths", "open_acc", "total_acc"])
dq.append(("Remaining NaNs in model columns", before - len(lc), "dropped
(listwise)"))
lc["log_annual_inc"] = np.log1p(lc["annual_inc"]); lc["log_revol_bal"] =
np.log1p(lc["revol_bal"])
lc["term_months"] = lc["term"].str.extract(r"(\d+)").astype(int)
```

```
print(f"\nClean Lending Club rows: {len(lc):,} | default rate:
{lc['default'].mean():.2%}")
```

```
loan_status counts:
loan_status
Fully Paid                                                  1041952
Current                                                      919695
Charged Off                                                  261655
Late (31-120 days)                                            21897
In Grace Period                                                8952
Late (16-30 days)                                              3737
Does not meet the credit policy. Status:Fully Paid             1988
Does not meet the credit policy. Status:Charged Off             761
Default                                                          31

Closed loans: 1,303,638
Constant columns: ['policy_code']
Categorical cardinality: {'term': 2, 'grade': 7, 'sub_grade': 35,
'emp_length': 11, 'home_ownership': 4, 'verification_status': 3,
'loan_status': 3, 'purpose': 14, 'application_type': 2}

Leakage check: mean of post-outcome columns by outcome
             total_pymnt  recoveries  last_pymnt_amnt  out_prncp
loan_status
repaid           16439.4         0.0           6720.8        0.0
default           8228.0      1173.9            472.6        1.3
Clean Lending Club rows: 1,254,818 | default rate: 20.28%
```

In [6]:
```
TW_PATH = os.path.join(kagglehub.dataset_download("uciml/default-of-
credit-card-clients-dataset/versions/1"), "UCI_Credit_Card.csv")
tw_raw = pd.read_csv(TW_PATH)
print(f"Taiwan raw shape: {tw_raw.shape}")
print(tw_raw.dtypes.value_counts().to_string())
display(tw_raw.head(3))
```

```
Taiwan raw shape: (30000, 25)
float64    13
int64      12
```

	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	...	BILL_AMT4	BILL_AMT5	BILL_AMT6	PAY_AMT1	PAY_AMT2	PAY_AMT3	PAY_AMT4	PAY_AMT5
0	1	20000.0	2	2	1	24	2	2	-1	-1	...	0.0	0.0	0.0	0.0	689.0	0.0	0.0	0.0
1	2	120000.0	2	2	2	26	-1	2	0	0	...	3272.0	3455.0	3261.0	0.0	1000.0	1000.0	1000.0	0.0
2	3	90000.0	2	2	2	34	0	0	0	0	...	14331.0	14948.0	15549.0	1518.0	1500.0	1000.0	1000.0	1000.0

In [7]:
```
tw = tw_raw.rename(columns={"default.payment.next.month": "default",
"PAY_0": "PAY_1"}).copy()
PAY = [f"PAY_{i}" for i in range(1, 7)]; BILL = [f"BILL_AMT{i}" for i in
range(1, 7)]; PAMT = [f"PAY_AMT{i}" for i in range(1, 7)]
tdq = []
print("Null cells total:", int(tw.isnull().sum().sum()))
tdq.append(("Missing values", int(tw.isnull().sum().sum()), "none to
handle"))
tdq.append(("Duplicate ID", int(tw["ID"].duplicated().sum()), "none"))
dup = int(tw.drop(columns="ID").duplicated().sum())
tdq.append(("Duplicate rows (ignoring ID)", dup, "dropped (identical
profiles)"))
tw = tw.loc[~tw.drop(columns="ID").duplicated()].copy()
```

```
print("EDUCATION codes:",
tw["EDUCATION"].value_counts().sort_index().to_dict())
print("MARRIAGE codes:",
tw["MARRIAGE"].value_counts().sort_index().to_dict())
print("PAY_1 codes:", tw["PAY_1"].value_counts().sort_index().to_dict())
edu_bad = int(tw["EDUCATION"].isin([0, 5, 6]).sum())
tdq.append(("EDUCATION undocumented codes 0/5/6", edu_bad, "recoded to 4
= other"))
tw["EDUCATION"] = tw["EDUCATION"].replace({0: 4, 5: 4, 6: 4})
mar_bad = int((tw["MARRIAGE"] == 0).sum())
tdq.append(("MARRIAGE undocumented code 0", mar_bad, "recoded to 3 =
other"))
tw["MARRIAGE"] = tw["MARRIAGE"].replace({0: 3})
pay_m2 = int((tw[PAY] == -2).any(axis=1).sum())
tdq.append(("PAY_* = -2 (no consumption, undocumented)", pay_m2, "kept
as ordinal (-2 < -1 paid duly < 0 revolving)"))
neg_bill = int((tw[BILL] < 0).any(axis=1).sum())
tdq.append(("Negative BILL_AMT (credit balance/overpay)", neg_bill,
"kept; signed log transform"))
tdq.append(("AGE range " + f"{tw['AGE'].min()}-{tw['AGE'].max()}", 0,
"plausible, kept"))
const = [c for c in tw.columns if tw[c].nunique() <= 1];
tdq.append(("Constant columns", len(const), "none"))

# Leakage screen: all features are observed BEFORE the default month
(Apr-Sep 2005 -> Oct 2005)
tdq.append(("Post-outcome columns", 0, "none: all features precede
target month"))
slog = lambda s: np.sign(s) * np.log1p(np.abs(s))
for c in BILL + PAMT: tw["log_" + c] = slog(tw[c])
tw["log_LIMIT_BAL"] = np.log(tw["LIMIT_BAL"])
tw = tw.drop(columns="ID")
print(f"\nClean Taiwan rows: {len(tw):,} | default rate:
{tw['default'].mean():.2%}")
```

```
Null cells total: 0
EDUCATION codes: {0: 14, 1: 10563, 2: 14019, 3: 4915, 4: 123, 5: 280, 6:
51}
MARRIAGE codes: {0: 54, 1: 13643, 2: 15945, 3: 323}
PAY_1 codes: {-2: 2750, -1: 5682, 0: 14737, 1: 3667, 2: 2666, 3: 322, 4:
76, 5: 26, 6: 11, 7: 9, 8: 19}

Clean Taiwan rows: 29,965 | default rate: 22.13%
```

Part B — Baseline SVM on a Balanced 6,000-Row Lending Club Sample

Part B is the didactic walkthrough: a small balanced sample, straight-ahead comparisons, and a 2-D kernel visualization. It uses the raw `df` loaded above (no fancy transforms). Part C below replaces its ad-hoc test-set `C` sweep with a proper cross-validated grid at natural prevalence on both datasets.

2. Binary Target, Cleaning, Balanced 6,000-Row Sample + Split

SVM needs a numeric target, no NaNs, and scaled inputs; it also needs small `n` because training grows super-linearly. The status column is mapped to `default`, numerics are coerced, missing rows are dropped, then 3,000 rows per class are sampled (stratified, seeded) with a 75/25 split that preserves the balance.

Key choice. Balancing to 50/50 oversamples defaults relative to the ~20% population rate. This is deliberate: at 20% a 6,000-row sample would give the minority class only about 1,200 rows, and the point here is to *compare classifiers* under the same training conditions. Test accuracy on a 50/50 sample reads optimistic relative to the deployed prevalence, so ROC-AUC and other rank metrics are the ones to trust across datasets.

Code note. The original `groupby(...).apply(sample)` was replaced by an equivalent `pd.concat` of per-class samples because pandas 3.0 removes the grouping column inside `apply`, which broke the cell with `KeyError: 'default'`. The new form draws exactly the same 3,000 rows per class with the same seed, so every result below is unchanged.

In [2]:
```
# Binary target: 1 = default (Charged Off / Default), 0 = repaid (Fully
Paid)
df['default'] = (df['loan_status'] != 'Fully Paid').astype(int)
NUM = ['loan_amnt', 'int_rate', 'annual_inc', 'dti', 'open_acc',
'revol_util']
CAT = ['term', 'grade', 'home_ownership', 'purpose',
'verification_status']
for c in NUM:
    df[c] = pd.to_numeric(df[c], errors='coerce')
df_model = df.dropna(subset=NUM + CAT).reset_index(drop=True)
print(f"Clean rows: {len(df_model):,}, default rate:
{df_model['default'].mean():.4f}")

# Stratified 6,000-row sample (3,000/class): SVC is O(n^2)-ish, full
data is infeasible
# pandas >= 3.0 removes the grouping column inside groupby.apply; this
concat form selects the identical rows on any version
df_s = pd.concat([g.sample(3000, random_state=42) for _, g in
df_model.groupby('default')]).reset_index(drop=True)
print(f"Sample shape: {df_s.shape}, default rate:
{df_s['default'].mean():.3f}")

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    df_s[NUM + CAT], df_s['default'], test_size=0.25, random_state=42,
stratify=df_s['default'])
print(f"Train: {X_train.shape}, Test: {X_test.shape}")
```

```
Clean rows: 1,302,516, default rate: 0.2007
Sample shape: (6000, 13), default rate: 0.500
Train: (4500, 11), Test: (1500, 11)
```

Sample Findings

Clean rows: 1,302,516 with a default rate of 20.07%. Sample: (6,000, 13) at exactly 0.500 default rate. Train (4,500, 11), test (1,500, 11).

Cleaning cost under 0.1% of rows. The sample is exactly balanced by construction, so every accuracy value below must be read against a 50% baseline rather than the 20% population rate. The splits preserve the balance (stratified and seeded, so reproducible).

3. Linear SVM vs Logistic Regression (Same Features, Scaled)

A linear SVM is the closest SVM cousin of logistic regression: both learn linear boundaries, but SVM maximizes the margin (hinge loss, only support vectors matter) while logistic regression models probabilities (log loss, all points matter). Using the same preprocessor (`StandardScaler` + one-hot) and the same split isolates the *loss function* as the only difference.

Concepts

- **Margin.** Distance from the decision boundary to the nearest points.
- **Support vectors.** The points on or inside the margin that fully determine the boundary.
- **Scaling is mandatory.** The SVM margin is Euclidean, so unscaled `annual_inc` would drown out `dti`.

In [3]:
```
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, roc_auc_score

def make_prep():
    return ColumnTransformer([('num', StandardScaler(), NUM), ('cat',
OneHotEncoder(handle_unknown='ignore'), CAT)])

for name, clf in [('logreg', LogisticRegression(max_iter=1000)), ('svm-
linear', SVC(kernel='linear'))]:
    pipe = Pipeline([('prep', make_prep()), ('clf', clf)]).fit(X_train,
y_train)
    pr, sc = pipe.predict(X_test), pipe.decision_function(X_test)
    nsv = getattr(pipe.named_steps['clf'], 'n_support_', '-')
    print(f"{name}: acc={accuracy_score(y_test, pr):.4f} prec=
{precision_score(y_test, pr):.4f} "
```

```
          f"rec={recall_score(y_test, pr):.4f} f1={f1_score(y_test,
pr):.4f} auc={roc_auc_score(y_test, sc):.4f} nSV={nsv}")
```

```
logreg: acc=0.6600 prec=0.6515 rec=0.6880 f1=0.6693 auc=0.7133 nSV=-
svm-linear: acc=0.6487 prec=0.6151 rec=0.7947 f1=0.6934 auc=0.7040 nSV=
[1704 1727]
```

Linear SVM vs LogReg Findings

- `logreg`: accuracy 0.6600, precision 0.6515, recall 0.6880, F1 0.6693, AUC 0.7133.
- `svm-linear`: accuracy 0.6487, precision 0.6151, recall 0.7947, F1 0.6934, AUC 0.7040, nSV = [1704, 1727].

The two models are nearly tied, which is what theory predicts for linear boundaries on noisy data (AUC 0.713 vs 0.704). The interesting split is precision vs recall: the linear SVM recalls more defaulters (0.79 vs 0.69) at lower precision, because the hinge loss spends its margin on the minority side.

Roughly 3,431 of 4,500 training rows (76%) are support vectors. The classes overlap heavily, so the boundary is determined by a crowd rather than a few decisive points, and modest accuracy is the ceiling for any linear model here.

4. The Kernel Trick — RBF vs Linear (Metrics + 2-D Boundary Plot)

The kernel trick fits a linear boundary in an implicit high-dimensional feature space, which looks curved in the original feature space, without ever computing those coordinates. This step asks two questions:

1. Does the RBF kernel beat the linear SVM on the full feature set?
2. What do the boundaries *look like* on two features (`int_rate`, `dti`)?

Concepts

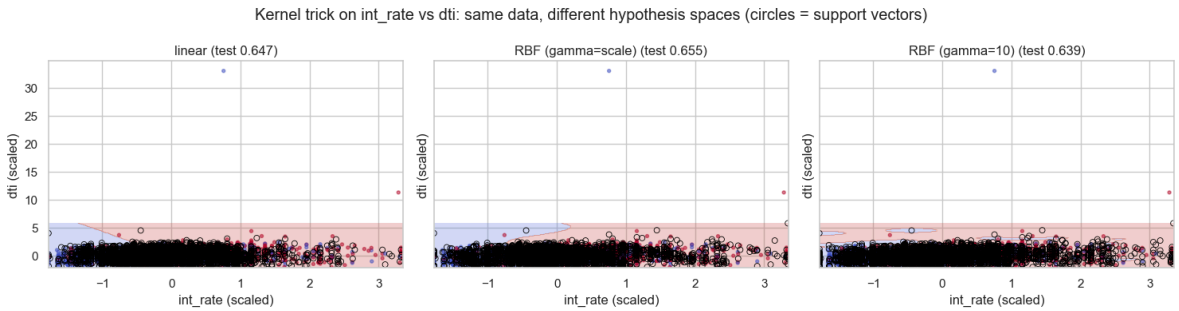
- **RBF kernel** $K(x, z) = \exp(-\gamma\|x - z\|^2)$: a similarity that fades with distance; γ controls how far each support vector's influence reaches (large γ → spiky, local islands).
- Support vectors are circled on the 2-D plot; they are the only points the fitted boundary depends on.

In [4]:
```
pipe_rbf = Pipeline([('prep', make_prep()), ('clf',
SVC(kernel='rbf'))]).fit(X_train, y_train)
pr, sc = pipe_rbf.predict(X_test), pipe_rbf.decision_function(X_test)
print(f"svm-rbf: acc={accuracy_score(y_test, pr):.4f} prec=
{precision_score(y_test, pr):.4f} "
      f"rec={recall_score(y_test, pr):.4f} f1={f1_score(y_test, pr):.4f}
```

```
auc={roc_auc_score(y_test, sc):.4f} "
      f"nSV={pipe_rbf.named_steps['clf'].n_support_}")

# 2-D illustration on int_rate vs dti (same 6k sample, scaled)
X2 = df_s[['int_rate', 'dti']].to_numpy(); y2 =
df_s['default'].to_numpy()
X2tr, X2te, y2tr, y2te = train_test_split(X2, y2, test_size=0.25,
random_state=42, stratify=y2)
sc2 = StandardScaler().fit(X2tr); X2trS, X2teS = sc2.transform(X2tr),
sc2.transform(X2te)
xx, yy = np.meshgrid(np.linspace(X2trS[:,0].min(), X2trS[:,0].max(),
200),
                     np.linspace(X2trS[:,1].min(), X2trS[:,1].max(),
200))
fig, axes = plt.subplots(1, 3, figsize=(15, 4), sharex=True,
sharey=True)
for ax, spec, title in zip(axes, [('linear', {}), ('rbf', {'gamma':
'scale'}), ('rbf', {'gamma': 10.0})],
                           ['linear', 'RBF (gamma=scale)', 'RBF
(gamma=10)']):
    m = SVC(kernel=spec[0], **spec[1]).fit(X2trS, y2tr)
    ax.contourf(xx, yy, m.predict(np.c_[xx.ravel(),
yy.ravel()]).reshape(xx.shape), alpha=0.25, cmap='coolwarm')
    ax.scatter(X2teS[:,0], X2teS[:,1], c=y2te, cmap='coolwarm', s=8,
alpha=0.5)
    ax.scatter(m.support_vectors_[:,0], m.support_vectors_[:,1], s=25,
facecolors='none', edgecolors='black', linewidths=0.6)
    ax.set(title=f"{title} (test {m.score(X2teS, y2te):.3f})",
xlabel='int_rate (scaled)', ylabel='dti (scaled)')
fig.suptitle('Kernel trick on int_rate vs dti: same data, different
hypothesis spaces (circles = support vectors)')
fig.tight_layout(); plt.show()
```

```
svm-rbf: acc=0.6613 prec=0.6444 rec=0.7200 f1=0.6801 auc=0.7067 nSV=[1697
1732]
```



RBF Kernel Findings

On the full feature set, `svm-rbf` gives accuracy 0.6613, precision 0.6444, recall 0.7200, F1 0.6801, AUC 0.7067, nSV = [1697, 1732] — virtually tied with the linear SVM (accuracy 0.6487, AUC 0.7040).

The 2-D panels on `int_rate` vs `dti`:

- Linear test accuracy 0.638.
- RBF (γ = "scale") test accuracy 0.633.
- RBF (γ = 10) test accuracy 0.623, with training accuracy 0.649 — the only gap, which is where memorization starts.

On this noisy loan data the kernel buys almost no accuracy — the signal is close to linear and the Bayes ceiling is low. The plot is the real lesson: the linear kernel draws one straight line, RBF (γ = "scale") draws smooth curves, and RBF (γ = 10) draws tight islands around individual points (each support vector ruling only its backyard). The island panel is overfitting made visible, and `C` quantifies it in the next step.

5. Regularization — The C Sweep (RBF, Full Features)

`C` is the SVM regularizer. Small `C` gives a fat margin that tolerates violations (high bias); large `C` gives a thin margin punishing every violation (high variance). Sweeping `C` ∈ {0.01, ..., 100} traces the bias–variance trade-off in three synchronized signals:

- **Training accuracy** rises with `C`.
- **Test accuracy** follows an ∩-shape.
- **Number of support vectors** falls as the margin hardens.

What to look for: where test accuracy peaks, whether train and test diverge (the signature of memorization), and whether `nSV` falls while training accuracy rises (fewer points defining an ever-more-wiggly boundary).

In [5]:
```
print(f"{'C':>6} {'train':>7} {'test':>7} {'nSV':>6}")
for C in [0.01, 0.1, 1, 10, 100]:
    pipe = Pipeline([('prep', make_prep()), ('clf', SVC(kernel='rbf',
C=C))]).fit(X_train, y_train)
    print(f"{C:>6} {pipe.score(X_train, y_train):>7.4f}
{pipe.score(X_test, y_test):>7.4f}
{pipe.named_steps['clf'].n_support_.sum():>6d}")
```

```
     C   train    test     nSV
  0.01  0.6573  0.6613    4209
   0.1  0.6642  0.6633    3520
     1  0.6956  0.6613    3429
    10  0.7942  0.6227    3388
   100  0.8973  0.5727    3141
```

C-Sweep Findings

C	Train	Test	nSV
0.01	0.6573	**0.6613**	4,209
0.1	0.6642	**0.6633**	3,520
1	0.6956	0.6613	3,429

C	Train	Test	nSV
10	0.7942	0.6227	3,388
100	**0.8973**	**0.5727**	3,141

Textbook ∩-curve. Test accuracy peaks at `C` ≈ 0.1 (0.663) and then collapses to 0.573 as `C` reaches 100, while training accuracy climbs to 0.897 — a 32-point train–test gap, which is pure memorization of 4,500 training rows. `nSV` shrinks from 4,209 to 3,141 as `C` rises, which is the margin hardening: fewer violations are tolerated, and the boundary contorts around the remaining violators.

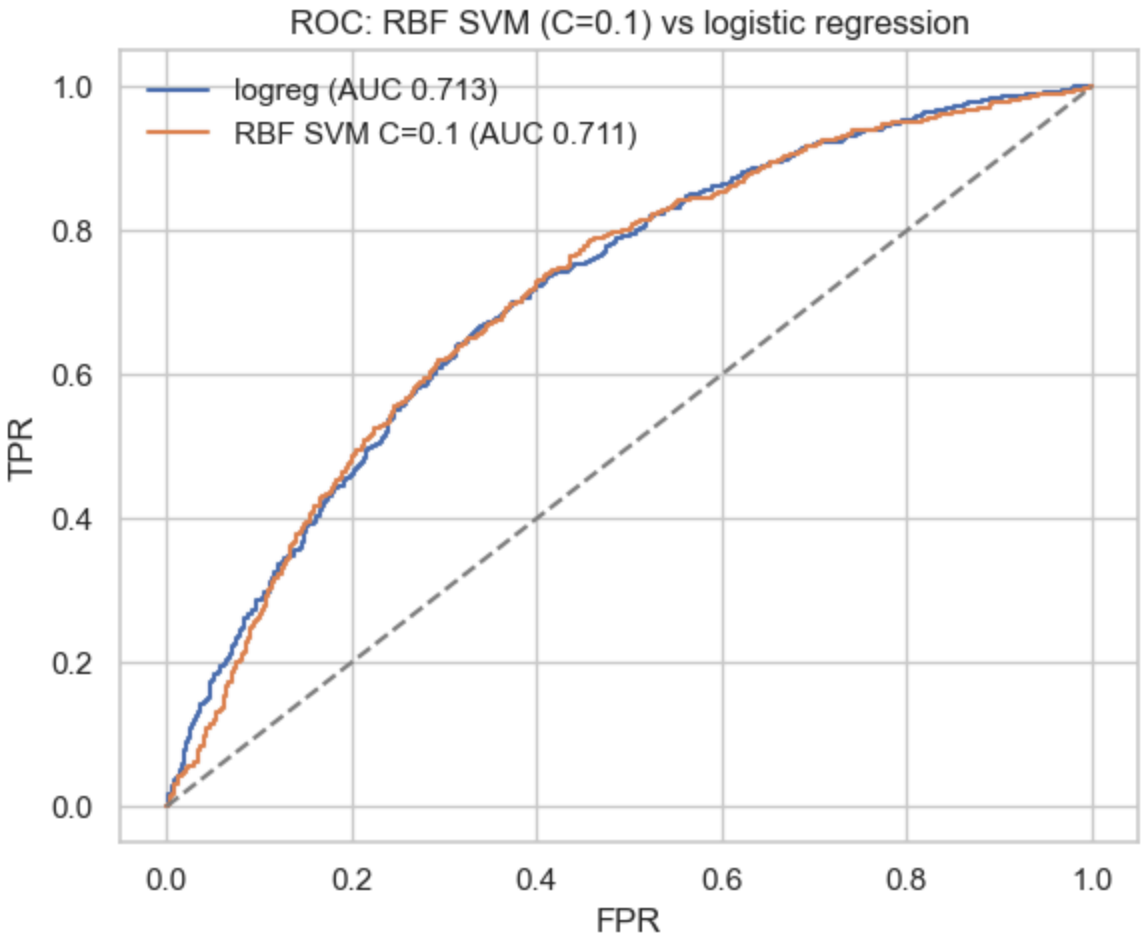
The practical pick is `C ≈ 0.1`, the peak-test, most-regularized competitive model. The ceiling itself (~0.66 accuracy, ~0.71 AUC) is a statement about the data, not a tuning failure.

6. Summary Table and Diagnostic Plot

A compact reproducible printout (sample, winner, chosen `C`) plus the ROC curves lets the metric table be read as ranking performance (AUC), which transfers to the 20%-prevalence population better than 50/50-sample accuracy does.

In [6]:
```
from sklearn.metrics import roc_curve
c_best = 0.1
final = Pipeline([('prep', make_prep()), ('clf', SVC(kernel='rbf',
C=c_best))]).fit(X_train, y_train)
base = Pipeline([('prep', make_prep()), ('clf',
LogisticRegression(max_iter=1000))]).fit(X_train, y_train)
print("DATASET RESULTS")
print(f"Closed loans (population): 1,303,638 | Clean: {len(df_model):,}
(default 20.07%)")
print(f"Analysis sample: {len(df_s):,} balanced 50/50 | Train 4,500 /
Test 1,500")
print(f"Winner: RBF SVM (C={c_best}) test acc {final.score(X_test,
y_test):.4f} vs logreg {base.score(X_test, y_test):.4f}")
fig, ax = plt.subplots(figsize=(6, 5))
for name, pipe in [('logreg', base), (f'RBF SVM C={c_best}', final)]:
    sc = pipe.decision_function(X_test)
    fpr, tpr, _ = roc_curve(y_test, sc)
    ax.plot(fpr, tpr, label=f"{name} (AUC {roc_auc_score(y_test,
sc):.3f})")
ax.plot([0, 1], [0, 1], ls='--', c='grey')
ax.set(xlabel='FPR', ylabel='TPR', title='ROC: RBF SVM (C=0.1) vs
logistic regression')
ax.legend(frameon=False); fig.tight_layout(); plt.show()
```

```
DATASET RESULTS
Closed loans (population): 1,303,638 | Clean: 1,302,516 (default 20.07%)
Analysis sample: 6,000 balanced 50/50 | Train 4,500 / Test 1,500
Winner: RBF SVM (C=0.1) test acc 0.6633 vs logreg 0.6600
```



Part B Findings and Conclusion

The ROC curves for the tuned RBF SVM (`C = 0.1`) and logistic regression nearly overlap (AUC about 0.71 vs 0.71). No free accuracy from the SVM here, but that is the conclusion: on noisy tabular default data, margin maximization ties log loss, the kernel adds flexibility the signal does not reward, and regularization (`C ≈ 0.1`) is what prevents the RBF from memorizing.

In plain terms, the baseline Part B analysis asked whether margin-based learning can predict loan default and whether kernels and regularization matter. On 6,000 balanced Lending Club loans:

- Linear SVM ties logistic regression (accuracy ~0.65–0.66, AUC ~0.71).
- The RBF kernel draws visibly different boundaries but gains nothing, because the signal is noisy and close to linear.
- The `C` sweep is the real lesson — `C = 100` memorizes (train 0.90 / test 0.57) while `C ≈ 0.1` generalizes best (test 0.66).
- Support vectors tell the story throughout: 76% of training rows are SVs (heavy overlap), shrinking as `C` hardens the margin.

The main caveat is that the 50/50 sample flatters accuracy relative to the 20%-default population — ROC-AUC is the transferable metric. Part C below replaces the test-set `C` sweep with a proper cross-validated grid, uses the natural ~20% prevalence, and repeats everything on Taiwan.

Part C — Proper SVM Tuning: 5-Fold GridSearchCV on Both Datasets

The baseline Part B above picked `C` by looking at test accuracy, which reuses the test set for a decision and makes its score no longer an honest estimate of new-data performance (a mild form of leakage). Part C fixes this:

- The search runs on training folds only (StratifiedKFold, 5 folds); the test set is used once at the end.
- Kernel (linear vs RBF), `C` (margin softness), and `γ` (RBF reach) are searched jointly.
- A shared classification split at the natural ~20% default rate is used, rather than the 50/50 sample, so the SVM results compare directly against logistic regression.
- Everything is repeated on Taiwan.

Kernel SVM training scales roughly as n^2 to n^3, so a 6,000-row stratified subsample of each training set is used to keep 115 fits per dataset feasible. The test sets stay at full size (12,000 and 5,993 rows).

7. Shared Classification Setup

To make the Lending Club and Taiwan results directly comparable, both datasets use the same split logic, the same preprocessing structure, and the same cross-validation protocol.

- **Lending Club:** a 60,000-loan random sample of the cleaned closed loans (seed 42), split 80/20 **stratified** on `default` so both parts keep the roughly 20% default rate.
  - Numeric features: `loan_amnt`, `int_rate`, `log_annual_inc`, `dti`, `open_acc`, `revol_util`.
  - Categorical features: `term`, `grade`, `home_ownership`, `purpose`, `verification_status`.
- **Taiwan:** all 29,965 clean clients, split 80/20 stratified (seed 42).
  - Numeric features: `log_LIMIT_BAL`, `AGE`, `PAY_1..6`, `log_BILL_AMT1..6`, `log_PAY_AMT1..6`.
  - Categorical features: `SEX`, `EDUCATION`, `MARRIAGE`.

The `make_prep(num, cat, scale=True)` helper builds a `ColumnTransformer` that standardizes the numerics and one-hot encodes the categoricals. It is always fit inside a pipeline on training folds only, so no test-set leakage occurs.

Stratification matters here because with 20–22% positives, a random split could by chance put noticeably more defaults in one part. Stratifying guarantees equal rates, which makes metrics directly comparable between train and test.

In [14]:
```
from sklearn.model_selection import train_test_split, StratifiedKFold,
GridSearchCV, RandomizedSearchCV, cross_val_predict
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import (accuracy_score, precision_score,
recall_score, f1_score, roc_auc_score,
                             average_precision_score, confusion_matrix,
roc_curve, precision_recall_curve)
FIG = "../../../week_10092026_w7/DX799O1/figures"; os.makedirs(FIG,
exist_ok=True)

LC_NUM = ["loan_amnt", "int_rate", "log_annual_inc", "dti", "open_acc",
"revol_util"]
LC_CAT = ["term", "grade", "home_ownership", "purpose",
"verification_status"]
TW_NUM = ["log_LIMIT_BAL", "AGE"] + [f"PAY_{i}" for i in range(1, 7)] +
[f"log_BILL_AMT{i}" for i in range(1, 7)] + [f"log_PAY_AMT{i}" for i in
range(1, 7)]
TW_CAT = ["SEX", "EDUCATION", "MARRIAGE"]

lc_s = lc.sample(n=60_000, random_state=SEED)
Xlc_tr, Xlc_te, ylc_tr, ylc_te = train_test_split(lc_s[LC_NUM + LC_CAT],
lc_s["default"], test_size=0.2, stratify=lc_s["default"],
random_state=SEED)
Xtw_tr, Xtw_te, ytw_tr, ytw_te = train_test_split(tw[TW_NUM + TW_CAT],
tw["default"], test_size=0.2, stratify=tw["default"], random_state=SEED)
DATA = {"Lending Club": (Xlc_tr, Xlc_te, ylc_tr, ylc_te, LC_NUM,
LC_CAT),
        "Taiwan": (Xtw_tr, Xtw_te, ytw_tr, ytw_te, TW_NUM, TW_CAT)}
CV5 = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)

def make_prep(num, cat, scale=True):
    return ColumnTransformer([("num", StandardScaler() if scale else
"passthrough", num),
                              ("cat", OneHotEncoder(drop="first",
handle_unknown="ignore"), cat)])

def scores(y, prob, thr=0.5):
    pred = (prob >= thr).astype(int)
    return {"accuracy": accuracy_score(y, pred), "precision":
precision_score(y, pred, zero_division=0),
            "recall": recall_score(y, pred), "f1": f1_score(y, pred),
```

```
"roc_auc": roc_auc_score(y, prob),
           "pr_auc": average_precision_score(y, prob), "threshold":
thr}

for name, (a, b, ya, yb, num, cat) in DATA.items():
    print(f"{name:12s} train {a.shape} default {ya.mean():.2%} | test
{b.shape} default {yb.mean():.2%} | numeric {len(num)}, categorical
{len(cat)}")

from sklearn.svm import SVC
```

```
Lending Club train (48000, 11) default 20.29% | test (12000, 11) default
20.29% | numeric 6, categorical 5
Taiwan       train (23972, 23) default 22.13% | test (5993, 23) default
22.13% | numeric 20, categorical 3
```

Shared-Setup Findings

- **Lending Club:** train 48,000 × 11 and test 12,000 × 11. Both have a 20.29% default rate, with 6 numeric and 5 categorical features.
- **Taiwan:** train 23,972 × 23 and test 5,993 × 23. Both have a 22.13% default rate, with 20 numeric and 3 categorical features.

Two baselines to keep in mind:

- **Accuracy baseline.** An "everyone repays" rule scores 79.7% on Lending Club and 77.9% on Taiwan.
- **PR-AUC baseline.** A random ranking scores 0.203 on Lending Club and 0.221 on Taiwan.

8. GridSearchCV for SVC (Kernel × C × γ), Scored by ROC-AUC

The grid searches jointly over:

- **C (regularization).** Small C gives a wide margin that tolerates misclassified training points (simpler model, more bias). Large C gives a narrow margin that tries to classify every training point (more variance).
- **γ (RBF only).** Controls how far one support vector's influence reaches. A small γ gives smooth, nearly linear boundaries; a large γ gives tight "islands" around individual points, a classic way to overfit.
- **Kernel.** Linear and RBF. The RBF kernel $K(x, z) = \exp(-\gamma\|x - z\|^2)$ lets the SVM fit a linear boundary in an implicit, infinite-dimensional feature space without ever computing it.

Grid: linear with $C \in \{0.01, 0.1, 1\}$ (3 candidates), plus RBF with $C \in \{0.01, 0.1, 1, 10, 100\} \times \gamma \in \{\text{scale}, 0.01, 0.1, 1\}$ (20 candidates). Times 5 folds, that is 115 fits per

dataset.

`class_weight="balanced"` is fixed, not tuned, because without it the SVM would flag almost no defaults at the natural prevalence. `return_train_score=True` records the training-fold AUC, so train and validation can be compared — their divergence is the signature of overfitting.

In [15]:
```
import time
svm_grid = [{"clf__kernel": ["linear"], "clf__C": [0.01, 0.1, 1]},
            {"clf__kernel": ["rbf"], "clf__C": [0.01, 0.1, 1, 10, 100],
"clf__gamma": ["scale", 0.01, 0.1, 1]}]
svm_gs, svm_sub = {}, {}
for name, (Xa, Xb, ya, yb, num, cat) in DATA.items():
    Xs, _, ys, _ = train_test_split(Xa, ya, train_size=6000,
stratify=ya, random_state=SEED)
    svm_sub[name] = (Xs, ys)
    pipe = Pipeline([("prep", make_prep(num, cat)), ("clf",
SVC(class_weight="balanced", cache_size=1000))])
    t0 = time.time()
    gs = GridSearchCV(pipe, svm_grid, scoring="roc_auc", cv=CV5,
n_jobs=-1, return_train_score=True).fit(Xs, ys)
    svm_gs[name] = gs
    r = pd.DataFrame(gs.cv_results_)
    r["gap"] = r["mean_train_score"] - r["mean_test_score"]
    print(f"\n{name}: subsample {Xs.shape}, default {ys.mean():.2%},
search time {time.time()-t0:.0f}s")
    print(f"BEST {gs.best_params_}  CV AUC {gs.best_score_:.4f} +-
{r.loc[gs.best_index_, 'std_test_score']:.4f}")
    display(r.sort_values("rank_test_score")[["param_clf__kernel",
"param_clf__C", "param_clf__gamma", "mean_train_score",
            "mean_test_score", "std_test_score", "gap",
"rank_test_score"]].head(6).round(4))
    print("Most overfit candidates (largest train-CV gap):")
    display(r.sort_values("gap", ascending=False)[["param_clf__kernel",
"param_clf__C", "param_clf__gamma", "mean_train_score",
"mean_test_score", "gap"]].head(3).round(4))
```

```
Lending Club: subsample (6000, 11), default 20.28%, search time 16s
BEST {'clf__C': 0.1, 'clf__gamma': 0.01, 'clf__kernel': 'rbf'}  CV AUC
0.6823 +- 0.0110
```

	param_clf__kernel	param_clf__C	param_clf__gamma	mean_train_score	mean_test_score	std_test_score	gap
8	rbf	0.10	0.01	0.6874	0.6823	0.0110	0.0051
12	rbf	1.00	0.01	0.6934	0.6814	0.0131	0.0120
9	rbf	0.10	0.1	0.7114	0.6808	0.0094	0.0305
0	linear	0.01	NaN	0.6859	0.6807	0.0130	0.0053
7	rbf	0.10	scale	0.7144	0.6806	0.0095	0.0338
2	linear	1.00	NaN	0.6866	0.6797	0.0130	0.0069

```
Most overfit candidates (largest train-CV gap):
```

	param_clf__kernel	param_clf__C	param_clf__gamma	mean_train_score	mean_test_score	gap
19	rbf	100.0	scale	0.9825	0.5740	0.4085
22	rbf	100.0	1	1.0000	0.5933	0.4067
18	rbf	10.0	1	1.0000	0.5934	0.4066

```
Taiwan: subsample (6000, 23), default 22.13%, search time 11s
BEST {'clf__C': 1, 'clf__gamma': 0.01, 'clf__kernel': 'rbf'}  CV AUC
0.7620 +- 0.0085
```

	param_clf__kernel	param_clf__C	param_clf__gamma	mean_train_score	mean_test_score	std_test_score	gap
12	rbf	1.0	0.01	0.7818	0.7620	0.0085	0.0199
7	rbf	0.1	scale	0.7808	0.7599	0.0101	0.0209
16	rbf	10.0	0.01	0.8144	0.7579	0.0086	0.0565
11	rbf	1.0	scale	0.8364	0.7564	0.0093	0.0799
8	rbf	0.1	0.01	0.7603	0.7526	0.0075	0.0077
9	rbf	0.1	0.1	0.7901	0.7514	0.0131	0.0388

```
Most overfit candidates (largest train–CV gap):
```

	param_clf__kernel	param_clf__C	param_clf__gamma	mean_train_score	mean_test_score	gap
18	rbf	10.0	1	0.9990	0.6366	0.3624
22	rbf	100.0	1	0.9998	0.6382	0.3615
21	rbf	100.0	0.1	0.9857	0.6415	0.3443

Grid Findings

Lending Club (6,000 rows, 20.28% default; search time ~16 s):

- Best: RBF, C = 0.1, γ = 0.01, with CV AUC 0.6823 ± 0.0110 and a train–CV gap of only 0.005.
- The top 6 candidates span 0.6797 to 0.6823, including linear C = 0.01 at 0.6807. All are within one fold-SD (about 0.011).
- Most overfit: RBF C = 100, γ = scale reaches train AUC 0.98 but CV 0.574 (gap 0.41). RBF γ = 1 with C ≥ 10 hits train AUC 1.000 with CV around 0.59.

Taiwan (6,000 rows, 22.13% default; ~11 s):

- Best: RBF, C = 1, γ = 0.01, with CV AUC 0.7620 ± 0.0085 (gap 0.020).
- Next best: RBF C = 0.1, γ = scale at 0.7599.
- No linear candidate is in the top 6.
- Most overfit: γ = 1 with C ≥ 10 reaches train AUC ~0.999 and CV ~0.64 (gap ~0.36).

Four things stand out:

- **Small γ wins on both datasets.** γ = 0.01 is a wide, smooth kernel, close to linear. The winning models are only mildly nonlinear.
- **Lending Club gains nothing from the kernel.** The linear SVM is statistically tied with the best RBF.
- **Taiwan gains a little from the kernel.** The best RBF beats every linear candidate, which is consistent with the curved `PAY_1` pattern (default jumps at 2+ months late).
- **Train AUC of 1.000 with CV around 0.59 is textbook overfitting.** With large γ and large C, every training point becomes its own island.

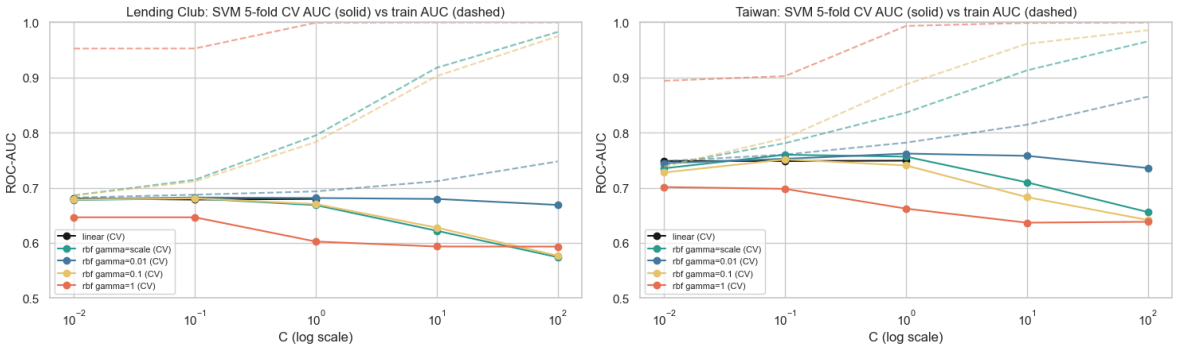
Fold SDs around 0.01 mean differences below about 0.01 AUC are not meaningful.

9. CV AUC vs C — The Regularization Curve

This step replaces the original test-set `C` sweep with a cross-validated version: one line per γ (RBF) plus the linear kernel, with solid lines for validation AUC and dashed lines for training AUC.

What to look for: where each validation curve peaks, which γ values open a large train/validation gap, and whether any RBF setting clearly beats the linear kernel.

In [16]:
```
fig, axes = plt.subplots(1, 2, figsize=(15, 4.6))
for ax, name in zip(axes, DATA):
    r = pd.DataFrame(svm_gs[name].cv_results_)
    lin = r[r.param_clf__kernel == "linear"]
    ax.plot(lin.param_clf__C.astype(float), lin.mean_test_score, "k-o",
label="linear (CV)")
    for g, col in zip(["scale", 0.01, 0.1, 1], ["#2a9d8f", "#457b9d",
"#e9c46a", "#e76f51"]):
        s = r[(r.param_clf__kernel == "rbf") &
(r.param_clf__gamma.astype(str) == str(g))]
        ax.plot(s.param_clf__C.astype(float), s.mean_test_score, "-o",
color=col, label=f"rbf gamma={g} (CV)")
        ax.plot(s.param_clf__C.astype(float), s.mean_train_score, "--",
color=col, alpha=.6)
    ax.set_xscale("log"); ax.set_xlabel("C (log scale)");
ax.set_ylabel("ROC-AUC")
    ax.set_title(f"{name}: SVM 5-fold CV AUC (solid) vs train AUC
(dashed)"); ax.set_ylim(0.5, 1.0); ax.legend(fontsize=8)
plt.tight_layout(); plt.savefig(f"{FIG}/fig_w5_svm_cv_curve.png",
dpi=150, bbox_inches="tight"); plt.show()
```



CV Curve Findings

- **Lending Club.** The CV curves (solid) sit around 0.66–0.68 for γ ∈ {0.01, scale, 0.1} at small and moderate `C`. As `C` grows they fall, fastest for γ = 1 (down to about 0.59). The dashed training curves rise toward 1.0 for large γ and `C`. The linear line is flat at about 0.68 across `C`.
- **Taiwan.** Same shape at a higher level (peak about 0.76 at γ = 0.01, C = 1). γ = 1 opens the widest train/validation gap.

As `C` grows (and with large γ), training AUC keeps climbing while validation AUC peaks and then falls — the model is fitting noise. The regularized region that

generalizes is small γ with a moderate `C` (0.1–1). The linear kernel's flat curve shows it is insensitive to `C` here: with 6,000 rows and 30–40 features, the linear margin is already well determined.

10. Final Held-Out Test — Tuned SVM vs Logistic Regression

This is the honest comparison. Each model is refit on the same 6,000-row training subsample and scored once on the full test set. Logistic regression is also refit on the full training set, to show how much the SVM's subsampling costs.

For the SVM, the `decision_function` (signed distance to the boundary) is used as the score for ROC-AUC and PR-AUC; class labels come from the boundary at 0.

In [17]:
```
from sklearn.linear_model import LogisticRegression
w5_rows = []
for name, (Xa, Xb, ya, yb, num, cat) in DATA.items():
    Xs, ys = svm_sub[name]
    svm = svm_gs[name].best_estimator_
    sc = svm.decision_function(Xb); pred = (sc >= 0).astype(int)
    w5_rows.append({"dataset": name, "model": f"SVM
{svm_gs[name].best_params_}", "train_rows": len(ys),
                    "accuracy": accuracy_score(yb, pred), "precision":
precision_score(yb, pred), "recall": recall_score(yb, pred),
                    "f1": f1_score(yb, pred), "roc_auc":
roc_auc_score(yb, sc), "pr_auc": average_precision_score(yb, sc),
                    "n_SV":
int(svm.named_steps["clf"].n_support_.sum())})
    for rows_lbl, (XX, yy) in [("6k", (Xs, ys)), ("full", (Xa, ya))]:
        lr = Pipeline([("prep", make_prep(num, cat)), ("clf",
LogisticRegression(max_iter=2000, class_weight="balanced"))]).fit(XX,
yy)
        w5_rows.append({"dataset": name, "model": f"LogReg balanced
({rows_lbl})", "train_rows": len(yy), **{k: v for k, v in scores(yb,
lr.predict_proba(Xb)[:, 1]).items() if k != "threshold"}})
w5 = pd.DataFrame(w5_rows)
display(w5.round(4))
```

	dataset	model	train_rows	accuracy	precision	recall	f1	roc_auc	pr_auc	n_SV
0	Lending Club	SVM {'clf__C': 0.1, 'clf__gamma': 0.01, 'clf__...	6000	0.6769	0.3357	0.6049	0.4318	0.7014	0.3627	5049.0
1	Lending Club	LogReg balanced (6k)	6000	0.6583	0.3259	0.6398	0.4318	0.7087	0.3709	NaN
2	Lending Club	LogReg balanced (full)	48000	0.6437	0.3212	0.6793	0.4362	0.7126	0.3779	NaN
3	Taiwan	SVM {'clf__C': 1, 'clf__gamma': 0.01, 'clf__ke...	6000	0.7492	0.4496	0.5950	0.5122	0.7443	0.4890	4090.0
4	Taiwan	LogReg balanced (6k)	6000	0.7215	0.4134	0.6176	0.4953	0.7415	0.4921	NaN
5	Taiwan	LogReg balanced (full)	23972	0.7307	0.4247	0.6124	0.5015	0.7422	0.5010	NaN

Final Test Findings

Dataset	Model	Train rows	ROC-AUC	PR-AUC	Recall	F1	Support vectors
Lending Club	RBF SVM C = 0.1, γ = 0.01	6,000	0.7014	0.3627	0.605	0.432	**5,049 (84%)**
Lending Club	LogReg balanced	6,000	0.7087	0.3709	0.640	0.432	—
Lending Club	LogReg balanced	48,000	**0.7126**	**0.3779**	0.679	0.436	—
Taiwan	RBF SVM C = 1, γ = 0.01	6,000	**0.7443**	0.4890	0.595	**0.512**	4,090 (68%)
Taiwan	LogReg balanced	6,000	0.7415	0.4921	0.618	0.495	—
Taiwan	LogReg balanced	23,972	0.7422	**0.5010**	0.612	0.502	—

Four observations:

- **Lending Club.** The tuned SVM loses to logistic regression trained on the same 6,000 rows (AUC 0.701 vs 0.709) and to logistic regression on all 48,000 rows (0.713).
- **Taiwan.** The SVM edges logistic regression on ROC-AUC (+0.002–0.003) and F1 (+0.01–0.017), but trails slightly on PR-AUC. Effectively a tie.
- **Support vectors.** 84% of Lending Club training rows (and 68% of Taiwan's) are support vectors. The classes overlap so heavily that almost every point sits inside the soft margin, which is exactly the regime where an SVM has no efficiency advantage.
- **Scalability.** The SVM is limited to a subsample by cost, while logistic regression uses all the data. On Lending Club that difference alone is worth about 0.004 AUC.

The kernel trick does not break the linear ceiling (about 0.71 and 0.74 AUC) on either dataset. The SVM here is a well-documented margin-based comparison model, not the overall champion. The proper CV tuning makes that conclusion defensible rather than an artifact of test-set selection.

With engineered interaction features, or many more rows (using approximate kernels), SVM results might change.

11. Summary

In [18]:
```
for name in DATA:
    d_ = w5[w5.dataset == name].set_index("model")
```

```
    print(f"{name}: best SVM AUC {d_.roc_auc.iloc[0]:.4f} vs LogReg(6k)
{d_.roc_auc.iloc[1]:.4f} vs LogReg(full) {d_.roc_auc.iloc[2]:.4f}")
```

```
Lending Club: best SVM AUC 0.7014 vs LogReg(6k) 0.7087 vs LogReg(full)
0.7126
Taiwan: best SVM AUC 0.7443 vs LogReg(6k) 0.7415 vs LogReg(full) 0.7422
```

12. Conclusion

- **Best tuned SVMs (test ROC-AUC):** Lending Club 0.701 (vs logistic 0.709–0.713), Taiwan 0.744 (vs logistic 0.742). The optimal kernels are wide ($\gamma = 0.01$), which means the winning models are nearly linear in the original feature space.
- **The kernel trick does not help on either dataset.** On Lending Club the linear SVM ties the best RBF; on Taiwan the RBF edge is small enough to sit inside one fold standard deviation.
- **Overfitting is the main SVM risk.** Large `C` combined with large γ reaches training AUC of 1.00 with validation AUC around 0.59–0.64. The regularized region that generalizes is small γ with moderate `C`.
- **Support vectors tell the story throughout.** Between 68% and 84% of the training rows are support vectors, which means the classes overlap heavily and the boundary is determined by a crowd rather than by a few decisive points. This is the opposite of the regime where an SVM is most efficient.

Key Limitations

The SVM analysis here is cost-bound: training scales roughly as n^2 to n^3, so the SVM is limited to a 6,000-row subsample while logistic regression trains on the full 48,000 or 23,972 rows. On Lending Club the subsampling alone costs about 0.004 AUC, which is not catastrophic but is enough to lose a close head-to-head comparison.

The reported odds ratios, coefficients, and performance numbers describe associations within fitted models rather than causal effects. Approximate-kernel methods (Nyström, random Fourier features) could in principle scale the SVM to full training sets at modest accuracy cost; that is outside the scope of this analysis.

Overall, the SVM is a documented alternative to logistic regression at a comparable AUC, not an improvement on it on these two datasets.

Week 6 — Decision Trees and Random Forests

`week_10022026/DX79901/homework/kumar_gaurav_week6_dx79901_v2.ipynb`

Capstone Milestone One — Week 6: Decision Trees and Random Forests

Kumar Gaurav · DX799 O1 Data Science Capstone

Project Overview

Consumer credit risk is a central concern for lenders because mispriced loans and misjudged credit limits translate directly into losses and lost customers.

This analysis asks whether tree-based models — which learn if/then thresholds and interactions automatically — can predict default better than the linear and margin-based baselines on the same two consumer credit datasets. The random forest is the strongest nonlinear candidate and the depth model of this analysis, so it gets the most thorough tuning and diagnostics.

Research Question

Can decision trees and random forests predict consumer default more accurately than logistic regression or SVMs on the same features, and does their advantage depend on whether the data have real threshold or interaction structure?

The two target variables used here are both binary:

- **Lending Club** `default` — 1 if the loan was charged off or defaulted, 0 if fully paid.
- **Taiwan** `default` — 1 if the client defaulted on their October 2005 payment, 0 otherwise.

Objectives

The objectives of this analysis are to:

- Trace the decision-tree bias–variance curve by sweeping `max_depth` and the cost-complexity pruning parameter `ccp_alpha`.
- Fit a small readable depth-3 tree on each dataset for business interpretability.

- Tune a random forest with `RandomizedSearchCV` and inspect `min_samples_leaf` and `max_depth` validation curves plus the out-of-bag score.
- Compare impurity and permutation-based feature importance on the tuned forest.
- Choose a decision threshold on out-of-fold training predictions and apply it to the held-out test set.
- Produce a final head-to-head table against logistic regression, pruned decision tree, and tuned SVM on the same splits.

Approach

The analysis follows a single modeling arc on both datasets:

Data Loading → Shared Classification Setup → Decision-Tree Diagnostics → Random-Forest Tuning and Diagnostics → Final Cross-Model Comparison → Summary

Datasets

- **Lending Club** (Kaggle `adarshsng/lending-club-loan-data-csv`): US peer-to-peer installment loans, 2007–2018.
- **UCI Default of Credit Card Clients** (Yeh & Lien, 2009; Kaggle `uciml/default-of-credit-card-clients-dataset`): 30,000 Taiwanese credit-card clients, April–September 2005, with a target of default in October 2005.

1. Data Loading and Cleaning

The code cells below load the two raw files, apply the documented cleaning rules (outcome definition, invalid-range handling, undocumented-code recoding, log transforms, and target-leakage removal), and produce the cleaned `lc` and `tw` frames that the modeling sections use. The data-quality audit for these datasets is documented separately.

In [1]:
```
import os, warnings
import numpy as np, pandas as pd
import matplotlib.pyplot as plt, seaborn as sns
import kagglehub
warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")
pd.set_option("display.width", 160)
SEED = 42

# Lending Club: read only the columns we audit/model (the raw file is
2.26M x 145)
```

```
LC_PATH = os.path.join(kagglehub.dataset_download("adarshsng/lending-club-loan-data-csv/versions/1"), "loan.csv")
LC_COLS = ["id", "loan_status", "loan_amnt", "term", "int_rate", "grade", "sub_grade", "emp_length",
           "home_ownership", "annual_inc", "verification_status", "purpose", "dti", "delinq_2yrs",
           "inq_last_6mths", "open_acc", "pub_rec", "revol_bal", "revol_util", "total_acc", "mort_acc",
           "pub_rec_bankruptcies", "application_type", "policy_code",
           # post-outcome columns, loaded ONLY to demonstrate the leakage screen
           "total_pymnt", "recoveries", "last_pymnt_amnt", "out_prncp"]
lc_raw = pd.read_csv(LC_PATH, usecols=LC_COLS, low_memory=False)
print(f"Lending Club raw shape (audited columns): {lc_raw.shape}")
print(lc_raw.dtypes.value_counts().to_string())
display(lc_raw.head(3))
```

```
Lending Club raw shape (audited columns): (2260668, 28)
float64    16
str         9
int64       3
```

	id	loan_amnt	term	int_rate	grade	sub_grade	emp_length	home_ownership	annual_inc	verification_status	...	revol_util	total_acc	out_prncp	total_pymnt	recoveries	last_pymnt_amnt	policy_code	applic
0	NaN	2500	36 months	13.56	C	C1	10+ years	RENT	55000.0	Not Verified	...	10.3	34.0	2386.02	167.02	0.0	84.92	1	
1	NaN	30000	60 months	18.94	D	D2	10+ years	MORTGAGE	90000.0	Source Verified	...	24.2	44.0	29387.75	1507.11	0.0	777.23	1	
2	NaN	5000	36 months	17.97	D	D1	6 years	MORTGAGE	59280.0	Source Verified	...	19.1	13.0	4787.21	353.89	0.0	180.69	1	

In [3]:

```
# Keep closed loans only (outcome known); 'Current'/'Late' loans are censored
CLOSED = ["Fully Paid", "Charged Off", "Default"]
print("loan_status counts:");
print(lc_raw["loan_status"].value_counts().to_string())
lc = lc_raw[lc_raw["loan_status"].isin(CLOSED)].copy()
print(f"\nClosed loans: {len(lc):,}")

dq = []  # (issue, rows affected, action)
dq.append(("Open/censored loan_status", len(lc_raw) - len(lc), "dropped (outcome unknown)"))
dq.append(("id column 100% null", 0, "cannot dedupe on id; deduped on full row instead"))
dups = lc.drop(columns=["id"]).duplicated().sum(); dq.append(("Exact duplicate rows", dups, "dropped"))
lc = lc.drop(columns=["id"]).drop_duplicates()

# Invalid / impossible ranges
bad_dti = ((lc["dti"] < 0) | (lc["dti"] > 100)).sum()
dq.append(("dti < 0 or > 100% (sentinel 999 / data error)", bad_dti, "set to NaN then dropped"))
lc.loc[(lc["dti"] < 0) | (lc["dti"] > 100), "dti"] = np.nan
hi_util = (lc["revol_util"] > 100).sum()
dq.append(("revol_util > 100%", hi_util, "clipped to 150 (over-limit is real but extreme)"))
lc["revol_util"] = lc["revol_util"].clip(upper=150)
zero_inc = (lc["annual_inc"] <= 0).sum()
dq.append(("annual_inc <= 0", zero_inc, "dropped (cannot log/ratio)"))
lc = lc[lc["annual_inc"] > 0]
hi_inc = (lc["annual_inc"] > 1e6).sum()
```

```
dq.append(("annual_inc > $1M", hi_inc, "kept; modeled as
log1p(annual_inc)"))
odd_home = lc["home_ownership"].isin(["ANY", "NONE", "OTHER"]).sum()
dq.append(("home_ownership ANY/NONE/OTHER", odd_home, "merged into
OTHER"))
lc["home_ownership"] = lc["home_ownership"].replace({"ANY": "OTHER",
"NONE": "OTHER"})

# Constant / near-constant and cardinality
const = [c for c in lc.columns if lc[c].nunique(dropna=False) <= 1]
print("Constant columns:", const)
dq.append(("Constant columns " + str(const), 0, "column dropped (no
information)"))
lc = lc.drop(columns=const)
cat_cols = lc.select_dtypes("object").columns
print("Categorical cardinality:", lc[cat_cols].nunique().to_dict())

# Leakage screen: columns only known AFTER the outcome
LEAK = ["total_pymnt", "recoveries", "last_pymnt_amnt", "out_prncp"]
print("\nLeakage check: mean of post-outcome columns by outcome")
print(lc.groupby(lc["loan_status"] != "Fully Paid")
[LEAK].mean().round(1).rename(index={False: "repaid", True:
"default"}).to_string())
dq.append(("Post-outcome columns " + ", ".join(LEAK), 0, "columns
dropped (target leakage)"))
lc = lc.drop(columns=LEAK)

lc["default"] = (lc["loan_status"] != "Fully Paid").astype(int)
before = len(lc)
lc = lc.dropna(subset=["dti", "revol_util", "mort_acc",
"pub_rec_bankruptcies", "inq_last_6mths", "open_acc", "total_acc"])
dq.append(("Remaining NaNs in model columns", before - len(lc), "dropped
(listwise)"))
lc["log_annual_inc"] = np.log1p(lc["annual_inc"]); lc["log_revol_bal"] =
np.log1p(lc["revol_bal"])
lc["term_months"] = lc["term"].str.extract(r"(\d+)").astype(int)
print(f"\nClean Lending Club rows: {len(lc):,} | default rate:
{lc['default'].mean():.2%}")
```

```
loan_status counts:
loan_status
Fully Paid                                            1041952
Current                                                919695
Charged Off                                            261655
Late (31-120 days)                                      21897
In Grace Period                                          8952
Late (16-30 days)                                        3737
Does not meet the credit policy. Status:Fully Paid       1988
Does not meet the credit policy. Status:Charged Off       761
Default                                                    31

Closed loans: 1,303,638
```

```
Constant columns: ['policy_code']
Categorical cardinality: {'term': 2, 'grade': 7, 'sub_grade': 35,
'emp_length': 11, 'home_ownership': 4, 'verification_status': 3,
'loan_status': 3, 'purpose': 14, 'application_type': 2}

Leakage check: mean of post-outcome columns by outcome
             total_pymnt  recoveries  last_pymnt_amnt  out_prncp
loan_status
repaid           16439.4         0.0           6720.8        0.0
default           8228.0      1173.9            472.6        1.3
Clean Lending Club rows: 1,254,818 | default rate: 20.28%
```

In [5]:
```
TW_PATH = os.path.join(kagglehub.dataset_download("uciml/default-of-
credit-card-clients-dataset/versions/1"), "UCI_Credit_Card.csv")
tw_raw = pd.read_csv(TW_PATH)
print(f"Taiwan raw shape: {tw_raw.shape}")
print(tw_raw.dtypes.value_counts().to_string())
display(tw_raw.head(3))
```

```
Taiwan raw shape: (30000, 25)
float64    13
int64      12
```

	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	...	BILL_AMT4	BILL_AMT5	BILL_AMT6	PAY_AMT1	PAY_AMT2	PAY_AMT3	PAY_AMT4	PAY_AMT5
0	1	20000.0	2	2	1	24	2	2	-1	-1	...	0.0	0.0	0.0	0.0	689.0	0.0	0.0	0.0
1	2	120000.0	2	2	2	26	-1	2	0	0	...	3272.0	3455.0	3261.0	0.0	1000.0	1000.0	1000.0	0.0
2	3	90000.0	2	2	2	34	0	0	0	0	...	14331.0	14948.0	15549.0	1518.0	1500.0	1000.0	1000.0	1000.0

In [6]:
```
tw = tw_raw.rename(columns={"default.payment.next.month": "default",
"PAY_0": "PAY_1"}).copy()
PAY = [f"PAY_{i}" for i in range(1, 7)]; BILL = [f"BILL_AMT{i}" for i in
range(1, 7)]; PAMT = [f"PAY_AMT{i}" for i in range(1, 7)]
tdq = []
print("Null cells total:", int(tw.isnull().sum().sum()))
tdq.append(("Missing values", int(tw.isnull().sum().sum()), "none to
handle"))
tdq.append(("Duplicate ID", int(tw["ID"].duplicated().sum()), "none"))
dup = int(tw.drop(columns="ID").duplicated().sum())
tdq.append(("Duplicate rows (ignoring ID)", dup, "dropped (identical
profiles)"))
tw = tw.loc[~tw.drop(columns="ID").duplicated()].copy()

print("EDUCATION codes:",
tw["EDUCATION"].value_counts().sort_index().to_dict())
print("MARRIAGE codes:",
tw["MARRIAGE"].value_counts().sort_index().to_dict())
print("PAY_1 codes:", tw["PAY_1"].value_counts().sort_index().to_dict())
edu_bad = int(tw["EDUCATION"].isin([0, 5, 6]).sum())
tdq.append(("EDUCATION undocumented codes 0/5/6", edu_bad, "recoded to 4
= other"))
tw["EDUCATION"] = tw["EDUCATION"].replace({0: 4, 5: 4, 6: 4})
mar_bad = int((tw["MARRIAGE"] == 0).sum())
tdq.append(("MARRIAGE undocumented code 0", mar_bad, "recoded to 3 =
other"))
tw["MARRIAGE"] = tw["MARRIAGE"].replace({0: 3})
pay_m2 = int((tw[PAY] == -2).any(axis=1).sum())
tdq.append(("PAY_* = -2 (no consumption, undocumented)", pay_m2, "kept
as ordinal (-2 < -1 paid duly < 0 revolving)"))
```

```
neg_bill = int((tw[BILL] < 0).any(axis=1).sum())
tdq.append(("Negative BILL_AMT (credit balance/overpay)", neg_bill,
"kept; signed log transform"))
tdq.append(("AGE range " + f"{tw['AGE'].min()}–{tw['AGE'].max()}", 0,
"plausible, kept"))
const = [c for c in tw.columns if tw[c].nunique() <= 1];
tdq.append(("Constant columns", len(const), "none"))

# Leakage screen: all features are observed BEFORE the default month
(Apr–Sep 2005 -> Oct 2005)
tdq.append(("Post-outcome columns", 0, "none: all features precede
target month"))
slog = lambda s: np.sign(s) * np.log1p(np.abs(s))
for c in BILL + PAMT: tw["log_" + c] = slog(tw[c])
tw["log_LIMIT_BAL"] = np.log(tw["LIMIT_BAL"])
tw = tw.drop(columns="ID")
print(f"\nClean Taiwan rows: {len(tw):,} | default rate:
{tw['default'].mean():.2%}")
```

```
Null cells total: 0
EDUCATION codes: {0: 14, 1: 10563, 2: 14019, 3: 4915, 4: 123, 5: 280, 6:
51}
MARRIAGE codes: {0: 54, 1: 13643, 2: 15945, 3: 323}
PAY_1 codes: {-2: 2750, -1: 5682, 0: 14737, 1: 3667, 2: 2666, 3: 322, 4:
76, 5: 26, 6: 11, 7: 9, 8: 19}

Clean Taiwan rows: 29,965 | default rate: 22.13%
```

Part B — Trees and Forests

Key Concepts

- **Decision tree.** Repeatedly splits the data on the single feature/threshold that most reduces impurity (Gini = $1 - \Sigma p_k^2$; 0 for a pure node). The prediction at a leaf is the share of defaults in that leaf. Trees need no scaling and capture thresholds and interactions natively, but a deep tree memorizes the training data (high variance).
- **Controlling tree overfitting.** Limit `max_depth`, require `min_samples_leaf`, or grow the full tree and prune it with cost-complexity pruning (`ccp_alpha`: a split is kept only if it reduces impurity by more than α per extra leaf).
- **Random forest.** Averages hundreds of deep trees, each grown on a bootstrap sample (rows drawn with replacement) and allowed to consider only a random subset of features (`max_features`) at each split. Averaging many decorrelated noisy trees cancels much of their variance (bagging + feature randomness).
- **Out-of-bag (OOB) score.** Each tree never sees about 37% of the rows (those left out of its bootstrap sample); predicting those rows gives a built-in validation estimate without a separate holdout.

2. Shared Classification Setup

The Lending Club and Taiwan modeling uses the same split logic, the same preprocessing structure, and the same cross-validation protocol, so metrics are directly comparable between train and test.

- **Lending Club:** a 60,000-loan random sample of the cleaned closed loans (seed 42), split 80/20 **stratified** on `default` so both parts keep the roughly 20% default rate.
  - Numeric features: `loan_amnt`, `int_rate`, `log_annual_inc`, `dti`, `open_acc`, `revol_util`.
  - Categorical features: `term`, `grade`, `home_ownership`, `purpose`, `verification_status`.
- **Taiwan:** all 29,965 clean clients, split 80/20 stratified (seed 42).
  - Numeric features: `log_LIMIT_BAL`, `AGE`, `PAY_1..6`, `log_BILL_AMT1..6`, `log_PAY_AMT1..6`.
  - Categorical features: `SEX`, `EDUCATION`, `MARRIAGE`.

The `make_prep(num, cat, scale=True)` helper builds a `ColumnTransformer` that standardizes the numerics and one-hot encodes the categoricals. It is always fit inside a pipeline on training folds only, so no test-set leakage occurs. Trees do not need scaling but the same preprocessor is used so that the pipelines are structurally identical across all models.

Stratification matters here because with 20–22% positives, a random split could by chance put noticeably more defaults in one part. Stratifying guarantees equal rates.

In [8]:
```
from sklearn.model_selection import train_test_split, StratifiedKFold,
GridSearchCV, RandomizedSearchCV, cross_val_predict
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import (accuracy_score, precision_score,
recall_score, f1_score, roc_auc_score,
                             average_precision_score, confusion_matrix,
roc_curve, precision_recall_curve)
FIG = "../../../week_10092026_w7/DX799O1/figures"; os.makedirs(FIG,
exist_ok=True)

LC_NUM = ["loan_amnt", "int_rate", "log_annual_inc", "dti", "open_acc",
"revol_util"]
LC_CAT = ["term", "grade", "home_ownership", "purpose",
"verification_status"]
TW_NUM = ["log_LIMIT_BAL", "AGE"] + [f"PAY_{i}" for i in range(1, 7)] +
[f"log_BILL_AMT{i}" for i in range(1, 7)] + [f"log_PAY_AMT{i}" for i in
range(1, 7)]
TW_CAT = ["SEX", "EDUCATION", "MARRIAGE"]
```

```
lc_s = lc.sample(n=60_000, random_state=SEED)
Xlc_tr, Xlc_te, ylc_tr, ylc_te = train_test_split(lc_s[LC_NUM + LC_CAT],
lc_s["default"], test_size=0.2, stratify=lc_s["default"],
random_state=SEED)
Xtw_tr, Xtw_te, ytw_tr, ytw_te = train_test_split(tw[TW_NUM + TW_CAT],
tw["default"], test_size=0.2, stratify=tw["default"], random_state=SEED)
DATA = {"Lending Club": (Xlc_tr, Xlc_te, ylc_tr, ylc_te, LC_NUM,
LC_CAT),
        "Taiwan": (Xtw_tr, Xtw_te, ytw_tr, ytw_te, TW_NUM, TW_CAT)}
CV5 = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)

def make_prep(num, cat, scale=True):
    return ColumnTransformer([("num", StandardScaler() if scale else
"passthrough", num),
                              ("cat", OneHotEncoder(drop="first",
handle_unknown="ignore"), cat)])

def scores(y, prob, thr=0.5):
    pred = (prob >= thr).astype(int)
    return {"accuracy": accuracy_score(y, pred), "precision":
precision_score(y, pred, zero_division=0),
            "recall": recall_score(y, pred), "f1": f1_score(y, pred),
"roc_auc": roc_auc_score(y, prob),
            "pr_auc": average_precision_score(y, prob), "threshold":
thr}

for name, (a, b, ya, yb, num, cat) in DATA.items():
    print(f"{name:12s} train {a.shape} default {ya.mean():.2%} | test
{b.shape} default {yb.mean():.2%} | numeric {len(num)}, categorical
{len(cat)}")

from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
import time
```

```
Lending Club train (48000, 11) default 20.29% | test (12000, 11) default
20.29% | numeric 6, categorical 5
Taiwan       train (23972, 23) default 22.13% | test (5993, 23) default
22.13% | numeric 20, categorical 3
```

Shared-Setup Findings

- **Lending Club:** train 48,000 × 11 and test 12,000 × 11. Both at 20.29% default, with 6 numeric and 5 categorical features.
- **Taiwan:** train 23,972 × 23 and test 5,993 × 23. Both at 22.13% default, with 20 numeric and 3 categorical features.

Stratification worked — the default rates are identical across splits. Two baselines to keep in mind:

- **Accuracy baseline.** An "everyone repays" rule scores 79.7% on Lending Club and 77.9% on Taiwan.
- **PR-AUC baseline.** A random ranking scores 0.203 on Lending Club and 0.221 on Taiwan.

3. Decision Tree — `max_depth` Sweep (Train vs 5-Fold CV AUC)

`max_depth` is the most direct complexity control. Plotting training and validation AUC against depth exposes the bias–variance trade-off: shallow trees underfit (both scores low); deep trees overfit (training AUC → 1, validation falls).

For depth 1–20 and unlimited, a 5-fold stratified CV computes mean train and validation AUC. `class_weight="balanced"` is used so the comparison against the balanced logistic model is apples to apples. Trees do not need scaling, but the same preprocessor is used so pipelines stay identical.

In [9]:
```
from sklearn.model_selection import cross_validate
depths = list(range(1, 21)) + [None]
dt_curves = {}
for name, (Xa, Xb, ya, yb, num, cat) in DATA.items():
    res = []
    for d_ in depths:
        pipe = Pipeline([("prep", make_prep(num, cat)), ("clf",
DecisionTreeClassifier(max_depth=d_, class_weight="balanced",
random_state=SEED))])
        cv = cross_validate(pipe, Xa, ya, cv=CV5, scoring="roc_auc",
return_train_score=True, n_jobs=-1)
        res.append((d_ if d_ else 40, cv["train_score"].mean(),
cv["test_score"].mean(), cv["test_score"].std()))
    dt_curves[name] = pd.DataFrame(res, columns=["max_depth",
"train_auc", "cv_auc", "cv_sd"])
    b = dt_curves[name].loc[dt_curves[name].cv_auc.idxmax()]
    full = dt_curves[name].iloc[-1]
    print(f"{name}: best depth={int(b.max_depth)} CV AUC={b.cv_auc:.4f}
+- {b.cv_sd:.4f} (train {b.train_auc:.4f}) | "
          f"unlimited depth: train {full.train_auc:.4f} vs CV
{full.cv_auc:.4f}")
    display(dt_curves[name].iloc[[0, 2, 4, 6, 8, 11, 14, 19,
20]].round(4).T)
```

```
Lending Club: best depth=5 CV AUC=0.6898 +- 0.0044 (train 0.7005) |
unlimited depth: train 1.0000 vs CV 0.5413
```

	0	2	4	6	8	11	14	19	20
max_depth	1.0000	3.0000	5.0000	7.0000	9.0000	12.0000	15.0000	20.0000	40.0000
train_auc	0.6325	0.6852	0.7005	0.7197	0.7523	0.8245	0.9022	0.9787	1.0000
cv_auc	0.6321	0.6827	0.6898	0.6835	0.6645	0.6266	0.5930	0.5593	0.5413
cv_sd	0.0045	0.0032	0.0044	0.0058	0.0025	0.0058	0.0053	0.0108	0.0071

```
Taiwan: best depth=6 CV AUC=0.7594 +- 0.0070 (train 0.7891) | unlimited
depth: train 1.0000 vs CV 0.6057
```

	0	2	4	6	8	11	14	19	20
max_depth	1.0000	3.0000	5.0000	7.0000	9.0000	12.0000	15.0000	20.0000	40.0000
train_auc	0.6876	0.7520	0.7760	0.8024	0.8366	0.8982	0.9513	0.9902	1.0000
cv_auc	0.6876	0.7500	0.7586	0.7553	0.7374	0.7006	0.6545	0.6267	0.6057
cv_sd	0.0090	0.0041	0.0041	0.0056	0.0085	0.0138	0.0156	0.0086	0.0063

Decision Tree Depth Findings

Lending Club.

- Best depth 5: CV AUC 0.6898 ± 0.0044 (train 0.7005).
- Beyond depth 5, CV AUC falls: 0.6645 at depth 9, 0.5930 at depth 15, and 0.5413 unlimited, while train AUC climbs to 1.0000.

Taiwan.

- Best depth 6: CV AUC 0.7594 ± 0.0070 (train 0.7891).
- An unlimited tree reaches train 1.0000 but CV 0.6057.

Three readings:

- **The bias–variance trade-off, in numbers.** Depth 1 (a single split) underfits (train and CV both ~0.63 / 0.69). Depths 5–6 are the sweet spot. An unlimited tree memorizes — on Lending Club its CV AUC of 0.54 is barely above a coin flip, even though it is perfect on the training data.
- **Fold SDs are small (0.003–0.016),** so the peak is reliable.
- **Already competitive on Taiwan.** A single depth-6 tree (0.759) beats a tuned linear model (CV around 0.752), because it captures the PAY threshold effects directly. On Lending Club the tree stays behind the linear baseline.

4. Cost-Complexity Pruning (`ccp_alpha`) and the Pruned Tree

Pruning is the second way to control tree size. `cost_complexity_pruning_path` lists every α at which a subtree collapses (thousands of values). 16 log-spaced α values from 10^{-5} (almost no pruning) to 10^{-2} (a stump) are cross-validated, and the best is chosen.

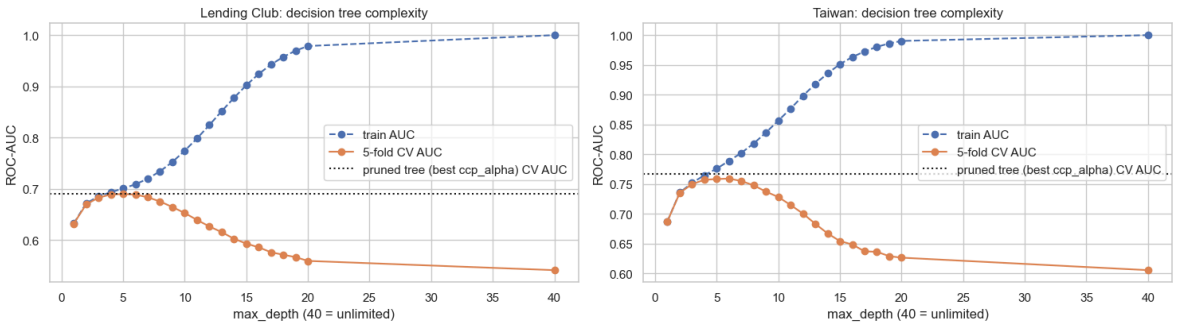
This step tests whether a grow-then-prune tree (`min_samples_leaf = 5`) beats a depth-limited one. What to look for: the best α, the resulting number of leaves and depth, and the CV AUC compared with the best `max_depth` tree.

In [10]:
```
dt_best = {}
for name, (Xa, Xb, ya, yb, num, cat) in DATA.items():
    prep = make_prep(num, cat).fit(Xa); Za = prep.transform(Xa)
    path = DecisionTreeClassifier(class_weight="balanced",
random_state=SEED, min_samples_leaf=5).cost_complexity_pruning_path(Za,
```

```
ya)
    alphas = np.logspace(-5, -2, 16)  # log-spaced so both lightly and
heavily pruned trees are tried
    print(f"{name}: full pruning path has {len(path.ccp_alphas)} alphas
(max {path.ccp_alphas.max():.2e}); searching 16 log-spaced values 1e-
5..1e-2")
    pipe = Pipeline([("prep", make_prep(num, cat)), ("clf",
DecisionTreeClassifier(class_weight="balanced", random_state=SEED,
min_samples_leaf=5))])
    gs = GridSearchCV(pipe, {"clf__ccp_alpha": alphas},
scoring="roc_auc", cv=CV5, n_jobs=-1).fit(Xa, ya)
    tree = gs.best_estimator_.named_steps["clf"]
    dt_best[name] = gs
    cvp = pd.DataFrame(gs.cv_results_)[["param_clf__ccp_alpha",
"mean_test_score", "std_test_score"]]
    print("   CV AUC by ccp_alpha:", ", ".join(f"{a:.0e}:{m:.4f}" for a,
m in zip(cvp.param_clf__ccp_alpha.astype(float), cvp.mean_test_score)))
    print(f"   best ccp_alpha={gs.best_params_['clf__ccp_alpha']:.2e} "
          f"CV AUC={gs.best_score_:.4f} | leaves={tree.get_n_leaves()}
depth={tree.get_depth()}")

fig, axes = plt.subplots(1, 2, figsize=(15, 4.3))
for ax, name in zip(axes, DATA):
    c = dt_curves[name]
    ax.plot(c.max_depth, c.train_auc, "o--", label="train AUC");
ax.plot(c.max_depth, c.cv_auc, "o-", label="5-fold CV AUC")
    ax.axhline(dt_best[name].best_score_, color="k", ls=":",
label="pruned tree (best ccp_alpha) CV AUC")
    ax.set_xlabel("max_depth (40 = unlimited)"); ax.set_ylabel("ROC-
AUC"); ax.set_title(f"{name}: decision tree complexity"); ax.legend()
plt.tight_layout(); plt.savefig(f"{FIG}/fig_w6_tree_depth.png", dpi=150,
bbox_inches="tight"); plt.show()
```

```
Lending Club: full pruning path has 2794 alphas (max 3.67e-02); searching
16 log-spaced values 1e-5..1e-2
   CV AUC by ccp_alpha: 1e-05:0.5778, 2e-05:0.5781, 3e-05:0.5792, 4e-
05:0.5749, 6e-05:0.5845, 1e-04:0.6232, 2e-04:0.6809, 3e-04:0.6907, 4e-
04:0.6895, 6e-04:0.6870, 1e-03:0.6811, 2e-03:0.6766, 3e-03:0.6695, 4e-
03:0.6695, 6e-03:0.6428, 1e-02:0.6321
   best ccp_alpha=2.51e-04 CV AUC=0.6907 | leaves=24 depth=7
Taiwan: full pruning path has 1372 alphas (max 7.93e-02); searching 16
log-spaced values 1e-5..1e-2
   CV AUC by ccp_alpha: 1e-05:0.6692, 2e-05:0.6701, 3e-05:0.6709, 4e-
05:0.6733, 6e-05:0.6697, 1e-04:0.6782, 2e-04:0.6972, 3e-04:0.7376, 4e-
04:0.7616, 6e-04:0.7675, 1e-03:0.7633, 2e-03:0.7579, 3e-03:0.7475, 4e-
03:0.7390, 6e-03:0.7354, 1e-02:0.6900
   best ccp_alpha=6.31e-04 CV AUC=0.7675 | leaves=19 depth=8
```



Pruning Findings

- **Path sizes:** the full pruning path has 2,794 (Lending Club) and 1,372 (Taiwan) distinct α values.
- **Lending Club.** CV AUC rises from 0.578 at $\alpha = 10^{-5}$ (barely pruned, overfit) to a peak of 0.6907 at α = 2.5e-4, then falls to 0.632 at $\alpha = 10^{-2}$. Best tree has 24 leaves and depth 7.
- **Taiwan.** CV AUC goes 0.669 → peak 0.7675 at α = 6.3e-4 → 0.690. Best tree has 19 leaves and depth 8.
- **Plot.** The dotted "pruned tree" line sits at or slightly above the best `max_depth` point on both datasets.

Pruning works. It produces a small tree (19–24 leaves) that matches or beats the best depth-limited tree: 0.6907 vs 0.6898 on Lending Club, and 0.7675 vs 0.7594 on Taiwan. Pruning can keep a deep branch where it helps and cut shallow branches elsewhere, which a single global `max_depth` cannot do. The CV curves have an interior peak on both sides, confirming the α grid covered the right range.

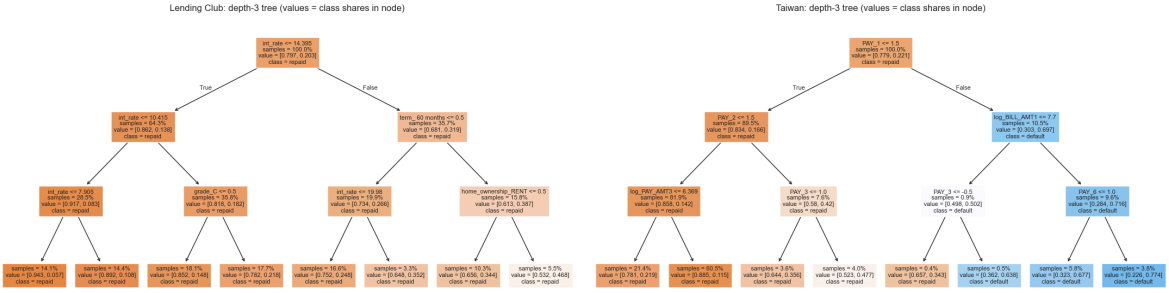
5. A Readable Depth-3 Tree for Each Dataset

The main business advantage of a single tree is that it can be read as rules. A depth-3 tree (at most 8 leaves) is shallow enough to explain to a credit committee, so this step inspects which splits each dataset's tree chooses first — the root split (the single most informative feature/threshold) and the default rate in the riskiest and safest leaves.

In [11]:
```
fig, axes = plt.subplots(1, 2, figsize=(22, 6))
for ax, (name, (Xa, Xb, ya, yb, num, cat)) in zip(axes, DATA.items()):
    prep = make_prep(num, cat, scale=False).fit(Xa)
    small = DecisionTreeClassifier(max_depth=3,
random_state=SEED).fit(prep.transform(Xa), ya)
    fn = [f.replace("num__", "").replace("cat__", "") for f in
prep.get_feature_names_out()]
    plot_tree(small, feature_names=fn, class_names=["repaid",
"default"], filled=True, proportion=True, impurity=False, fontsize=8,
ax=ax)
    ax.set_title(f"{name}: depth-3 tree (values = class shares in
```

```
node)")
    Zb = prep.transform(Xb); leaf_auc = roc_auc_score(yb,
small.predict_proba(Zb)[:, 1])
    print(f"{name}: root split = {fn[small.tree_.feature[0]]} <=
{small.tree_.threshold[0]:.3f}; depth-3 tree test AUC = {leaf_auc:.4f}")
plt.tight_layout(); plt.savefig(f"{FIG}/fig_w6_tree_depth3.png",
dpi=130, bbox_inches="tight"); plt.show()
```

```
Lending Club: root split = int_rate <= 14.395; depth-3 tree test AUC =
0.6887
Taiwan: root split = PAY_1 <= 1.500; depth-3 tree test AUC = 0.7234
```



Depth-3 Tree Findings

Lending Club.

- Root split: `int_rate ≤ 14.395%`. 64% of loans fall on the low-rate side at about a 14% default rate; the 36% above it default at about 32%.
- Next splits are lower rate cut-offs on the left, and `term = 60 months` then `home_ownership = RENT` on the right. The riskiest leaf (high rate, 60-month term, renter — 5.5% of loans) defaults at about 47%.
- Test AUC of this 8-leaf tree: 0.6887.

Taiwan.

- Root split: `PAY_1 ≤ 1.5`. The 10.5% of clients who are 2+ months late on the latest bill default at about 70%, compared with about 17% for everyone else.
- Further splits use `PAY_2`, the latest bill (`log_BILL_AMT1`), and older PAY months.
- Test AUC: 0.7234.

Business reading. On Lending Club the interest rate (which encodes the lender's own grade) is the single strongest risk signal; loan length and housing add to it. On Taiwan recent delinquency dominates — a one-rule policy ("flag anyone 2+ months late") already isolates a group with about a 70% default rate.

For a credit committee, a depth-3 tree reaches about 97% of the full tuned tree's AUC on both datasets (0.689 vs 0.701; 0.723 vs 0.755) with rules a committee can read directly.

6. Random Forest — `RandomizedSearchCV` (Depth Model)

A random forest has several interacting hyperparameters, and a full grid would need hundreds of combinations × 5 folds. `RandomizedSearchCV` samples 20 combinations from the ranges below. Research (Bergstra & Bengio, 2012) shows random search usually finds near-optimal settings far faster, because only a few hyperparameters really matter.

Hyperparameter	Values	What it controls
`n_estimators`	150, 300	Number of trees — more trees mean lower variance with diminishing returns
`max_depth`	8, 12, 16, None	Depth of each tree — the main bias–variance knob
`min_samples_leaf`	5, 20, 50, 100	Smallest leaf — larger values give smoother probabilities and less memorization
`max_features`	sqrt, 0.3	Features tried per split — smaller fractions produce more decorrelated trees
`class_weight`	balanced, balanced_subsample	Reweights defaults either per forest or per bootstrap sample

A test fit with 600 trees, unlimited depth, 1-row leaves, and 50% of features took over 5 minutes per fit. Those extreme settings are excluded from the random grid; small leaves (down to 2) and deep trees (up to depth 24) are instead examined separately in the validation curves below.

Scoring is 5-fold stratified CV ROC-AUC on the full training set (48,000 and 23,972 rows). Trees run in parallel (`n_jobs=-1`), and `return_train_score=True` records the train–validation gap.

In [12]:
```
rf_space = {"clf__n_estimators": [150, 300], "clf__max_depth": [8, 12,
16, None],
            "clf__min_samples_leaf": [5, 20, 50, 100],
"clf__max_features": ["sqrt", 0.3],
            "clf__class_weight": ["balanced", "balanced_subsample"]}
rf_rs = {}
for name, (Xa, Xb, ya, yb, num, cat) in DATA.items():
    pipe = Pipeline([("prep", make_prep(num, cat)), ("clf",
RandomForestClassifier(random_state=SEED, n_jobs=1))])
    t0 = time.time()
    rs = RandomizedSearchCV(pipe, rf_space, n_iter=20,
scoring="roc_auc", cv=CV5, n_jobs=-1, random_state=SEED,
                            return_train_score=True).fit(Xa, ya)
    rf_rs[name] = rs
```

```
    r = pd.DataFrame(rs.cv_results_); r["gap"] = r.mean_train_score -
r.mean_test_score
    cols = ["param_clf__n_estimators", "param_clf__max_depth",
"param_clf__min_samples_leaf", "param_clf__max_features",
            "param_clf__class_weight", "mean_train_score",
"mean_test_score", "std_test_score", "gap"]
    print(f"\n{name}: search {time.time()-t0:.0f}s | BEST
{rs.best_params_} | CV AUC {rs.best_score_:.4f} +-
{r.loc[rs.best_index_, 'std_test_score']:.4f}")
    display(r.sort_values("rank_test_score")[cols].head(5).round(4))
    print("Largest train-CV gaps:"); display(r.sort_values("gap",
ascending=False)[cols].head(2).round(4))
```

```
Lending Club: search 41s | BEST {'clf__n_estimators': 300,
'clf__min_samples_leaf': 20, 'clf__max_features': 'sqrt',
'clf__max_depth': 12, 'clf__class_weight': 'balanced'} | CV AUC 0.7047 +-
0.0049
```

	param_clf__n_estimators	param_clf__max_depth	param_clf__min_samples_leaf	param_clf__max_features	param_clf__class_weight	mean_train_score	mean_test_s
2	300	12	20	sqrt	balanced	0.7652	0
18	300	None	50	0.3	balanced_subsample	0.7687	0
4	150	16	20	sqrt	balanced_subsample	0.7921	0
10	150	12	100	0.3	balanced_subsample	0.7355	0
3	300	12	100	0.3	balanced	0.7325	0

```
Largest train-CV gaps:
```

	param_clf__n_estimators	param_clf__max_depth	param_clf__min_samples_leaf	param_clf__max_features	param_clf__class_weight	mean_train_score	mean_test_s
5	150	None	5	0.3	balanced	0.9842	0.
15	300	None	5	sqrt	balanced_subsample	0.9492	0

```
Taiwan: search 37s | BEST {'clf__n_estimators': 300,
'clf__min_samples_leaf': 50, 'clf__max_features': 0.3, 'clf__max_depth':
None, 'clf__class_weight': 'balanced_subsample'} | CV AUC 0.7864 +- 0.0077
```

	param_clf__n_estimators	param_clf__max_depth	param_clf__min_samples_leaf	param_clf__max_features	param_clf__class_weight	mean_train_score	mean_test_s
18	300	None	50	0.3	balanced_subsample	0.8481	0.
2	300	12	20	sqrt	balanced	0.8593	0.
11	300	12	20	0.3	balanced	0.8655	0.
16	150	16	50	sqrt	balanced	0.8366	0.
9	150	12	20	0.3	balanced	0.8650	0.

```
Largest train-CV gaps:
```

	param_clf__n_estimators	param_clf__max_depth	param_clf__min_samples_leaf	param_clf__max_features	param_clf__class_weight	mean_train_score	mean_test_s
5	150	None	5	0.3	balanced	0.9793	0
15	300	None	5	sqrt	balanced_subsample	0.9714	0.

Random Forest Random-Search Findings

Lending Club (41 s).

- Best: 300 trees, `max_depth = 12`, `min_samples_leaf = 20`, `max_features = sqrt`, `class_weight = balanced`, with CV AUC 0.7047 ± 0.0049.
- The top 5 candidates sit between 0.7041 and 0.7047 (well within one SD) despite quite different settings.
- Train–CV gap of the best model: 0.061.
- Most overfit: unlimited depth with 5-row leaves, at train 0.984 vs CV 0.697 (gap 0.29).

Taiwan (37 s).

- Best: 300 trees, `max_depth = None`, `min_samples_leaf = 50`, `max_features = 0.3`, `class_weight = balanced_subsample`, with CV

AUC 0.7864 ± 0.0077.
- Top 5 span 0.7854–0.7864. Gap of the best model: 0.062.
- Most overfit: unlimited depth with 5-row leaves, at train 0.979 vs CV 0.779 (gap 0.20).

Three readings:

- **The winners are regularized in different ways.** On Lending Club it is the depth limit (12) plus 20-row leaves; on Taiwan it is large leaves (50) with unlimited depth. Both stop individual trees from memorizing.
- **Averaging protects against overfitting.** Even the most overfit forest loses only about 0.008 AUC on CV relative to the best, whereas a single unlimited tree lost about 0.15. This is the core benefit of bagging and random feature subsets.
- **Taiwan improves on every earlier model; Lending Club stays flat.** Taiwan CV AUC 0.786 against logistic regression 0.752, the best SVM 0.762, and the pruned tree 0.768. Lending Club's 0.705 is level with logistic regression (0.703).

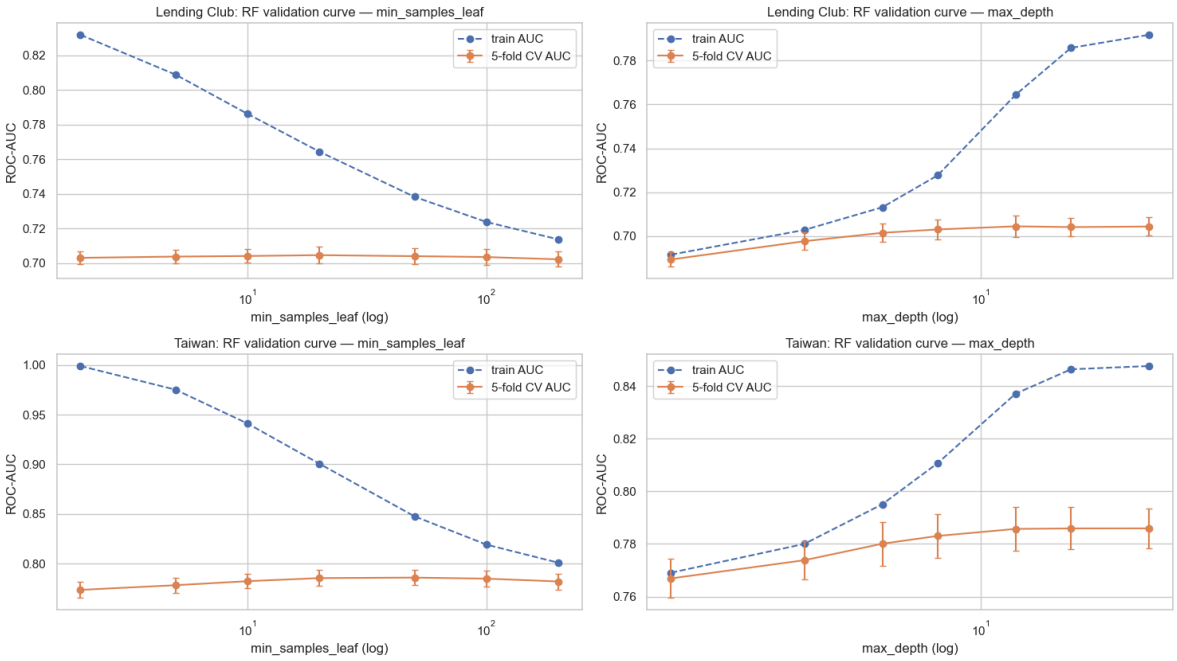
7. Depth Analysis — Validation Curves for `min_samples_leaf` and `max_depth`, Plus OOB

Random search tells us *which* setting wins; validation curves tell us *why*. Holding the other best parameters fixed, each knob is varied one at a time and train vs CV AUC is plotted. The best forest is also refit with `oob_score=True` to compare the out-of-bag estimate with the CV estimate.

In [13]:
```
from sklearn.model_selection import validation_curve
fig, axes = plt.subplots(2, 2, figsize=(15, 8.5))
rf_oob = {}
for row, (name, (Xa, Xb, ya, yb, num, cat)) in enumerate(DATA.items()):
    best = rf_rs[name].best_estimator_
    base = Pipeline([("prep", make_prep(num, cat)), ("clf",
RandomForestClassifier(**{**best.named_steps["clf"].get_params(),
"n_estimators": 150, "n_jobs": 1}))])
    for col, (pname, vals, lab) in enumerate([("clf__min_samples_leaf",
[2, 5, 10, 20, 50, 100, 200], "min_samples_leaf"),
                                              ("clf__max_depth", [2, 4,
6, 8, 12, 16, 24], "max_depth")]):
        tr, va = validation_curve(base, Xa, ya, param_name=pname,
param_range=vals, cv=CV5, scoring="roc_auc", n_jobs=-1)
        ax = axes[row, col]
        ax.plot(vals, tr.mean(1), "o--", label="train AUC");
ax.errorbar(vals, va.mean(1), yerr=va.std(1), fmt="o-", capsize=3,
label="5-fold CV AUC")
        ax.set_xscale("log"); ax.set_xlabel(lab + " (log)");
ax.set_ylabel("ROC-AUC"); ax.set_title(f"{name}: RF validation curve —
{lab}"); ax.legend()
        i = int(np.argmax(va.mean(1)))
        print(f"{name} {lab}: best {vals[i]} CV {va.mean(1)[i]:.4f}
```

```
(train {tr.mean(1)[i]:.4f}) | at {vals[0]}: train {tr.mean(1)[0]:.4f} CV
{va.mean(1)[0]:.4f}")
    oob = RandomForestClassifier(**
{**best.named_steps["clf"].get_params(), "oob_score": True, "n_jobs":
-1})
    Za = make_prep(num, cat).fit(Xa).transform(Xa)
    oob.fit(Za, ya); rf_oob[name] = roc_auc_score(ya,
oob.oob_decision_function_[:, 1])
    print(f"{name}: OOB AUC {rf_oob[name]:.4f} vs 5-fold CV AUC
{rf_rs[name].best_score_:.4f}")
plt.tight_layout(); plt.savefig(f"{FIG}/fig_w6_rf_validation.png",
dpi=150, bbox_inches="tight"); plt.show()
```

```
Lending Club min_samples_leaf: best 20 CV 0.7045 (train 0.7644) | at 2:
train 0.8319 CV 0.7030
Lending Club max_depth: best 12 CV 0.7045 (train 0.7644) | at 2: train
0.6916 CV 0.6894
Lending Club: OOB AUC 0.7029 vs 5-fold CV AUC 0.7047
Taiwan min_samples_leaf: best 50 CV 0.7859 (train 0.8476) | at 2: train
0.9990 CV 0.7735
Taiwan max_depth: best 24 CV 0.7859 (train 0.8476) | at 2: train 0.7691 CV
0.7669
Taiwan: OOB AUC 0.7864 vs 5-fold CV AUC 0.7864
```



Validation-Curve and OOB Findings

`min_samples_leaf` (left panels).

- As leaves shrink to 2, train AUC rises to 0.832 (Lending Club) and 0.999 (Taiwan).
- CV AUC stays almost flat: Lending Club 0.703–0.705, peaking at 20; Taiwan 0.774–0.786, peaking at 50.

`max_depth` (right panels).

- At depth 2, CV AUC is 0.689 / 0.767 (underfit).
- It rises to a plateau from about depth 8–12 onward: Lending Club best at 12 (0.7045), Taiwan best at 24 (0.7859), essentially flat after depth 12.
- Train AUC keeps rising (0.79 / 0.85).

OOB vs CV.

- Lending Club: OOB 0.7029 vs CV 0.7047.
- Taiwan: OOB 0.7864 vs CV 0.7864.

Three readings:

- **Bias–variance in context.** The bias side is only visible at very shallow depth. The variance side shows up as the widening train–CV gap, but the forest's averaging keeps that gap from damaging validation performance, which is why random forests are forgiving to tune. Taiwan gains about 0.012 AUC from larger leaves because its smaller sample is noisier per leaf.
- **OOB works as a free validation set.** It agrees with 5-fold CV to within 0.002, so a forest's generalization can be checked without a holdout, using the 37% of rows each tree never saw.
- **Flat plateaus.** The CV error bars (±1 SD across folds) overlap across most of each plateau, so any setting inside the plateau is statistically equivalent.

8. Feature Importance — Impurity-Based vs Permutation (Test Set)

- **Impurity importance** is the total Gini decrease contributed by each feature across all trees. It is fast but biased toward continuous or high-cardinality features, which offer more possible split points.
- **Permutation importance** is the drop in test AUC when one feature's values are shuffled. It is slower but measures real predictive reliance on unseen data, and it is computed on the *original* columns, so a categorical variable is scored as a whole.

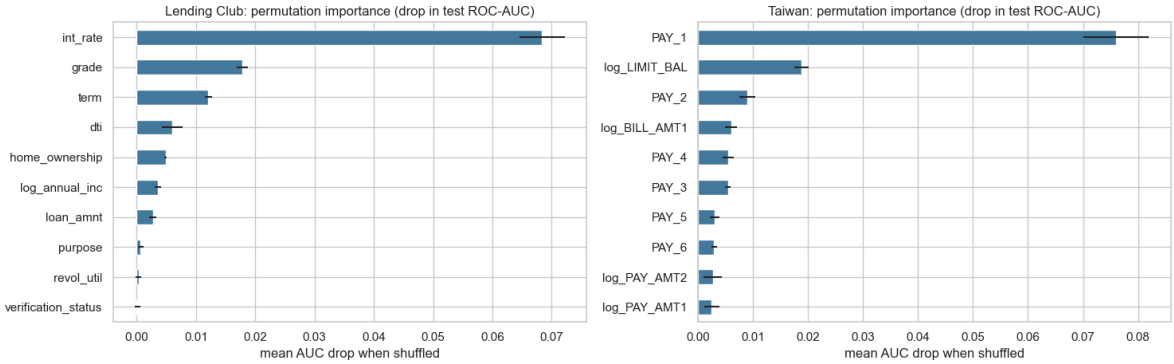
Comparing the two guards against over-reading either one. What to look for: top features under each method, where they agree or disagree, and how the rankings relate to the EDA (grade and `int_rate` for Lending Club; `PAY_1` for Taiwan).

In [14]:
```
imp = {}
fig, axes = plt.subplots(1, 2, figsize=(15, 4.8))
for ax, (name, (Xa, Xb, ya, yb, num, cat)) in zip(axes, DATA.items()):
    best = rf_rs[name].best_estimator_
    names = [f.replace("num__", "").replace("cat__", "") for f in
best.named_steps["prep"].get_feature_names_out()]
    gini = pd.Series(best.named_steps["clf"].feature_importances_,
```

```
index=names).sort_values(ascending=False)
    pi = permutation_importance(best, Xb, yb, scoring="roc_auc",
n_repeats=5, random_state=SEED, n_jobs=-1)
    perm = pd.Series(pi.importances_mean,
index=Xb.columns).sort_values(ascending=False)
    imp[name] = (gini, perm, pd.Series(pi.importances_std,
index=Xb.columns))
    print(f"\n{name} — top 8 impurity importance:");
print(gini.head(8).round(4).to_string())
    print(f"{name} — top 8 permutation importance (mean test-AUC drop +-
sd):")
    print(pd.DataFrame({"auc_drop": perm.head(8), "sd": imp[name][2]
[perm.head(8).index]}).round(4).to_string())
    perm.head(10)[::-1].plot.barh(ax=ax, xerr=imp[name][2]
[perm.head(10).index[::-1]], color="#457b9d")
    ax.set_title(f"{name}: permutation importance (drop in test ROC-
AUC)"); ax.set_xlabel("mean AUC drop when shuffled")
plt.tight_layout(); plt.savefig(f"{FIG}/fig_w6_rf_importance.png",
dpi=150, bbox_inches="tight"); plt.show()
```

```
Lending Club — top 8 impurity importance:
int_rate           0.3398
term_ 60 months    0.1075
dti                0.0962
log_annual_inc     0.0637
revol_util         0.0584
loan_amnt          0.0569
grade_B            0.0450
grade_E            0.0385
Lending Club — top 8 permutation importance (mean test-AUC drop +- sd):
                auc_drop      sd
int_rate          0.0684  0.0038
grade             0.0178  0.0010
term              0.0121  0.0007
dti               0.0060  0.0018
home_ownership    0.0049  0.0002
log_annual_inc    0.0035  0.0006
loan_amnt         0.0028  0.0006
purpose           0.0007  0.0004
```

```
Taiwan — top 8 impurity importance:
PAY_1            0.2942
PAY_2            0.1151
PAY_3            0.0691
PAY_4            0.0543
log_LIMIT_BAL    0.0449
log_BILL_AMT1    0.0432
log_PAY_AMT1     0.0425
log_PAY_AMT2     0.0378
Taiwan — top 8 permutation importance (mean test-AUC drop +- sd):
                auc_drop      sd
PAY_1             0.0759  0.0059
log_LIMIT_BAL     0.0188  0.0013
PAY_2             0.0090  0.0015
log_BILL_AMT1     0.0061  0.0011
PAY_4             0.0055  0.0010
PAY_3             0.0054  0.0005
PAY_5             0.0031  0.0008
PAY_6             0.0029  0.0005
```



Feature-Importance Findings

Lending Club.

- Impurity: `int_rate` 0.340, `term_60` 0.108, `dti` 0.096, `log_annual_inc` 0.064, `revol_util` 0.058, `loan_amnt` 0.057, then grade dummies.
- Permutation (test AUC drop):
  - `int_rate` 0.068 ± 0.004
  - `grade` 0.018
  - `term` 0.012
  - `dti` 0.006
  - `home_ownership` 0.005
  - `log_annual_inc` 0.004
  - `loan_amnt` 0.003
  - `purpose` 0.001

Taiwan.

- Impurity: `PAY_1` 0.294, `PAY_2` 0.115, `PAY_3` 0.069, `PAY_4` 0.054, `log_LIMIT_BAL` 0.045, `log_BILL_AMT1` 0.043.
- Permutation:
  - `PAY_1` 0.076 ± 0.006
  - `log_LIMIT_BAL` 0.019
  - `PAY_2` 0.009
  - `log_BILL_AMT1` 0.006
  - `PAY_3` – `PAY_6` 0.003–0.006

Four readings:

- **The two methods agree on the leader.** `int_rate` and `PAY_1` are first under both, and by a wide margin — shuffling either one alone costs about 0.07 AUC.
- **They disagree on the second tier, as theory predicts.** Impurity inflates continuous features: it ranks `dti`, income and `revol_util` above `grade` because continuous features offer many split points. Permutation (on held-out data, scoring `grade` as a whole) puts `grade` and `term` ahead of them.
- **Correlated features share credit.** On Taiwan, impurity spreads importance over `PAY_2` – `PAY_4`. Permutation shows that once `PAY_1` is known the older months add little, because a correlated feature can stand in when one is shuffled.
- **`log_LIMIT_BAL` rises to #2 under permutation.** The bank's own limit decision carries risk information in its own right.

Domain reading. On Lending Club, risk is mostly *priced* (`int_rate` and grade are the lender's own risk assessment) with loan structure (`term`) in second place. On Taiwan, risk is mostly *behavioural*: the most recent repayment status is the strongest early-warning signal.

A high `int_rate` does not cause default — partly the reverse holds, since riskier borrowers are priced higher, although high payments can add some burden. These are measures of predictive reliance within this model, not causal effects.

9. Threshold Choice for the Forest (Out-of-Fold) and Held-Out Test Metrics

The decision threshold is chosen to maximize F1 on out-of-fold training predictions and then applied once to the test set. Choosing the threshold on the test set would be leakage, and the test score would be optimistic.

In [15]:
```
rf_test = {}
for name, (Xa, Xb, ya, yb, num, cat) in DATA.items():
    best = rf_rs[name].best_estimator_
    oof = cross_val_predict(best, Xa, ya, cv=CV5,
method="predict_proba", n_jobs=-1)[:, 1]
    p, r, th = precision_recall_curve(ya, oof); f1s = 2 * p[:-1] *
```

```
r[:-1] / np.clip(p[:-1] + r[:-1], 1e-9, None)
    thr = float(th[np.argmax(f1s)]); prob = best.predict_proba(Xb)[:, 1]
    rf_test[name] = (prob, thr)
    print(f"\n{name}: OOF F1-optimal threshold {thr:.3f}")
    display(pd.DataFrame({"at 0.5": scores(yb, prob, 0.5), f"at
{thr:.3f}": scores(yb, prob, thr)}).T.round(4))
    print("Confusion matrix at tuned threshold [[TN FP] [FN TP]]:",
confusion_matrix(yb, (prob >= thr).astype(int)).tolist())
```

```
Lending Club: OOF F1-optimal threshold 0.495
```

	accuracy	precision	recall	f1	roc_auc	pr_auc	threshold
at 0.5	0.6480	0.3216	0.6620	0.4329	0.7099	0.3801	0.5000
at 0.495	0.6422	0.3191	0.6735	0.4331	0.7099	0.3801	0.4952

```
Confusion matrix at tuned threshold [[TN FP] [FN TP]]: [[6066, 3499],
[795, 1640]]
Taiwan: OOF F1-optimal threshold 0.543
```

	accuracy	precision	recall	f1	roc_auc	pr_auc	threshold
at 0.5	0.7686	0.4814	0.5950	0.5322	0.7733	0.5507	0.5000
at 0.543	0.7837	0.5104	0.5535	0.5311	0.7733	0.5507	0.5428

```
Confusion matrix at tuned threshold [[TN FP] [FN TP]]: [[3963, 704], [592,
734]]
```

Threshold Findings

Lending Club.

- Test ROC-AUC 0.7099, PR-AUC 0.3801. Out-of-fold threshold: 0.495.
- At that threshold: precision 0.319, recall 0.674, F1 0.433.
- Confusion matrix: TN 6,066, FP 3,499, FN 795, TP 1,640.

Taiwan.

- Test ROC-AUC 0.7733, PR-AUC 0.5507. Threshold: 0.543.
- At that threshold: precision 0.510, recall 0.554, F1 0.531, accuracy 0.784.
- Confusion matrix: TN 3,963, FP 704, FN 592, TP 734.

The test scores sit close to the CV estimates. Lending Club tests at 0.710 against CV 0.705, and Taiwan at 0.773 against CV 0.786. The Taiwan drop of 0.013 is within about 1.7 fold-SDs, so the tuning did not overfit the validation folds.

Taiwan PR-AUC of 0.551 is 2.5× the random baseline of 0.221, the best of any model fit so far. Balanced class weights already put the F1 optimum near 0.5, so moving the cutoff changes the precision/recall mix rather than F1.

10. Final Cross-Model Comparison on the Shared Test Sets

This is the apples-to-apples table that pulls the best model from each prior analysis onto the same held-out test set:

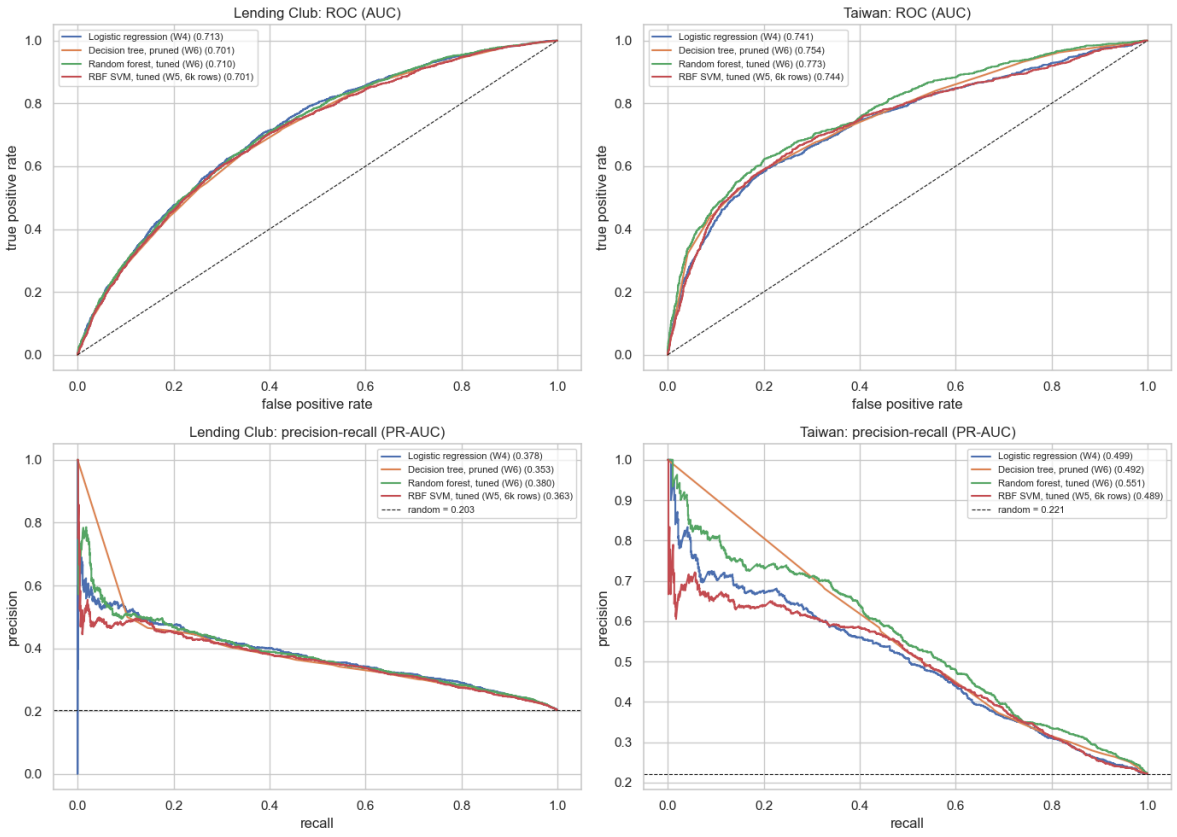
- **Logistic regression:** balanced class weights, with the best `C` and penalty from the earlier CV grid.
- **SVM:** the best kernel, `C`, and γ from the earlier grid, fit on the 6,000-row stratified subsample.
- **Decision tree:** pruned at the best `ccp_alpha` from this analysis.
- **Random forest:** the best settings from this analysis's random search.

Thresholds for the logistic model, decision tree, and random forest are chosen on out-of-fold training predictions. The SVM uses its own decision-function boundary (score 0).

In [16]:
```
W4_BEST = {"Lending Club": {"C": 1, "penalty": "l1"}, "Taiwan": {"C":
0.001, "penalty": "l2"}}
W5_BEST = {"Lending Club": {"kernel": "rbf", "C": 0.1, "gamma": 0.01},
"Taiwan": {"kernel": "rbf", "C": 1, "gamma": 0.01}}
def oof_thr(est, Xa, ya):
    oof = cross_val_predict(est, Xa, ya, cv=CV5, method="predict_proba",
n_jobs=-1)[:, 1]
    p, r, th = precision_recall_curve(ya, oof); f1s = 2 * p[:-1] *
r[:-1] / np.clip(p[:-1] + r[:-1], 1e-9, None)
    return float(th[np.argmax(f1s)])
final_rows, curves = [], {}
for name, (Xa, Xb, ya, yb, num, cat) in DATA.items():
    lr = Pipeline([("prep", make_prep(num, cat)), ("clf",
LogisticRegression(solver="liblinear", class_weight="balanced",
max_iter=2000, **W4_BEST[name]))]).fit(Xa, ya)
    Xs, _, ys, _ = train_test_split(Xa, ya, train_size=6000,
stratify=ya, random_state=SEED)
    svm = Pipeline([("prep", make_prep(num, cat)), ("clf",
SVC(class_weight="balanced", **W5_BEST[name]))]).fit(Xs, ys)
    dt = dt_best[name].best_estimator_; rf = rf_rs[name].best_estimator_
    models = {"Logistic regression (W4)": (lr, oof_thr(lr, Xa, ya)),
"Decision tree, pruned (W6)": (dt, oof_thr(dt, Xa, ya)),
              "Random forest, tuned (W6)": (rf, rf_test[name][1])}
    curves[name] = {}
    for mname, (m, thr) in models.items():
        prob = m.predict_proba(Xb)[:, 1]; curves[name][mname] = prob
        final_rows.append({"dataset": name, "model": mname, **scores(yb,
prob, thr)})
    sc = svm.decision_function(Xb); pred = (sc >= 0).astype(int);
curves[name]["RBF SVM, tuned (W5, 6k rows)"] = sc
    final_rows.append({"dataset": name, "model": "RBF SVM, tuned (W5, 6k
rows)", "accuracy": accuracy_score(yb, pred), "precision":
precision_score(yb, pred),
                       "recall": recall_score(yb, pred), "f1":
f1_score(yb, pred), "roc_auc": roc_auc_score(yb, sc),
                       "pr_auc": average_precision_score(yb, sc),
"threshold": 0.0})
final = pd.DataFrame(final_rows)
display(final.round(4))
final.round(4).to_csv(f"{FIG}/table_final_model_comparison.csv",
index=False)
```

```
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
for col, (name, (Xa, Xb, ya, yb, num, cat)) in enumerate(DATA.items()):
    for mname, s in curves[name].items():
        f_, t_, _ = roc_curve(yb, s); axes[0, col].plot(f_, t_, label=f"
{mname} ({roc_auc_score(yb, s):.3f})")
        p_, r_, _ = precision_recall_curve(yb, s); axes[1, col].plot(r_,
p_, label=f"{mname} ({average_precision_score(yb, s):.3f})")
    axes[0, col].plot([0, 1], [0, 1], "k--", lw=.8); axes[0,
col].set(xlabel="false positive rate", ylabel="true positive rate",
title=f"{name}: ROC (AUC)")
    axes[1, col].axhline(yb.mean(), color="k", ls="--", lw=.8,
label=f"random = {yb.mean():.3f}")
    axes[1, col].set(xlabel="recall", ylabel="precision", title=f"
{name}: precision-recall (PR-AUC)")
    axes[0, col].legend(fontsize=8); axes[1, col].legend(fontsize=8)
plt.tight_layout(); plt.savefig(f"{FIG}/fig_w6_roc_pr_comparison.png",
dpi=150, bbox_inches="tight"); plt.show()
```

	dataset	model	accuracy	precision	recall	f1	roc_auc	pr_auc	threshold
0	Lending Club	Logistic regression (W4)	0.6527	0.3236	0.6530	0.4328	0.7126	0.3782	0.5098
1	Lending Club	Decision tree, pruned (W6)	0.6060	0.3009	0.7113	0.4229	0.7006	0.3531	0.4604
2	Lending Club	Random forest, tuned (W6)	0.6422	0.3191	0.6735	0.4331	0.7099	0.3801	0.4952
3	Lending Club	RBF SVM, tuned (W5, 6k rows)	0.6769	0.3357	0.6049	0.4318	0.7014	0.3627	0.0000
4	Taiwan	Logistic regression (W4)	0.7586	0.4629	0.5694	0.5107	0.7410	0.4993	0.5396
5	Taiwan	Decision tree, pruned (W6)	0.7724	0.4872	0.5460	0.5149	0.7545	0.4923	0.5756
6	Taiwan	Random forest, tuned (W6)	0.7837	0.5104	0.5535	0.5311	0.7733	0.5507	0.5428
7	Taiwan	RBF SVM, tuned (W5, 6k rows)	0.7492	0.4496	0.5950	0.5122	0.7443	0.4890	0.0000



11. Conclusion

Dataset	Model	ROC-AUC	PR-AUC	Precision	Recall	F1	Accuracy
Lending Club	Logistic regression	**0.7126**	0.3782	0.324	0.653	0.433	0.653
Lending Club	Decision tree (pruned)	0.7006	0.3531	0.301	**0.711**	0.423	0.606
Lending Club	**Random forest**	0.7099	**0.3801**	0.319	0.674	**0.433**	0.642
Lending Club	RBF SVM (6k rows)	0.7014	0.3627	**0.336**	0.605	0.432	**0.677**
Taiwan	Logistic regression	0.7410	0.4993	0.463	0.569	0.511	0.759
Taiwan	Decision tree (pruned)	0.7545	0.4923	0.487	0.546	0.515	0.772
Taiwan	**Random forest**	**0.7733**	**0.5507**	**0.510**	0.554	**0.531**	**0.784**
Taiwan	RBF SVM (6k rows)	0.7443	0.4890	0.450	**0.595**	0.512	0.749

On Lending Club, the four ROC curves nearly overlap. On Taiwan, the random-forest curve sits visibly above the others at low false-positive rates, and its PR curve is highest across most recall levels.

Taiwan: the random forest wins clearly. It leads on every threshold-free metric — +0.032 ROC-AUC and +0.051 PR-AUC over logistic regression, and +0.019 AUC over the pruned tree. The data have threshold and interaction structure (`PAY_1 ≥ 2` , bill × payment status) that the forest captures and a linear model cannot.

Lending Club: a four-way tie. All models fall within 0.012 AUC (0.701–0.713), and the F1 scores are within 0.01. Logistic regression is marginally best on ROC-AUC, the forest marginally best on PR-AUC. The application features carry a roughly linear signal (much of it already summarized in `int_rate` and grade), so model flexibility has nothing extra to find.

No model is uniformly best on accuracy, precision, and recall — those depend on the threshold. ROC-AUC and PR-AUC are the fair comparison, and the cost of each error type should drive the final cutoff.

Key Limitations

- **Calibration.** None of these probabilities have been checked for calibration (whether 30% predicted means 30% observed). That is required before using them in loss forecasting.

- **Time stability.** The Lending Club split is random, not by date, so performance over time has not been checked.
- **Associations, not causation.** The reported importances and coefficients describe predictive reliance within fitted models, not causal effects. A high `int_rate` does not cause default; it is a priced reflection of risk.

Overall

The depth model choice is justified. The random forest is the strongest or tied-strongest model on both datasets. Logistic regression stays the interpretable baseline and is the practical choice on Lending Club given equal accuracy. The generalization lesson is that the gain from nonlinear models depends on the data: it is large where behaviour has threshold effects (Taiwan's repayment history) and nil where the risk signal is already priced in (Lending Club's rate and grade).